FOURIER ANALYSIS ALGEBRA ALGORITHMS

The Discrete Algebra of the Fourier Transform

Gabriel Peyré

CNRS & DMA
École Normale Supérieure, PSL University

Adapted English edition with Python
September 9, 2026



Cover: detail of a painting by Louis Peyré (1923–2012).

Adapted and translated from the author's French book
L'algèbre discrète de la transformée de Fourier
Ellipses, 2004 ISBN 9782729818678

https://mathematical-tours.github.io/daft/

Preface

This book develops the algebra behind the Fourier transform and shows how that algebra leads to practical algorithms. It is intended for readers who want to move between mathematics and computation: an engineer seeking to understand why a transform works, a mathematician looking for applications of finite groups, or a programmer implementing a spectral method.

About this edition. *The Discrete Algebra of the Fourier Transform* is an adapted English translation of Gabriel Peyré's French book *L'algèbre discrète de la transformée de Fourier*, published by Ellipses in 2004 (ISBN 9782729818678). The publisher's page describes the original French edition. The present edition revises the exposition, corrects mathematical and transcription errors, redraws the illustrations, and expresses the computational examples in Python. The previous English sources and their MATLAB and Maple programs are preserved in the accompanying `legacy/` directory.

The first two chapters explain Fourier analysis on finite abelian groups and its applications to duality, quadratic reciprocity, and the Walsh transform. Chapters 3 and 4 develop the discrete Fourier transform, fast algorithms, and applications to filtering, interpolation, and differential equations. Chapter 5 studies related transforms, including the Hartley, z-, and fractional Fourier transforms. Chapter 6 introduces Fourier transforms over finite fields and their use in error-correcting codes. The final two chapters extend the theory to noncommutative groups through linear representations and their characters.

The exposition assumes undergraduate linear algebra and basic familiarity with groups and complex numbers. Additional prerequisites are introduced where needed. The chapters on finite fields and representations require more algebraic maturity. Readers primarily interested in numerical algorithms may start with Chapter 3 and return to the algebraic foundations as needed.

Exercises connect the theory to explicit calculations and applications. They are numbered within each chapter, with detailed solutions at the end of that chapter. The two computational appendices explain the numerical and exact-arithmetic conventions used in the Python programs. Numerical examples use NumPy and SciPy, symbolic calculations use SymPy, and finite-field calculations use the `galois` package. Displayed programs are drawn from the accompanying Python source files so that the formulas, implementations, and executable checks can be studied together.

Convolution is a recurring theme. On a finite group it comes from multiplication in the group algebra; in signal processing it describes filtering; in probability it gives the law of a sum of independent variables. The Fourier transform turns each of these operations into a simpler calculation. Following that principle across different settings is the organizing thread of the book.

Contents

Preface ii

1 Fourier analysis on finite groups 1
- 1.1 Dual of a finite group . . . 1
- 1.2 Dual of an abelian group . . . 3
- 1.3 Dual of a noncommutative group . . . 6
- 1.4 The Fourier transform . . . 8
- 1.5 Exercises . . . 11
- 1.6 Solutions to the exercises . . . 16

2 Applications of Fourier duality 21
- 2.1 Gauss sums . . . 21
- 2.2 The Walsh transform . . . 26
- 2.3 Poisson summation . . . 29
- 2.4 Exercises . . . 33
- 2.5 Solutions to the exercises . . . 43

3 The discrete Fourier transform 50
- 3.1 The language of signal processing . . . 50
- 3.2 Fast Fourier transforms . . . 51
- 3.3 Circular and linear convolution . . . 56
- 3.4 Higher-dimensional transforms . . . 58
- 3.5 Symmetry and spectral structure . . . 59
- 3.6 Exercises . . . 61
- 3.7 Solutions to the exercises . . . 70

4 Applications of the discrete Fourier transform 78
- 4.1 Connection with the Fourier transform on $\mathbb{R}$ . . . 78
- 4.2 Filtering . . . 79
- 4.3 Geometric aspects of filtering . . . 82
- 4.4 Numerical solution of partial differential equations . . . 87
- 4.5 Fast products . . . 89
- 4.6 Exercises . . . 91
- 4.7 Solutions to the exercises . . . 99

5 Transforms related to the discrete Fourier transform 109
- 5.1 Hartley transform . . . 109
- 5.2 Z transform and applications . . . 111
- 5.3 Vector Z transform and the chirp algorithm . . . 115
- 5.4 Scaled-frequency Fourier transforms . . . 117
- 5.5 Exercises . . . 119
- 5.6 Solutions to the exercises . . . 124

6 Fourier transforms over finite fields and rings 131
- 6.1 Calculations over finite fields . . . 131
- 6.2 Calculations over rings . . . 133
- 6.3 Error-correcting codes . . . 135
- 6.4 Error-correcting codes and Fourier duality . . . 140

6.5 Exercises . . . 143
6.6 Solutions to the exercises . . . 148

7 Linear representations of finite groups **156**
7.1 Representations and their decomposition . . . 156
7.2 Invariant vectors and averaging . . . 159
7.3 Characters . . . 161
7.4 Multiplicities and dimension formulas . . . 162
7.5 Fourier analysis with matrix coefficients . . . 163
7.6 Exercises . . . 165
7.7 Solutions to the exercises . . . 170

8 Applications of linear representations **175**
8.1 Representations of familiar groups . . . 175
8.2 Normal subgroups from character values . . . 177
8.3 Spectral analysis on a noncommutative group . . . 178
8.4 Exercises . . . 180
8.5 Solutions to the exercises . . . 183

A Python programs for numerical computation **187**
A.1 Fast Walsh transform . . . 187
A.2 Fast Hartley transform . . . 188
A.3 Fast Fourier transforms . . . 189
A.4 Integer multiplication by FFT . . . 190
A.5 The Dirichlet Poisson equation . . . 192
A.6 The periodic heat equation . . . 193

B Python programs for exact arithmetic **194**
B.1 Fourier transforms over finite fields . . . 194
B.2 Fourier transforms over residue rings . . . 195
B.3 Multiplication of large integers . . . 196
B.4 BCH encoding and decoding . . . 197

References **200**

CHAPTER 1

Fourier analysis on finite groups

Fourier analysis represents a function by elementary components adapted to the symmetries of its domain. On a finite abelian group, these components are the characters, and all calculations involve finite sums. This chapter constructs the Fourier transform, proves inversion and orthogonality formulas, and explains why convolution becomes multiplication.

The finite setting separates the algebra of Fourier analysis from questions of convergence and regularity. It also provides the foundations of the fast transforms developed later. We begin with cyclic groups, extend the theory to finite abelian groups, and identify the obstruction in the noncommutative case. For complementary accounts, see Serre [55], Warusfel [66], and Dym and McKean [25].

1.1 Dual of a finite group

A function $f : G \to \mathbb{C}$ can always be represented by its values. To exploit the group structure, we instead seek elementary functions that respect multiplication in G.

1.1.1 Characters and function spaces

Characters capture the multiplicative structure of a group in scalar-valued functions. We introduce them alongside the space of all functions on the group, where they will eventually serve as Fourier coordinates.

Definition 1 (Characters and the dual group)**.** A *linear character* of a finite group G is a group homomorphism $\chi : G \to \mathbb{C}^*$. The set of these characters is denoted by $\hat{G}$ and is called the *dual group*. In the present chapter, "character" always means linear character. Chapter 7 introduces the more general characters of linear representations.

Pointwise multiplication makes $\hat{G}$ an abelian group:

$$(\chi_1\chi_2)(x) = \chi_1(x)\chi_2(x).$$

Its identity is the constant character 1, and the inverse of χ is $x \mapsto \chi(x)^{-1}$.

Because the group is finite, a character can take only a restricted set of complex values; the following properties will be used repeatedly in inner-product calculations.

Proposition 1. *If $|G| = n$, every character takes values in the group of nth roots of unity,*

$$\mathbb{U}_n = \{e^{2\pi ik/n} : 0 \le k < n\}.$$

Consequently, for every $g \in G$,

$$|\chi(g)| = 1, \qquad \chi(g^{-1}) = \chi(g)^{-1} = \overline{\chi(g)}.$$

Proof. Lagrange's theorem gives $g^n = 1$. Applying χ yields $\chi(g)^n = \chi(g^n) = 1$. □

These restrictions have two immediate consequences for the size of the dual and the information carried by a character.

Remark 1. There are only finitely many maps $G \to \mathbb{U}_n$, so $\hat{G}$ is finite. Every linear character is constant on conjugacy classes, since

$$\chi(h^{-1}gh) = \chi(h)^{-1}\chi(g)\chi(h) = \chi(g). \tag{1.1}$$

To discuss expansions in characters, we place them in a vector space with an inner product adapted to the uniform measure on the group.

Definition 2 (Functions on G)**.** Let $\mathbb{C}[G]$ denote the complex vector space of functions $G \to \mathbb{C}$. We equip it with the normalized Hermitian inner product

$$\langle f, g\rangle = \frac{1}{|G|}\sum_{x\in G} f(x)\overline{g(x)}, \qquad \|f\|_2^2 = \langle f, f\rangle. \tag{1.2}$$

The inner product is linear in its first argument.

The normalization in this inner product is natural from both the analytic and the symmetry viewpoints.

Remark 2. This inner product is the finite analogue of $\langle f, g\rangle = \int_{\mathbb{R}} f(x)\overline{g(x)}\,\mathrm{d}x$ on $L^2(\mathbb{R})$. Both are translation invariant. For example, setting $T_y f(x) = f(xy)$ gives $\langle T_y f, T_y g\rangle = \langle f, g\rangle$ because $x \mapsto xy$ permutes G.

Before constructing a frequency basis, we record the coordinate basis associated with the individual group elements.

Proposition 2. *The functions $(\delta_g)_{g\in G}$, defined by*

$$\delta_g(h) = \begin{cases} 1 & h = g, \\ 0 & h \neq g, \end{cases} \tag{1.3}$$

form a basis of $\mathbb{C}[G]$. In particular, $\dim_{\mathbb{C}} \mathbb{C}[G] = |G|$.

Proof. Evaluating a linear combination at each $h \in G$ proves linear independence. The identity

$$f = \sum_{g\in G} f(g)\delta_g \tag{1.4}$$

proves that the family spans the space. With the normalization (1.2), these basis vectors are orthogonal but have squared norm $1/|G|$. Thus $(\sqrt{|G|}\,\delta_g)_{g\in G}$ is orthonormal. □

The injection $g \mapsto \delta_g$ will later give the group algebra its multiplication. For Fourier analysis, we seek a different basis: one that respects translations and simplifies convolution.

1.1.2 Dual of a cyclic group

Cyclic groups are both the simplest examples and the domains of the one-dimensional discrete transforms used in signal processing.

A character on a cyclic group is determined by its value at a generator, which makes the entire dual explicit.

Proposition 3 (The cyclic case)**.** *Let $G = \langle g_0\rangle$ have order n, and set $\omega = e^{2\pi \mathrm{i}/n}$. Its characters are exactly*

$$\chi_j(g_0^k) = \omega^{jk} = e^{2\pi \mathrm{i} jk/n}, \qquad 0 \le j < n.$$

The map $j \mapsto \chi_j$ is an isomorphism $\mathbb{Z}/n\mathbb{Z} \to \hat{G}$.

Proof. A character is determined by its value at g_0, which must be an nth root of unity. Each choice $\chi(g_0) = \omega^j$ defines a character, since the formula is unchanged when k is replaced by $k + n$. The resulting characters are distinct and satisfy $\chi_j\chi_k = \chi_{j+k}$, with indices modulo n. □

Figure 1.1 shows χ_j for $j = 0, 1, 2, 3$ on $\mathbb{Z}/12\mathbb{Z}$. Their real and imaginary parts sample $\cos(2\pi jx/12)$ and $\sin(2\pi jx/12)$ at integer values of x.

The formula also shows precisely which choices enter the identification of a cyclic group with its dual.

Remark 3. An abstract cyclic group has no preferred generator. Choosing a generator and a primitive root of unity determines the isomorphism with its dual. This choice dependence is analogous to the choice of a basis when identifying a vector space with its dual.

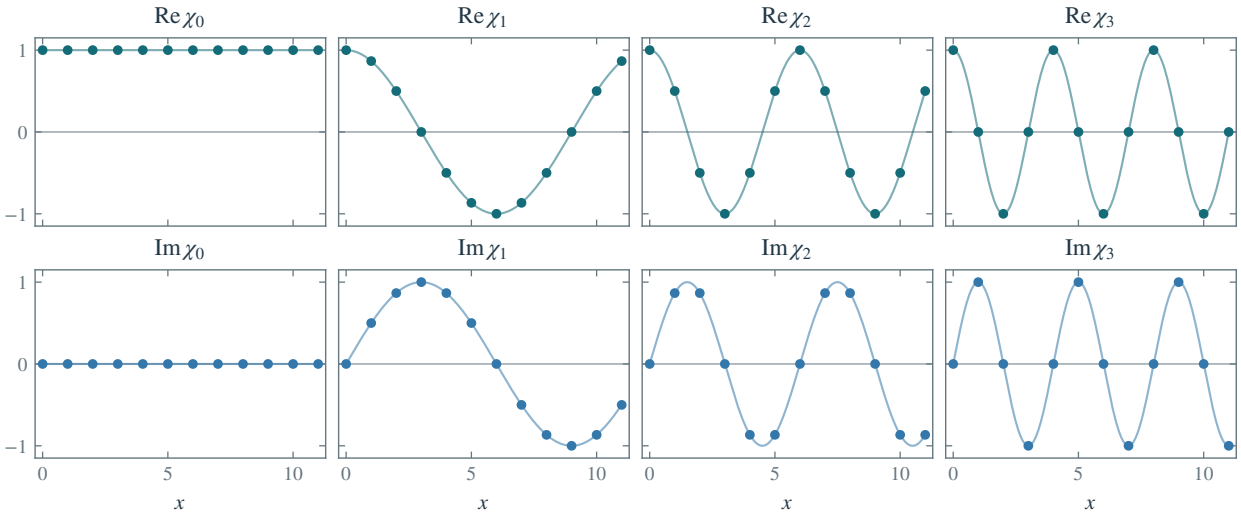


Figure 1.1. Real and imaginary parts of the first four characters of $\mathbb{Z}/12\mathbb{Z}$.

For the standard cyclic group, the usual residue coordinates provide a particularly convenient choice.

Remark 4. Every cyclic group of order n is isomorphic to $\mathbb{Z}/n\mathbb{Z}$. For this concrete group, the pairing $(j, k) \mapsto e^{2\pi ijk/n}$ gives the familiar identification

$$\widehat{\mathbb{Z}/n\mathbb{Z}} \simeq \mathbb{Z}/n\mathbb{Z}.$$

The structure theorem will extend the existence of such an isomorphism to all finite abelian groups.

These explicit characters supply not just a group isomorphism but an orthogonal coordinate system for functions.

Proposition 4. *For a cyclic group G, the characters form an orthonormal basis of $\mathbb{C}[G]$: $\langle \chi_p, \chi_q \rangle = \delta_p^q$, where δ_p^q is 1 if $p = q$ and 0 otherwise.*

Proof. Using the preceding notation,

$$\langle \chi_p, \chi_q \rangle = \frac{1}{n} \sum_{k=0}^{n-1} \omega^{(p-q)k} = \delta_p^q. \tag{1.5}$$

For $p \neq q$, the sum is a geometric series with ratio $\omega^{p-q} \neq 1$ and vanishes. For $p = q$, every summand is 1. There are n orthonormal characters in an n-dimensional space, so they form a basis. □

1.2 Dual of an abelian group

We now prove that the characters of any finite abelian group form an orthonormal basis. The key algebraic step is extension of a character from a subgroup to the whole group.

1.2.1 Character extension

To compare the frequencies of a group with those of a subgroup, we must know whether a character on the subgroup extends to the whole group. The extension argument below is the key to counting characters without assuming the structure theorem for finite abelian groups.

Lemma 1 (Character extension)**.** *Let H be a subgroup of a finite abelian group G. Every character of H extends to a character of G.*

Proof. We induct on the index $[G : H]$. There is nothing to prove when $H = G$. Otherwise, choose $x \in G \setminus H$ and let $K = \langle H, x \rangle$. Let n be the smallest positive integer such that $x^n \in H$; thus $n = [K : H] \geq 2$.

Every element of K has a unique expression yx^k, with $y \in H$ and $0 \leq k < n$. Existence follows by reducing the exponent modulo n. For uniqueness, $yx^k = y'x^{k'}$ implies $x^{k-k'} \in H$; the minimality of n and $|k - k'| < n$ force $k = k'$ and then $y = y'$.

Given $\chi \in \hat{H}$, choose a complex number ζ satisfying $\zeta^n = \chi(x^n)$. Such a root exists and has modulus one. Define

$$\tilde{\chi}(yx^k) = \chi(y)\zeta^k, \qquad y \in H, \quad 0 \le k < n. \tag{1.6}$$

This is well defined by uniqueness and agrees with χ on H. To check multiplicativity, write $k + k' = qn + r$, with $q \in \{0, 1\}$ and $0 \le r < n$. Since G is abelian,

$$\begin{aligned}\tilde{\chi}\big((yx^k)(y'x^{k'})\big) &= \chi(yy'x^{qn})\zeta^r \\ &= \chi(y)\chi(y')\chi(x^n)^q\zeta^r \\ &= \chi(y)\chi(y')\zeta^{k+k'} = \tilde{\chi}(yx^k)\tilde{\chi}(y'x^{k'}).\end{aligned}$$

Thus $\tilde{\chi}$ is a character of K. Finally, $[G : K] = [G : H]/n < [G : H]$, so the induction hypothesis extends it to G. □

The root must satisfy $\zeta^n = \chi(x^n)$; it need not be an nth root of *unity*. For example, if $G = \langle x \rangle$ has order 4, $H = \{1, x^2\}$, and $\chi(x^2) = -1$, then ζ must be i or −i. In general, an extension is not unique.

Restriction and pullback organize the extension lemma into an exact sequence, relating the duals of a group, a subgroup, and the quotient.

Lemma 2. *For a subgroup H of a finite abelian group G, restriction $\rho : \hat{G} \to \hat{H}$ and pullback $j : \widehat{G/H} \to \hat{G}$, $j(\eta)(x) = \eta(xH)$, give an exact sequence*

$$\{1\} \longrightarrow \widehat{G/H} \xrightarrow{\ j\ } \hat{G} \xrightarrow{\ \rho\ } \hat{H} \longrightarrow \{1\}.$$

Proof. The quotient map $G \to G/H$ is surjective, so its pullback j is injective. Character extension makes ρ surjective. A character lies in $\operatorname{Ker}\rho$ precisely when it is trivial on H, in which case it factors uniquely through G/H. Hence $\operatorname{Ker}\rho = \Im j$. □

Counting the kernel and image in this sequence gives the number of characters of any finite abelian group.

Corollary 1. *A finite abelian group and its dual have the same order.*

Proof. Induct on $|G|$. The result holds for cyclic groups, including the trivial group. Otherwise choose a nontrivial proper cyclic subgroup H. Exactness and the induction hypothesis give

$$|\hat{G}| = |\widehat{G/H}|\,|\hat{H}| = |G/H|\,|H| = |G|.$$

□

1.2.2 Isomorphism with the dual

A finite abelian group and its dual have not only the same order but the same group structure. We prove this by decomposing the group into cyclic factors, keeping track of the choices in the resulting isomorphism. The algebraic input is the following structure theorem; see Artin [3] for a proof.

Theorem 1 (Structure theorem for finite abelian groups)**.** *There is a unique sequence of integers $2 \le n_1 \mid n_2 \mid \cdots \mid n_r$ such that*

$$G \simeq \mathbb{Z}/n_1\mathbb{Z} \times \cdots \times \mathbb{Z}/n_r\mathbb{Z}.$$

For the trivial group, the sequence is empty. The integers are unique; the isomorphism generally is not.

Applying cyclic duality to each factor in the structure theorem yields the desired isomorphism.

Corollary 2 (Isomorphism theorem)**.** *Every finite abelian group G is isomorphic to $\hat{G}$.*

Proof. There is a canonical isomorphism $\widehat{G \times H} \simeq \hat{G} \times \hat{H}$: restrict a character to each factor. Its inverse sends (χ, η) to $(g, h) \mapsto \chi(g)\eta(h)$. Apply this to the cyclic factors in the structure theorem and use Proposition 3. □

It is useful to separate the existence of this isomorphism from the question of whether it is canonical.

Remark 5. The proof requires a choice of cyclic decomposition and generators. It gives an isomorphism $G \simeq \hat{G}$, but no preferred identification for an arbitrary finite abelian group. The situation changes when we pass to the bidual.

1.2.3 The bidual

An isomorphism between a group and its dual requires choices, but passing to the dual twice removes this ambiguity. We construct the bidual and show how evaluation recovers each group element in a canonical way.

Definition 3 (Bidual)**.** The *bidual* of a finite abelian group G is $\hat{\hat{G}} = \widehat{\hat{G}}$.

Evaluation against every character associates a bidual element with each group element, without choosing generators.

Proposition 5 (Canonical isomorphism)**.** *Evaluation gives a canonical isomorphism*

$$\Phi : G \longrightarrow \hat{\hat{G}}, \qquad \Phi(g)(\chi) = \chi(g). \tag{1.7}$$

Proof. For each g, evaluation is a character of $\hat{G}$, and $\Phi(gh) = \Phi(g)\Phi(h)$. To prove injectivity, let $g \neq 1$ have order m. On $\langle g \rangle$, the character sending g to $e^{2\pi \mathrm{i}/m}$ is nontrivial at g. Extend it to G using Lemma 1. Therefore $\Phi(g) \neq 1$. Since $|G| = |\hat{\hat{G}}|$, injectivity implies surjectivity. □

This construction has familiar linear-algebraic and broader analytic counterparts.

Remark 6. This is the analogue of the canonical map from a finite-dimensional vector space to its double dual. For infinite groups, the choice of characters and topology matters. With continuous unit-modulus characters, Pontryagin duality extends the bidual theorem to locally compact abelian groups. For example, the dual of $\mathbb{R}/(2\pi\mathbb{Z})$ is $\mathbb{Z}$, and the dual of $\mathbb{Z}$ is again the circle. The fact that a group differs from its *dual* does not prevent its canonical identification with its *bidual*.

1.2.4 Orthogonality relations

Orthogonality makes character expansions easy to compute and invert. We prove it by exploiting cancellation in character sums, then combine orthogonality with the count of characters to obtain a basis.

Lemma 3. *For a character χ of a finite abelian group G,*

$$\sum_{g \in G} \chi(g) = \begin{cases} |G| & \chi = 1, \\ 0 & \chi \neq 1. \end{cases} \tag{1.8}$$

Proof. The trivial character gives the first case. Otherwise choose t with $\chi(t) \neq 1$. Translation permutes the summation indices, so

$$\chi(t) \sum_{g \in G} \chi(g) = \sum_{g \in G} \chi(tg) = \sum_{g \in G} \chi(g).$$

Subtracting and dividing by $\chi(t) - 1$ gives the second case. □

Applying the cancellation lemma to the product of one character with the conjugate of another gives orthogonality.

Proposition 6 (Orthogonality of characters)**.** *For $\chi_1, \chi_2 \in \hat{G}$,*

$$\langle \chi_1, \chi_2 \rangle = \begin{cases} 1 & \chi_1 = \chi_2, \\ 0 & \chi_1 \neq \chi_2. \end{cases}$$

Proof. Apply the preceding lemma to the character $\chi_1 \overline{\chi_2}$. It is trivial exactly when $\chi_1 = \chi_2$. □

The character count now ensures that this orthogonal family is complete.

Corollary 3. *The characters of a finite abelian group G form an orthonormal basis of $\mathbb{C}[G]$.*

Proof. They are orthonormal, and their number is $|G| = \dim_{\mathbb{C}} \mathbb{C}[G]$. □

Biduality lets us exchange the roles of group elements and characters, giving a second orthogonality relation.

Proposition 7 (Dual orthogonality)**.** *For $g, h \in G$,*

$$\sum_{\chi \in \hat{G}} \chi(g)\overline{\chi(h)} = |G|\delta_g^h.$$

Proof. Apply character orthogonality to $\hat{G}$ and the evaluation characters $\Phi(g), \Phi(h)$. This gives

$$\sum_{\chi \in \hat{G}} \Phi(g)(\chi)\overline{\Phi(h)(\chi)} = |\hat{G}|\delta_{\Phi(g)}^{\Phi(h)} = |G|\delta_g^h. \tag{1.9}$$

□

In matrix form, the two relations express the same unitary change of coordinates.

Remark 7. Enumerate $G = \{g_1, \dots, g_n\}$ and $\hat{G} = \{\chi_1, \dots, \chi_n\}$. For the character table $M_{ij} = \chi_i(g_j)$, the two orthogonality relations read $MM^* = M^*M = nI$. Thus $M/\sqrt{n}$ is unitary. The two identities express orthogonality of the rows and columns, respectively.

1.3 Dual of a noncommutative group

For noncommutative groups, linear characters no longer suffice to represent all functions. The symmetric group gives a concrete example, and the abelianization of a general group explains the obstruction.

1.3.1 The symmetric group

The symmetric group gives a concrete test of how much information scalar characters retain. We construct the sign character and show that, apart from the trivial character, it is the only one available.

Definition 4 (Sign of a permutation)**.** Write $\sigma \in \mathfrak{S}_n$ as a product of disjoint cycles, including cycles of length one. If there are k cycles, its *sign* is

$$\epsilon(\sigma) = (-1)^{n-k}.$$

Uniqueness of disjoint cycle decomposition makes this definition unambiguous; see Lang [36].

The effect of a single transposition is the essential step in proving that the sign is multiplicative.

Lemma 4. *For a transposition τ, one has $\epsilon(\sigma\tau) = -\epsilon(\sigma)$.*

Proof. Write $\tau = (a, b)$. If a and b belong to the same cycle of σ, right multiplication by τ splits that cycle into two. Otherwise it joins their two cycles. In either case the number of cycles changes by one. □

Iterating this observation relates cycle decomposition to transposition parity and proves the character property.

Proposition 8. *If σ is a product of p transpositions, then $\epsilon(\sigma) = (-1)^p$. Moreover, ϵ is a homomorphism $\mathfrak{S}_n \to \{1, -1\}$.*

Proof. Starting from the identity, apply the preceding lemma p times. Concatenating decompositions of two permutations then proves multiplicativity. □

Although a transposition decomposition is not unique, its parity is therefore well defined. An alternative definition is the determinant of the permutation matrix.

We can now determine all scalar characters of the symmetric group.

Proposition 9. *For $n \geq 2$, the only nontrivial linear character of $\mathfrak{S}_n$ is its sign ϵ.*

Proof. Transpositions generate $\mathfrak{S}_n$ and are all conjugate, so a character has a common value a on them. Since a transposition squares to the identity, $a^2 = 1$. If $a = 1$, the character is trivial. If $a = -1$, its value on a product of p transpositions is $(-1)^p$, so it equals ϵ. □

Thus $\widehat{\mathfrak{S}_n} \simeq \mathbb{Z}/2\mathbb{Z}$ for $n \geq 2$. For $n = 1$, both $\mathfrak{S}_1$ and its dual are trivial.

1.3.2 The derived subgroup

The limitation seen for permutations has a general explanation: scalar characters cannot detect commutators. We identify the quotient that they do detect and use it to measure the information lost when the group is noncommutative.

Definition 5 (Derived subgroup)**.** The *commutator* of $x, y \in G$ is $[x, y] = xyx^{-1}y^{-1}$. The subgroup generated by all commutators,

$$D(G) = \langle [x, y] : x, y \in G \rangle,$$

is the *derived subgroup* (or commutator subgroup).

The derived subgroup is designed to remove exactly the obstruction to commutativity.

Proposition 10 (Properties of the derived subgroup)**.** *The subgroup $D(G)$ is normal, $G/D(G)$ is abelian, and $D(G) = \{1\}$ if and only if G is abelian.*

Proof. Every automorphism φ satisfies $\varphi([x, y]) = [\varphi(x), \varphi(y)]$ and therefore preserves $D(G)$. In particular, inner automorphisms preserve it, proving normality. In the quotient, $\overline{x}\,\overline{y}\,\overline{x}^{-1}\overline{y}^{-1} = 1$, so all elements commute. Finally, all commutators are trivial exactly when G is abelian. □

Since complex scalars commute, every character factors through the resulting abelian quotient.

Proposition 11. *For a finite group G, there is a canonical isomorphism $\hat{G} \simeq \widehat{G/D(G)}$. Consequently, $\hat{G} \simeq G/D(G)$, with the second isomorphism generally depending on choices.*

Proof. The target $\mathbb{C}^*$ is abelian, so every character is trivial on commutators and hence on $D(G)$. It therefore factors uniquely through the quotient:

$$\overline{\chi}(xD(G)) = \chi(x).$$

Conversely, pullback along $G \to G/D(G)$ gives a character of G. These constructions are inverse homomorphisms. Since $G/D(G)$ is finite abelian, Corollary 2 gives the final assertion. □

This factorization also gives a direct count of the scalar characters and explains why they cannot form a basis in the nonabelian case.

Remark 8. The factorization is expressed by the commutative diagram

$$\begin{array}{ccc} G & \xrightarrow{\chi} & \mathbb{C}^* \\ {\scriptstyle\pi}\downarrow & & \| \\ G/D(G) & \xrightarrow{\overline{\chi}} & \mathbb{C}^*. \end{array}$$

In particular, $|\hat{G}| = |G|/|D(G)|$. When G is nonabelian this is strictly less than $|G|$, so linear characters cannot span $\mathbb{C}[G]$.

The alternating group $\mathfrak{A}_n = \operatorname{Ker} \epsilon$ is the normal subgroup of even permutations.

To identify the derived subgroup of the symmetric group, we first need a small generating family for its even permutations.

Lemma 5. *For $n \geq 3$, the group $\mathfrak{A}_n$ is generated by 3-cycles.*

Proof. The transpositions $(1, i)$ generate $\mathfrak{S}_n$, since $(i, j) = (1, i)(1, j)(1, i)$ for distinct $i, j \neq 1$. An even permutation is a product of an even number of these transpositions. Group them into pairs. A pair of equal transpositions is the identity; a pair of distinct ones is a 3-cycle. □

These generators allow us to recover the earlier character calculation from the general quotient construction.

Proposition 12 (The symmetric group)**.** *For $n \geq 3$, $D(\mathfrak{S}_n) = \mathfrak{A}_n$ and $\widehat{\mathfrak{S}_n} \simeq \mathfrak{S}_n/\mathfrak{A}_n \simeq \mathbb{Z}/2\mathbb{Z}$.*

Proof. Since ϵ is a character, $D(\mathfrak{S}_n) \subset \mathfrak{A}_n$. For a 3-cycle σ, choose τ with $\tau\sigma\tau^{-1} = \sigma^2$; for instance, exchange two entries of the cycle. Then $[\tau, \sigma] = \sigma$. The preceding lemma gives the reverse inclusion. The quotient is isomorphic to $\{1, -1\}$ by the sign map. □

The missing Fourier coordinates require a richer notion of character, which will motivate the representation theory developed later.

Remark 9. To recover Fourier analysis on a noncommutative group, we replace scalar-valued homomorphisms by matrix-valued representations. Linear characters correspond to representations of dimension one. Higher-dimensional representations supply the missing components; this is the subject of Chapter 7.

1.4 The Fourier transform

Throughout this section, G is a finite abelian group. Its character basis now gives a systematic decomposition of every function on G.

1.4.1 Fourier coefficients and the Fourier transform

The character basis lets us replace function values by frequency coordinates. We distinguish the normalized coefficients of a basis expansion from the unnormalized Fourier transform used below, then derive reconstruction and energy identities with the corresponding constants.

Definition 6 (Fourier coefficients)**.** For $f \in \mathbb{C}[G]$ and $\chi \in \hat{G}$, define

$$c_f(\chi) = \langle f, \chi \rangle = \frac{1}{|G|} \sum_{x \in G} f(x)\overline{\chi(x)}.$$

This defines a linear map $c : \mathbb{C}[G] \to \mathbb{C}[\hat{G}]$, $f \mapsto c_f$.

For convolution and computation, it is convenient to package the same information in an unnormalized transform with the following sign convention.

Definition 7 (Fourier transform)**.** The Fourier transform is the linear map

$$\mathcal{F} : \mathbb{C}[G] \longrightarrow \mathbb{C}[\hat{G}], \qquad f \longmapsto \hat{f}, \qquad \hat{f}(\chi) = \sum_{x \in G} f(x)\chi(x). \tag{1.10}$$

Thus $\hat{f}(\chi) = |G| c_f(\overline{\chi})$.

This convention makes evaluation of $\hat{f}$ an algebra homomorphism for convolution. The numerical DFT in Chapter 3 uses a negative exponential: it is the same transform evaluated at the inverse cyclic characters $\chi_k^{-1}(x) = e^{-2\pi i k x/n}$. Keeping track of this indexing choice avoids sign ambiguities.

Figure 1.2 shows a bell-shaped function on $\mathbb{Z}/17\mathbb{Z}$ and its transform, with centered indices $-8, \ldots, 8$. The coefficient of the trivial character is the sum of the function values.

Expansion in the character basis gives an explicit formula for recovering the original function.

Proposition 13 (Inversion formula)**.** *For every $f \in \mathbb{C}[G]$,*

$$f = \sum_{\chi \in \hat{G}} c_f(\chi)\chi = \frac{1}{|G|} \sum_{\chi \in \hat{G}} \hat{f}(\chi)\chi^{-1}. \tag{1.11}$$

Proof. The first equality is expansion in the orthonormal character basis. For the second, substitute $c_f(\chi) = \hat{f}(\chi^{-1})/|G|$ and reindex by $\chi \mapsto \chi^{-1}$. □

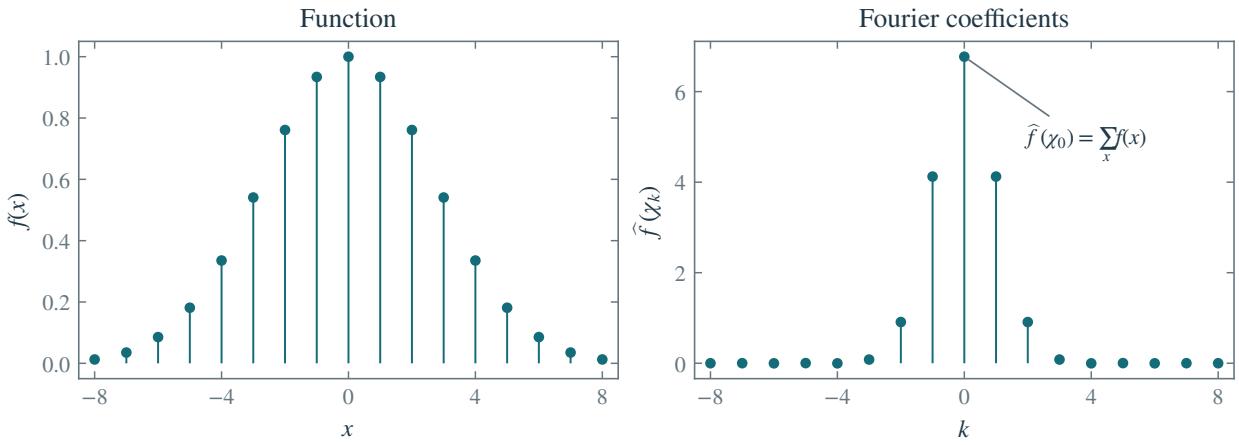


Figure 1.2. An even bell-shaped function on $\mathbb{Z}/17\mathbb{Z}$ and its real Fourier coefficients, with centered indices.

The reconstruction formula shows that either choice of Fourier coordinates retains all the information in the function.

Proposition 14 (Fourier isomorphism)**.** *The maps c and $\mathcal{F}$ are vector-space isomorphisms $\mathbb{C}[G] \to \mathbb{C}[\hat{G}]$.*

Proof. Inversion gives injectivity. The domain and codomain both have dimension $|G|$, so the maps are also surjective. □

Orthogonality also controls inner products and energy under this change of coordinates.

Proposition 15 (Plancherel's formula)**.** *For $f, g \in \mathbb{C}[G]$,*

$$\sum_{x \in G} f(x)\overline{g(x)} = |G| \sum_{\chi \in \hat{G}} c_f(\chi)\overline{c_g(\chi)} \tag{1.12}$$

$$= \frac{1}{|G|} \sum_{\chi \in \hat{G}} \hat{f}(\chi)\overline{\hat{g}(\chi)}. \tag{1.13}$$

Proof. Expand both functions in the orthonormal character basis. Orthogonality eliminates all terms involving distinct characters and gives the first identity. The relation between c_f and $\hat{f}$ gives the second. □

Keeping track of the normalization makes the geometric meaning of Plancherel's formula transparent.

Remark 10 (Normalization). With the normalized inner products on both $\mathbb{C}[G]$ and $\mathbb{C}[\hat{G}]$, Plancherel's formula reads

$$\langle f, g \rangle = \frac{1}{|G|} \langle \hat{f}, \hat{g} \rangle.$$

Thus $\mathcal{F}/\sqrt{|G|}$ is unitary. This is the finite analogue of conservation of the L^2 inner product by a suitably normalized continuous Fourier transform; see Section 4.1.

1.4.2 The group algebra

Convolution combines functions in a way that reflects the group law. We construct this product and its algebraic properties so that translation, multiplication of characters, and Fourier analysis fit into the same framework.

The pointwise product

$$(f_1 \cdot f_2)(g) = f_1(g) f_2(g) \tag{1.14}$$

turns $\mathbb{C}[G]$ into an algebra, but does not use multiplication in G. A second product does: set $\delta_g * \delta_h = \delta_{gh}$ and extend by bilinearity. Expanding two functions in the Dirac basis gives the following formula.

Definition 8 (Convolution)**.** For $f_1, f_2 \in \mathbb{C}[G]$, their convolution is

$$(f_1 * f_2)(g) = \sum_{hk=g} f_1(h) f_2(k) = \sum_{h \in G} f_1(h) f_2(h^{-1}g). \tag{1.15}$$

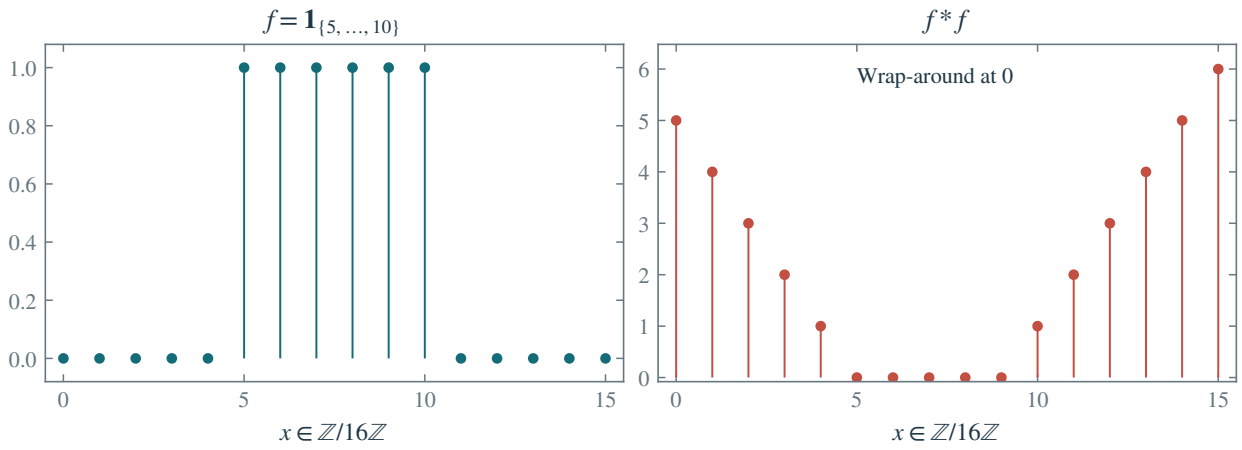


Figure 1.3. Circular self-convolution of $f = \mathbf{1}_{\{5,\ldots,10\}}$ on $\mathbb{Z}/16\mathbb{Z}$. The maximum occurs at 15 modulo 16; the remaining values wrap around at 0.

Convolution with a function concentrated at one point reveals its direct connection with translation.

Remark 11. Convolution with a Dirac function translates a function: $(f * \delta_g)(h) = f(hg^{-1})$. For cyclic groups this is circular translation, studied in Section 3.3.

The elementary properties of convolution make the function space into an algebra with an identity element.

Proposition 16. *Convolution is bilinear, associative, and commutative, with identity δ_1. It makes $\mathbb{C}[G]$ a commutative unital algebra.*

Proof. Bilinearity is part of the construction. On basis elements,

$$(\delta_g * \delta_h) * \delta_k = \delta_{(gh)k} = \delta_{g(hk)} = \delta_g * (\delta_h * \delta_k).$$

Bilinearity extends associativity to all functions. Commutativity follows from $gh = hg$, and $\delta_1 * \delta_g = \delta_g$ proves the identity property. □

Figure 1.3 illustrates self-convolution of a box signal on $\mathbb{Z}/16\mathbb{Z}$. The values count overlaps of translated supports. This viewpoint connects direct summation, FFT computation in Python, and multiplication of polynomials modulo $X^{16} - 1$.

Characters interact with this algebra in a particularly simple way: they extend uniquely to multiplicative linear functionals.

Proposition 17 (Extension to the group algebra)**.** *Every homomorphism $\rho : G \to \mathbb{C}^*$ extends uniquely to a unital $\mathbb{C}$-algebra homomorphism $\tilde{\rho} : \mathbb{C}[G] \to \mathbb{C}$.*

Proof. Linearity forces $\tilde{\rho}(f) = \sum_{g \in G} f(g)\rho(g)$. Since $\rho(gh) = \rho(g)\rho(h)$, this extension preserves products of basis elements and hence all convolution products. It maps δ_1 to 1. □

Conversely, a unital algebra homomorphism restricts to a group homomorphism on the invertible elements δ_g. The word *unital* excludes the zero homomorphism, which cannot take values in $\mathbb{C}^*$.

Convolution also describes the distribution obtained by combining independent random group elements.

Remark 12 (Probabilistic interpretation). Let X, Y be independent G-valued random variables, with probability mass functions P_X, P_Y. Independence gives

$$\mathbb{P}(XY = g) = \sum_{h \in G} \mathbb{P}(X = h)\mathbb{P}(Y = h^{-1}g) = (P_X * P_Y)(g).$$

In additive notation this is $P_{X+Y} = P_X * P_Y$. Analogous formulas hold for integer-valued variables and for real-valued variables with densities; see Ouvrard [45]. Exercises 1.9 and 1.10 develop finite-group applications.

1.4.3 Convolution and the Fourier transform

The practical strength of Fourier coordinates is that they turn convolution into multiplication. We first establish this property for a single character, then assemble the scalar identities into an isomorphism of algebras.

Proposition 18 (Fourier evaluation as an algebra homomorphism)**.** *For each $\chi \in \hat{G}$, the map $\mathcal{F}_\chi : f \mapsto \hat{f}(\chi)$ is the unique unital algebra homomorphism extending χ to $\mathbb{C}[G]$.*

Proof. The extension formula in Proposition 17 is precisely $\hat{f}(\chi) = \sum_g f(g)\chi(g)$. □

Collecting the multiplicative identities for all characters gives the convolution theorem in its two normalization conventions.

Theorem 2 (Convolution theorem)**.** *For $f, g \in \mathbb{C}[G]$,*

$$\widehat{f * g} = \hat{f} \cdot \hat{g}, \qquad c_{f*g} = |G| c_f \cdot c_g. \tag{1.16}$$

The Fourier transform is an algebra isomorphism from $(\mathbb{C}[G], *)$ to $(\mathbb{C}[\hat{G}], \cdot)$.

Proof. The first identity holds at every χ by the preceding proposition. The second follows from $\hat{f}(\chi) = |G| c_f(\overline{\chi})$. Inversion supplies the isomorphism assertion. □

Convolution has become pointwise multiplication. This simple structural fact will underlie fast filtering, polynomial multiplication, spectral solutions of differential equations, and the decoding of error-correcting codes.

1.5 Exercises

These exercises use Fourier coordinates to study counting, symmetry, random walks, and uncertainty. The first applies the character basis to a structured matrix, turning a determinant into a product of scalar quantities.

Exercise 1.1 (Circulant determinant)**.** Let G be cyclic and $f \in \mathbb{C}[G]$. Consider the convolution operator $\Phi^f(u) = f * u$ on $\mathbb{C}[G]$.

1. Show that every character is an eigenvector of Φ^f. Find the corresponding eigenvalue, keeping track of the Fourier sign convention.
2. Express the matrix A of Φ^f in the Dirac basis and deduce a formula for $\det A$.
3. With $\omega = e^{2\pi \mathrm{i}/n}$, show that

$$\det \begin{pmatrix} a_0 & a_1 & \cdots & a_{n-1} \\ a_{n-1} & a_0 & \cdots & a_{n-2} \\ \vdots & \vdots & \ddots & \vdots \\ a_1 & a_2 & \cdots & a_0 \end{pmatrix} = \prod_{i=0}^{n-1} \left(\sum_{j=0}^{n-1} a_j \omega^{ij} \right).$$

4. After reading Chapter 3, implement this determinant using an FFT. Discuss the numerical limitations of multiplying many eigenvalues.

Rotations provide another example of a noncommutative group whose scalar characters reveal very little.

Exercise 1.2 (Linear characters of $SO(3)$)**.** Let $SO(3)$ be the group of real orthogonal 3×3 matrices of determinant one. Show that every homomorphism $SO(3) \to \mathbb{C}^*$ is trivial.

1. Prove that rotations with the same angle in $[0, \pi]$ are conjugate.
2. Deduce that a character χ depends only on the rotation angle.
3. Let r_α and s_α be rotations through α about the first and second coordinate axes. Show that $r_\alpha s_\alpha^{-1}$ is a rotation through an angle β satisfying $\cos\beta = ((1 + \cos\alpha)^2 - 2)/2$. As α varies over $[0, \pi]$, show that β covers $[0, \pi]$.
4. Deduce that $\chi = 1$. No continuity assumption on χ is needed.

Character orthogonality can replace an equation constraint by a sum that is easier to manipulate algebraically.

Exercise 1.3 (Counting solutions)**.** Let G be finite abelian and $\varphi : G^n \to G$ any map. For $h \in G$, let $N(h)$ count the tuples satisfying $\varphi(g_1, \dots, g_n) = h$. Prove that

$$N(h) = \frac{1}{|G|} \sum_{g_1, \dots, g_n \in G} \sum_{\chi \in \hat{G}} \chi\big(\varphi(g_1, \dots, g_n)\big) \overline{\chi(h)}.$$

For a subset of a finite group, nontrivial Fourier coefficients measure deviations from uniformity.

Exercise 1.4 (Indicator functions)**.** Let G be finite abelian with $|G| \geq 2$, and let f_A be the indicator of $A \subset G$. Write χ_0 for the trivial character.

1. Show that $\|f_A\|_2 = \sqrt{|A|/|G|}$ and $\hat{f}_A(\chi_0) = |A|$.
2. Define
$$\Phi(A) = \max_{\chi \neq \chi_0} |\hat{f}_A(\chi)|. \tag{1.17}$$
If $|A| \leq |G|/2$, prove that
$$\sqrt{|A|/2} \leq \Phi(A) \leq |A|. \tag{1.18}$$
3. Show that $\Phi(A) = \Phi(G \setminus A)$ and obtain corresponding bounds when $|A| > |G|/2$.
4. Prove that $\Phi(\alpha(A)) = \Phi(A)$ for every automorphism α. Using additive notation, also prove $\Phi(A + a) = \Phi(A)$ for every $a \in G$.

Small nontrivial Fourier coefficients indicate that A has little correlation with any nonconstant character. This form of uniform distribution will be useful in counting problems and coding theory.

The preceding measure of uniformity gives quantitative estimates for equations whose variables lie in prescribed subsets.

Exercise 1.5 (Equations on a finite abelian group)**.** Use the notation of Exercise 1.4. The argument is adapted from Babai [4].

1. For subsets $A_1, \dots, A_k \subset G$, consider
$$x_1 + \cdots + x_k = a, \qquad x_i \in A_i. \tag{1.19}$$
Reduce to $a = 0$ by translating one set. For $a = 0$, show that the number of solutions is
$$N = \frac{|A_1| \cdots |A_k|}{|G|} + R, \qquad R = \frac{1}{|G|} \sum_{\chi \neq \chi_0} \prod_{i=1}^{k} \hat{f}_{A_i}(\chi).$$
2. For $k = 3$, use Cauchy–Schwarz and Plancherel to prove
$$|R| \leq \frac{\Phi(A_3)}{|G|} \sum_{\chi \in \hat{G}} |\hat{f}_{A_1}(\chi)|\,|\hat{f}_{A_2}(\chi)| \leq \Phi(A_3)\sqrt{|A_1|\,|A_2|}.$$
Explain why the same bound holds for general a.
3. For nonempty A_1, A_2, A_3, deduce that a solution exists for every $a \in G$ if
$$\frac{\Phi(A_3)}{|A_3|} < \frac{\sqrt{|A_1|\,|A_2|}}{|G|}.$$

Exercise 2.2 applies this criterion over a finite field to an analogue of Fermat's equation.

Translation and modulation jointly characterize the Fourier transform through a representation of the Heisenberg group.

Exercise 1.6 (The Heisenberg group)**.** Let G be finite abelian and $\mathcal{U} = \{z \in \mathbb{C} : |z| = 1\}$. On $\mathcal{H}(G) = \mathcal{U} \times G \times \hat{G}$, define

$$(\lambda, x, \chi)(\mu, y, \tau) = (\lambda\mu\tau(x), xy, \chi\tau).$$

1. Verify the group axioms and find the identity and inverses.
2. Show that $((\lambda, x, \chi) \cdot f)(z) = \lambda\chi(z)f(xz)$ defines an action on $\mathbb{C}[G]$.
3. Define scalar multiplication $D_\lambda f = \lambda f$, translation $T_x f(z) = f(xz)$, and modulation $M_\chi f(z) = \chi(z)f(z)$. Express the action using these operators and determine their Fourier transforms. Compare with the corresponding operations on $\mathbb{R}$.
4. Identify G with its bidual. Write the product on $\mathcal{H}(\hat{G}) = \mathcal{U} \times \hat{G} \times G$ and its action on $\mathbb{C}[\hat{G}]$.
5. Prove that
$$\alpha(\lambda, x, \chi) = (\lambda\chi(x)^{-1}, \chi, x^{-1})$$
is an isomorphism $\mathcal{H}(G) \to \mathcal{H}(\hat{G})$ and that
$$\mathcal{F}\big((\lambda, x, \chi) \cdot f\big) = \alpha(\lambda, x, \chi) \cdot \mathcal{F}(f).$$
6. Let $\Phi : \mathbb{C}[G] \to \mathbb{C}[G]$ be *complex-linear* and commute with the action of $\mathcal{H}(G)$. Prove that $\Phi = rI$ for some $r \in \mathbb{C}$. One may use the Dirac basis or Schur's lemma (Lemma 12). Deduce that a complex-linear map $\Psi : \mathbb{C}[G] \to \mathbb{C}[\hat{G}]$ satisfying $\Psi(h \cdot f) = \alpha(h) \cdot \Psi(f)$ is a scalar multiple of $\mathcal{F}$. When is this map invertible?

The following continuous analogue uses Fourier analysis to test when integer translates form an orthonormal family.

Exercise 1.7 (Fourier analysis and orthogonal translates)**.** Let $f \in L^2(\mathbb{R})$ and $\tau_r f = f(\cdot - r)$. Use the convention

$$\hat{f}(\omega) = \int_{\mathbb{R}} f(x)e^{-\mathrm{i}\omega x}\,\mathrm{d}x$$

for $f \in L^1 \cap L^2$, extended to L^2 by density.

1. Prove that $(\tau_n f)_{n\in\mathbb{Z}}$ is orthonormal if and only if
$$\sum_{k\in\mathbb{Z}} |\hat{f}(\omega + 2\pi k)|^2 = 1 \quad \text{for almost every } \omega.$$
2. Suppose $S(\omega) = \sum_k |\hat{f}(\omega + 2\pi k)|^2 \geq A > 0$ almost everywhere. Define φ by
$$\hat{\varphi}(\omega) = \frac{\hat{f}(\omega)}{\sqrt{S(\omega)}}.$$
Show that $\varphi \in L^2(\mathbb{R})$ and that its integer translates are orthonormal.

The next exercise gives a finite-group counterpart.

A parallel construction on finite groups orthogonalizes an orbit by adjusting its spectral components.

Exercise 1.8 (Orthogonalization on an abelian group)**.** Let V be a finite-dimensional complex inner-product space, with inner product linear in the first argument, and let G be a finite abelian group of unitary operators on V. For $b \in V$, call the *indexed family* $(Ab)_{A\in G}$ orthonormal when $\langle Ab, Bb\rangle = \delta_A^B$. The indexing matters: repeated orbit vectors do not count as distinct orthonormal vectors. See Bernardini and Kovacevic [7].

1. Set $\psi_b(A) = \langle Ab, b\rangle$ and
$$\mathcal{U}_\chi = \frac{1}{|G|}\sum_{A\in G} \chi(A)A.$$
Show that $(Ab)_{A\in G}$ is orthonormal if and only if $\hat{\psi}_b \equiv 1$, equivalently $|G|\langle \mathcal{U}_\chi b, b\rangle = 1$ for every χ.
2. Prove that the $\mathcal{U}_\chi$ are self-adjoint orthogonal projectors, $\mathcal{U}_{\chi_1}\mathcal{U}_{\chi_2} = 0$ for $\chi_1 \neq \chi_2$, and $\sum_\chi \mathcal{U}_\chi = I$. Deduce $\hat{\psi}_b(\chi) = |G|\|\mathcal{U}_\chi b\|^2 \geq 0$.

3. If $\hat{\psi}_b$ never vanishes, show that
$$\tilde{b} = \sum_{\chi \in \hat{G}} \frac{\mathcal{U}_\chi b}{\sqrt{\hat{\psi}_b(\chi)}}$$
has an orthonormal indexed orbit. Use the positive square roots.
4. Compare this construction with Exercise 1.7. Explain why an orthonormal orbit requires $|G| \le \dim V$.

Plancherel's identity turns spectral information about a probability distribution into a bound on its distance from uniformity.

Exercise 1.9 (Distance from the uniform distribution)**.** Let $P : G \to [0,1]$ be a probability mass function on a finite abelian group, and let $U(g) = 1/|G|$. Write χ_0 for the trivial character.

1. Compute $\hat{P}(\chi_0)$ and $\hat{U}(\chi)$. Express $\|P - U\|_2^2$ using the nontrivial Fourier coefficients of P.
2. Prove that, for every $g \in G$,
$$\left| P(g) - \frac{1}{|G|} \right|^2 \le \frac{1}{|G|} \sum_{\chi \neq \chi_0} |\hat{P}(\chi)|^2.$$

The nontrivial coefficients quantify the departure from uniformity.

For a random walk, convolution describes repeated steps and the Fourier transform separates their long-term effects.

Exercise 1.10 (Random walks)**.** Consider a time-homogeneous random walk (X_k) on $\mathbb{Z}/n\mathbb{Z}$, with $n \ge 3$. Write $p_{i,j} = \mathbb{P}(X_{k+1} = j \mid X_k = i)$, and let $p^{(k)}$ be the column vector with entries $p_i^{(k)} = \mathbb{P}(X_k = i)$.

1. Define $P_{j,i} = p_{i,j}$, so each *column* of P sums to one. Prove $p^{(k+1)} = Pp^{(k)}$ and $p^{(k)} = P^k p^{(0)}$. In the remaining questions take $p^{(0)} = \delta_0$.
2. Let $0 < p < 1$, with $p_{i,i-1} = p$, $p_{i,i+1} = 1 - p$, and all other transition probabilities zero. Show $Px = v * x$, where $v(1) = 1 - p$, $v(-1) = p$, and v vanishes elsewhere. Recover the formula by adding an independent random increment.
3. Express $\hat{p}^{(k)}$ in terms of $\hat{v}$ and $\hat{p}^{(0)}$. Show that $p^{(k)}$ tends to the uniform distribution if n is odd. Explain the parity obstruction when n is even.
4. Generalize to $p_{i,j} = v(j - i)$ for a probability mass function v. Give a necessary and sufficient condition for convergence to uniformity from every initial distribution in terms of $|\hat{v}(\chi)|$ for nontrivial characters. Interpret the role of characters that are constant on the support of v.

Figure 1.4 compares two increment distributions. The magnitude of the largest nontrivial Fourier coefficient controls the asymptotic rate of convergence when that magnitude is less than one.

The final exercise shows why a nonzero function and its Fourier transform cannot both be supported on very small sets.

Exercise 1.11 (The discrete uncertainty principle)**.** Let G be a finite *abelian* group and $f \in \mathbb{C}[G] \setminus \{0\}$. The goal is to prove
$$|\operatorname{Supp} f|\,|\operatorname{Supp} \hat{f}| \ge |G|. \tag{1.20}$$

1. For $G = \mathbb{Z}/n\mathbb{Z}$, suppose f has p nonzero values. Use a Vandermonde matrix to show that $\hat{f}$ cannot have p consecutive zeros with indices taken cyclically. Deduce (1.20). See Donoho and Stark [24].
2. For general finite abelian G, set $M = \max_{x \in G} |f(x)|$. Prove
$$\|f\|_2^2 \le \frac{M^2}{|G|} |\operatorname{Supp} f|, \qquad M \le \frac{1}{|G|} \sum_{\chi \in \hat{G}} |\hat{f}(\chi)|.$$
Use Cauchy–Schwarz and Plancherel to obtain $M^2 \le |\operatorname{Supp} \hat{f}|\,\|f\|_2^2$ and conclude.
3. Let $H \subset G$ be a subgroup, with indicator f_H, and define its annihilator $H^\sharp = \{\chi \in \hat{G} : \chi|_H = 1\}$. Show $\hat{f}_H = |H| f_{H^\sharp}$ and $|H|\,|H^\sharp| = |G|$. Deduce equality in (1.20) for f_H.

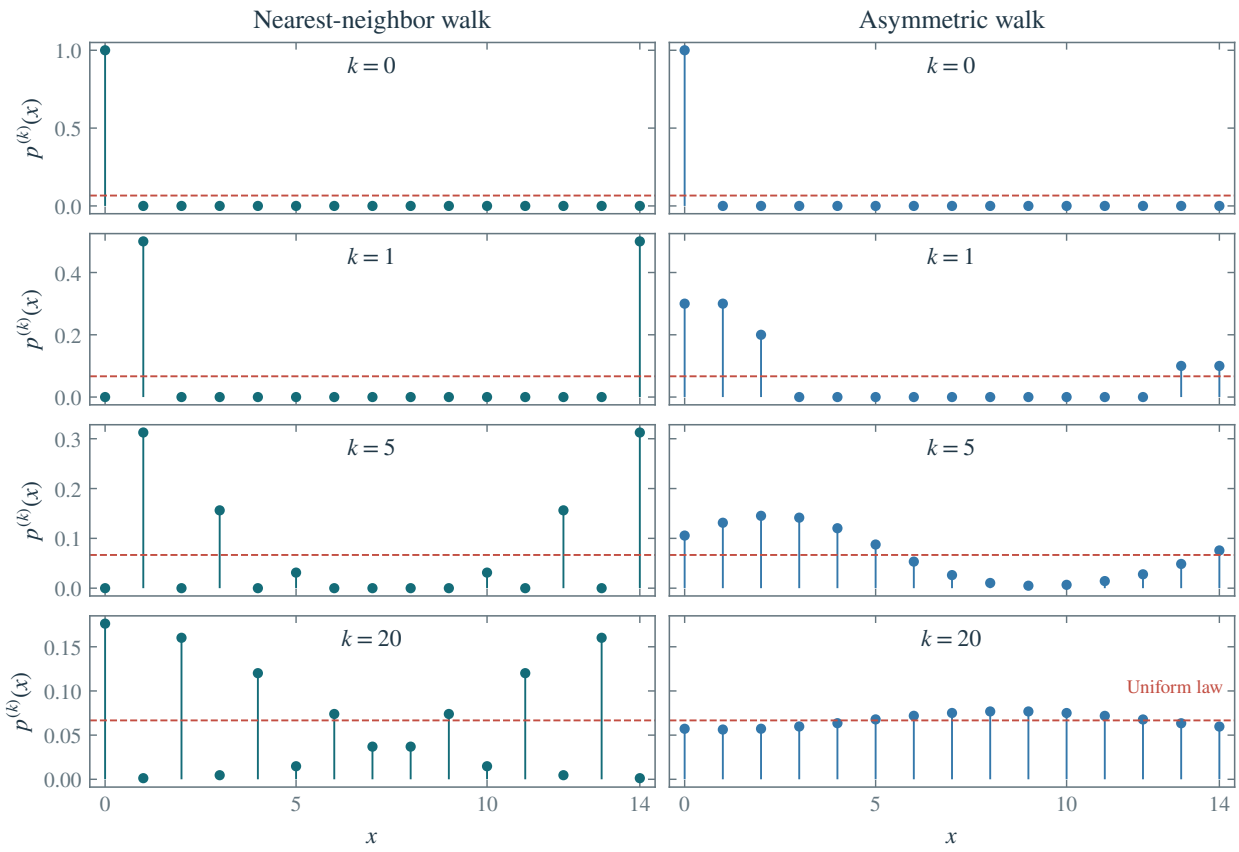


Figure 1.4. Random walks on $\mathbb{Z}/15\mathbb{Z}$ with increment laws $(0, 1/2, 0, \ldots, 0, 1/2)$ and $(0.3, 0.3, 0.2, 0, \ldots, 0, 0.1, 0.1)$. Dashed lines show the uniform distribution.

Translations, modulation by a character, and multiplication by a nonzero scalar preserve equality. In fact, every equality case has the form $c\chi(x)f_H(a^{-1}x)$ for a subgroup H, an element $a \in G$, a character χ, and $c \neq 0$; see Matusiak et al. [44]. The restriction on simultaneous concentration in space and frequency is also useful constructively, for example in the distance bound for BCH codes in Proposition 53.

1.6 Solutions to the exercises

These solutions use character orthogonality to turn determinants, counting constraints, and random-walk evolution into frequency calculations. The later arguments connect spectral normalization with orthogonal translates and establish the limits imposed by uncertainty.

Solution to Exercise 1.1

1. Directly from convolution and multiplicativity,
$$(f * \chi)(x) = \sum_{h \in G} f(h)\chi(h^{-1}x) = \chi(x) \sum_{h \in G} f(h)\chi(h)^{-1} = \hat{f}(\chi^{-1})\chi(x).$$
Thus the eigenvalue associated with χ is $\hat{f}(\chi^{-1})$.
2. The coefficient of δ_h in $f * \delta_g$ is $f(hg^{-1})$. In additive cyclic coordinates this gives $A_{ij} = f(i-j)$, with indices modulo n. Since the characters form an eigenbasis,
$$\det A = \prod_{\chi \in \hat{G}} \hat{f}(\chi^{-1}) = \prod_{\chi \in \hat{G}} \hat{f}(\chi).$$
3. For the matrix displayed in the exercise, choose $f(k) = a_{-k}$, so that $A_{ij} = a_{j-i}$. Its eigenvalue on χ_i is $\sum_j a_j \omega^{ij}$. Multiplying these eigenvalues gives the formula.
4. An FFT computes all eigenvalues in $O(n \log n)$ operations; their product takes a further $O(n)$ operations. Program 1 uses NumPy's negative-exponential convention. This changes the order of the eigenvalues, but not their product. For large matrices, the product can overflow or underflow; summing $\log|\lambda_j|$ and keeping track of the phase gives a more useful representation. Near-zero eigenvalues also make a determinant sensitive to rounding.

```
def circulant_determinant(first_row):
    eigenvalues = np.fft.fft(np.asarray(first_row, dtype=complex))
    return np.prod(eigenvalues)
```

Program 1. A circulant determinant from its first row.

Solution to Exercise 1.2

1. Choose positively oriented orthonormal frames adapted to the two rotation axes. The change-of-frame matrix Ω belongs to $SO(3)$, and conjugation by Ω takes one rotation to the other. Reversing an axis allows every rotation to be described by an angle in $[0, \pi]$.
2. A homomorphism to an abelian group is constant on conjugacy classes. Thus χ depends only on the angle.
3. Matrix multiplication gives
$$\operatorname{Tr}(r_\alpha s_\alpha^{-1}) = 2\cos\alpha + \cos^2\alpha.$$
Since a rotation through β has trace $1 + 2\cos\beta$,
$$\cos\beta = \frac{(1+\cos\alpha)^2 - 2}{2}.$$
This expression is continuous and strictly decreasing from 1 to -1 as α increases from 0 to π. Hence β covers $[0, \pi]$. Program 2 verifies the trace symbolically using SymPy, imported as sp.

```
def rotation_product_trace():
    a = sp.symbols('alpha', real=True)
    c, s = sp.cos(a), sp.sin(a)
    rx = sp.Matrix([[1, 0, 0], [0, c, -s], [0, s, c]])
    ry = sp.Matrix([[c, 0, s], [0, 1, 0], [-s, 0, c]])
    return sp.simplify(sp.trace(rx * ry.T))
```

Program 2. Symbolic trace of a product of rotations.

4. The rotations r_α and s_α are conjugate, so $\chi(r_\alpha s_\alpha^{-1}) = 1$. Every rotation angle occurs in such a product, and χ depends only on that angle. Therefore $\chi = 1$ on all of $SO(3)$.

Solution to Exercise 1.3

Dual orthogonality gives, for each tuple,

$$\mathbf{1}_{\{\varphi(g_1,\dots,g_n)=h\}} = \frac{1}{|G|}\sum_{\chi\in\hat{G}} \chi\big(\varphi(g_1,\dots,g_n)h^{-1}\big).$$

Sum over all tuples and use $\chi(h^{-1}) = \overline{\chi(h)}$. The factor $1/|G|$ remains in both sums.

Solution to Exercise 1.4

Write $n = |G|$ and $a = |A|$.

1. The definitions give $\|f_A\|_2^2 = a/n$ and $\hat{f}_A(\chi_0) = a$.
2. Plancherel yields $\sum_\chi |\hat{f}_A(\chi)|^2 = na$. Separating the trivial character and bounding the other $n-1$ terms gives
$$na \le a^2 + (n-1)\Phi(A)^2, \qquad \Phi(A)^2 \ge \frac{a(n-a)}{n-1} \ge \frac{a}{2}$$
when $a \le n/2$. Also, $|\hat{f}_A(\chi)| \le \sum_{x\in A}|\chi(x)| = a$.
3. Since $f_{G\setminus A} = 1 - f_A$, their nontrivial Fourier coefficients are negatives of one another. Thus $\Phi(G\setminus A) = \Phi(A)$. For $a > n/2$, apply the preceding bounds to obtain $\sqrt{(n-a)/2} \le \Phi(A) \le n - a$.
4. We have $\hat{f}_{\alpha(A)}(\chi) = \hat{f}_A(\chi\circ\alpha)$. Composition with an automorphism permutes the nontrivial characters. For a translation, instead use $\hat{f}_{A+b}(\chi) = \chi(b)\hat{f}_A(\chi)$ and $|\chi(b)| = 1$. A nonzero translation is generally *not* a group automorphism.

Solution to Exercise 1.5

1. Replace A_1 by $A_1 - a = \{x - a : x \in A_1\}$. This preserves the number of solutions and reduces the equation to $a = 0$. Character orthogonality then gives
$$N = \frac{1}{|G|}\sum_{\chi\in\hat{G}}\sum_{x_i\in A_i}\chi(x_1+\cdots+x_k) = \frac{1}{|G|}\sum_{\chi\in\hat{G}}\prod_{i=1}^{k}\hat{f}_{A_i}(\chi). \tag{1.21}$$
The trivial character contributes $|A_1|\cdots|A_k|/|G|$; the remaining characters contribute R.
2. For $k = 3$, bound the third factor by $\Phi(A_3)$ and enlarge the sum to all characters. Cauchy–Schwarz and Plancherel give
$$\begin{aligned} |R| &\le \frac{\Phi(A_3)}{|G|}\sum_\chi |\hat{f}_{A_1}(\chi)|\,|\hat{f}_{A_2}(\chi)| \\ &\le \frac{\Phi(A_3)}{|G|}\sqrt{\sum_\chi |\hat{f}_{A_1}(\chi)|^2}\sqrt{\sum_\chi |\hat{f}_{A_2}(\chi)|^2} \\ &= \Phi(A_3)\sqrt{|A_1|\,|A_2|}. \end{aligned}$$
For general a, the Fourier expression acquires factors $\overline{\chi(a)}$ of modulus one, so the bound is unchanged.

3. The hypothesis implies $|R| < |A_1||A_2||A_3|/|G|$. Therefore the main term in (1.21) is larger than the possible error, and $N > 0$.

Solution to Exercise 1.6

1. Associativity follows from multiplicativity of each character. The identity is $(1, 1, \chi_0)$, and
$$(\lambda, x, \chi)^{-1} = (\lambda^{-1}\chi(x), x^{-1}, \chi^{-1}).$$
For example, the scalar part of a triple product is $\lambda\mu\nu\tau(x)\eta(x)\eta(y)$ with either parenthesization.
2. Composing the proposed actions gives
$$\lambda\chi(z)\mu\tau(xz)f(yxz) = \lambda\mu\tau(x)(\chi\tau)(z)f(xyz),$$
which is the action of the product in $\mathcal{H}(G)$. The identity acts trivially.
3. The action is $D_\lambda M_\chi T_x f$. Its multiplication rule follows from $T_x M_\tau = \tau(x) M_\tau T_x$. Direct changes of variable give
$$\widehat{T_x f}(\tau) = \tau(x)^{-1}\hat{f}(\tau), \qquad \widehat{M_\chi f}(\tau) = \hat{f}(\chi\tau), \qquad \widehat{D_\lambda f} = \lambda\hat{f}.$$
Thus translation becomes modulation and modulation becomes translation, just as for the continuous Fourier transform with consistent sign conventions.
4. Evaluation identifies $x \in G$ with a character of $\hat{G}$. Consequently,
$$(\lambda, \chi, x)(\mu, \tau, y) = (\lambda\mu\chi(y), \chi\tau, xy), \qquad ((\lambda, \chi, x) \cdot \varphi)(\tau) = \lambda\tau(x)\varphi(\chi\tau).$$
5. Substitution in the two multiplication laws proves that α is a homomorphism. Its inverse is $\alpha^{-1}(\mu, \chi, y) = (\mu\chi(y)^{-1}, y^{-1}, \chi)$. Furthermore,
$$\widehat{D_\lambda M_\chi T_x} f(\tau) = \lambda\chi(x)^{-1}\tau(x)^{-1}\hat{f}(\chi\tau),$$
which is exactly the action of $\alpha(\lambda, x, \chi)$ on $\hat{f}$.
6. Commutation with all modulations implies that $\Phi\delta_a$ is supported at a: indeed, $M_\chi\delta_a = \chi(a)\delta_a$, and characters separate points. Thus Φ is diagonal in the Dirac basis. Commutation with translations forces all diagonal entries to be equal, giving $\Phi = rI$. This argument also proves irreducibility without assuming it in an appeal to Schur's lemma. Notice that $\mathcal{H}(G)$ itself is infinite because it contains $\mathcal{U}$; finite-dimensionality of the representation is the relevant feature.
If Ψ intertwines the two actions through α, then $\mathcal{F}^{-1}\Psi$ commutes with the original action. Hence $\Psi = r\mathcal{F}$. It is invertible exactly when $r \neq 0$. The intertwining isomorphism is therefore supplied by the Fourier transform itself.

Solution to Exercise 1.7

1. The nonnegative periodization $S(\omega) = \sum_k |\hat{f}(\omega + 2\pi k)|^2$ belongs to $L^1([0, 2\pi])$ by Tonelli's theorem and Plancherel. For $m = n - p$,
$$\begin{aligned}\langle \tau_n f, \tau_p f\rangle &= \frac{1}{2\pi}\int_{\mathbb{R}} |\hat{f}(\omega)|^2 e^{-\mathrm{i}m\omega}\,\mathrm{d}\omega \\ &= \frac{1}{2\pi}\int_0^{2\pi} S(\omega)e^{-\mathrm{i}m\omega}\,\mathrm{d}\omega. \end{aligned} \tag{1.22}$$
The second equality follows by partitioning $\mathbb{R}$ into intervals of length 2π; absolute integrability justifies the rearrangement. Thus orthonormality says that the Fourier coefficients of S equal those of the constant function 1. Uniqueness of Fourier coefficients in L^1 gives $S = 1$ almost everywhere, and the converse follows from the same formula.
2. The lower bound on S gives $|\hat{\varphi}| \le A^{-1/2}|\hat{f}|$, so $\varphi \in L^2$. Since S is 2π-periodic,
$$\sum_k |\hat{\varphi}(\omega + 2\pi k)|^2 = \frac{\sum_k |\hat{f}(\omega + 2\pi k)|^2}{S(\omega)} = 1$$
almost everywhere. Apply the first part.

Solution to Exercise 1.8

Write $n = |G|$.

1. Unitarity gives $\langle Ab,\, Bb\rangle = \psi_b(B^{-1}A)$. The indexed orbit is therefore orthonormal exactly when $\psi_b = \delta_I$, or equivalently $\hat{\psi}_b = 1$. Linearity in the first argument gives $\hat{\psi}_b(\chi) = n\langle \mathcal{U}_\chi b,\, b\rangle$.
2. Substituting $R = UV$ and using character orthogonality,

$$\begin{aligned}\mathcal{U}_\chi \mathcal{U}_\eta &= \frac{1}{n^2}\sum_{U,R\in G} \chi(U)\eta(U^{-1}R)R \\ &= \frac{1}{n^2}\left(\sum_U \chi(U)\overline{\eta(U)}\right)\left(\sum_R \eta(R)R\right) = \delta_\chi^\eta \mathcal{U}_\eta.\end{aligned}$$

   Also,

$$\mathcal{U}_\chi^* = \frac{1}{n}\sum_U \overline{\chi(U)}U^{-1} = \frac{1}{n}\sum_V \chi(V)V = \mathcal{U}_\chi.$$

   Dual orthogonality gives $\sum_\chi \mathcal{U}_\chi = I$. These are therefore mutually orthogonal spectral projectors, and $\langle \mathcal{U}_\chi b,\, b\rangle = \|\mathcal{U}_\chi b\|^2$.
3. Orthogonality of the projectors implies

$$\mathcal{U}_\chi \tilde{b} = \frac{\mathcal{U}_\chi b}{\sqrt{\hat{\psi}_b(\chi)}}, \qquad n\|\mathcal{U}_\chi \tilde{b}\|^2 = \frac{n\|\mathcal{U}_\chi b\|^2}{\hat{\psi}_b(\chi)} = 1.$$

   The criterion in the first part proves orthonormality. The normalization is $\|\mathcal{U}_\chi \tilde{b}\|^2 = 1/n$, not 1.
4. Both constructions normalize the spectral energy of an orbit. In the continuous exercise the acting group is $\mathbb{Z}$, through integer translations on $L^2(\mathbb{R})$. Here it is a finite group of unitary operators. An orthonormal family of n vectors requires $n \le \dim V$.

Solution to Exercise 1.9

1. We have $\hat{P}(\chi_0) = 1$ and $\hat{U} = \delta_{\chi_0}$. Plancherel, with the normalized inner products, gives

$$\|P - U\|_2^2 = \frac{1}{|G|^2}\sum_{\chi\neq\chi_0} |\hat{P}(\chi)|^2.$$

2. For each g,

$$|P(g) - U(g)|^2 \le \sum_{x\in G} |P(x) - U(x)|^2 = |G|\|P - U\|_2^2.$$

   Insert the expression from the first part. The square on the left is essential.

Solution to Exercise 1.10

1. Conditioning on X_k gives $p_j^{(k+1)} = \sum_i p_{i,j}p_i^{(k)}$. Since $P_{j,i} = p_{i,j}$, this is $p^{(k+1)} = Pp^{(k)}$. Iterate to obtain $p^{(k)} = P^k p^{(0)}$.
2. The nearest-neighbor transition satisfies $(Px)(j) = (1-p)x(j-1) + px(j+1) = (v * x)(j)$. Equivalently, $X_k = X_0 + Y_1 + \cdots + Y_k$ with independent increments of law v; convolution gives the law of the sum.
3. The convolution theorem yields $\hat{p}^{(k)}(\chi) = \hat{v}(\chi)^k \hat{p}^{(0)}(\chi)$. For $\chi_j(x) = e^{2\pi i jx/n}$,

$$\hat{v}(\chi_j) = (1-p)e^{2\pi i j/n} + pe^{-2\pi i j/n}.$$

   Since both weights are positive, its modulus is one exactly when the two unit complex numbers agree, that is, when $2j = 0$ modulo n. For odd n, only $j = 0$ qualifies, so all nontrivial coefficients decay and inversion gives convergence to uniformity. For even n, $\hat{v}(\chi_{n/2}) = -1$. Starting from 0, the walk alternates between even and odd sites, so its law does not converge.

4. For any increment law, $|\hat{v}(\chi)| \le \sum_x v(x) = 1$. Convergence to uniformity from every starting law is equivalent to $|\hat{v}(\chi)| < 1$ for every nontrivial χ. Sufficiency follows by decay of those coefficients. Necessity follows by starting at a point mass, whose Fourier coefficients all have modulus one.
   Equality $|\hat{v}(\chi)| = 1$ holds precisely when χ is constant on the support of v, by equality in the triangle inequality. A constant value 1 gives a persistent coefficient; another unit value gives an oscillating coefficient. For a particular initial law, an exceptional coefficient matters only if the corresponding initial Fourier coefficient is nonzero. Program 3 computes the law directly in Fourier coordinates. Real projection removes numerical roundoff in the imaginary part.

```
def random_walk(p0, v, k):
    """Law after k >= 0 steps; p0 and v are probability vectors of equal size."""
    spectrum = np.fft.fft(p0) * np.fft.fft(v)**k
    return np.fft.ifft(spectrum).real
```

Program 3. The probability law after k steps of a cyclic random walk.

Solution to Exercise 1.11

1. Write $\operatorname{Supp} f = \{a_1, \dots, a_p\}$ and $\omega = e^{2\pi i/n}$. If $\hat{f}(0), \dots, \hat{f}(p-1)$ all vanish, then the vector $(f(a_j))_{j=1}^{p}$ lies in the kernel of the Vandermonde matrix $V_{rj} = (\omega^{a_j})^r$, $0 \le r < p$. Its nodes are distinct, so V is invertible, a contradiction. For a block starting at another frequency, modulate f to reduce to this case without changing its support.
   Let $q = |\operatorname{Supp} \hat{f}|$. Between successive nonzero coefficients on the frequency circle there can be at most $p-1$ zeros, so the cyclic distance between them is at most p. Summing the q distances gives $n \le pq$. This also covers the case where p does not divide n.
2. The first bound follows from $|f(x)| \le M$ on its support. Inversion and $|\chi(x)| = 1$ give the second. If $B = \operatorname{Supp} \hat{f}$, Cauchy–Schwarz and Plancherel imply

$$M^2 \le \frac{1}{|G|^2} \left(\sum_{\chi \in B} |\hat{f}(\chi)| \right)^2 \le \frac{|B|}{|G|^2} \sum_{\chi} |\hat{f}(\chi)|^2 = |B| \|f\|_2^2.$$

   Combining with $\|f\|_2^2 \le M^2 |\operatorname{Supp} f| / |G|$ and canceling the positive quantity $\|f\|_2^2$ proves the result.
3. Character orthogonality on H gives $\hat{f}_H(\chi) = \sum_{h \in H} \chi(h) = |H|$ if $\chi|_H = 1$, and 0 otherwise. By the exact sequence in Lemma 2, $|H^\sharp| = |\widehat{G/H}| = |G|/|H|$. The support sizes therefore attain equality.

CHAPTER 2

Applications of Fourier duality

Character theory connects problems that initially look quite different: quadratic residues, fast transforms on binary data, and the weight distribution of a code. We develop Gauss sums and quadratic reciprocity, construct the Walsh transform, and derive Poisson summation and the MacWilliams identity. The exercises connect these ideas to wavelets, compression, and sampling.

2.1 Gauss sums

A finite field has both an additive group and a multiplicative group. Gauss sums combine their characters and reveal how the two structures interact. For a detailed treatment of finite fields and character sums, see Lidl and Niederreiter [40] and Langevin [37].

2.1.1 Quadratic residues

Solving a quadratic congruence requires knowing which residues are squares. We begin with a simple integer equation, then introduce the quadratic character that records this information multiplicatively.

Consider the integer equation

$$x^2 = 2y^2 + 7k, \qquad x, y, k \in \mathbb{Z}. \tag{2.1}$$

Modulo 7, the square roots of 2 are 3 and $4 = -3$. If $y \not\equiv 0 \pmod 7$, division by y^2 therefore gives $x \equiv 3y$ or $x \equiv 4y \pmod 7$. If $y \equiv 0$, the equation forces $x \equiv 0$ as well. The same conclusion follows uniformly from

$$x^2 - 2y^2 \equiv (x - 3y)(x - 4y) \pmod 7.$$

Thus the integer pairs are exactly those satisfying one of these two congruences, and $k = (x^2 - 2y^2)/7$. This does not assert that the rational number x/y equals an integer of the form $3 + 7m$ or $4 + 7m$.

For an odd prime p, the *Legendre symbol* records whether an integer is a square modulo p:

$$\left(\frac{a}{p}\right) = \begin{cases} 0 & p \mid a, \\ 1 & a \text{ is a nonzero square modulo } p, \\ -1 & a \text{ is a nonsquare modulo } p. \end{cases}$$

It depends only on the residue class in $\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$. Restricted to $\mathbb{F}_p^*$, it defines the quadratic character

$$\eta : \mathbb{F}_p^* \longrightarrow \{1, -1\}, \qquad \eta(a) = \left(\frac{a}{p}\right). \tag{2.2}$$

Its multiplicativity follows from Euler's criterion.

Lemma 6 (Euler's criterion)**.** *For an odd prime p and $a \in \mathbb{F}_p^*$,*

$$a^{(p-1)/2} = \begin{cases} 1 & a \text{ is a square}, \\ -1 & a \text{ is a nonsquare}. \end{cases}$$

Equivalently, for an integer a not divisible by p, $\left(\frac{a}{p}\right) \equiv a^{(p-1)/2} \pmod p$.

Proof. The homomorphism $x \mapsto x^2$ on $\mathbb{F}_p^*$ has kernel $\{1, -1\}$, so its image has $(p-1)/2$ elements. Each square $a = b^2$ satisfies $a^{(p-1)/2} = b^{p-1} = 1$. Since $X^{(p-1)/2} - 1$ has at most $(p-1)/2$ roots, these are all its roots. Finally, $(a^{(p-1)/2})^2 = a^{p-1} = 1$, so every nonsquare gives -1. □

Quadratic reciprocity will relate $\left(\frac{p}{r}\right)$ and $\left(\frac{r}{p}\right)$ for distinct odd primes. Combined with reduction modulo the denominator and the values of the symbols at -1 and 2, it gives an efficient way to compute Legendre symbols.

2.1.2 Additive and multiplicative characters

A finite field supports two different Fourier theories, one for addition and one for multiplication. We describe their characters explicitly and use the field trace to connect extension fields with the prime field.

Let $q = p^r$, where p is prime. The field $\mathbb{F}_q$ has characteristic p and dimension r over its prime subfield $\mathbb{F}_p$. Its additive group is isomorphic to $(\mathbb{Z}/p\mathbb{Z})^r$; its multiplicative group $\mathbb{F}_q^*$ is cyclic of order $q-1$. Further details appear in Section 6.1.

We distinguish the two kinds of character by the operation they preserve and by their domains of definition.

Definition 9 (Additive and multiplicative characters)**.** An *additive character* is a homomorphism $(\mathbb{F}_q, +) \to \mathbb{C}^*$, while a *multiplicative character* is a homomorphism $(\mathbb{F}_q^*, \cdot) \to \mathbb{C}^*$. We denote them by ψ and χ, respectively.

If ζ generates $\mathbb{F}_q^*$, its multiplicative characters are

$$\chi_j(\zeta^k) = e^{2\pi \mathrm{i} jk/(q-1)}, \qquad 0 \le j \le q-2.$$

Here ζ is a field element, whereas the values of χ_j are complex numbers. The description depends on the chosen generator. To describe additive characters without choosing a vector-space basis, we use the field trace.

Definition 10 (Field trace)**.** Let K contain a finite subfield k with $|k| = s$, and let $[K : k] = t$. The trace from K to k is

$$\mathrm{Tr}_{K/k}(\alpha) = \alpha + \alpha^s + \cdots + \alpha^{s^{t-1}}.$$

For $K = \mathbb{F}_q$ and $k = \mathbb{F}_p$, we abbreviate it to Tr.

The Frobenius automorphism explains why the trace takes its values in the smaller field.

Proposition 19 (Frobenius and subfields)**.** *Let K be a finite field of characteristic p. The Frobenius map $x \mapsto x^p$ is a field automorphism, as are its iterates. If $k \subset K$ has s elements, then $x \in k$ exactly when $x^s = x$.*

Proof. The binomial theorem in characteristic p gives $(x+y)^p = x^p + y^p$, and the other homomorphism properties are immediate. Injectivity and finiteness give bijectivity. The elements of k are roots of $X^s - X$; since this polynomial has degree s, there are no additional roots in K. □

These Frobenius identities give the properties needed to build additive characters from the trace.

Proposition 20 (Properties of the trace)**.** *The trace $\mathrm{Tr}_{K/k} : K \to k$ is a nonzero k-linear map, and hence is surjective.*

Proof. Since $\alpha^{s^t} = \alpha$, applying $x \mapsto x^s$ permutes the terms in the trace sum. Its value is therefore fixed by this map and lies in k. Additivity follows from Frobenius, and $\lambda^s = \lambda$ for $\lambda \in k$ gives k-linearity. Finally, $X + X^s + \cdots + X^{s^{t-1}}$ is a nonzero polynomial of degree $s^{t-1} < |K|$, so it cannot vanish at every element of K. A nonzero linear map to the one-dimensional k-space k is surjective. □

Composing the trace with the standard exponential on the prime field provides a distinguished additive character.

Definition 11 (Canonical additive character)**.** Define

$$\psi_1(x) = e^{2\pi \mathrm{i}\,\mathrm{Tr}(x)/p}, \qquad x \in \mathbb{F}_q.$$

In the exponential, a value of $\mathrm{Tr}(x) \in \mathbb{F}_p$ is represented by an integer; the result is independent of the representative.

Rescaling its input generates all additive characters, with no further choice of basis.

Proposition 21. *The additive characters of $\mathbb{F}_q$ are exactly $\psi_a(x) = \psi_1(ax)$, with $a \in \mathbb{F}_q$. The map $a \mapsto \psi_a$ is an isomorphism from the additive group to its dual.*

Proof. Each ψ_a is a character and $\psi_a \psi_b = \psi_{a+b}$. The trace is nonzero, so ψ_1 is nontrivial. If $a \neq b$, multiplication by $a - b$ permutes $\mathbb{F}_q$, and there is a c with

$$\frac{\psi_a(c)}{\psi_b(c)} = \psi_1((a-b)c) \neq 1.$$

Thus the q characters are distinct. Since the dual has order q, they exhaust it. □

Before combining the two character theories, we fix the conventions for their trivial characters, especially at zero.

Remark 13 (The two trivial characters). The trivial additive character ψ_0 equals 1 on all of $\mathbb{F}_q$. The trivial multiplicative character χ_0 equals 1 on $\mathbb{F}_q^*$. When extending multiplicative characters to the whole field below, we assign value 0 at the origin, including for χ_0.

The trace construction is also compatible with passing through intermediate fields.

Remark 14 (Compatibility with field extensions). For $K = \mathbb{F}_{q^t}$, the trace is transitive: $\mathrm{Tr}_{K/\mathbb{F}_p} = \mathrm{Tr}_{\mathbb{F}_q/\mathbb{F}_p} \circ \mathrm{Tr}_{K/\mathbb{F}_q}$. Indeed, expanding the two sums lists every Frobenius power from 0 to $rt - 1$ exactly once. Therefore the canonical character μ_1 of K satisfies

$$\mu_1(\beta) = \psi_1\big(\mathrm{Tr}_{K/\mathbb{F}_q}(\beta)\big). \tag{2.3}$$

The quadratic character introduced for prime fields extends naturally to every finite field of odd order.

Example 1 (The quadratic character). For an odd prime power q, define $\eta(x)$ to be 1 on the nonzero squares of $\mathbb{F}_q$ and -1 on its nonsquares. Then $\eta = \chi_{(q-1)/2}$. For $q = p$ prime, it is the Legendre symbol.

The orthogonality relations of Chapter 1 take the following forms.

Proposition 22 (Additive orthogonality). *For $a, b \in \mathbb{F}_q$,*

$$\sum_{x\in\mathbb{F}_q} \psi_a(x)\overline{\psi_b(x)} = q\delta_a^b, \tag{2.4}$$

$$\sum_{x\in\mathbb{F}_q} \psi_a(x) = 0 \quad (a \neq 0), \tag{2.5}$$

$$\sum_{x\in\mathbb{F}_q} \psi_x(a)\overline{\psi_x(b)} = q\delta_a^b. \tag{2.6}$$

For multiplication, the same orthogonality principle applies on the nonzero elements, so the normalization uses $q - 1$.

Proposition 23 (Multiplicative orthogonality). *For $a, b \in \mathbb{F}_q^*$ and multiplicative characters χ, τ,*

$$\sum_{x\in\mathbb{F}_q^*} \chi(x)\overline{\tau(x)} = (q-1)\delta_\chi^\tau, \tag{2.7}$$

$$\sum_{x\in\mathbb{F}_q^*} \chi(x) = 0 \quad (\chi \neq \chi_0), \tag{2.8}$$

$$\sum_{\chi\in\widehat{\mathbb{F}_q^*}} \chi(a)\overline{\chi(b)} = (q-1)\delta_a^b. \tag{2.9}$$

The first and third identities in each proposition express orthogonality of the rows and columns of the corresponding character table.

2.1.3 Gauss sums

Gauss sums pair an additive character with a multiplicative one. Their Fourier interpretation lets us compute their magnitudes and symmetries, preparing the arithmetic applications that follow.

Definition 12 (Gauss sum). For a multiplicative character χ and an additive character ψ, define

$$G(\chi, \psi) = \sum_{x\in\mathbb{F}_q^*} \chi(x)\psi(x). \tag{2.10}$$

A Gauss sum can be viewed in two ways. It is the multiplicative Fourier transform of the restriction of ψ to $\mathbb{F}_q^*$. Alternatively, extend χ to a function $\tilde{\chi}$ on $\mathbb{F}_q$ by setting $\tilde{\chi}(0) = 0$. With the additive Fourier transform

$$\mathcal{F}_{\mathrm{add}} f(\psi) = \sum_{x\in\mathbb{F}_q} f(x)\psi(x), \tag{2.11}$$

we have

$$G(\chi, \psi) = \mathcal{F}_{\mathrm{add}}\tilde{\chi}(\psi). \tag{2.12}$$

The extension is a function on the additive group; it is not an additive character.

Fourier inversion therefore expresses a multiplicative character as a combination of additive characters, with Gauss sums as coefficients.

Proposition 24 (Additive Fourier expansion)**.** *For a multiplicative character extended by zero,*

$$\tilde{\chi} = \frac{1}{q} \sum_{\psi \in \widehat{\mathbb{F}_q}} G(\chi, \overline{\psi})\psi.$$

Proof. Its Fourier coefficient along ψ is $\langle \tilde{\chi}, \psi \rangle = G(\chi, \overline{\psi})/q$. Expand in the additive character basis. □

Changes of variable give a few identities that reduce many Gauss-sum calculations to a single reference sum.

Proposition 25 (Elementary identities)**.** *For $a \in \mathbb{F}_q^*$, $b \in \mathbb{F}_q$, and characters χ, ψ,*

1. $G(\chi, \psi_{ab}) = \overline{\chi(a)} G(\chi, \psi_b)$;
2. $G(\chi, \overline{\psi}) = \chi(-1) G(\chi, \psi)$;
3. $G(\overline{\chi}, \psi) = \chi(-1)\overline{G(\chi, \psi)}$.

Proof. In the first identity, substitute $y = ax$ in the defining sum and use $\chi(a^{-1}) = \overline{\chi(a)}$. The second follows by taking $a = -1$, because $\psi(-x) = \overline{\psi(x)}$. Conjugate this identity to obtain the third, using $\chi(-1) \in \{1, -1\}$. □

Orthogonality and Plancherel's formula then determine the magnitude, including the exceptional cases involving trivial characters.

Proposition 26 (Magnitudes of Gauss sums)**.** *If at least one character is trivial, then*

$$G(\chi, \psi) = \begin{cases} q - 1 & \chi = \chi_0,\ \psi = \psi_0, \\ -1 & \chi = \chi_0,\ \psi \neq \psi_0, \\ 0 & \chi \neq \chi_0,\ \psi = \psi_0. \end{cases}$$

If both characters are nontrivial, then $|G(\chi, \psi)| = \sqrt{q}$ and

$$G(\chi, \psi) G(\overline{\chi}, \psi) = q\chi(-1). \tag{2.13}$$

Proof. The three special cases follow immediately from character orthogonality. Now assume $\chi \neq \chi_0$. Since the zero extension has squared normalized norm $(q - 1)/q$, Plancherel gives

$$\frac{1}{q} \sum_{a \in \mathbb{F}_q} |G(\chi, \psi_a)|^2 = q \|\tilde{\chi}\|_2^2 = q - 1. \tag{2.14}$$

The coefficient at $a = 0$ is zero. By the first elementary identity, the other $q - 1$ coefficients have the same magnitude, say c. Consequently $(q - 1)c^2 = q(q - 1)$, so $c = \sqrt{q}$. Finally, the conjugation identity gives $G(\chi, \psi) G(\overline{\chi}, \psi) = \chi(-1)|G(\chi, \psi)|^2$. □

The remaining factor $\chi(-1)$ can be read directly from the order of the character.

Proposition 27. *If a multiplicative character χ has order m, then $\chi(-1) = -1$ exactly when m is even and $(q - 1)/m$ is odd.*

Proof. The order m divides $q - 1$. If m is odd, the image of χ does not contain -1, so $\chi(-1) = 1$. If m is even, q is odd. Choose a generator g_0 of $\mathbb{F}_q^*$ and put $\zeta = \chi(g_0)$, a primitive mth root. Then

$$\chi(-1) = \zeta^{(q-1)/2} = \left(\zeta^{m/2}\right)^{(q-1)/m} = (-1)^{(q-1)/m}.$$

□

2.1.4 Quadratic reciprocity

Quadratic reciprocity relates the solvability of two congruences with their prime moduli exchanged. Gauss sums give an algebraic proof; we first refine their evaluation by determining the phase of the quadratic sum.

Restrict the additive transform to nonzero inputs and outputs:

$$T : \mathbb{C}[\mathbb{F}_q^*] \longrightarrow \mathbb{C}[\mathbb{F}_q^*], \qquad (Tf)(a) = \sum_{x \in \mathbb{F}_q^*} f(x)\psi_a(x).$$

The elementary scaling identity gives

$$(T\chi)(a) = G(\chi, \psi_a) = \overline{\chi(a)}G(\chi, \psi_1), \qquad a \neq 0. \tag{2.15}$$

For the determinant argument, take $q = p$ with p an odd prime and write $\zeta = e^{2\pi \mathrm{i}/p}$.

Computing the determinant in the multiplicative character basis isolates the quadratic Gauss sum as the only undetermined factor.

Lemma 7. *For the quadratic character η of $\mathbb{F}_p^*$,*

$$\det T = (-1)^{(p-1)/2}\mathrm{i}^{(p-1)(p-3)/4}p^{(p-3)/2}G(\eta, \psi_1).$$

Proof. Use the multiplicative character basis, grouping χ_j with $\overline{\chi}_j$ for $1 \leq j \leq (p-3)/2$. The two unpaired real characters are χ_0 and η. Their eigenvalues are -1 and $G(\eta, \psi_1)$. On the ordered pair $(\chi, \overline{\chi})$, the matrix of T is

$$\begin{pmatrix} 0 & G(\overline{\chi}, \psi_1) \\ G(\chi, \psi_1) & 0 \end{pmatrix},$$

whose determinant is $-p\chi(-1)$. Choose the cyclic indexing of characters so that $\chi_j(-1) = (-1)^j$. With $s = (p-3)/2$, multiplication of the blocks therefore gives

$$\det T = -G(\eta, \psi_1)(-p)^s(-1)^{s(s+1)/2}.$$

Since $s + 1 = (p-1)/2$ and $(-1)^{s(s+1)/2} = \mathrm{i}^{(p-1)(p-3)/4}$, this is the claimed formula. □

A second determinant calculation fixes the phase and selects the correct square root.

Proposition 28 (The sign of the quadratic Gauss sum)**.** *With the canonical additive character of $\mathbb{F}_p$,*

$$G(\eta, \psi_1) = \begin{cases} \sqrt{p} & p \equiv 1 \pmod 4, \\ \mathrm{i}\sqrt{p} & p \equiv 3 \pmod 4. \end{cases}$$

The square root is the positive real square root.

Proof. Since $\eta = \overline{\eta}$, the product formula gives $G(\eta, \psi_1)^2 = \eta(-1)p = (-1)^{(p-1)/2}p$. Thus, for an undetermined $\epsilon_p \in \{1, -1\}$,

$$G(\eta, \psi_1) = \epsilon_p \mathrm{i}^{(p-1)^2/4}\sqrt{p}.$$

Inserting this into the preceding determinant formula yields

$$\det T = \epsilon_p(-1)^{(p-1)/2}\mathrm{i}^{(p-1)(p-2)/2}p^{(p-2)/2}. \tag{2.16}$$

We determine its phase independently in the Dirac basis. The matrix is $(\zeta^{jk})_{1 \leq j,k \leq p-1}$. Factoring ζ^j from row j gives a Vandermonde determinant, and $\prod_{j=1}^{p-1} \zeta^j = 1$. Hence

$$\det T = \prod_{1 \leq m < n \leq p-1} (\zeta^n - \zeta^m).$$

Every difference can be written

$$\zeta^n - \zeta^m = 2\mathrm{i}\, e^{\pi \mathrm{i}(m+n)/p} \sin\left(\frac{\pi(n-m)}{p}\right).$$

The sine is positive because $0 < n - m < p$. There are $N = (p-1)(p-2)/2$ factors, and each index appears in $p-2$ pairs, so

$$\sum_{m<n} (m+n) = (p-2) \sum_{m=1}^{p-1} m = \frac{p(p-1)(p-2)}{2}.$$

It follows that

$$\det T = (-1)^{(p-1)/2} \mathrm{i}^N A, \qquad A = \prod_{m<n} 2\sin\left(\frac{\pi(n-m)}{p}\right) > 0.$$

Comparison with (2.16) forces $\epsilon_p = 1$. □

For comparison, the corresponding evaluation over extension fields contains an additional phase factor.

Remark 15 (Extension fields). For $q = p^s$ with p odd, let η be the quadratic character of $\mathbb{F}_q^*$ and ψ_1 its canonical additive character. The Hasse–Davenport relation implies

$$G(\eta, \psi_1) = (-1)^{s-1} \mathrm{i}^{s(p-1)^2/4} \sqrt{q}.$$

Equivalently, the phase is $(-1)^{s-1}$ when $p \equiv 1 \pmod 4$, and $(-1)^{s-1}\mathrm{i}^s$ when $p \equiv 3 \pmod 4$. The distinction uses p, not q. For instance, the sum over $\mathbb{F}_9$ is $+3$. See Lidl and Niederreiter [40] for the extension formula.

The quadratic reciprocity law follows from the square of the Gauss sum; its proof does not require the more delicate determination of its sign. For historical background and other proofs, see Lemmermeyer [38].

Theorem 3 (Quadratic reciprocity)**.** *For distinct odd primes p and r,*

$$\left(\frac{p}{r}\right)\left(\frac{r}{p}\right) = (-1)^{(p-1)(r-1)/4}. \tag{2.17}$$

Proof. Set $\tau = G(\eta, \psi_1)$ over $\mathbb{F}_p$, and $p^* = (-1)^{(p-1)/2} p$, so that $\tau^2 = p^*$. Let $R = \mathbb{Z}[\zeta]$, where $\zeta = e^{2\pi\mathrm{i}/p}$. This ring contains τ, and reduction modulo rR has characteristic r. Also $rR \cap \mathbb{Z} = r\mathbb{Z}$: if an integer m belongs to rR, then the rational number m/r is an algebraic integer and hence an integer. Here elements of R are algebraic integers because ζ is a root of the monic polynomial $X^p - 1$; a rational root of a monic integer polynomial is an integer.

In characteristic r, the binomial theorem gives

$$\begin{aligned} \tau^r &\equiv \sum_{x \in \mathbb{F}_p^*} \eta(x)^r \zeta^{rx} = \sum_{x \in \mathbb{F}_p^*} \eta(x) \zeta^{rx} \\ &= G(\eta, \psi_r) = \eta(r)\tau \pmod{rR}. \end{aligned}$$

On the other hand, $\tau^r = (p^*)^{(r-1)/2}\tau$. Since p^* is an integer coprime to r, it is a unit modulo rR; $\tau^2 = p^*$ therefore makes τ a unit there as well. Canceling it gives

$$(p^*)^{(r-1)/2} \equiv \eta(r) \pmod{rR}.$$

Both sides are integers, so the congruence holds modulo r. Euler's criterion then gives

$$(-1)^{(p-1)(r-1)/4}\left(\frac{p}{r}\right) \equiv \left(\frac{r}{p}\right) \pmod r.$$

The two sides belong to $\{1, -1\}$ and r is odd, so they are equal as integers. Multiplying by $\left(\frac{p}{r}\right)$ proves the result. □

2.2 The Walsh transform

The Walsh transform is Fourier analysis on the Boolean cube $G = (\mathbb{Z}/2\mathbb{Z})^k$. Unlike a cyclic group of order 2^k, this group adds coordinates independently modulo two. Its characters take only the values 1 and -1, so the transform can be computed using additions and subtractions. Figure 2.1 shows the four-dimensional cube, with edges joining vectors that differ in one coordinate.

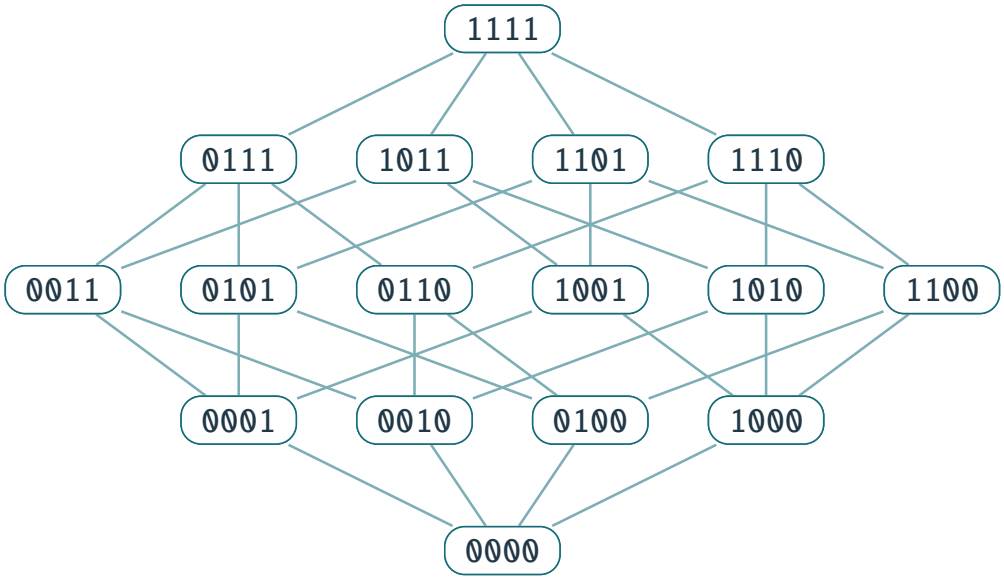


Figure 2.1. The Boolean cube $\mathbb{F}_2^4$, arranged by Hamming weight. Each edge changes exactly one coordinate.

2.2.1 Characters of the Boolean cube

On the Boolean cube, each character tests the parity of a chosen set of coordinates. Writing these tests in a fixed order produces the Walsh matrix and a concrete transform for binary-indexed data.

For $a = (a_0, \dots, a_{k-1}) \in \mathbb{F}_2^k$, define

$$\chi_a(x) = (-1)^{\langle a, x\rangle}, \qquad \langle a,\, x\rangle = \sum_{j=0}^{k-1} a_j x_j \quad \text{in } \mathbb{F}_2. \tag{2.18}$$

These are exactly the characters of the cube. Identify a binary vector x with the integer $\sum_j x_j 2^j$ in $\{0, \dots, 2^k - 1\}$. This gives a *natural order* of both the group elements and their characters. The identification is a bijection of sets; binary-vector addition corresponds to bitwise exclusive-or, not integer addition modulo 2^k.

The three-dimensional cube makes the sign pattern and the indexing convention explicit.

Example 2. For $k = 3$, the character table is the Walsh matrix

$$W_8 = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 & 1 & -1 & -1 & 1 \\ 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 \\ 1 & -1 & 1 & -1 & -1 & 1 & -1 & 1 \\ 1 & 1 & -1 & -1 & -1 & -1 & 1 & 1 \\ 1 & -1 & -1 & 1 & -1 & 1 & 1 & -1 \end{pmatrix}. \tag{2.19}$$

Its entry (i, j) is $\chi_i(j)$. Figure 2.2 shows W_{64}, with white entries for 1 and dark teal entries for -1.

Multiplication by this character table defines the Walsh transform with the same unnormalized convention as before.

Definition 13 (Walsh transform)**.** For a vector $f \in \mathbb{C}^{2^k}$, define

$$\mathcal{W}_k(f)[i] = \sum_{j=0}^{2^k-1} f[j]\chi_i(j) = 2^k \langle f,\, \chi_i\rangle, \qquad 0 \le i < 2^k. \tag{2.20}$$

Thus $\mathcal{W}_k(f) = W_{2^k} f$.

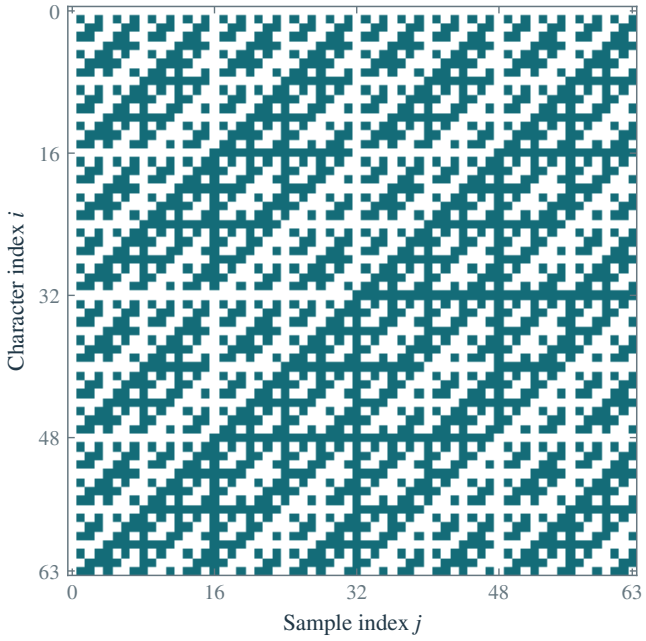


Figure 2.2. The Walsh matrix W_{64} in natural order. White represents +1 and dark teal represents −1.

Because the characters are real-valued, this is also $\hat{f}(\chi_i)$ under the convention of Chapter 1. No conjugation changes the result.

On the cube, the *degree* of a character is the number of nonzero coordinates in its index a: a character of degree d is the product of d coordinate characters. Small degree is important in Boolean function analysis. A different ordering, by the number of sign changes along the integer-indexed row, is called *sequency order*. The two notions should not be confused; Exercise 2.5 develops the latter.

Orthogonality makes inversion especially simple: applying the same transform twice recovers the data up to a scalar factor.

Proposition 29 (Inversion)**.** *The Walsh matrix is real and symmetric and satisfies* $W_{2^k}^2 = 2^k I$*. Consequently,*

$$\mathcal{W}_k^{-1} = 2^{-k}\mathcal{W}_k, \qquad f[i] = 2^{-k}\sum_{j=0}^{2^k-1} \mathcal{W}_k(f)[j]\chi_j(i).$$

Proof. Symmetry follows from $\langle i, j\rangle = \langle j, i\rangle$. Character orthogonality gives $W_{2^k}W_{2^k}^{\mathsf{T}} = 2^k I$, which proves the assertions. □

2.2.2 A fast algorithm

The binary product structure makes it possible to compute the Walsh transform recursively. Splitting one coordinate at a time reduces the cost from a dense matrix multiplication to a sequence of additions and subtractions.

Split f into its first and second halves, f_1 and f_2, each of length 2^{k-1}. Separating the highest binary coordinate in (2.20) gives

$$\mathcal{W}_k(f) = \begin{pmatrix} \mathcal{W}_{k-1}(f_1) + \mathcal{W}_{k-1}(f_2) \\ \mathcal{W}_{k-1}(f_1) - \mathcal{W}_{k-1}(f_2) \end{pmatrix}.$$

Equivalently,

$$W_{2^k} = \begin{pmatrix} W_{2^{k-1}} & W_{2^{k-1}} \\ W_{2^{k-1}} & -W_{2^{k-1}} \end{pmatrix}, \qquad W_1 = (1).$$

This recursion is the *fast Walsh transform* (FWT). For $n = 2^k$, its arithmetic cost satisfies $C(n) = 2C(n/2) + n$, with $C(1) = 0$. There are $\log_2 n$ levels, each requiring n additions or subtractions, so $C(n) = n \log_2 n$. Program 61 implements the recursion in Python. Exercise 2.7 explains its tensor-product structure. The FFT of the next chapter uses the same divide-and-conquer principle.

2.2.3 Uses and limitations

The Walsh transform decomposes functions into a simple orthogonal basis and is particularly convenient for binary data. It supports factorial contrasts in statistics (Exercise 2.3), signal compression (Exercise 2.5), and Boolean function analysis (Exercises 6.4 and 6.5).

As a group Fourier transform, it also satisfies a convolution theorem. Writing $x \oplus y$ for bitwise exclusive-or, define

$$(f *_{\oplus} g)[x] = \sum_{y} f[y]g[x \oplus y].$$

Then $\mathcal{W}_k(f *_{\oplus} g) = \mathcal{W}_k(f) \cdot \mathcal{W}_k(g)$. The Walsh transform generally does *not* diagonalize the circular convolution based on subtraction modulo 2^k. That operation belongs to the cyclic group and is diagonalized by the DFT. The distinction is the group law, not a failure of the convolution theorem.

2.3 Poisson summation

Poisson summation relates restriction to a subgroup and restriction of the Fourier transform to its annihilator. In a vector space over $\mathbb{F}_2$, this becomes a statement about orthogonal subspaces and leads to the MacWilliams identity for weight enumerators.

2.3.1 The finite-group formula

Summing a function over a subgroup preserves only those frequencies that are trivial on that subgroup. We formalize this observation through annihilators, starting with their linear-algebraic analogue.

Recall first the linear-algebraic pairing between a finite-dimensional k-vector space E and its dual $E^* = \mathrm{Hom}_k(E, k)$: $\langle x, \ell \rangle = \ell(x)$. For $A \subset E$, its annihilator is

$$A^{\perp} = \{\ell \in E^* : \ell(x) = 0 \text{ for every } x \in A\}. \tag{2.21}$$

It is a subspace of E^*. Conversely, for $B \subset E^*$, $B^0 = \{x \in E : \ell(x) = 0 \text{ for every } \ell \in B\}$ is a subspace of E. On subspaces, these operations are inverse, inclusion-reversing bijections. See Ramis, Deschamps, and Odoux [49].

For a group, characters that are trivial on a subgroup play the role of linear forms vanishing on a subspace.

Definition 14 (Annihilator of a subgroup)**.** For a subgroup H of a finite abelian group G, define

$$H^{\sharp} = \{\chi \in \hat{G} : \chi(h) = 1 \text{ for every } h \in H\}.$$

This subgroup of $\hat{G}$ is also called the orthogonal of H.

By Lemma 2, pullback from G/H gives a canonical identification $H^{\sharp} \simeq \widehat{G/H}$ and $|H^{\sharp}| = |G|/|H|$. The operation reverses inclusions, and $(H^{\sharp})^{\sharp} = H$ after identifying G with its bidual. Indeed, the double annihilator contains H and has the same order.

The annihilator identifies exactly the frequencies that survive a sum over a coset.

Theorem 4 (Finite Poisson summation)**.** *For $f : G \to \mathbb{C}$, a subgroup $H \subset G$, and $g \in G$,*

$$\sum_{h \in H} f(gh) = \frac{|H|}{|G|} \sum_{\chi \in H^{\sharp}} \hat{f}(\overline{\chi})\chi(g). \tag{2.22}$$

Proof. Periodize f over H by setting $\tilde{f}(gH) = \sum_{h \in H} f(gh)$. Replacing g by gh_0, with $h_0 \in H$, merely permutes the sum, so this is a well-defined function on G/H. Its Fourier expansion is

$$\tilde{f}(gH) = \sum_{\chi \in \widehat{G/H}} c_{\tilde{f}}(\chi)\chi(gH). \tag{2.23}$$

Choose representatives S for G/H. Identifying each quotient character with its pullback, its Fourier coefficient is

$$\begin{aligned} c_{\tilde{f}}(\chi) &= \frac{1}{|G/H|} \sum_{s \in S} \sum_{h \in H} f(sh)\overline{\chi(s)} \\ &= \frac{|H|}{|G|} \sum_{x \in G} f(x)\overline{\chi(x)} = \frac{|H|}{|G|} \hat{f}(\overline{\chi}). \end{aligned} \tag{2.24}$$

Here $(s,h) \mapsto sh$ is a bijection $S \times H \to G$, and $\chi(sh) = \chi(s)$. Substitute in the Fourier expansion and use $\widehat{G/H} \simeq H^\sharp$. □

At $g = 1$, inversion permutes $H^\sharp$, giving the common form

$$\sum_{h \in H} f(h) = \frac{|H|}{|G|} \sum_{\chi \in H^\sharp} \hat{f}(\chi). \tag{2.25}$$

There is also a useful proof by comparing the trace of a convolution operator in two bases. Pullback along $G \to G/H$ identifies $\mathbb{C}[G/H]$ with the subspace $V_H \subset \mathbb{C}[G]$ of functions constant on cosets of H.

A trace proof. Let $A_f : u \mapsto f * u$. Translation invariance of convolution implies $A_f(V_H) \subset V_H$. A basis of V_H consists of the coset indicators $e_{sH} = \mathbf{1}_{sH}$, for $s \in S$. These are not point masses on G. For each diagonal entry,

$$(A_f e_{sH})(s) = \sum_{x \in G} f(x) e_{sH}(x^{-1}s) = \sum_{h \in H} f(h).$$

Consequently,

$$\operatorname{Tr}(A_f|_{V_H}) = \frac{|G|}{|H|} \sum_{h \in H} f(h).$$

A second basis of V_H is the character family $H^\sharp$. The eigenvalue associated with χ is $\hat{f}(\chi^{-1})$, as shown in Exercise 1.1. Thus

$$\operatorname{Tr}(A_f|_{V_H}) = \sum_{\chi \in H^\sharp} \hat{f}(\chi^{-1}) = \sum_{\chi \in H^\sharp} \hat{f}(\chi).$$

Comparing the traces proves (2.25). For the full formula, apply it to $f_g(x) = f(gx)$ and use $\hat{f}_g(\chi) = \chi(g)^{-1}\hat{f}(\chi)$, then reindex by inversion. □

2.3.2 The MacWilliams identity

Poisson summation links the weight distribution of a binary linear code to that of its orthogonal code. We translate group annihilators into vector-space orthogonality and encode the resulting counting information in a polynomial.

Take $G = \mathbb{F}_2^k$. Every additive subgroup is an $\mathbb{F}_2$-linear subspace, because multiplication by either scalar 0 or 1 preserves it. Under the pairing $a \mapsto \chi_a$, we have

$$\chi_a \in H^\sharp \iff \langle a, h \rangle = 0 \text{ for every } h \in H \iff a \in H^\perp. \tag{2.26}$$

Thus the group annihilator corresponds to the linear orthogonal. The dot product over $\mathbb{F}_2$ is nondegenerate but not positive definite: for example, $(1,1,1,1)$ is orthogonal to itself. In particular, $H^\perp$ need not be a complementary subspace to H.

For extension fields, an additive subgroup need not be a linear subspace over the whole field. Exercise 2.8 treats the generalization with the required linearity assumption.

In binary coordinates, finite Poisson summation consequently takes the following symmetric pair of forms.

Proposition 30 (Vector-space Poisson summation)**.** *For an $\mathbb{F}_2$-linear subspace $H \subset \mathbb{F}_2^k$ and a function $f : \mathbb{F}_2^k \to \mathbb{C}$,*

$$\sum_{a \in H} f(a) = \frac{|H|}{2^k} \sum_{u \in H^\perp} \hat{f}(\chi_u), \tag{2.27}$$

$$\sum_{a \in H^\perp} f(a) = \frac{1}{|H|} \sum_{u \in H} \hat{f}(\chi_u). \tag{2.28}$$

Proof. The first identity is finite Poisson summation under (2.26). For the second, replace H by $H^\perp$, using $|H^\perp| = 2^k/|H|$ and $(H^\perp)^\perp = H$. □

To apply these formulas to codes, we first measure how many coordinates distinguish two binary vectors.

Definition 15 (Hamming weight and distance)**.** For $z \in \mathbb{F}_2^k$, its weight is the number of nonzero coordinates, $w(z) = |\{i : z_i \neq 0\}|$. The Hamming distance is

$$d(x, y) = w(x - y) = |\{i : x_i \neq y_i\}|.$$

In characteristic two, $x - y = x + y$.

In Figure 2.1, edges join vectors at Hamming distance one. To record the distribution of weights in a subspace, introduce a polynomial.

Definition 16 (Weight enumerator)**.** For $H \subset \mathbb{F}_2^k$, define

$$A_H(X, Y) = \sum_{c \in H} X^{k-w(c)} Y^{w(c)} = \sum_{i=0}^{k} A_i X^{k-i} Y^i,$$

where A_i is the number of vectors in H of weight i.

Poisson summation determines the weight enumerator of the orthogonal code by a simple change of variables.

Theorem 5 (MacWilliams identity)**.** *For a linear subspace $H \subset \mathbb{F}_2^k$,*

$$A_{H^\perp}(X, Y) = \frac{1}{|H|} A_H(X + Y, X - Y).$$

Proof. Fix $x, y \in \mathbb{C}$ and set $f(t) = x^{k-w(t)} y^{w(t)}$. The Walsh transform factors coordinate by coordinate:

$$\begin{aligned} \hat{f}(\chi_a) &= \sum_{t \in \mathbb{F}_2^k} \prod_{j=0}^{k-1} x^{1-t_j} y^{t_j} (-1)^{a_j t_j} \\ &= \prod_{j=0}^{k-1} (x + (-1)^{a_j} y) = (x + y)^{k-w(a)} (x - y)^{w(a)}. \end{aligned}$$

Insert this expression in (2.28). The equality holds for every $(x, y) \in \mathbb{C}^2$, so it is a polynomial identity. □

This relates the entire weight distribution of a linear code to that of its dual. Section 6.4 develops its coding-theoretic consequences.

2.3.3 Continuous Poisson summation

The same relation between sampling and periodization survives on the real line, where convergence becomes an essential part of the argument. We prove a continuous Poisson formula under explicit decay assumptions and apply it to Gaussian sums.

For this subsection, use the frequency convention

$$\hat{f}(\xi) = \int_{\mathbb{R}} f(t) e^{-2\pi i t \xi} \, \mathrm{d}t. \tag{2.29}$$

The finite-group proof extends by periodization, provided we justify convergence and the interchange of sums and integrals.

Theorem 6 (Continuous Poisson summation)**.** *Let $f : \mathbb{R} \to \mathbb{C}$ be continuous and suppose that*

$$|f(x)| \leq M(1 + |x|)^{-\alpha} \quad \text{for some } M > 0,\ \alpha > 1, \tag{2.30}$$

$$\sum_{n \in \mathbb{Z}} |\hat{f}(n)| < \infty. \tag{2.31}$$

Then

$$\sum_{n \in \mathbb{Z}} f(n) = \sum_{n \in \mathbb{Z}} \hat{f}(n). \tag{2.32}$$

Proof. Define $f_1(x) = \sum_{n\in\mathbb{Z}} f(x+n)$. For $|x| \le A$ and $|n| \ge 2A$,

$$|f(x+n)| \le M(1+|n|-A)^{-\alpha} \le M(1+|n|/2)^{-\alpha}.$$

The last bound is summable. Hence the periodization converges uniformly on compact intervals, is continuous, and is 1-periodic. Its Fourier coefficient at $m \in \mathbb{Z}$ is

$$\begin{aligned} c_m(f_1) &= \int_0^1 \sum_{n\in\mathbb{Z}} f(t+n)e^{-2\pi imt}\,\mathrm{d}t \\ &= \sum_{n\in\mathbb{Z}} \int_n^{n+1} f(u)e^{-2\pi imu}\,\mathrm{d}u = \hat{f}(m). \end{aligned}$$

Uniform convergence justifies the first interchange, and $f \in L^1(\mathbb{R})$ justifies the sum over intervals. The second hypothesis makes $\sum_m \hat{f}(m)e^{2\pi imx}$ uniformly convergent. Its sum and f_1 have identical Fourier coefficients; uniqueness and continuity imply

$$\sum_{n\in\mathbb{Z}} f(x+n) = \sum_{m\in\mathbb{Z}} \hat{f}(m)e^{2\pi imx}.$$

Set $x = 0$. □

The quotient interpretation explains why the finite and continuous arguments have the same structure.

Remark 16 (The quotient viewpoint). Here the subgroup is $\mathbb{Z} \subset \mathbb{R}$, and periodization produces a function on $\mathbb{R}/\mathbb{Z}$. Its characters are $x \mapsto e^{2\pi imx}$, indexed by $m \in \mathbb{Z}$. Thus the same quotient-and-dual construction underlies the finite and continuous formulas. In the continuous setting the constants are determined by the chosen Haar measures and Fourier convention.

A particularly useful application concerns a sum of Gaussian samples, recorded by the theta function.

Definition 17 (Jacobi's theta function)**.** For $t > 0$, define

$$\theta(t) = \sum_{n\in\mathbb{Z}} e^{-\pi n^2 t}.$$

To apply Poisson summation to this series, we need the Gaussian Fourier transform with the chosen frequency convention.

Lemma 8 (Fourier transform of a Gaussian)**.** *For $t > 0$, let $g_t(x) = e^{-x^2/(2t)}$. Under* (2.29)*,*

$$\hat{g}_t(\xi) = \sqrt{2\pi t}\, e^{-2\pi^2 t\xi^2}.$$

Proof. Differentiation under the integral is justified by the integrability of $|x|g_t(x)$. Since $xg_t(x) = -tg_t'(x)$, integration by parts gives

$$\frac{\mathrm{d}}{\mathrm{d}\xi}\hat{g}_t(\xi) = -4\pi^2 t\xi\hat{g}_t(\xi).$$

The boundary term vanishes by Gaussian decay. Solving this differential equation gives $\hat{g}_t(\xi) = \hat{g}_t(0)e^{-2\pi^2 t\xi^2}$. The substitution $x = \sqrt{t}\,u$ gives $\hat{g}_t(0) = \sqrt{t}\,I$, where $I = \int_{\mathbb{R}} e^{-u^2/2}\,\mathrm{d}u$. Tonelli's theorem and polar coordinates yield

$$I^2 = \int_{\mathbb{R}^2} e^{-(u^2+v^2)/2}\,\mathrm{d}u\,\mathrm{d}v = 2\pi\int_0^\infty re^{-r^2/2}\,\mathrm{d}r = 2\pi.$$

Since $I > 0$, the constant is $\sqrt{2\pi t}$. □

Combining the Gaussian transform with Poisson summation relates the theta series at reciprocal scales.

Theorem 7 (Jacobi's identity)**.**

$$\theta(t) = \frac{1}{\sqrt{t}}\theta(1/t), \qquad t > 0. \tag{2.33}$$

Proof. Apply Poisson summation to $h_t(x) = e^{-\pi tx^2} = g_{1/(2\pi t)}(x)$. The preceding lemma gives $\hat{h}_t(\xi) = t^{-1/2}e^{-\pi\xi^2/t}$, and both decay hypotheses hold. □

For small t, this identity greatly accelerates evaluation. On the right, keeping only the term $n = 0$ approximates $\theta(t)$ by $t^{-1/2}$, with relative error bounded by

$$0 < \frac{\theta(t) - t^{-1/2}}{t^{-1/2}} = 2\sum_{n=1}^{\infty} e^{-\pi n^2/t} \le \frac{2e^{-\pi/t}}{1 - e^{-3\pi/t}}.$$

For $t = 0.001$ this truncation error is far below double-precision rounding. The computed value is still subject to floating-point rounding; the identity reduces the number of terms, not the machine's rounding error.

2.4 Exercises

These exercises develop the arithmetic, computational, and analytic consequences of duality. We start with a lattice-point argument that gives a geometric alternative to the Gauss-sum proof of quadratic reciprocity.

Exercise 2.1 (A geometric proof of quadratic reciprocity)**.** Let p and r be distinct odd primes, and let $[x]_p \in \{0, \dots, p-1\}$ denote the least nonnegative residue of an integer x modulo p. We will prove (2.17) by counting lattice points.

1. Set $E = \{2, 4, \dots, p-1\}$ and $B_r = \{[ra]_p : a \in E\}$. Prove that $E = \{[(-1)^b b]_p : b \in B_r\}$.
2. Multiply the elements in these sets and use Euler's criterion to obtain
$$\left(\frac{r}{p}\right) = (-1)^{\sum_{b\in B_r} b}.$$
3. Use Euclidean division of ra by p to prove
$$\sum_{a\in E} ra = p\sum_{a\in E}\left\lfloor \frac{ra}{p}\right\rfloor + \sum_{b\in B_r} b, \qquad \left(\frac{r}{p}\right) = (-1)^{\sum_{a\in E}\lfloor ra/p\rfloor}.$$
4. In Figure 2.3, let $A = (0,0)$, $D = (p,0)$, $B = (p,r)$, $K = (p/2,0)$, $L = (0,r/2)$, and $H = (p/2, r/2)$. Show that the open diagonal segment AB contains no lattice point. Interpret $t_a = \lfloor ra/p\rfloor$ as the number of lattice points with abscissa a strictly between the horizontal axis and AB.
5. Show that $t_{p-a} = r - 1 - t_a$, so $t_{p-a} \equiv t_a \pmod 2$. Each pair $\{a, p-a\}$ contains exactly one even integer. Deduce that
$$\sum_{a\in E} t_a \equiv \sum_{a=1}^{(p-1)/2} t_a \pmod 2.$$
Interpret the right-hand side as the number of lattice points strictly inside the triangle AKH.
6. Interchange p and r. The two resulting triangles partition the lattice points strictly inside $AKHL$. Deduce quadratic reciprocity.

Gauss-sum bounds can guarantee nonzero solutions to polynomial equations over sufficiently large finite fields.

Exercise 2.2 (Fermat's equation over a finite field)**.** Let q be a prime power and $k \ge 1$ an integer. Using the counting estimates of Exercise 1.5, prove that $q \ge k^4 + 4$ guarantees a solution of

$$x^k + y^k = z^k, \qquad x, y, z \in \mathbb{F}_q^*. \tag{2.34}$$

All Fourier transforms in this exercise are on the additive group of $\mathbb{F}_q$.

1. Suppose first that $k \mid q-1$. Show that the unique multiplicative subgroup of index k is $H_k = \{x^k : x \in \mathbb{F}_q^*\}$.
2. Let $\chi_0, \dots, \chi_{k-1}$ be the characters of $\mathbb{F}_q^*/H_k$, pulled back to $\mathbb{F}_q^*$. Show that for every additive character ψ,
$$\widehat{f_{H_k}}(\psi) = \frac{1}{k}\sum_{i=0}^{k-1} G(\chi_i, \psi).$$
Deduce from the Gauss-sum estimates that $\Phi(H_k) < \sqrt{q}$, where Φ is defined in (1.17).

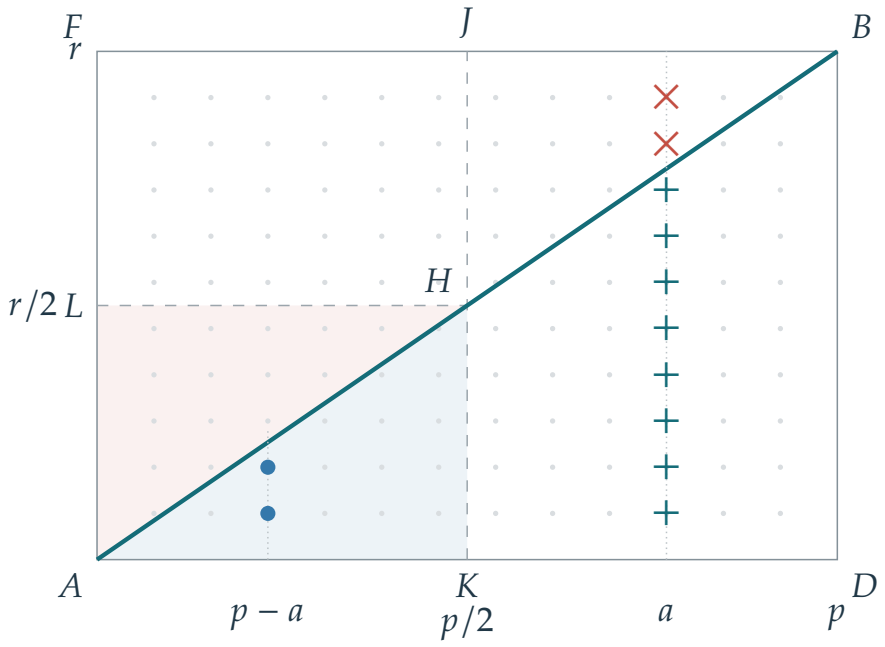


Figure 2.3. Lattice-point pairing in the geometric proof of quadratic reciprocity, illustrated with $p = 13$, $r = 11$, and $a = 10$.

3. For nonempty subsets $A_1, A_2 \subset \mathbb{F}_q^*$, let N and N' count, respectively, the solutions of

$$\begin{aligned} x + y &= z^k, && x \in A_1, \quad y \in A_2, \quad z \in \mathbb{F}_q^*, \\ x + y &= u, && x \in A_1, \quad y \in A_2, \quad u \in H_k. \end{aligned} \tag{2.35}$$

   Show that $N = kN'$ and, by applying the counting estimate to $A_1, A_2, -H_k$, prove

$$\left| N - \frac{|A_1||A_2|(q-1)}{q} \right| < k\sqrt{|A_1||A_2|q}. \tag{2.36}$$

4. Write $l_i = (q-1)/|A_i|$. Prove that $q \geq k^2 l_1 l_2 + 4$ implies $N > 0$. For general k, show that the sets of nonzero kth powers and dth powers coincide, where $d = \gcd(k, q-1)$, and extend the criterion.
5. Apply it with $A_1 = A_2 = H_d$ to prove the claimed sufficient condition for (2.34).

For a factorial experiment, Walsh coefficients separate the contributions of individual factors and their interactions.

Exercise 2.3 (Walsh transforms and factorial contrasts)**.** This example, adapted from Maslen and Rockmore [43], illustrates Yates's algorithm for a two-level factorial experiment. A farmer measures wheat height under three factors: lighting a, herbicide b, and fertilizer c. Each factor is set high (+) or low (−). The observed heights, in centimeters, are

a	b	c	α_{abc}	a	b	c	α_{abc}
+	+	+	69	+	+	−	61
−	+	+	81	−	+	−	92
+	−	+	63	+	−	−	54
−	−	+	77	−	−	−	89

Encode + by 0 and − by 1, with a the least significant bit. The data vector is therefore $f = (69, 81, 63, 77, 61, 92, 54, 89)$. The grand mean is $\mu = \frac{1}{8}\sum \alpha_{abc}$. For a nonempty subset $S \subset \{a, b, c\}$, define its factorial effect by

$$\mu_S = \frac{1}{4} \sum_{(a,b,c) \in \{+1,-1\}^3} \alpha_{abc} \prod_{s \in S} s.$$

For instance, μ_a is the mean at high lighting minus the mean at low lighting, averaged over the other two factors. Higher-order contrasts measure interactions under the same normalization. Express all seven effects

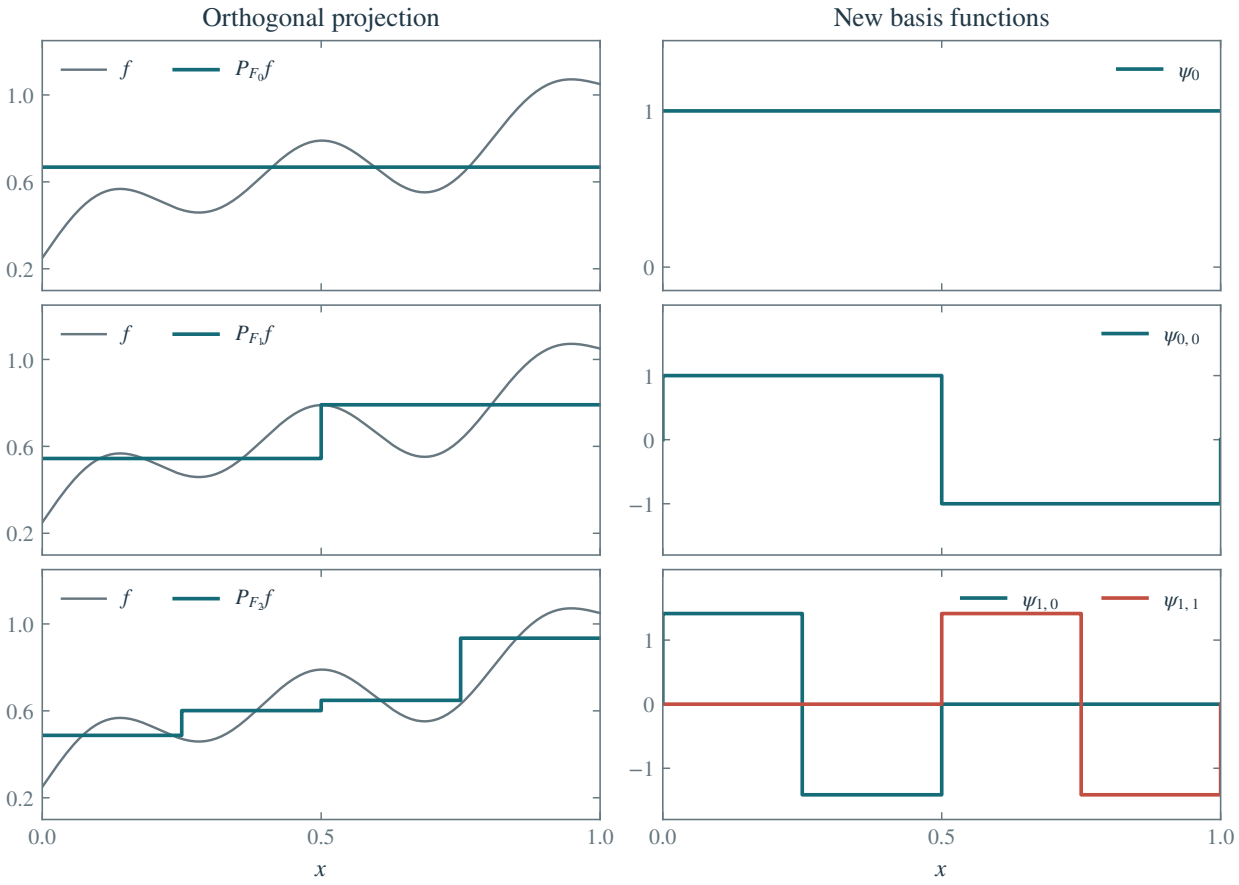


Figure 2.4. Successive Haar projections onto F_0, F_1, and F_2 (left), and the orthonormal basis functions added at each stage (right).

and the mean using a single Walsh transform, and compute them for the given data. These contrasts describe the observations; assessing statistical significance would also require a model for experimental variability.

The Haar basis offers a different decomposition of binary-scale data, organized by location and resolution.

Exercise 2.4 (The Haar basis and its fast transform)**.** Let $\psi_0 = \mathbf{1}_{[0,1)}$ and $\psi = \mathbf{1}_{[0,1/2)} - \mathbf{1}_{[1/2,1)}$. Define

$$\psi_{j,k}(x) = 2^{j/2}\psi(2^j x - k), \qquad j \geq 0, \quad 0 \leq k < 2^j, \qquad \psi_n = \psi_{j,k} \quad \text{when } n = 2^j + k.$$

Let F_J be the space of functions constant on each dyadic interval $[k2^{-J}, (k+1)2^{-J})$, for $0 \leq k < 2^J$. All inner products in this exercise are the usual integral inner product on $L^2([0,1])$. For pointwise statements on $[0,1]$, assign each step function its left limit at 1.

1. Prove that $\psi_0, \ldots, \psi_{2^J-1}$ form an orthonormal basis of F_J.
2. For $f \in C([0,1])$, its orthogonal projection onto F_J is

$$f_J = \langle f, \psi_0\rangle\psi_0 + \sum_{j=0}^{J-1}\sum_{k=0}^{2^j-1}\langle f, \psi_{j,k}\rangle\psi_{j,k}.$$

   Show that it is the average of f on each dyadic interval and converges uniformly to f. Prove the same convergence for every partial sum $\tilde{f}_n = \sum_{m=0}^{n}\langle f, \psi_m\rangle\psi_m$. Hint: a partial sum within one wavelet level averages f on a partition in which only some of the dyadic intervals have been bisected. Deduce that the Haar family is a complete orthonormal basis of $L^2([0,1])$.
3. Define $\varphi_{j,k}(x) = 2^{j/2}\psi_0(2^j x - k)$. Show that these functions form an orthonormal basis of F_j, and that

$$F_{j+1} = F_j \overset{\perp}{\oplus} G_j, \qquad G_j = \operatorname{span}\{\psi_{j,k} : 0 \leq k < 2^j\}.$$

   Express $\varphi_{j,k}$ and $\psi_{j,k}$ using $\varphi_{j+1,2k}$ and $\varphi_{j+1,2k+1}$.

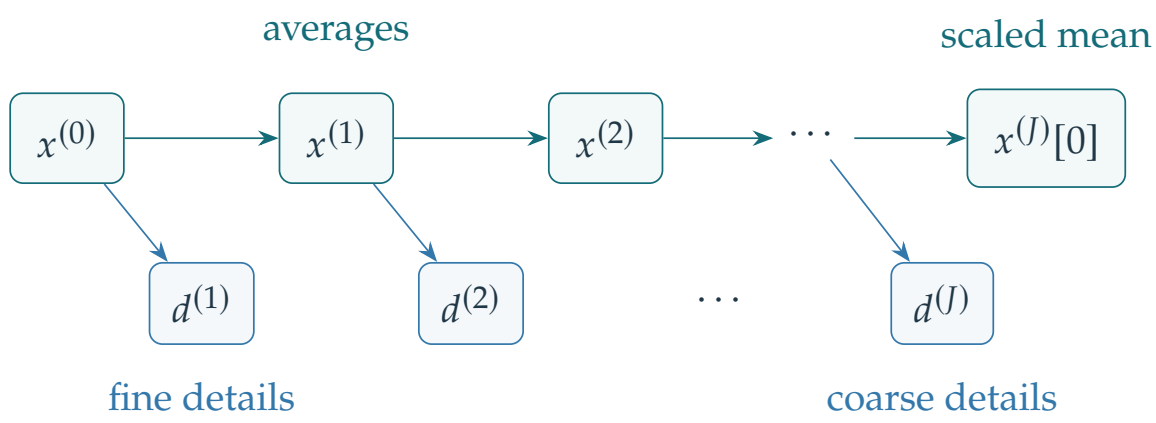


Figure 2.5. The Haar cascade: each stage separates scaling coefficients from details, which are stored from fine to coarse.

4. Let $f \in F_J$ and set $x^{(0)}[k] = \langle f,\, \varphi_{J,k}\rangle$. Relate these coefficients to the values of f on its dyadic cells. For $i = 1, \dots, J$, define vectors of length 2^{J-i} by

$$x^{(i)}[k] = \frac{x^{(i-1)}[2k] + x^{(i-1)}[2k+1]}{\sqrt{2}}, \qquad d^{(i)}[k] = \frac{x^{(i-1)}[2k] - x^{(i-1)}[2k+1]}{\sqrt{2}}.$$

   Show that each stage is an orthogonal transformation. Is its elementary 2×2 block a rotation? Prove that $x^{(J)}[0] = 2^{J/2}\,\mathrm{mean}(x^{(0)})$.
5. Prove that $x^{(i)}[k] = \langle f,\, \varphi_{J-i,k}\rangle$ and $d^{(i)}[k] = \langle f,\, \psi_{J-i,k}\rangle$. In particular, $x^{(J)}[0] = \langle f,\, \psi_0\rangle$. Estimate the arithmetic cost of all stages.
6. For $N = 2^J$, let Γ map the input vector to the concatenation $(d^{(1)}, d^{(2)}, \dots, d^{(J)}, x^{(J)})$. Show that Γ is orthogonal and identify the basis vectors whose coefficients it computes. Compare their supports, and the cost of the transform, with those of the Walsh basis.

Figure 2.6 uses this fine-to-coarse packing. For a smooth signal, neighboring fine-scale values differ little, so their Haar detail coefficients are small. The last coefficient records the mean with the orthonormal scaling specified above. Haar analysis is the simplest example of the multiresolution filter-bank construction developed by Mallat [42]. See also Exercise 7.11 for wavelets over finite fields.

Ordering Walsh functions by their oscillations makes it possible to explore approximation and compression.

Exercise 2.5 (Compression with Walsh functions). 1. For $N = 2^k$, define the two-dimensional basis functions $\chi_{i,j}(s,t) = \chi_i(s)\chi_j(t)$, for $0 \le i, j, s, t < N$. Give the forward and inverse transforms, and implement the forward transform by successive one-dimensional transforms along the two axes.
2. The *sequency* of a Walsh row is the number of sign changes between consecutive entries, without wrapping the last entry to the first. Show that the sequencies are precisely $0, 1, \dots, N-1$. Compare natural and sequency ordering in Figure 2.7.
3. Compare the spectra of a smooth signal in these two orders. Explain in what sense small sequency corresponds to slowly varying functions, and why it differs from small Hamming degree on the Boolean cube.
4. Write $i_0, \dots, i_{N-1}$ for the indices in increasing sequency. For $0 \le M \le N$, define

$$f_M = \sum_{m=0}^{M-1} \langle f,\, \chi_{i_m}\rangle \chi_{i_m} = \frac{1}{N} \sum_{m=0}^{M-1} \mathcal{W}_k(f)[i_m]\chi_{i_m},$$

   with the normalized discrete inner product. Explain how storing these M coefficients gives a linear approximation scheme and why their quantization is also needed for a finite bit representation. Compare it with low-frequency DFT approximation after reading Chapter 3.
5. For the two-dimensional functions, suggest an order based on their horizontal and vertical sequencies. One choice is increasing sum of the two sequencies, with ties broken by the vertical sequency. Implement the corresponding image approximation in Python.

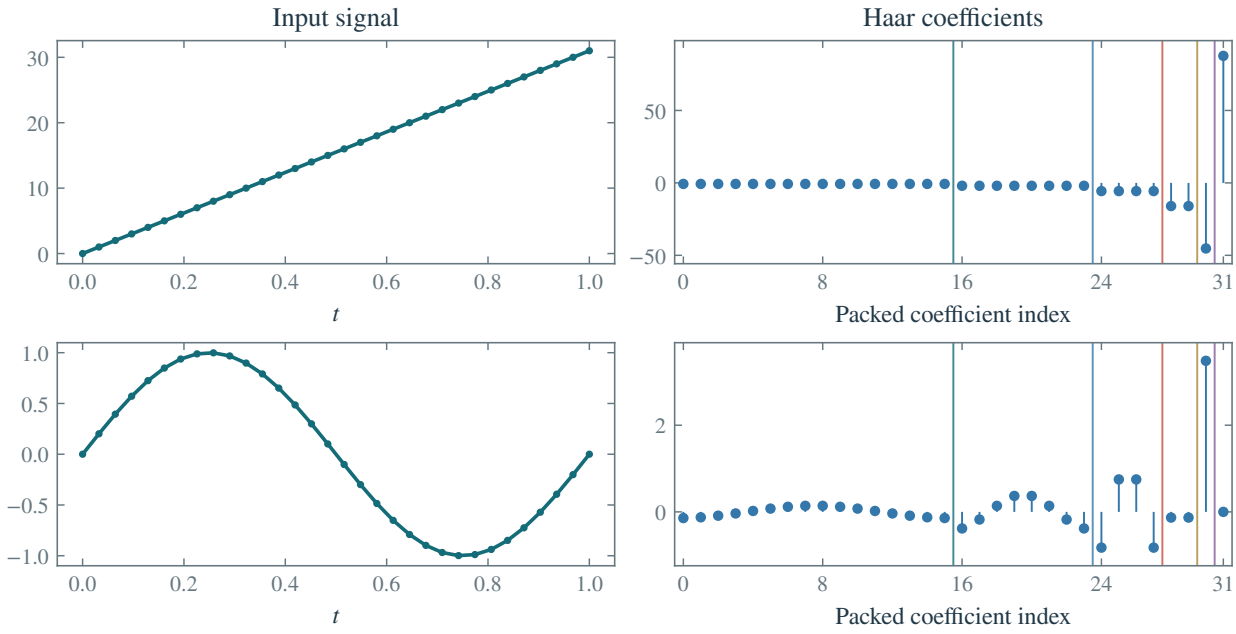


Figure 2.6. Two length-32 signals and their orthonormal Haar coefficients. Colored separators mark the detail scales; the final coefficient is the scaled mean.

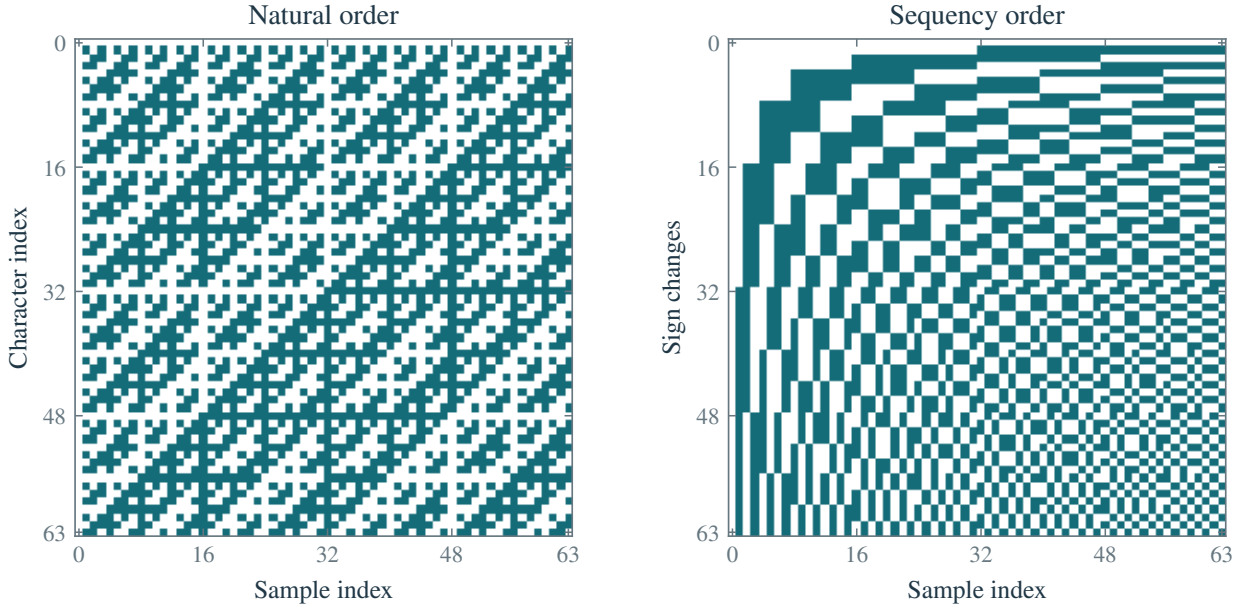


Figure 2.7. Walsh functions in natural order and in increasing sequency, with one basis function per row.

Walsh matrices belong to a broader family of orthogonal sign matrices whose possible sizes have strong arithmetic constraints.

Exercise 2.6 (Hadamard matrices)**.** A real $N \times N$ matrix H_N with entries in $\{1, -1\}$ is a *Hadamard matrix* if $H_N H_N^{\mathsf{T}} = N I_N$.

1. Show that the Walsh matrix W_{2^k} is a Hadamard matrix.
2. Show that changing the sign of a row or column, or permuting rows or columns, preserves this property. Normalize the first row and column to 1. If $N \geq 3$, group the columns by the four sign patterns of rows two and three. If their multiplicities are i, j, k, l, prove that $i = j = k = l = N/4$. Thus a Hadamard order is 1, 2, or a multiple of 4.

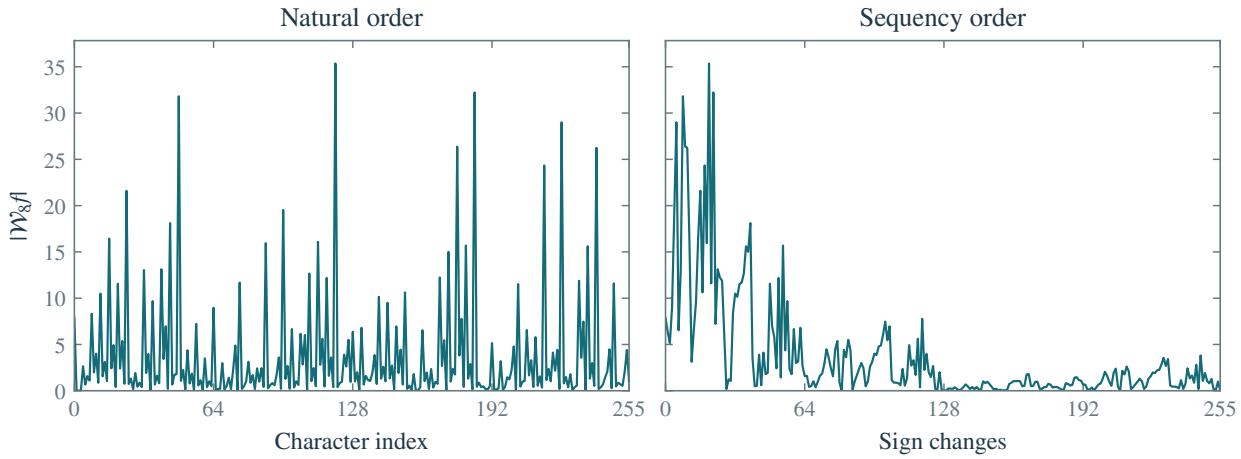


Figure 2.8. The Walsh spectrum of the oscillatory signal in Figure 2.9, in natural and sequency order.

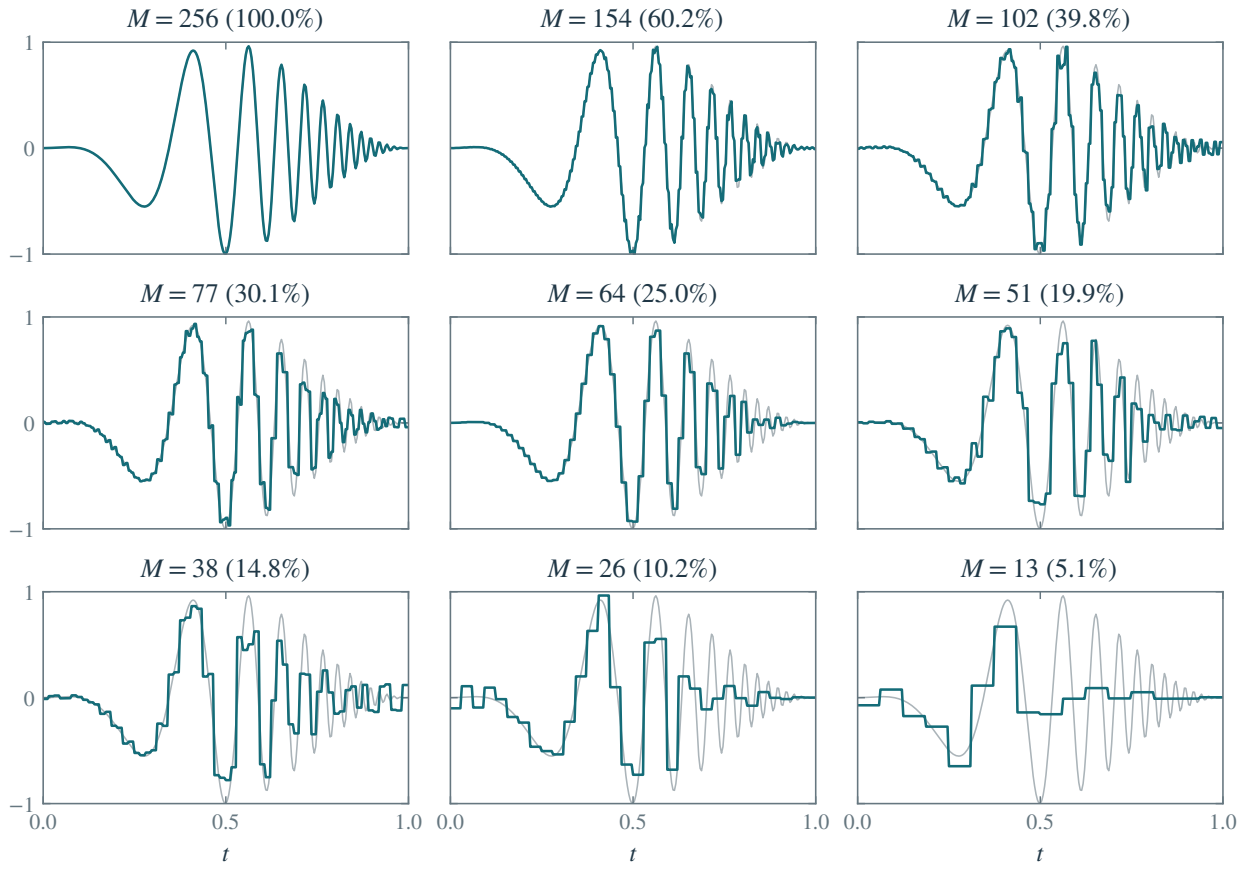


Figure 2.9. Progressive approximation of a signal by initial segments of the sequency-ordered Walsh basis. Each panel gives the exact retained coefficient count.

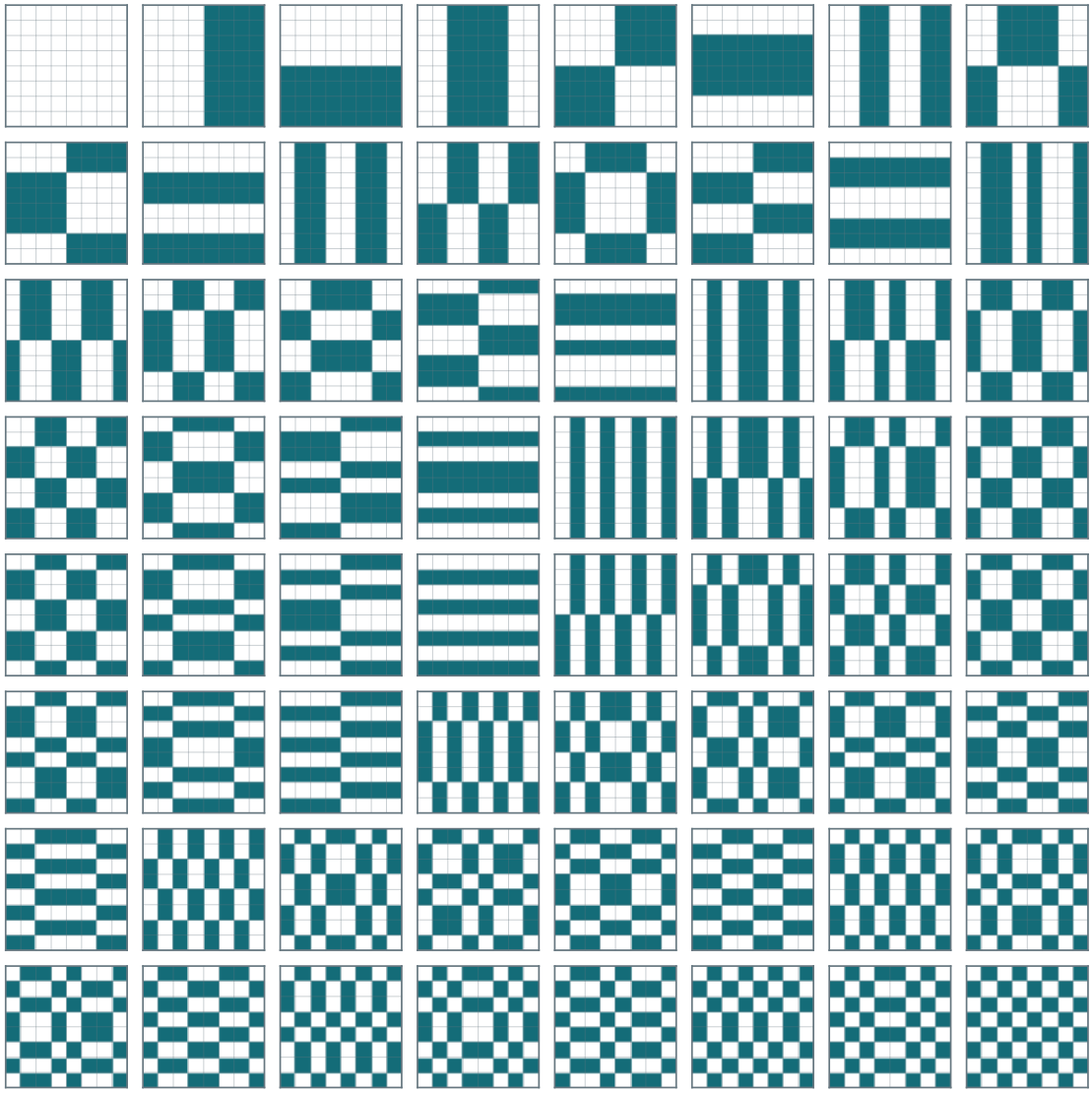


Figure 2.10. The 64 two-dimensional Walsh functions for 8×8 images, ordered by total sequency and then vertical sequency, from left to right and top to bottom.

3. For a prime $p \equiv 3 \pmod 4$, extend the quadratic character by $\eta(0) = 0$ and define

$$Q_{ij} = \eta(j - i), \qquad 0 \le i, j < p.$$

   Prove that $Q^{\mathsf{T}} = -Q$, $QQ^{\mathsf{T}} = pI_p - J$, and $QJ = JQ = 0$, where J has every entry equal to one.

4. With $\mathbf{1} \in \mathbb{R}^p$ a column vector of ones, prove that the Paley construction

$$H_{p+1} = \begin{pmatrix} 1 & \mathbf{1}^{\mathsf{T}} \\ \mathbf{1} & Q - I_p \end{pmatrix}$$

   is a Hadamard matrix. Compute it for $p = 7$ and check its Gram matrix. This supplies a different construction from the tensor powers of W_2.

5. If $A \in \mathbb{R}^{N \times N}$ and $|a_{ij}| \le 1$, prove

$$|\det A| \le N^{N/2}. \tag{2.37}$$

   Show that a Hadamard matrix attains equality. Also show that a maximizer of $|\det A|$ on the cube of all such matrices can be chosen with every entry in $\{1, -1\}$, even when the bound is not attained.

The determinant is the volume of the parallelepiped spanned by the columns. Its volume is at most the product of their lengths. At a Hadamard matrix, the columns are mutually orthogonal cube diagonals, each of



Figure 2.11. Walsh approximations of a synthetic letter A, with exact retained coefficient counts. Every panel uses the same grayscale range.

length $\sqrt{N}$. The condition on the order is necessary; the constructions above establish existence for particular families of orders. Exercise 6.11 applies these matrices to error-correcting codes.

Tensor products explain the recursive structure of Walsh matrices and suggest a wider family of fast transforms.

Exercise 2.7 (Tensor products of matrices)**.** For $A \in \mathbb{C}^{s\times s}$ and $B \in \mathbb{C}^{t\times t}$, their tensor product is the $st \times st$ block matrix

$$A \otimes B = (a_{ij}B)_{1\le i,j\le s}.$$

1. If $AA^* = sI_s$, prove that the tensor power with n factors satisfies $A^{\otimes n}(A^{\otimes n})^* = s^n I_{s^n}$.
2. Express W_{2^n} as a tensor power.
3. Generalize the FWT recursion to compute $y = A^{\otimes n}x$ without forming the full matrix. Give the inverse transform and its arithmetic cost when the base size s is fixed.

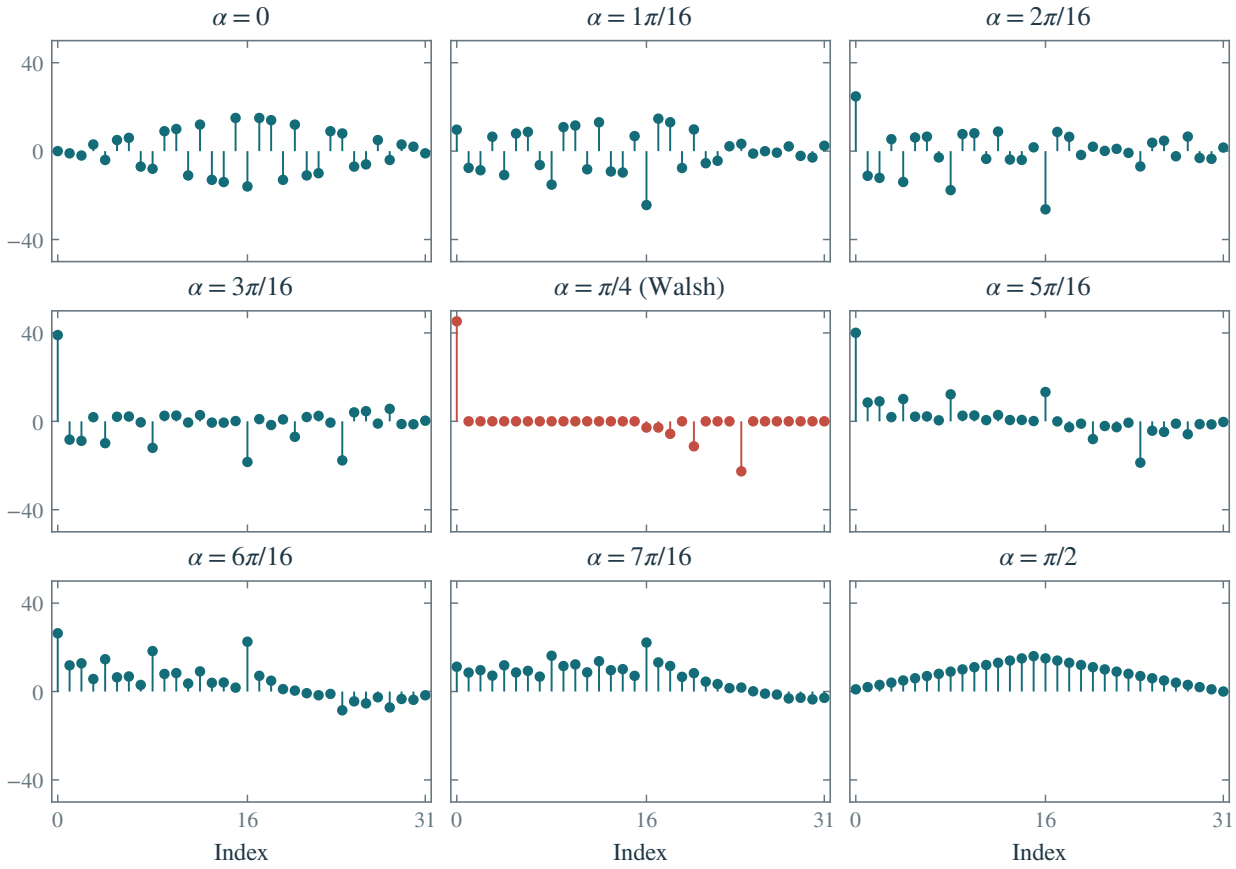


Figure 2.12. Normalized tensor transforms $2^{-n/2}A_\alpha^{\otimes n}$ of a triangular signal, with $n = 5$. The middle panel is the Walsh transform; the last reverses the input entries.

4. For
$$A_\alpha = \sqrt{2}\begin{pmatrix}\cos\alpha & \sin\alpha \\ \sin\alpha & -\cos\alpha\end{pmatrix},$$
show that $U_\alpha = 2^{-n/2}A_\alpha^{\otimes n}$ is orthogonal. Identify its actions at $\alpha = 0, \pi/4, \pi/2$. In particular, distinguish bitwise complementation from reversal of the *order* of the bits within an index.

Figure 2.12 shows the normalized transforms of a triangular signal. At $\alpha = \pi/4$ the transform is Walsh; at $\pi/2$ it reverses the order of the signal entries, sending index j to $2^n - 1 - j$. Exercise 8.7 constructs another possible base matrix using representation theory.

The binary MacWilliams identity extends to larger alphabets once field linearity and the trace pairing are handled carefully.

Exercise 2.8 (The MacWilliams identity over a finite field)**.** Let $q = p^r$ be a prime power, and set $E = \mathbb{F}_q^k$.

1. Show that the bilinear form $\langle a, x\rangle = \sum_{i=0}^{k-1} a_i x_i$ is nondegenerate and identifies E with its $\mathbb{F}_q$-linear dual.
2. Let ψ_1 be the canonical additive character of $\mathbb{F}_q$ (Definition 11). For $a \in E$, set $\chi_a(x) = \psi_1(\langle a, x\rangle)$. Use the nondegeneracy of the trace pairing to show that $a \mapsto \chi_a$ is an isomorphism from the additive group E to its character group.
3. Assume that $H \subset E$ is an $\mathbb{F}_q$-linear subspace. Prove that its additive annihilator corresponds to its linear orthogonal $H^\perp$. Explain precisely where closure under multiplication by all scalars in $\mathbb{F}_q$ is needed. Give an additive subgroup of $\mathbb{F}_4$ that is not an $\mathbb{F}_4$-linear subspace.
4. With $w(c)$ the number of nonzero coordinates and $A_H(X,Y) = \sum_{c\in H} X^{k-w(c)}Y^{w(c)}$, prove
$$A_{H^\perp}(X,Y) = \frac{1}{|H|}A_H\big(X + (q-1)Y, X - Y\big).$$

The language of distributions packages all lattice samples into a single object and gives a concise form of Poisson summation.

Exercise 2.9 (Poisson summation for distributions)**.** Use the angular-frequency convention $\hat{f}(\omega) = \int_{\mathbb{R}} f(x)e^{-\mathrm{i}\omega x}\,\mathrm{d}x$ and its extension to tempered distributions. For $s > 0$, the Dirac comb is

$$\Pi_s = \sum_{n\in\mathbb{Z}} \delta_{ns}, \qquad \langle \Pi_s, \varphi\rangle = \sum_{n\in\mathbb{Z}} \varphi(ns) \quad (\varphi \in \mathcal{S}(\mathbb{R})). \tag{2.38}$$

Explain why it is tempered. Use Poisson summation, with the appropriate rescaling, to prove $\hat{\Pi}_s = (2\pi/s)\Pi_{2\pi/s}$. Here the distribution pairing is bilinear and contains no complex conjugation.

For band-limited functions, frequency orthogonality turns this sampling structure into an exact reconstruction theorem.

Exercise 2.10 (Shannon sampling)**.** Use the same angular-frequency convention. For $T > 0$, set

$$I_T = [-\pi/T, \pi/T], \qquad E_T = \{f \in L^2(\mathbb{R}) : \hat{f} = 0 \text{ almost everywhere outside } I_T\}.$$

Define $\mathrm{sinc}_T(t) = \sin(\pi t/T)/(\pi t/T)$ for $t \neq 0$, with $\mathrm{sinc}_T(0) = 1$. We will establish

$$f(t) = \sum_{n\in\mathbb{Z}} f(nT)\,\mathrm{sinc}_T(t - nT). \tag{2.39}$$

1. Prove that each element of E_T has a unique smooth representative, which we use to define its samples. Establish $\|f\|_\infty \le T^{-1/2}\|f\|_2$.
2. Expand $\hat{f}|_{I_T}$ in the orthonormal exponential basis of $L^2(I_T)$. Express its Fourier-series coefficients in terms of $f(nT)$, and deduce $T\sum_n |f(nT)|^2 = \|f\|_2^2$. For $f_d = \sum_n f(nT)\delta_{nT}$, show in the sense of tempered distributions that
$$\hat{f}_d(\omega) = \frac{1}{T}\sum_{k\in\mathbb{Z}} \hat{f}(\omega - 2\pi k/T).$$
This implies $\hat{f}_d = \hat{f}/T$ almost everywhere in the interior of I_T.
3. Compute the inverse transform of $\mathbf{1}_{I_T}$ and deduce (2.39), with convergence in $L^2(\mathbb{R})$ and uniformly on $\mathbb{R}$.
4. Show that $\{T^{-1/2}\,\mathrm{sinc}_T(\cdot - nT)\}_{n\in\mathbb{Z}}$ is an orthonormal basis of E_T. Give the orthogonal projection of any $g \in L^2(\mathbb{R})$ onto E_T, both in frequency and in this basis.

2.5 Solutions to the exercises

The common tool in these solutions is a change of viewpoint between a group and its dual. We use it to count quadratic residues, organize Walsh and Haar computations, and pass from finite Poisson summation to distributional and sampling formulas.

Solution to Exercise 2.1

1. For $1 \le b < p$, the representative of $(-1)^b b$ is b if b is even and $p-b$ if b is odd; it is therefore even in both cases. If two representatives coming from $a, a' \in E$ coincide, then $ra \equiv \pm ra' \pmod p$, so $a \equiv \pm a' \pmod p$. The plus sign gives $a = a'$. For the minus sign, $a + a'$ would be p, which is impossible because the sum is even. The map is injective, and both sets have $(p-1)/2$ elements.
2. Taking products modulo p gives
$$\prod_{b\in B_r} b \equiv r^{(p-1)/2} \prod_{a\in E} a, \qquad \prod_{a\in E} a \equiv (-1)^{\sum_{b\in B_r} b} \prod_{b\in B_r} b.$$
All factors are nonzero modulo p, so they can be canceled. Euler's criterion gives the assertion, first as a congruence and then as an equality of signs, since p is odd.
3. Sum $ra = p\lfloor ra/p \rfloor + [ra]_p$ over E. The left-hand side is even, while p is odd. Thus the sum of the remainders and the sum of the quotients have the same parity.
4. A lattice point on the open segment would satisfy $py = rx$ with $0 < x < p$. Coprimality forces $p \mid x$, a contradiction. For each integer $a \in \{1, \dots, p-1\}$, the height ra/p is not an integer, so the positive integer ordinates below it are $1, \dots, t_a$.
5. Since ra/p is not an integer,
$$t_{p-a} = \left\lfloor r - \frac{ra}{p} \right\rfloor = r - 1 - t_a.$$
The two terms have the same parity because $r-1$ is even. Choose one representative $a \le (p-1)/2$ from each pair $\{a, p-a\}$. Exactly one member of the pair is even, so summing over the even members or over these smaller representatives gives the same parity. For such a, $ra/p < r/2$, hence $\sum_{a=1}^{(p-1)/2} t_a$ counts exactly the interior lattice points of AKH. This pairing is the needed counting argument; reflection across the oblique diagonal is not a lattice-preserving symmetry in general.
6. Interchanging the coordinate axes identifies the analogous count for $\left(\frac{p}{r}\right)$ with the interior lattice points of AHL. There is no lattice point on their common diagonal, nor on the half-integer outer edges $x = p/2$ and $y = r/2$. Together the two triangles contain $(p-1)(r-1)/4$ lattice points. Consequently,
$$\left(\frac{r}{p}\right)\left(\frac{p}{r}\right) = (-1)^{(p-1)(r-1)/4}.$$

Solution to Exercise 2.2

1. The cyclic group $\mathbb{F}_q^* = \langle g \rangle$ has a unique subgroup of each order dividing $q-1$. If $k \mid q-1$, its kth powers are $H_k = \langle g^k \rangle$, of order $(q-1)/k$.
2. Orthogonality in the quotient gives $\sum_{i=0}^{k-1} \chi_i(x) = k\mathbf{1}_{H_k}(x)$ for $x \neq 0$. Multiplying by $\psi(x)$ and summing proves the formula. If ψ is nontrivial, the trivial multiplicative character contributes a Gauss sum of modulus one, while the other $k-1$ sums have modulus $\sqrt{q}$. Thus
$$|\hat{f_{H_k}}(\psi)| \le \frac{1 + (k-1)\sqrt{q}}{k} < \sqrt{q}.$$
3. Each element of H_k has exactly k nonzero kth roots, so $N = kN'$. Count $x + y + v = 0$ with $v \in -H_k$. The Fourier coefficients of f_{-H_k} are those of f_{H_k} with the characters inverted; hence $\Phi(-H_k) = \Phi(H_k)$. Exercise 1.5 yields
$$\left| N' - \frac{|A_1||A_2||H_k|}{q} \right| \le \Phi(H_k)\sqrt{|A_1||A_2|} < \sqrt{|A_1||A_2|q}.$$
Multiply by k and use $k|H_k| = q-1$.

4. The main term in (2.36) exceeds its error bound provided $k^2 l_1 l_2 q^3 < (q-1)^4$. Under the proposed hypothesis,
$$k^2 l_1 l_2 q^3 \le (q-4)q^3 < (q-1)^4,$$
because the difference on the right is $6q^2 - 4q + 1 > 0$. Thus $N > 0$. For general k, in the cyclic group of order $q-1$, $\langle g^k \rangle = \langle g^d \rangle$ for $d = \gcd(k, q-1)$. Apply the result with $d \le k$. A nonzero dth-power right-hand side is also a kth power, which gives the required solution for the original exponent.
5. With $A_1 = A_2 = H_d$, we have $l_1 = l_2 = d \le k$. Thus $q \ge k^4 + 4$ implies the preceding criterion. All three entries are nonzero powers, so taking their kth roots gives a solution of (2.34) in $(\mathbb{F}_q^*)^3$.

Solution to Exercise 2.3

The character indexed by the bit mask of S is precisely $\prod_{s \in S} s$ under the encoding $+ = 0$, $- = 1$. In natural binary order,
$$W_8 f = (8\mu, 4\mu_a, 4\mu_b, 4\mu_{ab}, 4\mu_c, 4\mu_{ac}, 4\mu_{bc}, 4\mu_{abc}) = (586, -92, 20, 6, -6, 40, 0, -2).$$
An FWT followed by the indicated rescalings computes all quantities:

	μ	μ_a	μ_b	μ_{ab}	μ_c	μ_{ac}	μ_{bc}	μ_{abc}
cm	73.25	−23	5	1.5	−1.5	10	0	−0.5

For example, high lighting is associated with a mean height 23 cm below that at low lighting, after averaging over the other factors. The relatively large ac contrast indicates that the lighting contrast changes with the fertilizer level. These are descriptive effects, without an uncertainty estimate from this unreplicated table.

Solution to Exercise 2.4

1. Each wavelet has squared norm $2^j 2^{-j} = 1$ and integral zero. Wavelets at the same scale have disjoint supports. At different scales, the coarser wavelet is constant on the support of the finer one, so their inner product vanishes. The constant ψ_0 is orthogonal to them all. The first 2^J functions belong to F_J and form an orthonormal family with $\dim F_J = 2^J$ members; they are therefore a basis.
2. For a dyadic cell I of length 2^{-J}, orthogonal projection onto constants gives $f_J|_I = |I|^{-1} \int_I f$. Define the modulus of continuity $\omega_f(h) = \sup_{|x-y| \le h} |f(x) - f(y)|$. Then
$$|f_J(x) - f(x)| \le \frac{1}{|I|} \int_I |f(y) - f(x)| \, dy \le \omega_f(2^{-J}).$$
The same bound holds at the endpoint under our convention. Adding a wavelet coefficient at level J replaces the average on its support by the two averages on the half intervals. Distinct wavelets at that level have disjoint supports. Every intermediate partial sum is therefore an averaging operator on a partition whose largest cell has length 2^{-J}, and satisfies the same bound. As $\omega_f(2^{-J}) \to 0$, all partial sums converge uniformly. Continuous functions are dense in $L^2([0,1])$, so the span of the Haar family is dense there. Together with orthonormality, this proves completeness.
3. The normalized indicators $\varphi_{j,k}$ are an orthonormal basis of F_j. Directly on the two half intervals,
$$\varphi_{j,k} = \frac{\varphi_{j+1,2k} + \varphi_{j+1,2k+1}}{\sqrt{2}}, \qquad \psi_{j,k} = \frac{\varphi_{j+1,2k} - \varphi_{j+1,2k+1}}{\sqrt{2}}. \tag{2.40}$$
The 2^j wavelets at level j are orthogonal to F_j and belong to F_{j+1}. Dimension counting gives the asserted orthogonal decomposition.
4. If f has value v_k on cell k, then $x^{(0)}[k] = 2^{-J/2} v_k$. Each pair is transformed by
$$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$
This matrix is orthogonal and has determinant -1: it is a reflection, not a rotation. Applying it to disjoint pairs and rearranging the outputs is still orthogonal. Also,
$$\sum_k x^{(i)}[k] = 2^{-i/2} \sum_k x^{(0)}[k], \qquad x^{(J)}[0] = 2^{-J/2} \sum_k x^{(0)}[k].$$
Since the input has 2^J entries, the last expression is $2^{J/2} \operatorname{mean}(x^{(0)})$.

5. Take inner products with f in (2.40) and proceed by induction on i. The initial statement holds by definition of $x^{(0)}$. Each stage produces the coefficients of $\varphi_{J-i,k}$ and $\psi_{J-i,k}$. The final scaling function is $\varphi_{0,0} = \psi_0$. The total number of input entries processed is $N + N/2 + \cdots + 2 = 2N - 2$, so the cost is $O(N)$.
6. Orthogonality at each stage gives

$$\|x^{(0)}\|_2^2 = \sum_{i=1}^{J} \|d^{(i)}\|_2^2 + |x^{(J)}[0]|^2 = \|\Gamma x^{(0)}\|_2^2.$$

   Thus $\Gamma^{\mathsf{T}}\Gamma = I$. Its rows, or equivalently the vectors $\Gamma^{\mathsf{T}} e_m$, form the basis whose inner products are the output coefficients. They are the discrete Haar wavelets, followed by the normalized constant vector, in the specified packing. Fine wavelets are localized on short dyadic blocks; Walsh functions have full support. Haar analysis costs $O(N)$ operations, compared with $O(N \log N)$ for the FWT.

Program 4 implements precisely these signs and normalizations. The accompanying Python module also provides the inverse.

```
def haar(x):
    x = np.asarray(x, dtype=complex)
    n = x.size
    if x.ndim != 1 or n == 0 or n & (n-1):
        raise ValueError('Expected a nonempty vector of power-of-two length.')
    details = []
    while x.size > 1:
        even, odd = x[::2], x[1::2]
        details.append((even - odd) / np.sqrt(2))
        x = (even + odd) / np.sqrt(2)
    return np.concatenate(details + [x])
```

Program 4. Orthonormal Haar analysis, with details stored from fine to coarse.

Solution to Exercise 2.5

1. For an $N \times N$ array f, the transform is $W_N f W_N^{\mathsf{T}}$, with entries

$$\hat{f}[i,j] = \sum_{s,t=0}^{N-1} f[s,t]\chi_i(s)\chi_j(t).$$

   Since $W_N^2 = NI$, the inverse applies the same transform and divides by N^2. Transforming rows and then columns costs $O(N^2 \log N)$. Program 5 also handles rectangular arrays when each side length is a power of two; the inverse divisor is the total number of pixels.

```
def fwt2d(x):
    rows = np.apply_along_axis(fwt, 1, x)
    return np.apply_along_axis(fwt, 0, rows)
```

```
def walsh_approx2d(x, M):
    x = np.asarray(x)
    if x.ndim != 2 or not 0 <= M <= x.size:
        raise ValueError('Expected an image and a valid coefficient count.')
    sy, sx = walsh_sequency(x.shape[0]), walsh_sequency(x.shape[1])
    vertical, horizontal = np.meshgrid(sy, sx, indexing='ij')
    order = np.lexsort((vertical.ravel(), (vertical+horizontal).ravel()))
    spectrum = fwt2d(x).ravel()
    spectrum[order[M:]] = 0
    result = fwt2d(spectrum.reshape(x.shape)) / x.size
    return np.real_if_close(result) if np.isrealobj(x) else result
```

Program 5. A separable two-dimensional Walsh transform and a sequency approximation.

2. Write $n_i^{(k)}$ for the sequency of row i of W_{2^k}. The two rows above index i at the next level are concatenations (χ_i, χ_i) and $(\chi_i, -\chi_i)$. Every row starts with 1; let $\epsilon_i^{(k)}$ be zero or one according as its last entry is 1 or -1. Counting internal and middle sign changes,

$$n_i^{(k+1)} = 2n_i^{(k)} + \epsilon_i^{(k)}, \qquad n_{i+2^k}^{(k+1)} = 2n_i^{(k)} + 1 - \epsilon_i^{(k)}. \tag{2.41}$$

   The pair supplies exactly $2n_i^{(k)}$ and $2n_i^{(k)} + 1$. Starting from $n_0^{(0)} = 0$, induction proves that the sequencies are all distinct and exhaust $\{0, \dots, 2^k - 1\}$.
3. The endpoint parities satisfy $\epsilon_i^{(k+1)} = \epsilon_i^{(k)}$ and $\epsilon_{i+2^k}^{(k+1)} = 1 - \epsilon_i^{(k)}$. Program 6 computes the sequencies from these recurrences. Sorting them gives the permutation to apply to the spectrum. For $N = 8$, their natural-order values are $(0, 7, 3, 4, 1, 6, 2, 5)$. Small sequency means few changes along the ordered sample axis; small Hamming degree instead counts the factors in a Boolean character.

```
def walsh_sequency(n):
    if n < 1 or n & (n-1):
        raise ValueError('Expected a positive power of two.')
    counts, parity = np.array([0]), np.array([0])
    for _ in range(n.bit_length()-1):
        parity = np.r_[parity, 1-parity]
        counts = 2*np.tile(counts, 2) + parity
    return counts
```

Program 6. The number of sign changes in each naturally ordered Walsh row.

4. The approximation is the orthogonal projection onto the first M sequency-ordered functions. Its normalized squared error is

$$\|f - f_M\|_2^2 = \frac{1}{N^2} \sum_{m=M}^{N-1} |\mathcal{W}_k(f)[i_m]|^2.$$

   Only the retained coefficients need be stored because their positions are fixed by the ordering. Walsh reconstruction is piecewise constant and can introduce visible steps. Fourier reconstruction uses smooth trigonometric functions but can oscillate near discontinuities. The relative error depends on the signal; neither basis is uniformly better. Finite-bit compression also quantizes the retained coefficients.
5. If s_i and s_j are the vertical and horizontal sequencies, respectively, order coefficient (i, j) by the pair $(s_i + s_j, s_i)$. Retain the first M coefficients and invert the transform. The function `walsh_approx2d` in Program 5 implements this rule, resolving ties by vertical sequency.

Solution to Exercise 2.6

1. This is the identity $W_{2^k} W_{2^k}^{\mathsf{T}} = 2^k I$.
2. Sign changes and permutations preserve pairwise row orthogonality. Normalize the first row by column sign changes, then the first column by row sign changes. Group columns by the sign pairs in rows two and three. The counts obey

$$i + j + k + l = N, \quad i + j - k - l = 0, \quad i - j + k - l = 0, \quad i - j - k + l = 0.$$

   Solving gives $i = j = k = l = N/4$. For orders below three, the matrices (1) and W_2 supply the two exceptions.
3. Since $\eta(-1) = -1$, we have $Q_{ji} = -Q_{ij}$. For $c \neq 0$, substitute $z = (b + c)/b$ on $\mathbb{F}_p^*$ to obtain

$$\sum_{b \in \mathbb{F}_p} \eta(b)\eta(b + c) = \sum_{z \in \mathbb{F}_p \setminus \{1\}} \eta(z) = -1.$$

   For $c = 0$ the sum is $p - 1$, since the nonzero values of η have square one. Therefore the diagonal entries of QQ^{T} are $p - 1$ and its off-diagonal entries are -1, giving $pI_p - J$. Each row and column of Q sums to $\sum_b \eta(b) = 0$, so $QJ = JQ = 0$.

4. The off-diagonal blocks in HH^T vanish because $Q\mathbf{1} = Q^\mathsf{T}\mathbf{1} = 0$. The top-left entry is $p + 1$, and the bottom-right block is
$$J + (Q - I_p)(Q^\mathsf{T} - I_p) = J + (pI_p - J) - Q - Q^\mathsf{T} + I_p = (p + 1)I_p.$$
The two Python programs below construct H and check its Gram matrix using exact integer arithmetic; $p = 7$ gives an order-eight example.
5. By the determinant bound from Gram–Schmidt, the volume is at most the product of the column norms. Each norm is at most $\sqrt{N}$, proving (2.37). For a Hadamard matrix, $|\det H|^2 = \det(HH^\mathsf{T}) = N^N$. Finally, compactness gives a maximizer on the entrywise cube. With all other entries fixed, the determinant is affine in one entry, so its absolute value is convex and attains a maximum at an endpoint of that entry's interval. Move entries successively to ± 1 without decreasing $|\det A|$. The resulting vertex is also a maximizer.

```
def paley_hadamard(p):
    if p % 4 != 3 or not sp.isprime(p):
        raise ValueError('Expected a prime congruent to 3 modulo 4.')
    Q = np.array([[int(sp.legendre_symbol((j-i) % p, p))
                   for j in range(p)] for i in range(p)])
    H = np.ones((p+1, p+1), dtype=int)
    H[1:, 1:] = Q - np.eye(p, dtype=int)
    return H
```

Program 7. The Paley construction for a prime $p \equiv 3 \pmod 4$.

```
def paley_example():
    H = paley_hadamard(31)
    return H @ H.T
```

Program 8. Verifying the Paley construction at order 32.

Solution to Exercise 2.7

1. Block multiplication gives $(A \otimes B)(C \otimes D) = AC \otimes BD$ whenever the dimensions match, and $(A \otimes B)^* = A^* \otimes B^*$. Thus $A^{\otimes n}(A^{\otimes n})^* = (AA^*)^{\otimes n} = s^n I_{s^n}$.
2. With $A = \left(\begin{smallmatrix} 1 & 1 \\ 1 & -1 \end{smallmatrix}\right)$, the Walsh recursion gives $W_{2^n} = A^{\otimes n}$.
3. Split the input into s consecutive blocks. Recursively apply $A^{\otimes(n-1)}$ to each, then mix the resulting blocks by A. Program 9 implements this recursion. For $N = s^n$, its cost is $O(sN \log_s N)$ with dense base multiplication, or $O(N \log_s N)$ for fixed s. The inverse is
$$x = s^{-n}(A^*)^{\otimes n} y.$$
Use the same algorithm with A^* and divide the result by s^n.

```
def tensor_transform(x, A):
    x, A = np.asarray(x, dtype=complex), np.asarray(A, dtype=complex)
    s = A.shape[0]
    if A.shape != (s, s) or s < 2 or x.ndim != 1 or x.size == 0:
        raise ValueError('Expected a square base matrix and a nonempty vector.')
    if x.size == 1:
        return x.copy()
    if x.size % s:
        raise ValueError('Vector length must be a power of the base size.')
    blocks = np.stack([tensor_transform(part, A) for part in np.split(x, s)])
    return (A @ blocks).ravel()
```

Program 9. Applying a tensor power without forming its matrix.

4. The normalized base matrix $A_\alpha/\sqrt{2}$ is orthogonal, hence so is its tensor power. At $\alpha = \pi/4$, $U_\alpha = 2^{-n/2}W_{2^n}$. At $\alpha = 0$ it is diagonal, multiplying entry j by $(-1)^{w(j)}$, where $w(j)$ is its binary Hamming weight. At $\alpha = \pi/2$ the base interchanges 0 and 1 in each coordinate. Its tensor power complements every bit: $j \mapsto 2^n - 1 - j$, which reverses the sample vector. It does not reverse the order of bits within j.

Solution to Exercise 2.8

1. If $a \neq 0$, choose a coordinate i with $a_i \neq 0$ and set $x = a_i^{-1}e_i$. Then $\langle a, x\rangle = 1$. The linear map $a \mapsto \langle a, \cdot\rangle$ is injective between spaces of dimension k, so it is an isomorphism onto E^*.
2. The character map is a group homomorphism. If $a \neq 0$, the same coordinate choice lets $\langle a, x\rangle$ take every value of $\mathbb{F}_q$. The trace is surjective onto $\mathbb{F}_p$, so some x satisfies $\mathrm{Tr}(\langle a, x\rangle) = 1$ and $\chi_a(x) = e^{2\pi i/p} \neq 1$. Thus the map is injective; both groups have q^k elements, so it is bijective. The character ψ_1 itself need not be injective when q is not prime.
3. If $a \in H^\perp$, then clearly $\chi_a(h) = 1$ on H. Conversely, if $\chi_a(h) = 1$ for every $h \in H$, scalar closure gives
$$\mathrm{Tr}(\lambda\langle a, h\rangle) = 0 \quad \text{for every } \lambda \in \mathbb{F}_q \text{ and } h \in H.$$
The trace pairing is nondegenerate, so $\langle a, h\rangle = 0$ for every h. Without scalar closure this inference fails. For example, the subfield $\{0, 1\} \subset \mathbb{F}_4$ is an additive subgroup but not an $\mathbb{F}_4$-linear subspace: multiplication of 1 by an element outside $\mathbb{F}_2$ leaves it.
4. Apply finite Poisson summation to $f(t) = x^{k-w(t)}y^{w(t)}$. The transform factors over coordinates. At a coordinate with $a_i = 0$, the factor is $x + (q-1)y$. At a coordinate with $a_i \neq 0$, character orthogonality gives $\sum_{t_i \neq 0} \psi_1(a_i t_i) = -1$, so the factor is $x - y$. Therefore
$$\hat{f}(\chi_a) = \big(x + (q-1)y\big)^{k-w(a)}(x-y)^{w(a)}.$$
Since $|H^\perp| = q^k/|H|$, Poisson summation over $H^\perp$ gives the claimed identity for every $x, y \in \mathbb{C}$, and hence as a polynomial identity.

Solution to Exercise 2.9

For a Schwartz function φ, the series $\sum_n |\varphi(ns)|$ is bounded by a constant times the Schwartz seminorm $\sup_x (1 + |x|)^2|\varphi(x)|$. Thus Π_s is tempered. Under the stated Fourier convention, rescaled Poisson summation gives
$$\sum_{n\in\mathbb{Z}} \hat{\varphi}(ns) = \frac{2\pi}{s}\sum_{k\in\mathbb{Z}} \varphi(2\pi k/s).$$
Both sums converge absolutely. By the definition of the distributional transform, the left side is $\langle \Pi_s, \hat{\varphi}\rangle = \langle \hat{\Pi}_s, \varphi\rangle$; the right side is $\langle (2\pi/s)\Pi_{2\pi/s}, \varphi\rangle$. Equality for all Schwartz test functions proves the formula.

Solution to Exercise 2.10

1. Compact support and Cauchy–Schwarz imply $\hat{f} \in L^1$. Fourier inversion therefore gives the continuous representative
$$f(t) = \frac{1}{2\pi}\int_{I_T} \hat{f}(\omega)e^{i\omega t}\,\mathrm{d}\omega.$$
Multiplication by any power of ω preserves integrability on I_T, so differentiation under the integral proves smoothness. Two continuous representatives equal almost everywhere must agree everywhere. Cauchy–Schwarz and Plancherel also give
$$|f(t)| \le \frac{1}{2\pi}\sqrt{\frac{2\pi}{T}}\|\hat{f}\|_2 = T^{-1/2}\|f\|_2.$$
2. The functions $e_n(\omega) = \sqrt{T/(2\pi)}e^{-inT\omega}$ form an orthonormal basis of $L^2(I_T)$. Their coefficients are $\langle \hat{f}, e_n\rangle = \sqrt{2\pi T}\, f(nT)$, so
$$\hat{f}(\omega) = T\sum_{n\in\mathbb{Z}} f(nT)e^{-inT\omega} \quad \text{in } L^2(I_T), \qquad \|f\|_2^2 = T\sum_{n\in\mathbb{Z}} |f(nT)|^2.$$

Extend this series periodically with period $2\pi/T$. It defines the periodization of $\hat{f}$, since $\hat{f}$ vanishes outside one period up to null sets. On the other hand, the transform of $f_d = \sum_n f(nT)\delta_{nT}$ is the series $\sum_n f(nT)e^{-inT\omega}$ in tempered distributions. The ℓ^2 bound on the samples justifies this distributional series. Comparing gives the stated periodization identity; in the interior of the base interval only its unshifted term is nonzero almost everywhere. This argument also supplies the convergence needed to use the sampling identity.

3. Direct integration gives $\mathcal{F}^{-1}(\mathbf{1}_{I_T}) = T^{-1}\operatorname{sinc}_T$. Invert the $L^2(I_T)$ Fourier series from the previous question, extending each term by zero outside I_T. This gives (2.39) in $L^2(\mathbb{R})$. Every finite sum and the remainder belong to E_T; the bound from the first question makes L^2 convergence imply uniform convergence.
4. The transform of $g_n = \operatorname{sinc}_T(\cdot - nT)$ is $T\mathbf{1}_{I_T}(\omega)e^{-inT\omega}$. Plancherel gives $\langle g_n, g_m\rangle = T\delta_{nm}$, and completeness follows from the exponential basis. Thus $g_n/\sqrt{T}$ is orthonormal and complete. For arbitrary $g \in L^2(\mathbb{R})$, the orthogonal projection is

$$P_{E_T} g = \mathcal{F}^{-1}(\mathbf{1}_{I_T}\hat{g}) = \frac{1}{T}\sum_{n\in\mathbb{Z}} \langle g, g_n\rangle\, g_n.$$

When $f \in E_T$, inversion also gives $\langle f, g_n\rangle = Tf(nT)$. The factor T is essential unless the sample spacing is one.

CHAPTER 3

The discrete Fourier transform

The discrete Fourier transform turns cyclic structure into computations on frequencies. Its fast algorithms make spectral methods practical for signals, images, and convolution. We derive the main FFT factorizations, explain their indexing and boundary conventions, and use the transform's symmetries to construct eigenvectors and intermediate transforms.

The DFT is the Fourier transform of a cyclic group, but its numerical use raises additional questions: how should computations be factored, how are samples stored, and what boundary conditions does a convolution impose? The FFT is an algorithm for evaluating the DFT, not a different transform. For historical context see Rockmore [50]; for algorithmic background see Brigham [11], Arndt [2], and *Numerical Recipes* [48]. The Python implementations in Section A.3 accompany the derivations below.

3.1 The language of signal processing

Using the group Fourier transform on sampled data requires a precise convention for signs, scaling, and frequency indices. We establish that convention here and record the inversion and energy formulas that numerical implementations must respect.

A sampled signal is a vector $f = (f[0], \dots, f[N-1]) \in \mathbb{C}^N$. It may come from a continuous signal $\tilde{f}$ by $f[n] = \tilde{f}(t_n)$. For equally spaced samples on $[a, b)$, $t_n = a + (b-a)n/N$, so the right endpoint is not sampled a second time. The algebraic DFT is defined for every vector; interpreting its indices as physical frequencies requires the sample spacing.

Definition 18 (Discrete Fourier transform)**.** Let $\omega_N = e^{2\pi i/N}$. The discrete Fourier transform (DFT) is

$$\hat{f}[k] = \mathcal{F}(f)[k] = \sum_{n=0}^{N-1} f[n]\omega_N^{-nk}, \qquad 0 \le k < N. \tag{3.1}$$

Unless otherwise stated, $\mathcal{F}$ in this chapter denotes this unnormalized negative-exponential transform on $\mathbb{C}^N$.

This is the convention used by NumPy's `fft`; its inverse includes $1/N$. Replacing ω_N by another primitive root ω_N^r, with $\gcd(r, N) = 1$, permutes the frequency labels by $k \mapsto rk \pmod N$. It can also be interpreted as a change of generator of the cyclic group, provided the signal coordinates are changed consistently.

The connection with Chapter 1 is exact once the cyclic characters are indexed with this sign convention.

Remark 17 (Relation to the group Fourier transform). For this chapter, index the characters by the negative exponential:

$$\chi_k(n) = \omega_N^{-kn}, \qquad n \in \mathbb{Z}/N\mathbb{Z}. \tag{3.2}$$

These are the inverses of the positively indexed cyclic characters in Chapter 1. If $f_1 : \mathbb{Z}/N\mathbb{Z} \to \mathbb{C}$ is the function represented by f, then $\hat{f}[k] = \hat{f}_1(\chi_k)$ under the group-transform convention (1.10). The transform itself is unchanged; we have specified its frequency indexing.

Character orthogonality gives reconstruction with the opposite exponential sign and a factor of $1/N$.

Proposition 31 (Inversion)**.** *For every $f \in \mathbb{C}^N$,*

$$f[n] = \frac{1}{N}\sum_{k=0}^{N-1} \hat{f}[k]\omega_N^{nk}, \qquad 0 \le n < N. \tag{3.3}$$

Proof. Substitute (3.1) and use $\sum_{k=0}^{N-1} \omega_N^{k(n-m)} = N\delta_{nm}$ for $0 \le n, m < N$. □

In particular, no information is lost when a vector is replaced by its DFT.

Corollary 4. *The DFT is a linear isomorphism of $\mathbb{C}^N$.*

The same orthogonality calculation tracks inner products, and hence energy, under the transform.

Proposition 32 (Plancherel identity)**.** *For $f, g \in \mathbb{C}^N$,*

$$\sum_{n=0}^{N-1} f[n]\overline{g[n]} = \frac{1}{N}\sum_{k=0}^{N-1} \hat{f}[k]\overline{\hat{g}[k]}.$$

Thus $N^{-1/2}\mathcal{F}$ is unitary, for either the usual or the normalized discrete inner product. The factor in the unnormalized transform must be retained when using it to transfer norms or orthonormal bases.

3.2 Fast Fourier transforms

An FFT evaluates the same finite sums as the DFT, rearranging their operations to share intermediate results. We begin with power-of-two lengths and then extend the factorization to general products. The derivations require only complex arithmetic; the Good–Thomas exercise exposes the group-theoretic structure behind one important variant.

3.2.1 The radix-two algorithm

For a power-of-two signal length, splitting even and odd sample indices exposes two transforms of half the size. We derive the recursion and show how the two results produce every output frequency.

Direct evaluation of

$$\hat{f}[k] = \sum_{n=0}^{N-1} f[n]e^{-2\pi i k n/N} \tag{3.4}$$

uses N^2 complex multiplications and $N(N-1)$ additions if every term is counted, including trivial multiplications. This is $O(N^2)$ arithmetic. For $N = 2^p$, the radix-two Cooley–Tukey algorithm reduces it to $O(N \log N)$ by repeatedly splitting the input into even and odd indices.

An inverse transform needs no fundamentally different algorithm:

$$\mathcal{F}^{-1}f = \frac{1}{N}\overline{\mathcal{F}(\overline{f})} = \mathcal{F}(f_1), \qquad f_1[n] = \frac{1}{N} f[(-n) \bmod N].$$

Equivalently, reverse the sign of every exponential and divide once by N. In particular, index zero stays fixed in the cyclic reversal.

For even N, split the sum according to input parity:

$$\hat{f}[k] = \sum_{n=0}^{N/2-1} f[2n]\omega_{N/2}^{-nk} + \omega_N^{-k}\sum_{n=0}^{N/2-1} f[2n+1]\omega_{N/2}^{-nk}. \tag{3.5}$$

Define the even and odd subvectors

$$f^0[n] = f[2n], \qquad f^1[n] = f[2n+1], \qquad 0 \le n < N/2. \tag{3.6}$$

Their DFTs have length $N/2$ and are periodic in k with that period. Using $\omega_N^{-(k+N/2)} = -\omega_N^{-k}$ gives

$$\hat{f}[k] = \hat{f}^0[k] + \omega_N^{-k}\hat{f}^1[k], \tag{3.7}$$

$$\hat{f}[k+N/2] = \hat{f}^0[k] - \omega_N^{-k}\hat{f}^1[k], \qquad 0 \le k < N/2. \tag{3.8}$$

The same two shorter transforms therefore produce every output coefficient.

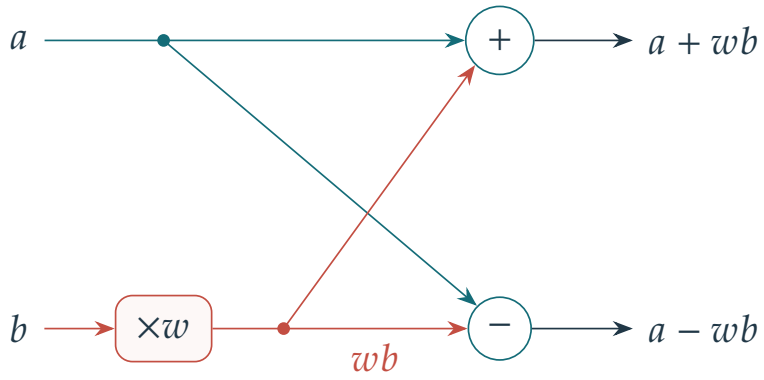


Figure 3.1. An FFT butterfly: the same weighted value wb enters the sum and the difference, with $w = \omega_N^{-k}$.

To write the recursive step compactly, we introduce notation for output halves and phase multiplication.

Definition 19 (Halves and phase multiplication)**.** For a vector h of even length N, write h_g and h_d for its first and second halves. In particular,

$$\hat{f}_g[k] = \hat{f}[k], \qquad \hat{f}_d[k] = \hat{f}[k + N/2], \qquad 0 \le k < N/2. \tag{3.9}$$

For $a \in \mathbb{C}^M$ and $x \in \mathbb{R}$, define the phase operator

$$(\mathcal{S}_M^x a)[j] = e^{-2\pi i x j/M} a[j], \qquad 0 \le j < M. \tag{3.10}$$

The two recurrence equations become

$$\hat{f}_g = \hat{f}^0 + \mathcal{S}_{N/2}^{1/2} \hat{f}^1, \tag{3.11}$$

$$\hat{f}_d = \hat{f}^0 - \mathcal{S}_{N/2}^{1/2} \hat{f}^1. \tag{3.12}$$

They are often called the Danielson–Lanczos relations. This is *decimation in time* (DIT), because the input sample indices are split by parity. Section 3.2.6 instead splits output frequencies.

The elementary operation $(a, b) \mapsto (a + wb, a - wb)$ is a *butterfly*, with $w = \omega_N^{-k}$ the *twiddle factor*. The crossing lines in Figure 3.1 indicate which input contributes to each output. Figure 3.2 shows the four butterflies that combine two length-four transforms. A recursive implementation follows the equations directly. The function `fft_recursive` in Section A.3 uses NumPy arrays and an optional inverse flag; `phase_multiply` implements $\mathcal{S}_M^x$. The recursion is useful for understanding the factorization. Iterative versions expose its memory layout more explicitly.

3.2.2 Arithmetic cost

The recursive factorization is useful because each level shares work across all frequencies. Counting that work makes the $O(N \log N)$ gain precise and separates arithmetic complexity from actual running time.

Proposition 33 (FFT complexity)**.** *If a combine stage costs KN operations and a length-one transform costs zero, the recurrence $C(N) = 2C(N/2) + KN$ gives $C(N) = KN \log_2 N$ for $N = 2^p$. More generally, an $O(N)$ combine stage gives $C(N) = O(N \log N)$.*

Proof. Set $D(p) = C(2^p)/2^p$. Then $D(p) = D(p-1) + K$ and $D(0) = 0$. Hence $D(p) = Kp$. Equivalently, each of the p recursion levels costs KN. □

This arithmetic estimate explains the gain over direct evaluation. Actual running time also depends on data movement, implementation, and hardware; a count of complex operations alone does not determine it.

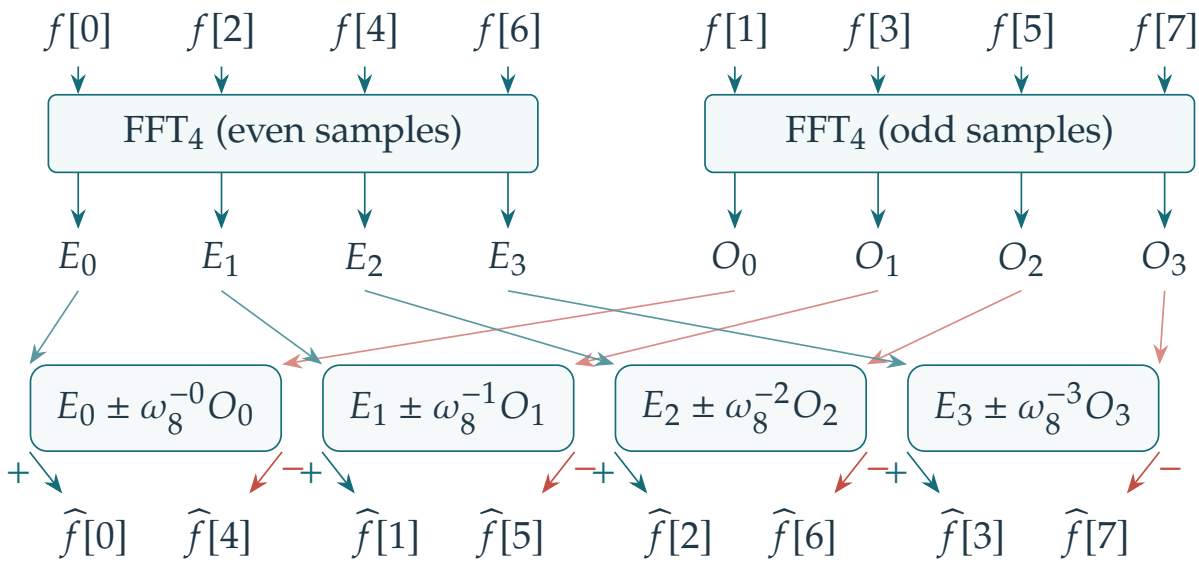


Figure 3.2. The combination stage of an eight-point DIT FFT. Two four-point transforms produce E_k and O_k; each butterfly combines E_k with $\omega_8^{-k} O_k$.

3.2.3 Zero padding and other radices

An inconvenient signal length raises two distinct possibilities: change the data by padding, or factor the transform at its original length. We explain what padding changes before extending the radix-two construction to other radices.

Remark 18 (What zero padding changes). Appending zeros to a length-N vector and taking a length-M DFT evaluates the same trigonometric sum $\sum_{n=0}^{N-1} f[n]e^{-in\theta}$ at $\theta = 2\pi k/M$. It does *not* compute the original length-N DFT unless the required frequencies coincide. Padding is useful for denser frequency sampling and for linear convolution. When the length-N DFT itself is required, use an algorithm for that length.

Keeping the original transform length instead leads to factorizations based on its available integer factors.

Remark 19 (Radix four and a general radix). A radix-r split groups input indices by their residue modulo r. For $N = rM$, set $f^b[n] = f[rn + b]$, for $0 \le b < r$ and $0 \le n < M$. If $\sigma = 1$ denotes the forward transform and $\sigma = -1$ the unnormalized inverse, let $\hat{f^b}_\sigma$ use that same sign and set $z_b[k] = \omega_N^{-\sigma bk} \hat{f^b}_\sigma[k]$. Then

$$\hat{f}_\sigma[k + qM] = \sum_{b=0}^{r-1} \omega_r^{-\sigma bq} z_b[k], \qquad 0 \le k < M, \quad 0 \le q < r.$$

Both the shorter transforms and the twiddle factors must change sign for the inverse. Divide its final output by N. For $r = 4$, the four outputs at a fixed k are

$$\begin{aligned} &z_0 + z_1 + z_2 + z_3, \quad z_0 - \mathrm{i}\sigma z_1 - z_2 + \mathrm{i}\sigma z_3, \\ &z_0 - z_1 + z_2 - z_3, \quad z_0 + \mathrm{i}\sigma z_1 - z_2 - \mathrm{i}\sigma z_3. \end{aligned}$$

Multiplication by ± 1 and $\pm \mathrm{i}$ uses only sign changes and swaps of real and imaginary parts. The split-radix method in Exercise 3.3 combines radix-two and radix-four splits.

3.2.4 The Cooley–Tukey factorization

A composite transform length can be decomposed even when its factors are not coprime. We derive the general Cooley–Tukey step, keeping track of the index maps and the phase factors that connect the shorter transforms.

Suppose $N = pq$, without requiring p and q to be prime or coprime. Starting again from

$$\hat{f}[k] = \sum_{n=0}^{N-1} f[n]\omega_N^{-nk}, \tag{3.13}$$

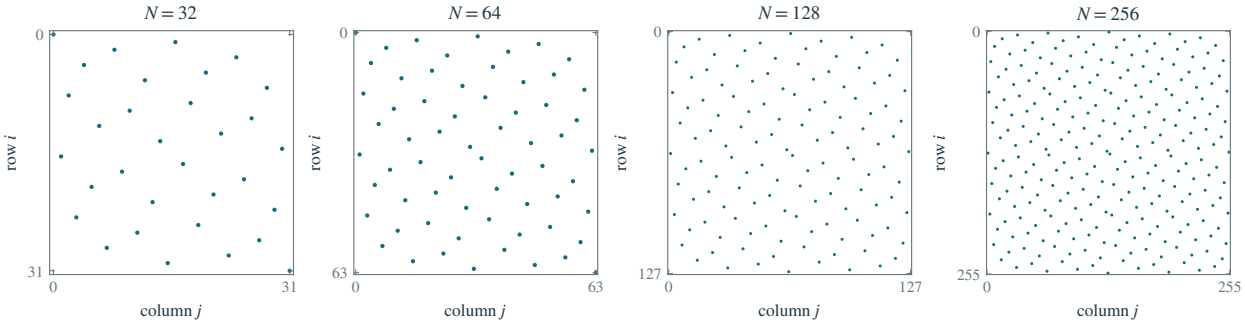


Figure 3.3. Bit-reversal permutation matrices for $N = 32, 64, 128, 256$. A dot in row i and column j means that the bits of j are those of i in reverse order.

write $n = ap + b$ and $k = cq + d$, with $0 \le a, d < q$ and $0 \le b, c < p$. Both index maps are bijections by Euclidean division. Expanding the product,

$$(ap + b)(cq + d) = acN + adp + bcq + bd.$$

The first term contributes a factor one. Define the length-q vectors $f_b[a] = f[ap + b]$. Then

$$\hat{f}[cq + d] = \sum_{b=0}^{p-1} \omega_p^{-bc} \left(\omega_N^{-bd} \hat{f}_b[d]\right). \tag{3.14}$$

This gives a concrete three-step algorithm: compute p length-q DFTs, multiply their entries by the twiddle factors ω_N^{-bd}, and compute q length-p DFTs. The input array $(f[ap+b])_{a,b}$ has q rows and p columns; specifying this orientation avoids ambiguous reshaping. With direct shorter transforms, the cost is $O(pq(p+q))$. Recursing through $N = p_1 \cdots p_s$ gives $O(N \sum_j p_j)$ operations. Bounded factors therefore give $O(N \log N)$, while a large prime factor calls for another method. Good–Thomas removes twiddle factors for coprime factors (Exercise 3.2); chirp methods offer another route for lengths with inconvenient factorizations.

3.2.5 Bit reversal and iterative implementation

To turn the recursive FFT into an iterative algorithm, we need to understand the order in which it visits the data. Binary digits reveal that order and explain the bit-reversal permutation used in the implementation.

The recursive DIT algorithm splits even and odd indices at every level. For $N = 2^p$, following these choices down the recursion tree reads the bits of an input index from least to most significant. The resulting leaf order is therefore the bit-reversal permutation

$$n_p(i) = \sum_{t=0}^{p-1} i_t 2^{p-1-t}, \qquad i = \sum_{t=0}^{p-1} i_t 2^t.$$

For example, $n_3(6) = n_3((110)_2) = (011)_2 = 3$. It is an involution. If $B_p f$ denotes the permuted vector $(B_p f)[i] = f[n_p(i)]$, then

$$B_p f = \left(B_{p-1} f^0,\ B_{p-1} f^1\right).$$

After this initial permutation, successive stages combine contiguous blocks of lengths $2, 4, \ldots, N$. The butterfly recurrence is still

$$\hat{f}[k] = \hat{f}^0[k] + \omega_N^{-k} \hat{f}^1[k] \quad (0 \le k < N/2), \tag{3.15}$$

with the corresponding minus sign for the second half. We are changing storage order, not taking the DFT of the permuted signal as a new input.

Program 69 generates the permutation in $O(N)$ arithmetic and index operations by the recursion of Exercise 3.1. An iterative DIT transform can then overwrite its working array stage by stage. The Python implementation copies the input and uses temporary arrays for each butterfly block; it does not claim the strict constant-workspace behavior of a low-level scalar implementation.

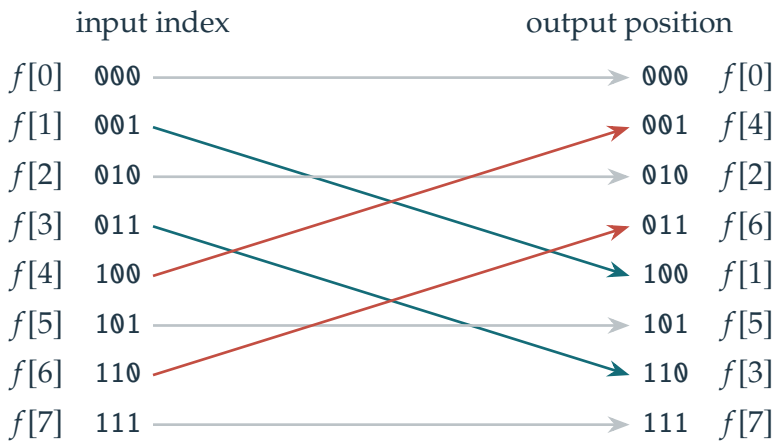


Figure 3.4. Bit reversal on eight entries. The binary labels make the swaps $1 \leftrightarrow 4$ and $3 \leftrightarrow 6$ explicit; the other four indices are fixed.

3.2.6 Decimation in frequency

The factorization can also begin by separating output frequencies. This alternative gives the decimation-in-frequency (DIF) algorithm and changes where phase multiplication and reordering occur.

Write $f_g[n] = f[n]$ and $f_d[n] = f[n + N/2]$ for $0 \le n < N/2$. Then

$$\hat{f}[k] = \sum_{n=0}^{N/2-1} \big(f_g[n] + (-1)^k f_d[n]\big)\omega_N^{-nk}.$$

For $0 \le k < N/2$, the even and odd outputs are

$$\hat{f}[2k] = \sum_{n=0}^{N/2-1} (f_g[n] + f_d[n])\omega_{N/2}^{-nk},$$
$$\hat{f}[2k+1] = \sum_{n=0}^{N/2-1} \omega_N^{-n}(f_g[n] - f_d[n])\omega_{N/2}^{-nk}.$$

In the notation of (3.10),

$$(\hat{f})^0 = \mathcal{F}(f_g + f_d), \qquad (\hat{f})^1 = \mathcal{F}\big(\mathcal{S}_{N/2}^{1/2}(f_g - f_d)\big).$$

The phase in the odd branch depends on the input summation index n. Also, $(\hat{f})^0$ denotes the even entries of the output, whereas $\hat{f^0}$ denotes the transform of the even input entries; these differ.

An iterative DIF implementation combines blocks of lengths $N, N/2, \dots, 2$. Starting with naturally ordered data, it leaves its outputs in bit-reversed order. Applying B_p at the end restores natural frequency order. Program 67 follows this convention. DIT and DIF express dual factorizations: DIT permutes the input first, while DIF permutes the output last. Their practical performance depends on the implementation and the data layout, not solely on this distinction. For broader surveys see Burrus [12].

3.2.7 Matrix form

Matrix notation exposes the FFT as a factorization of a structured linear map. It also connects the DFT to polynomial evaluation, which will be useful for fast multiplication.

The matrix of the DFT is

$$\Omega_N = (\omega_N^{-kn})_{0 \le k,n < N}. \tag{3.16}$$

It is a symmetric Vandermonde matrix. Applied to polynomial coefficients, it evaluates the unique polynomial of degree *at most* $N-1$ at the N distinct nodes $1, \omega_N^{-1}, \dots, \omega_N^{-(N-1)}$. Section 4.5 uses this connection for polynomial multiplication and interpolation.

Writing $A^* = \overline{A}^{\mathsf{T}}$ for the adjoint, $\Omega_N^{-1} = N^{-1}\Omega_N^*$ and $\Omega_N\Omega_N^* = NI_N$. Thus $U_N = \Omega_N/\sqrt{N}$ is unitary. The matrix is symmetric but in general is *not* Hermitian. If $P_{\mathrm{eo}}f = (f^0, f^1)$ and $\Delta = \mathrm{diag}(1, \omega_N^{-1}, \dots, \omega_N^{-(N/2-1)})$, then the radix-two factorization is

$$\Omega_N = \begin{pmatrix} I & \Delta \\ I & -\Delta \end{pmatrix} \begin{pmatrix} \Omega_{N/2} & 0 \\ 0 & \Omega_{N/2} \end{pmatrix} P_{\mathrm{eo}}.$$

The permutation is part of the factorization and cannot be omitted.

3.3 Circular and linear convolution

The convolution theorem of Chapter 1 becomes an efficient algorithm on sampled signals. Its interpretation depends on whether indices wrap around or signals are extended by zero.

3.3.1 Circular convolution

On a finite cyclic grid, translating a signal wraps indices around the boundary. Circular convolution is the product compatible with this geometry and with the length-N DFT.

Definition 20 (Circular convolution)**.** For $f, g \in \mathbb{C}^N$, define

$$(f * g)[n] = \sum_{k=0}^{N-1} f[k]g[(n-k) \bmod N], \qquad 0 \le n < N. \tag{3.17}$$

This is convolution on $\mathbb{Z}/N\mathbb{Z}$. It is bilinear, associative, and commutative, with unit δ_0. These properties follow from the group algebra, or directly by changing summation indices. Without modulo notation,

$$(f * g)[n] = \sum_{k=0}^{n} f[k]g[n-k] + \sum_{k=n+1}^{N-1} f[k]g[n-k+N].$$

Both sums contain a difference of indices, not a product.

The group convolution theorem now takes a particularly simple form in array notation.

Proposition 34 (DFT convolution theorem)**.** *For two length-N vectors,*

$$f \hat{*} g[k] = \hat{f}[k]\hat{g}[k], \qquad 0 \le k < N. \tag{3.18}$$

Proof. As functions f_1, g_1 on the cyclic group,

$$\hat{f}[k] = \hat{f}_1(\chi_k), \qquad \hat{g}[k] = \hat{g}_1(\chi_k). \tag{3.19}$$

The group convolution theorem gives the result. Alternatively, set $m = n - j$ modulo N in the double sum for $f \hat{*} g[k]$; the factor $\omega_N^{-k(m+j)}$ separates into a product, leaving the two Fourier sums. □

Circular convolution treats a finite vector as one period of a signal. This is a boundary model; it need not agree with extending the signal by zero.

3.3.2 Computing convolution with an FFT

The convolution theorem gives a practical algorithm by moving the expensive operation into frequency coordinates. Its cost depends on whether the filter transform can be reused and how long the filter is.

The theorem and inversion give

$$f * g = \mathcal{F}^{-1}(\hat{f} \cdot \hat{g}).$$

Two forward FFTs, N pointwise multiplications, and an inverse FFT cost $O(N \log N)$, compared with $O(N^2)$ for direct convolution. If one filter is fixed, its transform can be reused. For a very short filter, direct convolution may still be preferable because it avoids the overhead of the transforms.

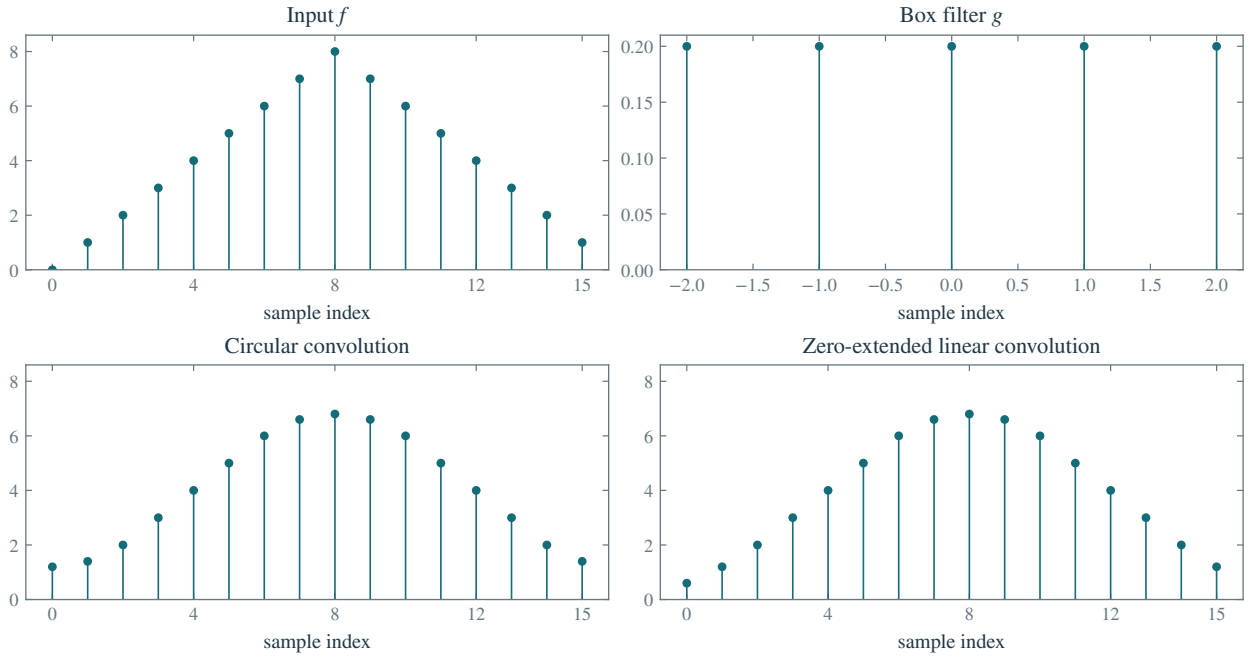


Figure 3.5. Circular and zero-extended linear convolution of a triangular signal with a unit-sum five-tap box filter. The outputs agree away from the endpoints.

3.3.3 Linear convolution and zero padding

Signals extended by zero require a convolution without wraparound. We determine the output support and the padding needed to compute this linear convolution with a circular FFT.

For finitely supported sequences $f_1, f_2 : \mathbb{Z} \to \mathbb{C}$, define

$$(f_1 \star f_2)[n] = \sum_{k \in \mathbb{Z}} f_1[k] f_2[n-k]. \tag{3.20}$$

This is *linear* or *acyclic* convolution. Its support satisfies $\mathrm{Supp}(f_1 \star f_2) \subset \mathrm{Supp}(f_1) + \mathrm{Supp}(f_2)$. If the input supports are contained in intervals of lengths N_1 and N_2, the output is contained in an interval of length $N_1 + N_2 - 1$. These are interval lengths, not the numbers of nonzero entries in sparse supports.

Suppose first that the intervals start at zero. Pad both vectors with zeros to a common length $L \geq N_1 + N_2 - 1$. Their length-L circular convolution agrees with their linear convolution on $0, \ldots, N_1 + N_2 - 2$, and is zero at the remaining indices. Indeed, no two potentially nonzero output indices are congruent modulo L. This is the condition that prevents wrap-around. An FFT at a convenient such length computes the linear convolution exactly in exact arithmetic; padding introduces no approximation here.

For a kernel supported on $\{-P, \ldots, Q\}$, store each coefficient at its residue modulo L: positive indices at the beginning and negative indices at the end. With f_1 supported on $0, \ldots, N_1 - 1$ and $L \geq N_1 + P + Q$, the output support is contained in $\{-P, \ldots, N_1 - 1 + Q\}$, and

$$(f_1 \star f_2)[n] = (\tilde{f}_1 * \tilde{f}_2)[n \bmod L] \quad \text{throughout this interval.}$$

In particular, negative output indices occupy the final slots. One must not append padding after a negative-index tail without moving that tail to the end of the new array.

Figure 3.5 compares periodic and zero boundary conditions for a short averaging filter. Their interior outputs agree where the whole filter support remains inside the sample interval. Near an edge, periodic convolution draws values from the opposite edge, while zero extension reduces the average. The appropriate boundary model depends on the application; zero extension is not inherently free of boundary artifacts. Exercise 3.4 develops overlap-add convolution for a long signal and a short filter. Chapter 4 uses these operations for filtering.

3.4 Higher-dimensional transforms

A rectangular array corresponds to a function on a product of cyclic groups. The characters factor by coordinate, so both the formulas and the algorithms separate into one-dimensional transforms.

3.4.1 The two-dimensional DFT

For images and other rectangular arrays, a frequency has one coordinate for each axis. We define the two-dimensional transform and recover inversion by applying one-dimensional orthogonality in both directions.

Definition 21 (Two-dimensional DFT)**.** For $f \in \mathbb{C}^{N\times P}$, define

$$\hat{f}[k,l] = \sum_{i=0}^{N-1}\sum_{j=0}^{P-1} f[i,j]\omega_N^{-ik}\omega_P^{-jl}. \tag{3.21}$$

Here $0 \le k < N$ and $0 \le l < P$.

The characters of $\mathbb{Z}/N\mathbb{Z}\times\mathbb{Z}/P\mathbb{Z}$ are $\chi_{k,l}(i,j) = \omega_N^{-ik}\omega_P^{-jl}$, so the formula is the corresponding group Fourier transform.

Applying the one-dimensional inversion formula in each coordinate determines the two-dimensional normalization.

Proposition 35 (Two-dimensional inversion)**.** *For $0 \le i < N$ and $0 \le j < P$,*

$$f[i,j] = \frac{1}{NP}\sum_{k=0}^{N-1}\sum_{l=0}^{P-1} \hat{f}[k,l]\omega_N^{ik}\omega_P^{jl}.$$

The proof applies character orthogonality along each coordinate. To compute the forward transform, first transform every row, then every column of the result. The order of these two passes can be reversed. In matrix form,

$$\hat{f} = \Omega_N f \Omega_P^{\mathsf{T}} = \Omega_N f \Omega_P,$$

because the Fourier matrices are symmetric. The cost is $O(NP(\log N + \log P))$. The same separation works in any dimension.

Figure 3.6 displays an image and the magnitude of its spectrum, with zero frequency shifted to the center for readability. For signed frequency indices (k,l) and unit pixel spacing, the frequency vector is $(k/N, l/P)$; its Euclidean norm is the spatial frequency, and it points in the direction of variation, normal to the stripes of a plane wave. The complex coefficient also carries phase, which a magnitude display omits.

3.4.2 Two-dimensional convolution

Filtering an array combines translations along both axes. With periodic boundary conditions, the two-dimensional DFT again replaces this operation by pointwise multiplication.

Definition 22 (Circular convolution of arrays)**.** For $f, g \in \mathbb{C}^{N\times P}$,

$$(f * g)[i,j] = \sum_{k=0}^{N-1}\sum_{l=0}^{P-1} f[k,l]g[(i-k) \bmod N, (j-l) \bmod P]. \tag{3.22}$$

This uses periodic boundaries along both axes.

The corresponding convolution theorem provides the basis of FFT filtering for images.

Proposition 36 (Two-dimensional convolution theorem)**.**

$$\widehat{f * g}[k,l] = \hat{f}[k,l]\hat{g}[k,l]. \tag{3.23}$$

Proof. Apply the group convolution theorem to $\mathbb{Z}/N\mathbb{Z}\times\mathbb{Z}/P\mathbb{Z}$, or separate the exponential after substituting the two difference indices. □

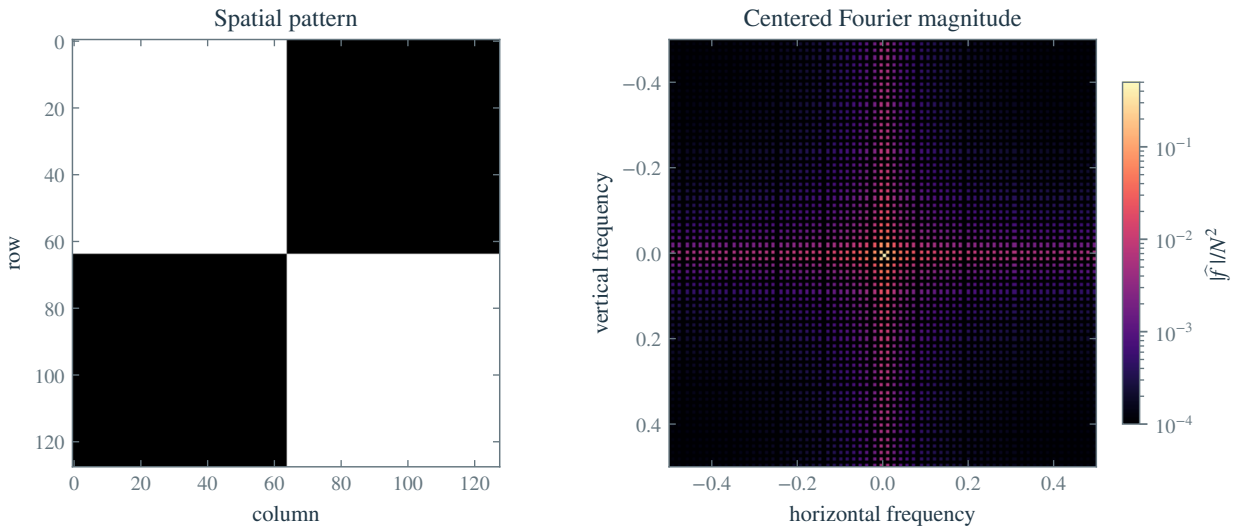


Figure 3.6. A two-dimensional checker pattern and its centered Fourier magnitude. The logarithmic color scale reveals the weaker frequencies.

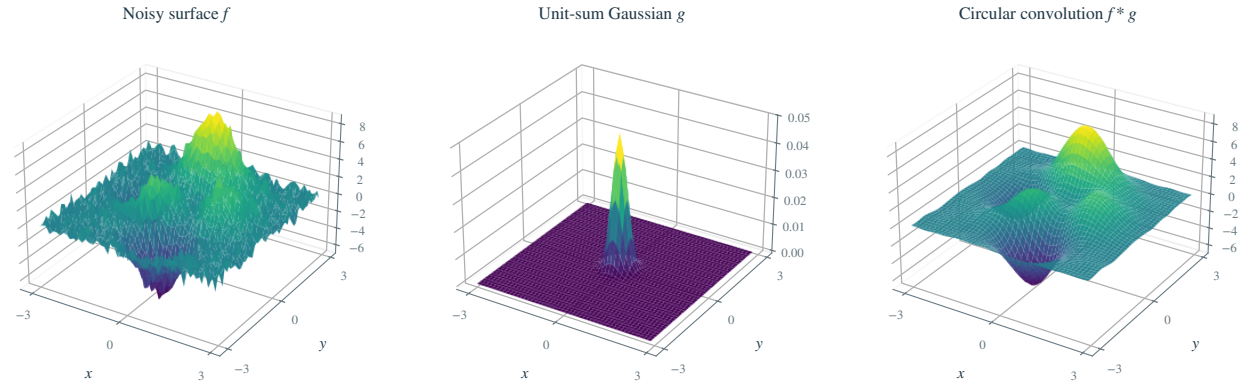


Figure 3.7. Circular Gaussian smoothing of a noisy surface. The discrete kernel has unit sum; its center is shifted to index $(0, 0)$ before the FFT.

Two forward FFTs, a pointwise product, and an inverse FFT therefore compute the convolution. For linear convolution of arrays, pad along each axis to at least the sum of the two side lengths minus one. A centered kernel's origin must be moved to index $(0, 0)$ in the padded array before transformation.

Convolution with a nonnegative kernel whose *discrete sum* is one forms a weighted average. Figure 3.7 illustrates the smoothing of an irregular surface. Its continuum analogue uses a kernel of integral one; the discrete formula uses a sum, without an implicit grid-spacing factor.

3.5 Symmetry and spectral structure

The fourth power of a normalized DFT is the identity. This simple relation restricts its spectrum and gives explicit projections onto its eigenspaces. It also reveals the choices involved in defining fractional powers.

3.5.1 Symmetry identities

Reversal and complex conjugation organize several useful DFT identities. We define reversal on the cyclic grid, split a signal by symmetry, and use these operations to describe repeated transforms and real-valued data.

Definition 23 (Cyclic reversal)**.** For $f \in \mathbb{C}^N$, define

$$f^\sharp[n] = f[(-n) \bmod N]. \tag{3.24}$$

Thus $f^\sharp = (f[0], f[N-1], \dots, f[1])$. A vector is symmetric if $f^\sharp = f$ and antisymmetric if $f^\sharp = -f$.

Averaging a signal with its reversal isolates its symmetric component; taking their difference isolates the antisymmetric component.

Definition 24 (Symmetric and antisymmetric parts)**.** Set $f_S = (f + f^\sharp)/2$ and $f_A = (f - f^\sharp)/2$. Then $f = f_S + f_A$ is an orthogonal decomposition.

These components behave predictably under the DFT, as the following identities show.

Proposition 37 (Symmetry of the DFT)**.** *For every $f \in \mathbb{C}^N$,*

$$\mathcal{F}(f^\sharp) = (\mathcal{F}f)^\sharp = N\mathcal{F}^{-1}f, \qquad \mathcal{F}^2 f = Nf^\sharp, \qquad \mathcal{F}(\overline{f}) = \overline{(\mathcal{F}f)^\sharp}.$$

Consequently, the DFT preserves the symmetric and antisymmetric subspaces. For real symmetric input its output is real; for real antisymmetric input its output is purely imaginary.

Proof. Substitute $m = -n$ modulo N in the Fourier sum for $f^\sharp$ to reverse the exponential's sign. Applying inversion gives the first two identities. Conjugating the Fourier sum gives the third. If f is real, this last identity states that $\hat{f}[-k] = \overline{\hat{f}[k]}$; combining it with symmetry or antisymmetry proves the final claims. □

3.5.2 Eigenvectors and fractional powers

The identity obtained after four normalized transforms restricts the possible eigenvalues to four phases. We use this finite spectrum to construct explicit spectral components and to explain the choices involved in fractional powers.

Write $U = N^{-1/2}\mathcal{F}$ for the unitary DFT. Then $U^2 f = f^\sharp$ and $U^4 = I$. Since U is normal, it has an orthonormal eigenbasis, with all eigenvalues in $\{1, -1, \mathrm{i}, -\mathrm{i}\}$. Some eigenspaces can be zero-dimensional for small N. The unnormalized transform has eigenvalues among $\{\sqrt{N}, -\sqrt{N}, \mathrm{i}\sqrt{N}, -\mathrm{i}\sqrt{N}\}$ and satisfies $\mathcal{F}^4 = N^2 I$.

The symmetry decomposition gives explicit formulas for extracting each possible eigenvalue component from a vector.

Theorem-Definition 1 (Explicit spectral components)**.** *For $f \in \mathbb{C}^N$, define*

$$\mathcal{U}_+(f) = \sqrt{N} f_S + \mathcal{F}(f_S), \qquad \mathcal{U}_-(f) = \sqrt{N} f_S - \mathcal{F}(f_S),$$
$$\mathcal{V}_+(f) = \sqrt{N} f_A + \mathrm{i}\mathcal{F}(f_A), \qquad \mathcal{V}_-(f) = \sqrt{N} f_A - \mathrm{i}\mathcal{F}(f_A).$$

They belong, respectively, to the eigenspaces of $\mathcal{F}$ with eigenvalues $\sqrt{N}, -\sqrt{N}, -\mathrm{i}\sqrt{N}, \mathrm{i}\sqrt{N}$. A nonzero component is an eigenvector; a zero component is not. Their properly normalized decomposition is

$$f = \frac{1}{2\sqrt{N}}\big(\mathcal{U}_+(f) + \mathcal{U}_-(f) + \mathcal{V}_+(f) + \mathcal{V}_-(f)\big).$$

Proof. On the symmetric subspace $\mathcal{F}^2 = NI$, so applying $\mathcal{F}$ to $\sqrt{N} f_S \pm \mathcal{F}(f_S)$ gives $\pm\sqrt{N}$ times that vector. On the antisymmetric subspace $\mathcal{F}^2 = -NI$, which gives the two remaining eigenvalue identities. Adding the four definitions gives $2\sqrt{N} f$. □

The orthogonal spectral projectors of U are therefore

$$P_1 = \frac{\mathcal{U}_+}{2\sqrt{N}}, \quad P_{-1} = \frac{\mathcal{U}_-}{2\sqrt{N}}, \quad P_{-\mathrm{i}} = \frac{\mathcal{V}_+}{2\sqrt{N}}, \quad P_{\mathrm{i}} = \frac{\mathcal{V}_-}{2\sqrt{N}}.$$

Equivalently, $P_\lambda = \frac{1}{4}\sum_{j=0}^{3} \lambda^{-j} U^j$. These formulas provide the components without selecting eigenvectors inside a repeated eigenspace.

To define a consistent family of fractional powers, fix one real phase for each eigenvalue, for example

$$\theta_1 = 0, \quad \theta_{-1} = \pi, \quad \theta_{\mathrm{i}} = \pi/2, \quad \theta_{-\mathrm{i}} = -\pi/2.$$

For every $\alpha \in \mathbb{R}$, set

$$U^\alpha = \sum_{\lambda \in \{1,-1,\mathrm{i},-\mathrm{i}\}} e^{\mathrm{i}\alpha\theta_\lambda} P_\lambda, \qquad \mathcal{F}^\alpha = N^{\alpha/2} U^\alpha.$$

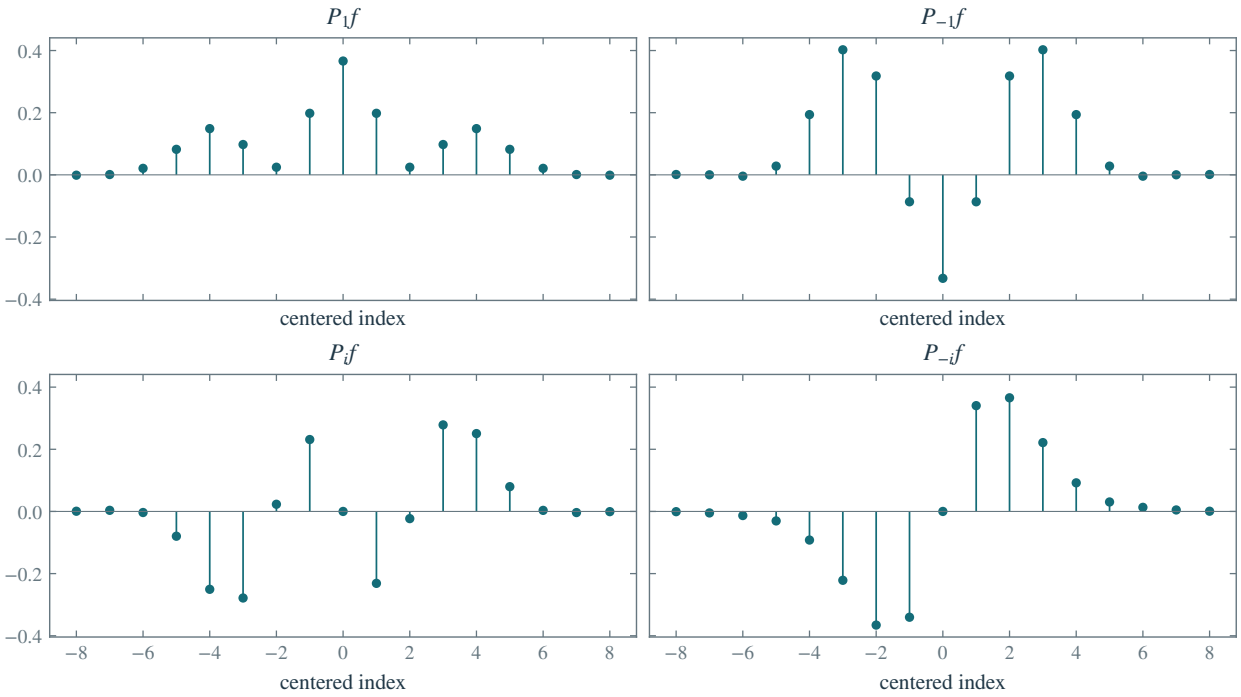


Figure 3.8. The four orthogonal components $P_\lambda f$ of a real signal for the normalized DFT U. Their sum is f; even and odd components appear on separate rows.

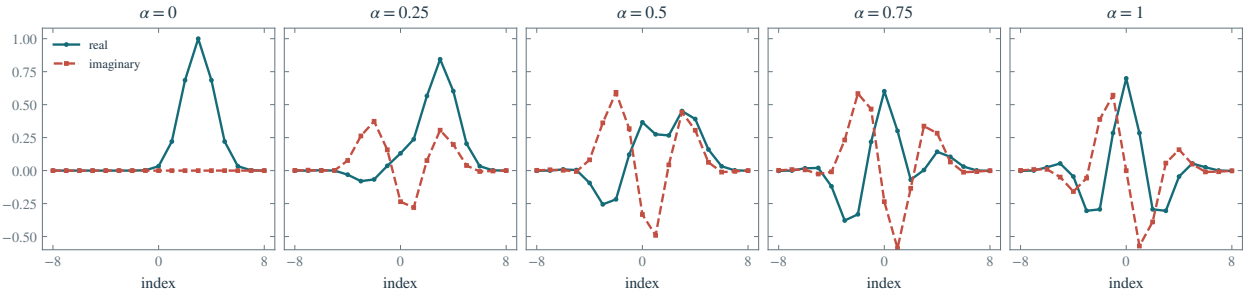


Figure 3.9. The unitary transforms $U^\alpha f$ for $0 \le \alpha \le 1$, using the fixed phases in the text. Real and imaginary parts are shown separately.

The projector identities give $U^0 = I$, $U^1 = U$, and $U^\alpha U^\beta = U^{\alpha+\beta}$; the same composition rule holds for $\mathcal{F}^\alpha$. In particular, $(\mathcal{F}^{1/2})^2 = \mathcal{F}$. The normalized family preserves norms, whereas $\|\mathcal{F}^\alpha f\|_2 = N^{\alpha/2}\|f\|_2$. The figures use U^α so that changes in shape can be compared at a fixed energy. Choosing other phases modulo 2π gives other continuous families. The choices must be fixed for all exponents; choosing a new root independently at each exponent need not obey the composition law. Within an eigenspace, changing the orthonormal basis alone has no effect when its phase is kept the same. Exercise 3.9 explores these choices, while Exercise 3.10 constructs a reproducible eigenbasis using a second commuting operator. For further background see Cariolaro [13].

3.6 Exercises

These exercises develop alternative FFT factorizations, applications of convolution, and the spectral structure of the transform. The first makes the indexing behind an iterative FFT explicit through a recursion on binary digits.

Exercise 3.1 (Bit reversal)**.** Define index vectors recursively by $u^{(0)} = (0)$ and $u^{(n)} = (2u^{(n-1)}, 2u^{(n-1)} + 1)$.

1. Compute $u^{(n)}$ for $n = 1, 2, 3$.
2. Prove that $u^{(n)}[k]$ reverses the n binary digits of k.

3. For a length-2^n vector, let $\tilde{f}[k] = f[u^{(n)}[k]]$. Explain its role in the iterative DIT algorithm.
4. Show that the first and second halves satisfy $\tilde{f}_g = \widetilde{f^0}$ and $\tilde{f}_d = \widetilde{f^1}$.
5. Implement this last relation recursively in Python. Compare its array copying cost with generating $u^{(n)}$ directly by the index-vector recursion.

For coprime factors, the Chinese remainder theorem gives a factorization in which permutations replace the usual twiddle factors.

Exercise 3.2 (The Good–Thomas algorithm)**.** Let $N = pq$, with $p, q \geq 2$ coprime integers. They need not be prime.

1. Use the Chinese remainder theorem to invert $\varphi : n \mapsto (n \bmod p, n \bmod q)$ from $\mathbb{Z}/N\mathbb{Z}$ to $\mathbb{Z}/p\mathbb{Z} \times \mathbb{Z}/q\mathbb{Z}$. Write its inverse as ψ.
2. For $F[k_1, k_2] = f[\psi(k_1, k_2)]$, prove

$$\hat{F}[s_1, s_2] = \hat{f}[(s_1 q + s_2 p) \bmod N].$$

3. Show that $(s_1, s_2) \mapsto (s_1 q + s_2 p) \bmod N$ is a bijection. Give its inverse explicitly and deduce the output permutation of the algorithm.
4. Explain why this factorization eliminates the twiddle factors in (3.14). Implement one Good–Thomas step in Python. Discuss how it can be combined recursively with Cooley–Tukey and with other algorithms for inconvenient lengths; distinguish arithmetic savings from the cost of permutations.

Combining radix-two and radix-four decompositions reduces the arithmetic count in a more flexible recursion.

Exercise 3.3 (The split-radix algorithm)**.** Let $N = 2^p \geq 4$. Split-radix keeps the length-$N/2$ transform for the even frequencies and computes the odd frequencies using two length-$N/4$ transforms.

1. Starting with a DIF decomposition, prove, for $j = 0, 1$ and $0 \leq k < N/4$, that

$$\hat{f}[4k + 2j + 1] = \sum_{a=0}^{N/4-1} \omega_{N/4}^{-ka} \omega_N^{-a(2j+1)} \sum_{b=0}^{3} f[a + bN/4] \omega_4^{-b(2j+1)}.$$

   Express the inner sums using additions, subtractions, and multiplication by ±i. Identify the remaining twiddle factors.
2. Derive the DIT version by separating the input subsequences $f[2k]$, $f[4k + 1]$, and $f[4k + 3]$. Give four formulas for the output quarters and implement the resulting recursion.
3. Compare this recursion with radix two in a specified arithmetic model: count complex additions and general twiddle multiplications, but regard multiplication by ±i as a sign change and component swap. Use direct length-two butterflies as the base case. Count every displayed twiddle multiplication, even when its factor happens to be one. Derive recurrences for these counts and compare their leading terms. Explain what extra work an iterative implementation requires to lay out its unequal subproblems and permute the data.

The comparison concerns this arithmetic model, not a universal optimality claim or a guarantee about running time. For further variants see Vetterli and Nussbaumer [64].

When a filter is short compared with the signal, processing overlapping output blocks can reduce the convolution cost.

Exercise 3.4 (Overlap-add convolution)**.** Let f and g have lengths N and M, with $M \ll N$, and extend both by zero outside their index intervals starting at zero.

1. If $N = pM$, let $f_j[k] = f[k + jM]$ for $0 \leq k < M$, and extend each block by zero. Prove

$$(f \star g)[k] = \sum_{j=0}^{p-1} (f_j \star g)[k - jM].$$

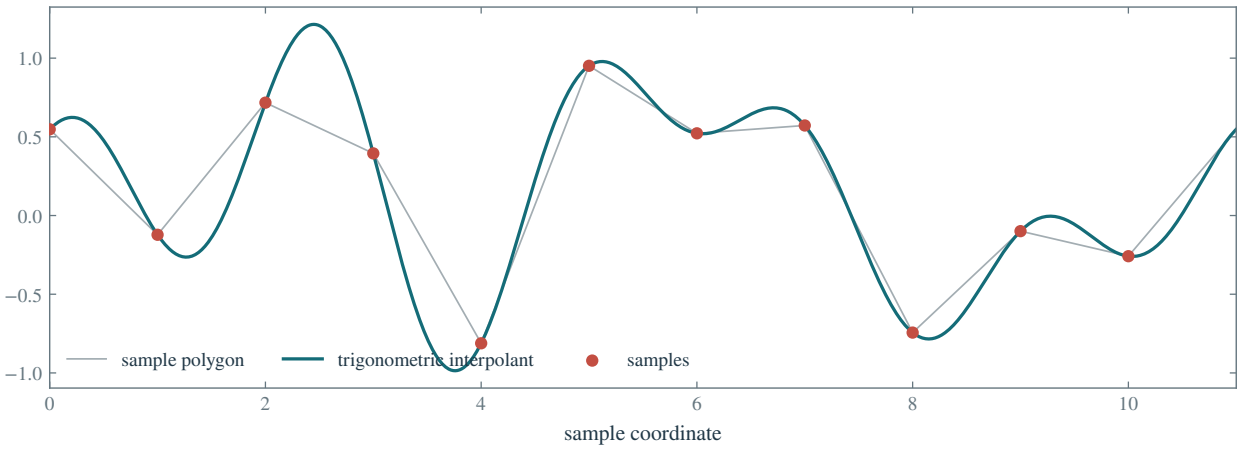


Figure 3.10. Periodic trigonometric interpolation of eleven samples. The faint polygon joins the samples for reference; it is not an underlying continuous signal.

2. Compute each block convolution using a common FFT length $L \geq 2M - 1$, reuse the transform of g, and add the overlapping outputs. Show that choosing $L = O(M)$ gives total cost $O(N \log M)$ for $M \geq 2$.
3. Handle a final block of fewer than M samples when $M \nmid N$. Implement the method and compare its output with direct linear convolution.

Circulant matrices provide the linear-algebraic form of circular convolution and its Fourier diagonalization.

Exercise 3.5 (Circulant matrices)**.** For $c \in \mathbb{C}^N$, let $C_{ij} = c[(i - j) \bmod N]$. Its first column is c, and its first row is $(c_0, c_{N-1}, \ldots, c_1)$. This convention differs from specifying the first row, as in Exercise 1.1.

1. Let $Re_j = e_{(j+1) \bmod N}$ for the standard basis indexed from zero. Prove
$$\Omega_N R \Omega_N^{-1} = D = \operatorname{diag}(1, \omega_N^{-1}, \ldots, \omega_N^{-(N-1)}).$$
2. Show that $C = \sum_{j=0}^{N-1} c_j R^j$ and deduce $\Omega_N C \Omega_N^{-1} = \operatorname{diag}(\hat{c})$. Hence show that $Cx = \Omega_N^{-1}((\Omega_N c) \cdot (\Omega_N x))$.
3. Verify directly that $Cx = c * x$ for every $x \in \mathbb{C}^N$. Recover the diagonalization from the convolution theorem.

Padding the spectrum, with the frequency labels and scaling preserved, produces samples of a trigonometric interpolant.

Exercise 3.6 (Trigonometric interpolation)**.** Let $N = 2N_0 + 1$ be odd and $f \in \mathbb{C}^N$. Define the trigonometric polynomial
$$p(t) = \frac{1}{N} \sum_{k=-N_0}^{N_0} \hat{f}[k \bmod N] e^{2\pi i k t}.$$
For an integer $\eta \geq 1$, let $P = \eta N$ and construct the length-P spectrum by inserting zeros between its nonnegative- and negative-frequency parts:
$$\hat{f}_0 = \eta\big(\hat{f}[0], \ldots, \hat{f}[N_0], 0, \ldots, 0, \hat{f}[N_0 + 1], \ldots, \hat{f}[N - 1]\big).$$
Prove that $f_0[j] = p(j/P)$ and $f_0[\eta k] = f[k]$. Implement the construction for real and complex inputs. Explain why padding the spectrum must respect the signed frequency indices, and why the factor η is needed. The odd-length assumption avoids a choice for the Nyquist coefficient that arises at even lengths.

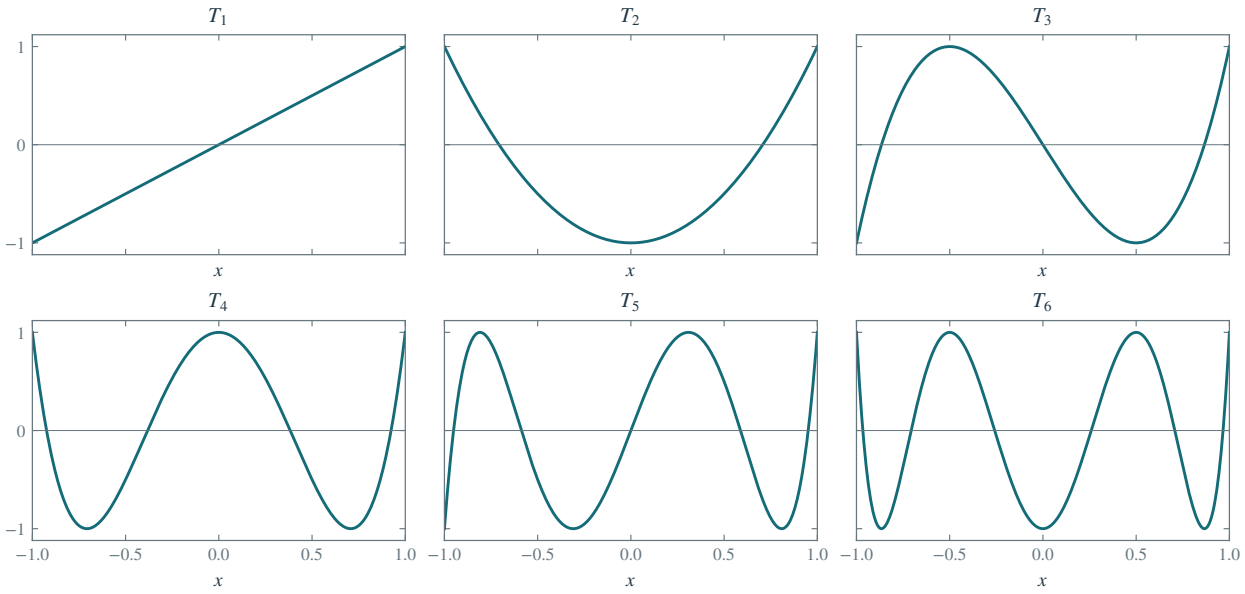


Figure 3.11. The Chebyshev polynomials $T_k(x) = \cos(k \arccos x)$ for $k = 1, \dots, 6$.

A cosine change of variables connects Fourier computation with polynomial interpolation at Chebyshev nodes.

Exercise 3.7 (Chebyshev interpolation)**.** For $x \in [-1, 1]$, define $T_k(x) = \cos(k \arccos x)$. For $f \in C([-1, 1])$, we seek the polynomial of degree at most $N - 1$ interpolating f at the Chebyshev nodes $x_j = \cos((j + 1/2)\pi/N)$, $0 \le j < N$.

1. Prove the recurrence $T_{k+1} = 2xT_k - T_{k-1}$, with $T_0 = 1$ and $T_1 = x$. Deduce that T_k is a polynomial of degree k and that the x_j are exactly the roots of T_N.
2. Show that the interpolant has a unique expansion $P_{N-1} = \sum_{k=0}^{N-1} \alpha_k T_k$.
3. Define the cosine transforms, with the normalization used here, by

$$C_2(v)[j] = \sum_{k=0}^{N-1} v[k] \cos((k + 1/2)j\pi/N),$$

$$C_3(v)[j] = \tfrac{1}{2} v[0] + \sum_{k=1}^{N-1} v[k] \cos((j + 1/2)k\pi/N).$$

   Prove that $C_2^{-1} = (2/N)C_3$. Implement both for real data using a length-$4N$ FFT, specifying the positions of the inserted zeros. These conventions differ by a factor of two from SciPy's default unnormalized DCT-II and DCT-III.
4. Express the coefficients α_k in terms of $v[j] = f(x_j)$. Use the cosine transform to compute them, then evaluate the Chebyshev expansion.

The DCT-II also underlies the block transform used in JPEG; Strang [56] discusses its mathematical structure. Figure 3.12 compares the two node choices for $f(x) = 1/(\alpha^2 + x^2)$ with $N = 11$ and $\alpha = 0.3$. Both interpolants are Lagrange interpolation polynomials; one uses equally spaced nodes and the other uses Chebyshev nodes. Explore how their maximum errors change with N and α. A finite plot suggests behavior but does not prove uniform convergence. Chebyshev expansions are also useful in spectral methods for differential equations; see Boyd [9].

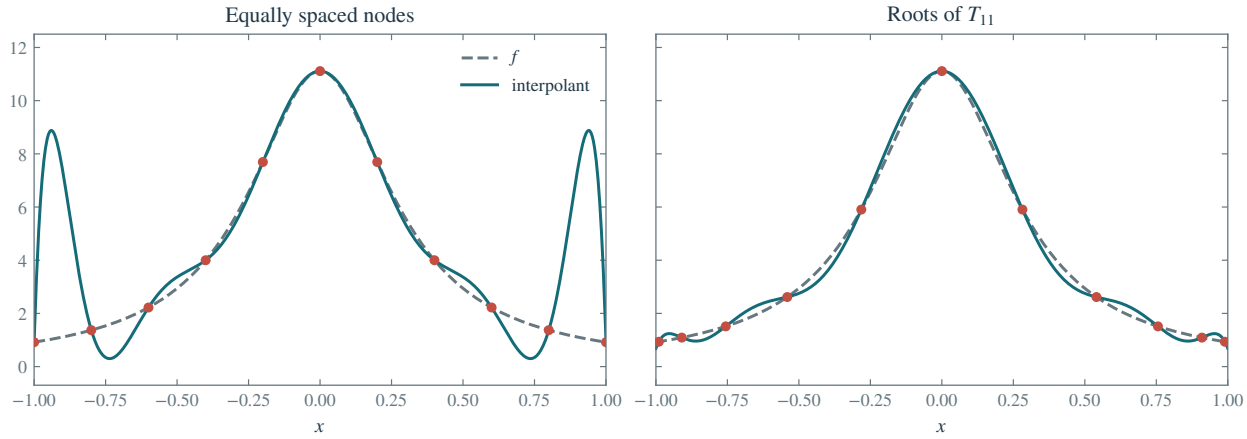


Figure 3.12. Degree-ten polynomial interpolation of $f(x) = 1/(0.3^2 + x^2)$ at eleven equally spaced points and at the roots of T_{11}. Both are Lagrange interpolants; the choice of nodes controls the endpoint oscillations.

Fourier multipliers extend integer-order differentiation to fractional orders once a phase convention is fixed.

Exercise 3.8 (Fractional differentiation)**.** Let f be a Schwartz function on $\mathbb{R}$, with $\hat{f}(\xi) = \int f(t)e^{-\mathrm{i}\xi t}\,\mathrm{d}t$. Show that $\widehat{f^{(m)}}(\xi) = (\mathrm{i}\xi)^m \hat{f}(\xi)$ for integers $m \geq 0$. For $\alpha \geq 0$, define one fractional derivative by the multiplier

$$(\mathrm{i}\xi)^\alpha = |\xi|^\alpha e^{\mathrm{i}\pi\alpha\,\mathrm{sgn}(\xi)/2} \quad (\xi \neq 0),$$

setting its value at zero to zero when $\alpha > 0$, and to one when $\alpha = 0$. Explain the branch choice and why this operator maps real input to real output. Implement its periodic discrete analogue on an *odd* number of uniformly spaced samples over a specified period. Compare integer orders with known derivatives and discuss periodization and sampling errors when using it to approximate differentiation on $\mathbb{R}$.

A related spectral construction defines continuous families of powers of the DFT, with an explicit choice of eigenvalue phases.

Exercise 3.9 (Intermediate Fourier transforms)**.** Let $U = \Omega_N/\sqrt{N}$ be the unitary DFT.

1. Prove that U is normal and $U^4 = I$. List the possible eigenvalues, and explain why normality, rather than a false Hermitian assumption, gives an orthonormal eigenbasis.
2. Let P_λ be its spectral projectors. Fix phases $e^{\mathrm{i}\theta_\lambda} = \lambda$ and define $U^\alpha = \sum_\lambda e^{\mathrm{i}\alpha\theta_\lambda} P_\lambda$. Prove $U^\alpha U^\beta = U^{\alpha+\beta}$, $U^1 = U$, and unitarity. Why must the phases be chosen consistently for all exponents?
3. Show that U^2 is cyclic reversal and explain the identity and reversal diagonals visible in Figure 3.14. Give the corresponding formulas for the unnormalized Ω_N^α.
4. Explain why changing a basis inside an eigenspace leaves the result unchanged when the phase is fixed for that whole eigenspace. Construct distinct families by changing a phase by an integer multiple of 2π, and implement the transform using the explicit projectors. Check the composition law numerically for noninteger exponents.

Figure 3.15 uses the phases fixed in Section 3.5.2. A symmetric input returns to itself at $\alpha = 2$ under this normalized transform. The unnormalized transform would return N times that input.

A second operator commuting with the DFT helps choose a reproducible eigenbasis inside its repeated eigenspaces.

Exercise 3.10 (A structured eigenbasis of the DFT)**.** For $N \geq 3$, let $Re_j = e_{j+1 \bmod N}$ and define the real symmetric matrix

$$S = R + R^{\mathsf{T}} + \operatorname{diag}\bigl(2\cos(2\pi k/N) - 4\bigr)_{k=0}^{N-1}.$$

The construction is based on Dickinson and Steiglitz [23]; Candan, Kutay, and Ozaktas [15] relate such eigenvectors to discrete fractional transforms.

1. Explain why S admits a real orthonormal eigenbasis.

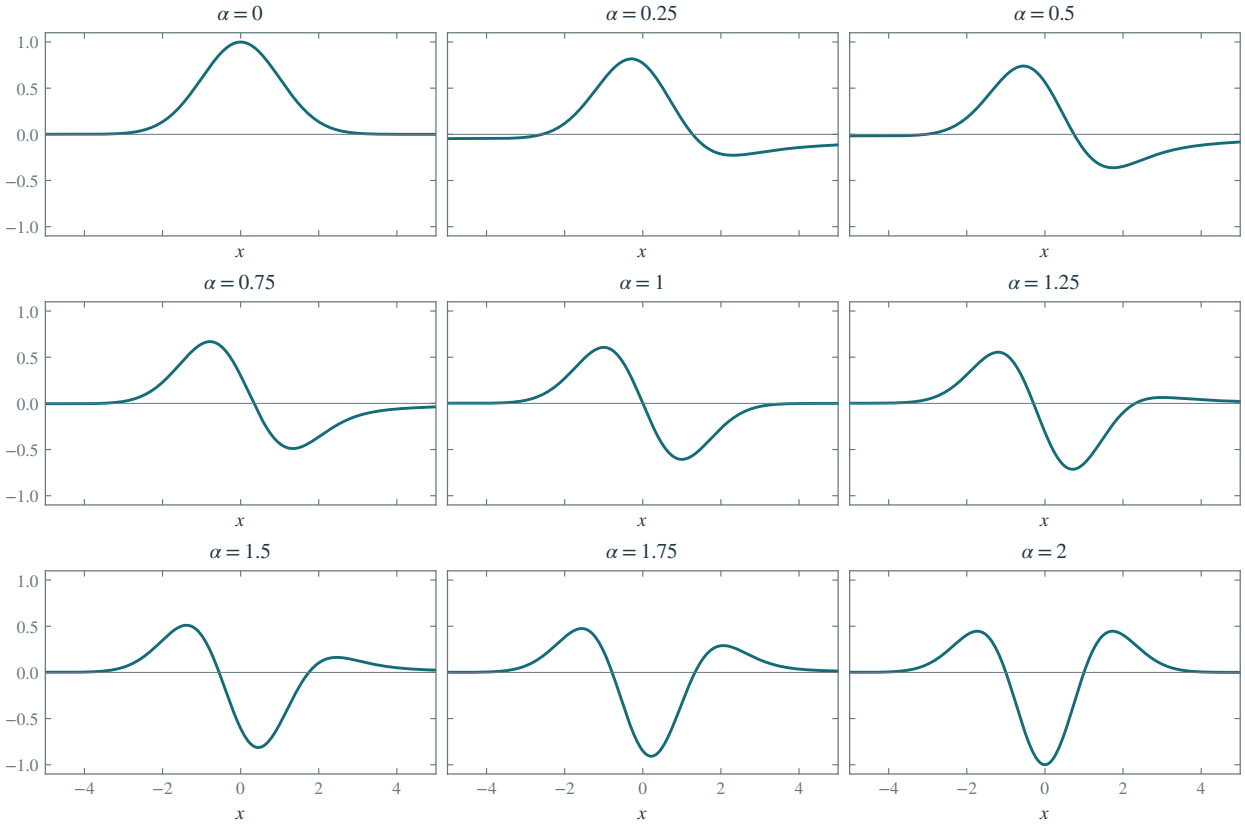


Figure 3.13. Fractional derivatives of $e^{-x^2/2}$ for orders $0 \le \alpha \le 2$, computed from the multiplier $(\mathrm{i}\xi)^\alpha$ with the phase convention in the text.

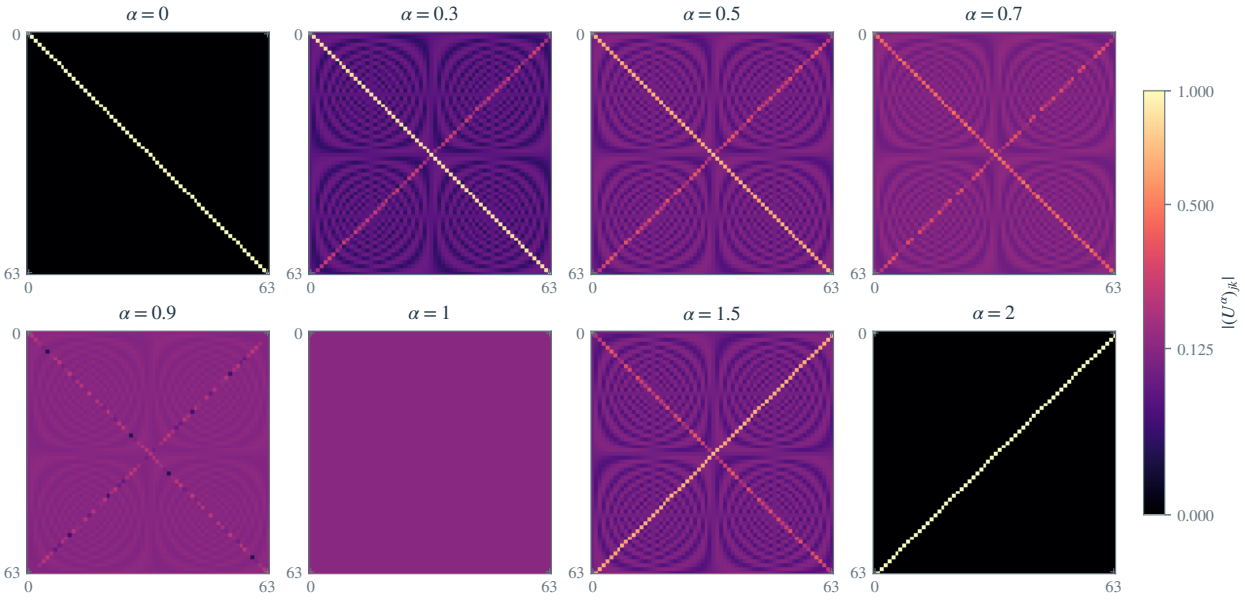


Figure 3.14. Entrywise magnitudes of the unitary matrices U^α, on one common nonlinear color scale. At $\alpha = 1$ every magnitude equals $1/\sqrt{N}$; at $\alpha = 2$ the matrix is cyclic reversal.

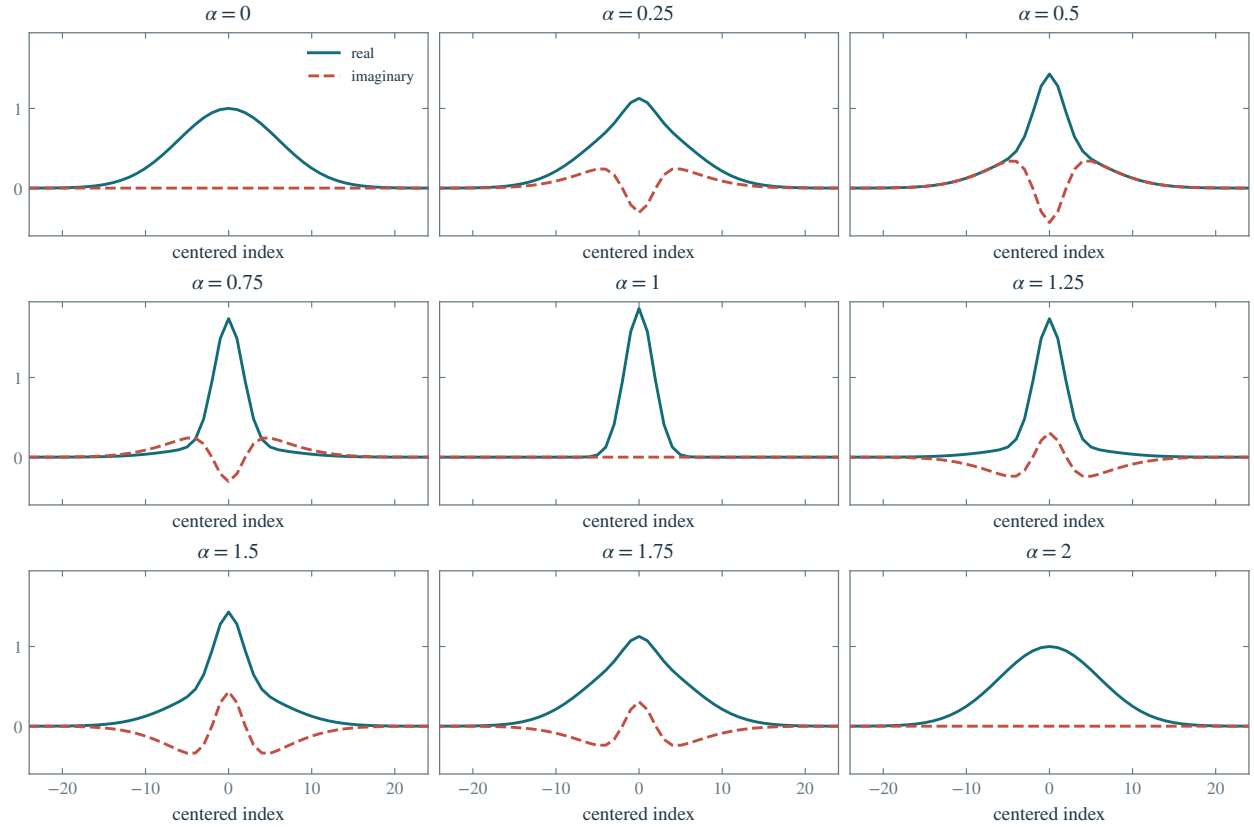


Figure 3.15. Unitary fractional Fourier transforms of a discrete Gaussian. The fixed eigenspace phases give a continuous family; at $\alpha = 2$ this even input is recovered.

2. Set $\Gamma = R + R^{\mathsf{T}} - 2I$ and $\Delta = \operatorname{diag}(2\cos(2\pi k/N) - 2)$. Prove $\Omega_N \Gamma = \Delta \Omega_N$ and $\Omega_N \Delta = \Gamma \Omega_N$, and conclude that S commutes with Ω_N.
3. Prove that commuting diagonalizable operators on $\mathbb{C}^N$ have a common eigenbasis. Explain why common eigenspaces need further care if an eigenvalue of one operator is repeated.
4. Restrict S to the symmetric and antisymmetric subspaces for cyclic reversal. Use the symmetric basis e_0, $(e_k + e_{N-k})/\sqrt{2}$ for $1 \le k < N/2$, and also $e_{N/2}$ if N is even; use $(e_k - e_{N-k})/\sqrt{2}$ for the antisymmetric basis. Show that each restriction is a real symmetric tridiagonal matrix with every subdiagonal entry nonzero, except when its dimension is one.
5. Prove that each such tridiagonal matrix has distinct eigenvalues by showing that an eigenvector is determined by its first coordinate. Explain why this conclusion applies separately to the two parity blocks, and does not imply that all eigenvalues of S are distinct. Check that S has the repeated eigenvalue -4 when $N = 4$.
6. Deduce that diagonalizing each parity block yields common eigenvectors of S and Ω_N, including when their eigenvalues of S overlap. Choose an ordering and signs to make the numerical output reproducible, and check the eigenvector residuals and orthogonality.

The figures use decreasing eigenvalue of S, with a specified tie convention. This makes the construction reproducible; it does not remove the freedom inherent in choosing an additional commuting operator and an ordering.

The final exercise uses a flat Fourier magnitude to construct orthonormal families of cyclic translates.

Exercise 3.11 (Orthogonalizing cyclic translates)**.** On $G = \mathbb{Z}/n\mathbb{Z}$, use the normalized inner product $\langle f, g \rangle = n^{-1} \sum_x f[x]\overline{g[x]}$. Define translation and modulation by

$$(k\top f)[x] = f[x - k], \qquad (k\perp f)[x] = \omega_n^{-kx} f[x].$$

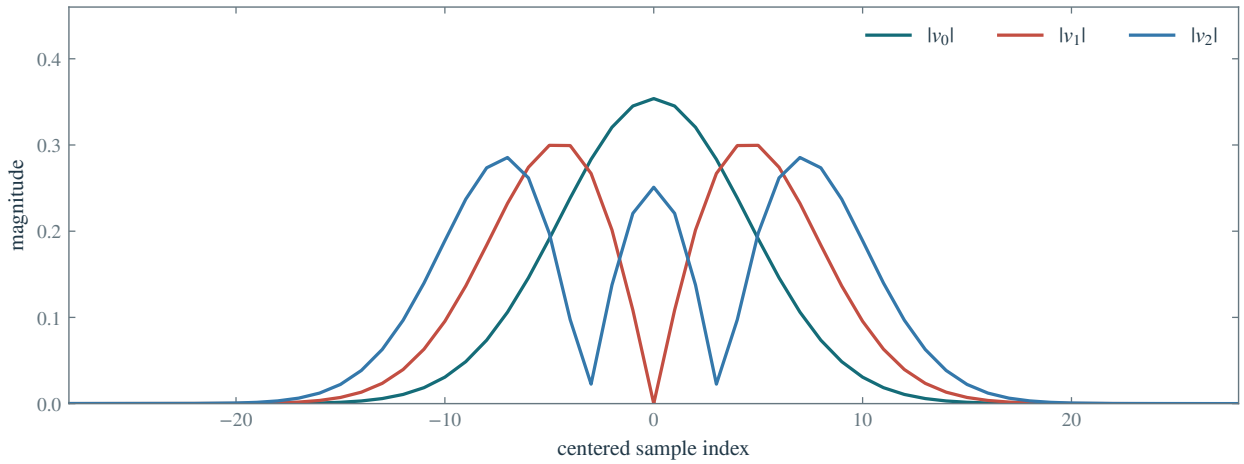


Figure 3.16. Magnitudes of the first three vectors in the structured DFT eigenbasis for $N = 128$, ordered by decreasing eigenvalue of S. The index is centered for display.

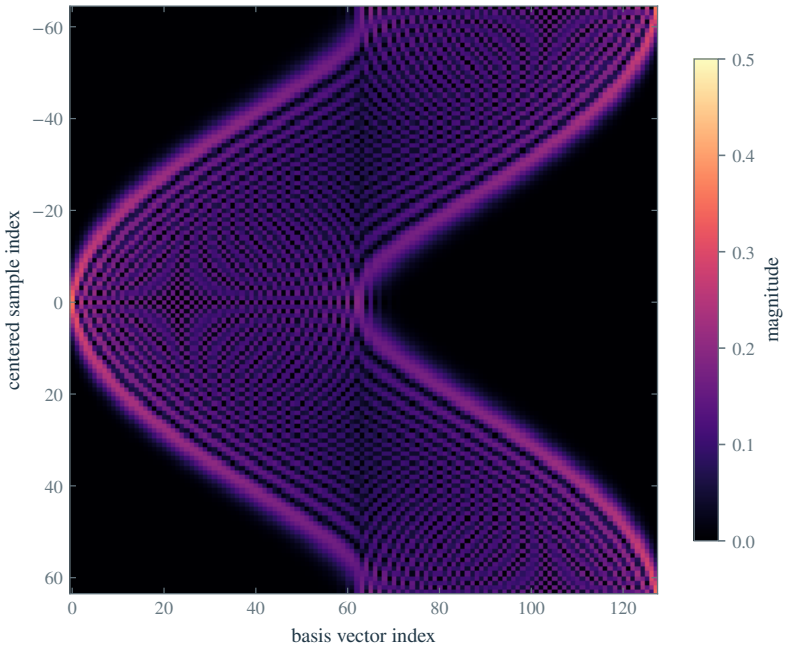


Figure 3.17. Magnitudes of the structured orthonormal eigenbasis for $N = 128$. Columns follow decreasing eigenvalues of S, with even parity first in a tie; rows show centered sample indices.

All indices are taken modulo n.

1. Prove $\mathcal{F}(k \top f) = k \perp \mathcal{F}(f)$ and $\mathcal{F}(k \perp f) = (-k) \top \mathcal{F}(f)$.
2. Explain how the *unitary* transform $n^{-1/2}\mathcal{F}$ maps an orthonormal translation orbit to an orthonormal modulation orbit, and conversely.
3. Show that $\{k \top f\}_{k \in G}$ is orthonormal if and only if $|\hat{f}[k]| = \sqrt{n}$ for every k.
4. When $\hat{f}$ has no zero entry, define $\hat{f}_0[k] = \sqrt{n}\, \hat{f}[k]/|\hat{f}[k]|$. Show that the translates of f_0 are orthonormal. Give the analogous pointwise normalization for a modulation orbit, assuming $f[x] \neq 0$ everywhere.
5. If the translates of g are orthonormal, express $\mathcal{G}(\varphi)[k] = \langle \varphi,\, k \top g \rangle$ as a circular correlation:

$$\mathcal{G}(\varphi) = \frac{1}{n} \varphi * \tilde{g}, \qquad \tilde{g}[x] = \overline{g[-x]}.$$

   Deduce an $O(n \log n)$ coefficient algorithm and the reconstruction of φ from these coefficients.

Figure 3.18 shows how replacing Fourier magnitudes by the constant $\sqrt{n}$ changes the signal. The effect

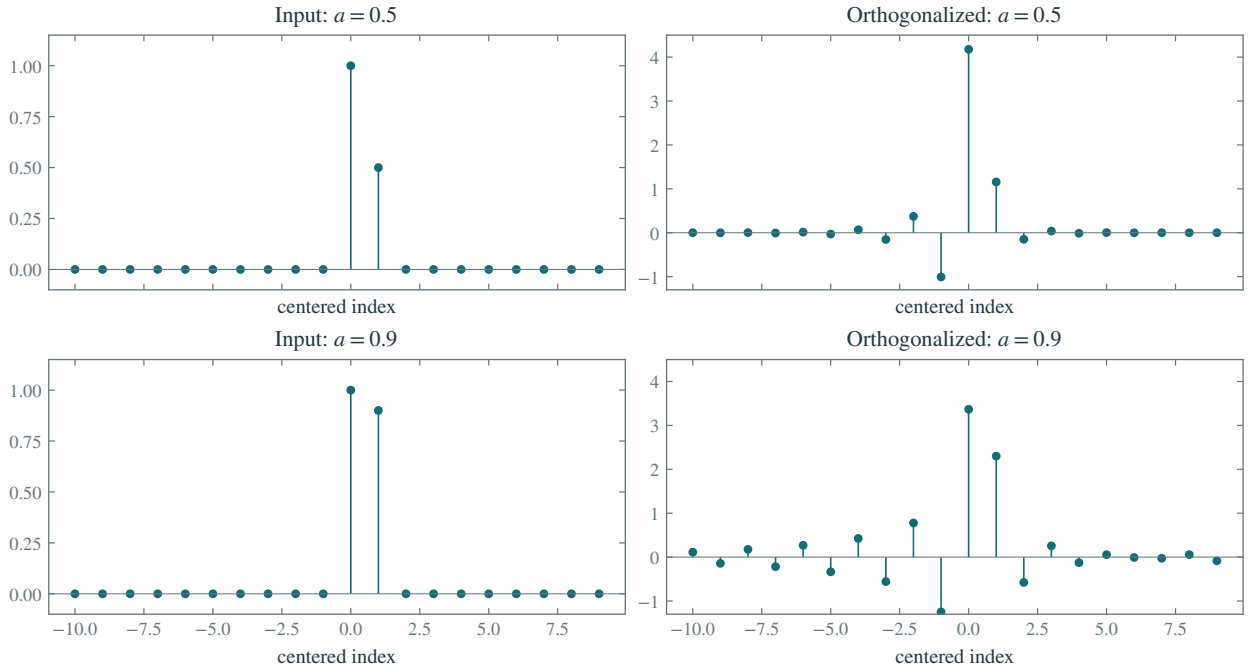


Figure 3.18. Orthogonalizing the cyclic translates of $f = \delta_0 + a\delta_1$ on $\mathbb{Z}/20\mathbb{Z}$, for $a = 0.5$ and $a = 0.9$. The output g satisfies $|\widehat{g}| = \sqrt{20}$, as required by the normalized inner product.

depends on its spectral phases; it is not a universal increase in oscillation or a smoothing operation.

3.7 Solutions to the exercises

These solutions follow the index maps and normalization factors that make fast-transform identities into correct algorithms. They also explain how Fourier coordinates support interpolation, convolution, fractional operators, and the construction of orthonormal bases.

Solution to Exercise 3.1

1. The first vectors are $u^{(1)} = (0, 1)$, $u^{(2)} = (0, 2, 1, 3)$, and $u^{(3)} = (0, 4, 2, 6, 1, 5, 3, 7)$.
2. When the most significant input bit is zero, reversal appends a zero to the reversed lower bits, giving $2u^{(n-1)}$. When it is one, reversal appends a one, giving $2u^{(n-1)} + 1$. This is exactly the defining recursion.
3. In this order, the leaves of the DIT recursion are adjacent in memory. The iterative algorithm combines contiguous blocks from size two upward; Section 3.2.5 explains why this evaluates the DFT of the original signal, rather than the DFT of a different signal.
4. A first-half output index has leading bit zero, so its reversed index is even. Reversing its remaining bits gives $\tilde{f}_g = \widetilde{f^0}$. The same argument with a leading one gives $\tilde{f}_d = \widetilde{f^1}$.
5. Program 10 implements this relation. At every level, concatenation copies N entries in total, so this recursive array construction costs $O(N \log N)$ element moves. In contrast, the direct index recursion creates arrays of lengths $1, 2, 4, \ldots, N$, for $O(N)$ total work, followed by a single $O(N)$ gather. The two implementations do not have the same copying complexity.

```
def bit_reverse_recursive(x):
    x = np.asarray(x)
    n = x.size
    if x.ndim != 1 or n == 0 or n & (n-1):
        raise ValueError('Expected a vector of power-of-two length.')
    if n == 1:
        return x.copy()
    return np.r_[bit_reverse_recursive(x[::2]), bit_reverse_recursive(x[1::2])]
```

Program 10. Recursive permutation of a vector into bit-reversed order.

Solution to Exercise 3.2

1. Choose integers u, v with $up + vq = 1$. Then $\psi(k_1, k_2) = (k_1 vq + k_2 up) \bmod N$ has the required two residues. The Chinese remainder theorem makes it the inverse of φ.
2. Since $n = \psi(k_1, k_2)$ satisfies $n \equiv k_1 \pmod{p}$ and $n \equiv k_2 \pmod{q}$,

$$n(s_1 q + s_2 p) \equiv k_1 s_1 q + k_2 s_2 p \pmod{N}.$$

   This follows either by reducing modulo both coprime factors or by expanding ψ. Substitute in the two-dimensional Fourier sum and reindex by n:

$$\hat{F}[s_1, s_2] = \sum_{n=0}^{N-1} f[n] \omega_N^{-n(s_1 q + s_2 p)} = \hat{f}[(s_1 q + s_2 p) \bmod N].$$

3. For a given output index m, the inverse mapping is $s_1 = mq^{-1} \bmod p$ and $s_2 = mp^{-1} \bmod q$. These inverses exist by coprimality, proving bijectivity. The integer $s_1 q + s_2 p$ can exceed $N - 1$; the reduction modulo N is essential.
4. The character pairing factors with no additional phase, so there is no analogue of ω_N^{-bd} in (3.14). Program 11 scatters the input by its two residues, applies a separable two-dimensional FFT, and scatters the output by the proved map. Further factoring the row and column transforms combines this method with Cooley–Tukey or other algorithms. The permutations still cost memory traffic; removing multiplications does not by itself establish a running-time advantage.

```
def good_thomas(x, p):
    x = np.asarray(x, dtype=complex)
    if x.ndim != 1 or p < 2:
        raise ValueError('Expected a vector and a factor p >= 2.')
    n, q = x.size, x.size//p
    if p < 2 or q < 2 or p*q != n or gcd(p, q) != 1:
        raise ValueError('Expected coprime factors p and q, both at least two.')
    F = np.empty((p, q), dtype=complex)
    indices = np.arange(n)
    F[indices % p, indices % q] = x
    spectrum = np.fft.fft2(F)
    s1, s2 = np.indices((p, q))
    y = np.empty(n, dtype=complex)
    y[(s1*q+s2*p) % n] = spectrum
    return y
```

Program 11. A Good–Thomas factorization with explicit input and output permutations.

Solution to Exercise 3.3

1. Write the input index as $a + bN/4$. Expanding the exponent for frequency $4k + 2j + 1$ gives

$$\omega_N^{-(a+bN/4)(4k+2j+1)} = \omega_{N/4}^{-ak}\omega_N^{-a(2j+1)}\omega_4^{-b(2j+1)},$$

since the omitted factor $e^{-2\pi ibk}$ is one. With $u_a = f[a] - f[a + N/2]$ and $v_a = f[a + N/4] - f[a + 3N/4]$, the two inner sums are $u_a - \mathrm{i}v_a$ and $u_a + \mathrm{i}v_a$. Their general twiddle factors are ω_N^{-a} and ω_N^{-3a}. The even-output branch is the length-$N/2$ DFT of $f_g + f_d$, without twiddles.

2. Let E be the length-$N/2$ DFT of the even input entries, and let O_1, O_3 be the length-$N/4$ DFTs of the entries at indices $4j + 1$ and $4j + 3$. For $0 \le k < N/4$, put $u = \omega_N^{-k}O_1[k]$, $v = \omega_N^{-3k}O_3[k]$. Then

$$\hat{f}[k] = E[k] + u + v, \qquad \hat{f}[k + N/2] = E[k] - u - v,$$
$$\hat{f}[k + N/4] = E[k + N/4] - \mathrm{i}(u - v), \qquad \hat{f}[k + 3N/4] = E[k + N/4] + \mathrm{i}(u - v).$$

These follow by substituting the four frequency quarters into the three input-subsequence sums. The function `split_radix` in the accompanying Python module follows these formulas; sizes one and two are the base cases.

3. Let $M(N)$ count the displayed general complex multiplications. The split-radix combine stage uses $N/2$ such products, giving

$$M(N) = M(N/2) + 2M(N/4) + N/2, \qquad M(1) = M(2) = 0.$$

For $N = 2^p$, solving the recurrence yields $M(N) = Np/3 - 2N/9 + (2/9)(-1)^p$. With the same length-two base case, radix two gives $Np/2 - N/2$. Thus the leading coefficient drops from $1/2$ to $1/3$ in this model. Each split-radix quartet uses six complex additions after sharing $u + v$ and $u - v$, so

$$A(N) = A(N/2) + 2A(N/4) + 3N/2, \qquad A(1) = 0, \quad A(2) = 2.$$

It follows that $A(N) = N \log_2 N$, the same count as radix two. Further savings are possible by special-casing trivial twiddles; these were excluded from this count. An iterative version must schedule subproblems of two different sizes and track where the three subsequences and four output quarters are stored. Its permutations must follow that factorization, rather than assuming equal-length blocks at every stage.

Solution to Exercise 3.4

1. As zero-extended sequences, $f = \sum_j f_j[\cdot - jM]$. Bilinearity and translation covariance of linear convolution give the asserted sum of shifted block convolutions.
2. Each block output has length at most $2M-1$. Choose one FFT length $L \geq 2M-1$, for example the next power of two, and compute the padded transform of g once. For every block, take one forward FFT, multiply by this stored transform, and take an inverse FFT. Add the first $2M-1$ entries at offset jM. With $L = O(M)$, each block costs $O(M \log M)$, and there are N/M blocks. Including the overlap additions gives $O(N \log M)$ for $M \geq 2$. For a one-sample kernel the operation is simply scalar multiplication.
3. If the last block has $r < M$ entries, pad it to the same FFT length but add only its $r + M - 1$ output entries. No output beyond index $N + M - 2$ should be retained. Program 12 handles this case directly.

```
def overlap_add(f, g):
    f, g = np.asarray(f), np.asarray(g)
    N, M = f.size, g.size
    if f.ndim != 1 or g.ndim != 1 or min(N, M) == 0:
        raise ValueError('Expected two nonempty vectors.')
    L = 1 << (2*M-2).bit_length()
    spectrum = np.fft.fft(g, L)
    result = np.zeros(N+M-1, dtype=complex)
    for start in range(0, N, M):
        block = f[start:start+M]
        values = np.fft.ifft(np.fft.fft(block, L)*spectrum)
        length = block.size+M-1
        result[start:start+length] += values[:length]
    return np.real_if_close(result) if np.isrealobj(f) and np.isrealobj(g) else result
```

Program 12. Overlap-add convolution, including an incomplete final block.

Solution to Exercise 3.5

1. Since $(Rx)[j] = x[j-1]$ modulo N,

$$(\Omega_N R x)[k] = \sum_j x[j-1]\omega_N^{-kj} = \omega_N^{-k} \sum_j x[j]\omega_N^{-kj} = (D\Omega_N x)[k].$$

   Multiply on the right by Ω_N^{-1}.
2. The first column of R^j is e_j, and every column is a cyclic shift of it; hence $C = \sum_{j=0}^{N-1} c_j R^j$. Conjugating by Ω_N gives $\sum_j c_j D^j$. Its kth diagonal entry is $\sum_j c_j \omega_N^{-kj} = \hat{c}[k]$. Applying this diagonalization to x gives the multiplication formula.
3. Directly, $(Cx)[i] = \sum_j c[i-j]x[j] = (c * x)[i]$. The DFT of this vector is $\hat{c} \cdot \hat{x}$, so inversion yields the same result.

Solution to Exercise 3.6

A negative signed frequency $k \in \{-N_0, \ldots, -1\}$ is stored at index $N + k$ in the original spectrum and at index $P + k$ in the padded spectrum. Both indices represent the same exponential after restriction to the appropriate grid. Thus inversion gives

$$f_0[j] = \frac{\eta}{P} \sum_{k=-N_0}^{N_0} \hat{f}[k \bmod N] e^{2\pi i k j/P} = p(j/P).$$

At $j = \eta m$, $j/P = m/N$, so DFT inversion gives $p(m/N) = f[m]$. The factor $\eta = P/N$ compensates for the inverse transform's new divisor. The result is the unique interpolant with signed frequencies $-N_0, \ldots, N_0$. Program 13 preserves complex input; it removes only roundoff-level imaginary parts when the input is real.

```
def trigonometric_interpolate(x, eta):
    x = np.asarray(x)
    n = x.size
    if x.ndim != 1 or n % 2 != 1 or not isinstance(eta, int) or eta < 1:
        raise ValueError('Expected an odd-length vector and an integer factor.')
    spectrum, middle = np.fft.fft(x), (n+1)//2
    padded = eta*np.r_[spectrum[:middle], np.zeros((eta-1)*n), spectrum[middle:]]
    y = np.fft.ifft(padded)
    return np.real_if_close(y) if np.isrealobj(x) else y
```

Program 13. Trigonometric interpolation by correctly scaled spectral padding.

Solution to Exercise 3.7

1. The identity $\cos((k+1)\theta) + \cos((k-1)\theta) = 2\cos\theta\cos(k\theta)$ gives the recurrence. Induction proves the degree assertion, with leading coefficient 2^{k-1} for $k \geq 1$. Substitution gives $T_N(x_j) = \cos((j+1/2)\pi) = 0$. These are N distinct points, so they exhaust the roots.
2. The polynomials $T_0, \ldots, T_{N-1}$ have distinct degrees and nonzero leading coefficients. They form a basis of the polynomials of degree at most $N-1$, giving the unique expansion of the unique interpolant.
3. Put $\theta_j = (j+1/2)\pi/N$. A finite geometric sum, or pairing conjugate exponentials, shows

$$\sum_{j=0}^{N-1} \cos(k\theta_j)\cos(l\theta_j) = \begin{cases} N, & k = l = 0, \\ N/2, & k = l > 0, \\ 0, & k \neq l, \end{cases} \qquad 0 \leq k, l < N.$$

   Thus the cosine rows, scaled by $N^{-1/2}$ for row zero and $(2/N)^{1/2}$ for the other rows, form an orthonormal matrix. This proves $C_2^{-1} = (2/N)C_3$. For C_2, place $v[k]$ at index $2k+1$ of a length-$4N$ zero array. The real part of its DFT at index j is exactly $C_2(v)[j]$. For C_3, pad v at the end to length $4N$, take the real DFT values at indices $2j+1$, and subtract $v[0]/2$. Programs 14 and 15 implement these two embeddings.

```
def dct2(x):
    x = np.asarray(x)
    if x.ndim != 1 or x.size == 0 or not np.isrealobj(x):
        raise ValueError('Expected a nonempty real vector.')
    n = x.size
    padded = np.zeros(4*n)
    padded[1:2*n:2] = x
    return np.fft.fft(padded).real[:n]
```

Program 14. The cosine transform C_2 via a length-$4N$ FFT.

```
def dct3(x):
    x = np.asarray(x)
    if x.ndim != 1 or x.size == 0 or not np.isrealobj(x):
        raise ValueError('Expected a nonempty real vector.')
    n = x.size
    return np.fft.fft(x, 4*n).real[1:2*n:2]-x[0]/2
```

Program 15. The cosine transform C_3 via a length-$4N$ FFT.

4. The interpolation conditions are

$$v[j] = \sum_{k=0}^{N-1} \alpha_k \cos(k\theta_j) = C_3(\alpha)[j] + \frac{\alpha_0}{2}. \tag{3.25}$$

Orthogonality gives $\alpha_0 = C_2(v)[0]/N$ and $\alpha_k = 2C_2(v)[k]/N$ for $k > 0$. The coefficients cost $O(N \log N)$ to compute. Evaluating the degree-$N-1$ series at each new point costs $O(N)$ using a recurrence, as in NumPy's `chebval`. Program 16 includes the Runge example from the exercise.

```
def chebyshev_interpolate(values, points):
    coefficients = 2*dct2(values)/len(values)
    coefficients[0] /= 2
    return np.polynomial.chebyshev.chebval(points, coefficients)
```

```
def chebyshev_example(n=16, alpha=.3):
    nodes = np.cos((np.arange(n)+.5)*np.pi/n)
    values = 1/(alpha**2+nodes**2)
    grid = np.linspace(-1, 1, 201)
    return nodes, values, grid, chebyshev_interpolate(values, grid)
```

Program 16. Chebyshev interpolation and the Runge example.

The Chebyshev nodes cluster near the endpoints, reducing the large endpoint oscillations seen with equally spaced nodes. Both constructions are polynomial interpolation; the difference is the node distribution, not whether Lagrange interpolation is being used.

Solution to Exercise 3.8

Integration by parts gives $\hat{f}'(\xi) = \mathrm{i}\xi\hat{f}(\xi)$, since the boundary term vanishes for a Schwartz function. Iterating gives $\widehat{f^{(m)}} = (\mathrm{i}\xi)^m \hat{f}$. The proposed multiplier extends these integer powers by fixing arguments $\pi/2$ and $-\pi/2$ on the positive and negative frequency axes. Its values at opposite frequencies are complex conjugates. Since real input also has conjugate-symmetric Fourier coefficients, the output is real. For noninteger α, this is a particular fractional differentiation operator; choosing a different multiplier would define a different operator.

For a period L, the discrete frequencies are $2\pi k/L$ with $k = -(N-1)/2, \ldots, (N-1)/2$ for odd N. Program 17 multiplies the DFT by the specified powers and inverts it. At integer orders this differentiates the trigonometric interpolant exactly in exact arithmetic. The odd sample count ensures that each nonzero frequency has a distinct negative partner, avoiding an ambiguous Nyquist slot. The case $\alpha = 0$ is explicitly the identity, including on the mean.

```
def fractional_derivative(f, alpha, period):
    f = np.asarray(f)
    n = f.size
    if f.ndim != 1 or n % 2 != 1 or alpha < 0 or period <= 0:
        raise ValueError('Use odd sample count, nonnegative order, positive period.')
    omega = 2*np.pi*np.fft.fftfreq(n, d=period/n)
    multiplier = np.ones(n, dtype=complex)
    if alpha > 0:
        multiplier = abs(omega)**alpha*np.exp(.5j*np.pi*alpha*np.sign(omega))
    y = np.fft.ifft(multiplier*np.fft.fft(f))
    return y.real if np.isrealobj(f) else y
```

Program 17. A periodic fractional derivative with explicit order and period.

For a rapidly decaying function on $\mathbb{R}$, the period must be large enough to make interactions with periodic copies small, and the spacing fine enough to resolve its Fourier content. A Gaussian is not strictly bandlimited; its decay allows an approximation, not an exact appeal to Shannon's theorem. Positive derivative order also amplifies high-frequency noise and roundoff. Negative orders require a separate treatment of the zero frequency and are outside the definition in this exercise.

Solution to Exercise 3.9

1. Unitarity implies normality. Since $U^2 f = f^\sharp$ and reversal is an involution, $U^4 = I$. Its spectrum is contained in $\{1, -1, \mathrm{i}, -\mathrm{i}\}$. The spectral theorem for normal matrices gives an orthonormal eigenbasis. The DFT matrix is symmetric but generally not Hermitian.

2. The orthogonal projectors satisfy $P_\lambda P_\mu = \delta_{\lambda\mu} P_\lambda$ and $\sum_\lambda P_\lambda = I$. Therefore

$$U^\alpha U^\beta = \sum_\lambda e^{\mathrm{i}(\alpha+\beta)\theta_\lambda} P_\lambda = U^{\alpha+\beta}.$$

   The multipliers have modulus one, proving unitarity, and $e^{\mathrm{i}\theta_\lambda} = \lambda$ gives $U^1 = U$. A fixed real phase is a consistent logarithm on each eigenspace; independently chosen roots need not have this additive dependence on the exponent.
3. The matrix J of cyclic reversal has entries $J_{ij} = 1$ when $i + j \equiv 0 \pmod N$, and zero otherwise. Thus $U^0 = I$ and $U^2 = J$: index zero stays fixed, and the other nonzero entries lie along the reversed diagonal. The intermediate matrices combine these two sparse patterns with U and U^3 through the projector formulas. For the unnormalized transform, $\Omega_N^\alpha = N^{\alpha/2} U^\alpha$ and $\Omega_N^2 = NJ$.
4. When the same phase is assigned throughout an eigenspace, its contribution is the scalar $e^{\mathrm{i}\alpha\theta_\lambda}$ times its orthogonal projector, which is basis-independent. Replacing θ_λ by $\theta_\lambda + 2\pi m_\lambda$ for fixed integers m_λ produces different families at noninteger exponents, while preserving the integer powers and the composition law. The program exposes these optional integer branch shifts. One could also choose different logarithm branches along separately chosen orthogonal directions within a repeated eigenspace; that is an additional choice, not an automatic consequence of diagonalization.

Program 18 uses $P_\lambda = \frac{1}{4}\sum_{j=0}^{3} \lambda^{-j} U^j$ and cyclic reversal. It avoids relying on a general eigenvalue routine to return orthogonal vectors inside repeated eigenspaces. Its projector computation needs only one FFT, since U^2 is reversal and U^3 is the reversal of U.

```
def fourier_components(x):
    x = np.asarray(x, dtype=complex)
    n = x.size
    if x.ndim != 1 or n == 0:
        raise ValueError('Expected a nonempty vector.')
    transformed = np.fft.fft(x)/np.sqrt(n)
    reverse = (-np.arange(n)) % n
    powers = np.stack([x, transformed, x[reverse], transformed[reverse]])
    eigenvalues = np.array([1, -1, 1j, -1j])
    weights = eigenvalues[:, None]**(-np.arange(4)[None, :])/4
    return weights @ powers
```

```
def fractional_dft(x, alpha, turns=(0, 0, 0, 0)):
    turns = np.asarray(turns)
    if turns.shape != (4,) or np.any(turns != np.round(turns)):
        raise ValueError('Choose one integer branch shift per eigenspace.')
    phases = np.array([0, np.pi, np.pi/2, -np.pi/2])+2*np.pi*turns
    return np.exp(1j*alpha*phases) @ fourier_components(x)
```

Program 18. Unitary intermediate DFTs from explicit spectral projectors.

Solution to Exercise 3.10

1. This follows from the real symmetric spectral theorem.
2. The circulant matrix Γ has first column $(-2, 1, 0, \ldots, 0, 1)$. Its DFT is $2\cos(2\pi k/N) - 2$, the diagonal of Δ. The convolution theorem gives $\Omega_N \Gamma = \Delta \Omega_N$. Taking transposes, using symmetry of all three matrices, gives $\Gamma \Omega_N = \Omega_N \Delta$. Adding shows $\Omega_N S = S \Omega_N$.
3. If A and B commute, each eigenspace of A is invariant under B. The restriction of a diagonalizable operator to an invariant subspace is diagonalizable: its minimal polynomial divides that of the operator, which has distinct linear factors. Diagonalize B on each eigenspace of A and combine the resulting bases. This proves simultaneous diagonalizability. It does not assert that an arbitrary eigenbasis chosen for A already consists of eigenvectors of B.
4. Cyclic reversal commutes with S: the two shifts are interchanged and the diagonal cosine is even. Thus the two parity subspaces are invariant. In the bases specified in the exercise, the shifts connect only

neighboring indices k and $k+1$. Their interior off-diagonal entries are one; coupling to e_0 or, for even N, to $e_{N/2}$ gives $\sqrt{2}$ in the symmetric block. When N is odd, coupling across the last pair contributes $+1$ or -1 to the final diagonal entry of the symmetric or antisymmetric block. Hence both blocks are tridiagonal, with every existing subdiagonal entry nonzero. There is a zero connection between the two blocks in the full transformed matrix; it is not an irreducible tridiagonal matrix.

5. Let T be a real symmetric tridiagonal matrix with subdiagonal entries $b_i \neq 0$. In $(T-\lambda I)v = 0$, the first row determines v_2 from v_1, and each successive row determines v_{i+1} from the preceding two coordinates. If $v_1 = 0$, every coordinate is zero. Thus each eigenspace has dimension at most one. Symmetry gives a full eigenbasis, so the eigenvalues are distinct. The condition concerns the *subdiagonal*, not the diagonal entries. It applies separately to each parity block, whose spectra may overlap. For $N = 4$, the independent vectors $(0,1,0,-1)$ and $(1,-1,-1,-1)$ both satisfy $Sv = -4v$, explicitly disproving simplicity of the full spectrum.
6. The Fourier matrix also preserves parity and commutes with S. It therefore preserves every one-dimensional eigenspace of each restricted block. The union of the two real orthonormal block eigenbases is consequently a common eigenbasis, even at eigenvalues shared between the blocks.

Program 19 uses this construction. It orders vectors by decreasing eigenvalue of S, resolving numerically equal values by putting the symmetric block first. It fixes each sign by making a largest-magnitude coordinate positive, using the first such coordinate in a tie. The rounding used solely for sorting prevents tiny roundoff differences from reordering an exact tie. The eigenvectors themselves are not rounded.

```
def fourier_eigenbasis(n):
    if n < 3:
        raise ValueError('This construction uses n >= 3.')
    eye = np.eye(n)
    S = np.roll(eye, 1, axis=0)+np.roll(eye, -1, axis=0)
    S += np.diag(2*np.cos(2*np.pi*np.arange(n)/n)-4)
    even, odd = [eye[:, 0]], []
    for k in range(1, (n+1)//2):
        even.append((eye[:, k]+eye[:, -k])/np.sqrt(2))
        odd.append((eye[:, k]-eye[:, -k])/np.sqrt(2))
    if n % 2 == 0:
        even.append(eye[:, n//2])
    vectors, values, parity = [], [], []
    for sign, basis in enumerate([even, odd]):
        B = np.column_stack(basis)
        eigenvalues, V = np.linalg.eigh(B.T @ S @ B)
        vectors.append(B @ V); values.extend(eigenvalues)
        parity.extend([sign]*len(eigenvalues))
    order = np.lexsort((parity, -np.round(values, 12)))
    V = np.column_stack(vectors)[:, order]
    pivots = np.argmax(abs(V), axis=0)
    V *= np.sign(V[pivots, np.arange(n)])
    return V
```

Program 19. A common eigenbasis obtained by diagonalizing the two parity blocks.

Solution to Exercise 3.11

1. Substitute $r = x - k$ in the translation sum to obtain

$$\mathcal{F}(k\top f)[t] = \sum_x f[x-k]\omega_n^{-tx} = \omega_n^{-tk}\hat{f}[t] = (k\perp\hat{f})[t].$$

   Multiplying $f[x]$ by ω_n^{-kx} instead gives $\hat{f}[t+k] = ((-k)\top\hat{f})[t]$. The sign in the second identity is essential.
2. The unitary transform maps each member of the translation orbit to the corresponding modulation of $\mathcal{F}f/\sqrt{n}$, preserving all inner products. In the reverse direction, the minus sign merely permutes the orbit indices. Using the unnormalized DFT without its scaling would multiply squared norms by n.

3. Write $\tilde{f}[x] = \overline{f[-x]}$. Then

$$\langle f,\, k\top f\rangle = \frac{1}{n}\sum_x f[x]\overline{f[x-k]} = \frac{1}{n}(f * \tilde{f})[k].$$

   Translation is unitary, so the whole orbit is orthonormal precisely when $f * \tilde{f} = n\delta_0$. Since $\hat{\tilde{f}} = \overline{\hat{f}}$, the convolution theorem makes this equivalent to $|\hat{f}[k]|^2 = n$ for all k.
4. The prescribed normalization gives exactly this constant magnitude. For modulations, the inner products are the Fourier coefficients of $|f|^2$ with normalized sums, so the orbit is orthonormal precisely when $|f[x]| = 1$ for every x. Thus $f[x]/|f[x]|$ is the analogous construction when no sample vanishes. Very small denominators make either phase normalization sensitive to perturbations, even when exact zeros are absent.
5. The same correlation calculation gives

$$\mathcal{G}(\varphi) = \frac{1}{n}\mathcal{F}^{-1}(\hat{\varphi}\cdot\overline{\hat{g}}).$$

   It costs $O(n\log n)$ and its output consists of the coefficients in the orthonormal basis of translates. Therefore $\varphi = \sum_k \mathcal{G}(\varphi)[k](k\top g) = \mathcal{G}(\varphi) * g$.

Program 20 implements the translation normalization with the required factor $\sqrt{n}$ and preserves complex-valued input.

```
def orthogonalize_translates(x):
    x = np.asarray(x)
    spectrum = np.fft.fft(x)
    if np.any(abs(spectrum) == 0):
        raise ValueError('Every Fourier coefficient must be nonzero.')
    y = np.fft.ifft(np.sqrt(x.size)*spectrum/abs(spectrum))
    return np.real_if_close(y) if np.isrealobj(x) else y
```

Program 20. Orthonormalizing cyclic translates for the normalized inner product.

CHAPTER 4

Applications of the discrete Fourier transform

The FFT turns Fourier identities into practical algorithms. This chapter connects the discrete and continuous transforms, then uses convolution to filter signals, analyze polygonal shapes, solve differential equations, and multiply polynomials and integers. Each application makes explicit which operations are exact and which introduce an approximation.

4.1 Connection with the Fourier transform on $\mathbb{R}$

The DFT is an exact invertible map on a finite-dimensional space. It can also approximate a continuous Fourier transform, provided the sampling interval, observation window, and normalization are chosen consistently. Keeping these two roles separate prevents several common numerical errors.

4.1.1 The continuous Fourier transform

To compare a finite DFT with a continuous transform, we first fix the latter's normalization and reconstruction formula. The character interpretation explains why the same exponential functions appear in both settings.

For $f \in L^1(\mathbb{R})$, use the angular-frequency convention

$$\hat{f}(\xi) = \int_{\mathbb{R}} f(t) e^{-\mathrm{i}\xi t} \mathrm{d}t. \tag{4.1}$$

The transform describes the complex amplitude of each frequency; its magnitude and phase carry different information. It extends by density to $L^2(\mathbb{R})$, with the Plancherel identity

$$\|\hat{f}\|_{L^2}^2 = 2\pi \|f\|_{L^2}^2, \qquad \|\hat{f}\|_{L^2} = \sqrt{2\pi}\|f\|_{L^2}.$$

Thus energy is conserved after normalization by $1/\sqrt{2\pi}$. See Rudin [52] for the analytic construction.

With the frequency convention fixed, integrability of the transform gives the following pointwise reconstruction formula almost everywhere.

Proposition 38 (Fourier inversion)**.** *If $f \in L^2(\mathbb{R})$ and $\hat{f} \in L^1(\mathbb{R})$, then almost everywhere*

$$f(x) = \frac{1}{2\pi} \int_{\mathbb{R}} \hat{f}(\xi) e^{\mathrm{i}x\xi} \mathrm{d}\xi. \tag{4.2}$$

The right-hand side gives a continuous representative of f.

The exponentials in this formula are precisely the continuous characters of the additive group of real numbers.

Proposition 39 (Characters of the real line)**.** *Every continuous homomorphism from $(\mathbb{R}, +)$ to $\Gamma = \{z \in \mathbb{C} : |z| = 1\}$ has the form $e_\gamma(t) = e^{\mathrm{i}\gamma t}$ for a unique $\gamma \in \mathbb{R}$. Consequently, $\gamma \mapsto e_\gamma$ is a group isomorphism $\mathbb{R} \simeq \hat{\mathbb{R}}$.*

Proof. Let χ be such a homomorphism. Since $\chi(0) = 1$, choose $h > 0$ small enough that $C = \int_0^h \chi(t)\mathrm{d}t \neq 0$. Integrating $\chi(s + t) = \chi(s)\chi(t)$ gives

$$\chi(s) = C^{-1} \int_s^{s+h} \chi(t)\mathrm{d}t.$$

The right-hand side is continuously differentiable. Differentiating the homomorphism identity at $t = 0$ yields $\chi'(s) = \chi(s)\chi'(0)$. With $\lambda = \chi'(0)$ and $\chi(0) = 1$, this differential equation gives $\chi(s) = e^{\lambda s}$. Differentiating $|\chi(s)|^2 = 1$ shows $\lambda + \overline{\lambda} = 0$, so $\lambda = \mathrm{i}\gamma$ for a unique real γ. □

For the circle $\mathbb{R}/(2\pi\mathbb{Z})$, periodicity restricts γ to the integers. Fourier inversion then becomes a Fourier series, with its convergence interpreted in an appropriate function space. These examples extend the finite-group inversion formula of Chapter 1; see also Dym and McKean [25, Chapter 4].

4.1.2 Approximating a continuous transform

A numerical Fourier transform depends on both the observation window and the spacing of the samples. We derive the DFT approximation from a quadrature rule so that its scale factors and sources of error are explicit.

Suppose that the observed samples are $f[n] = f(n\Delta)$ for $0 \le n < N$, where $\Delta > 0$ and $T = N\Delta$. If the contribution outside $[0, T]$ is negligible, a quadrature approximation is

$$\hat{f}(\xi) \approx \Delta \sum_{n=0}^{N-1} f[n] e^{-\mathrm{i}\xi n\Delta}.$$

This involves both truncation of the observation window and quadrature of the remaining integral. At $\xi_k = 2\pi k/T$ it becomes

$$\hat{f}(\xi_k) \approx \Delta \sum_{n=0}^{N-1} f[n] e^{-2\pi \mathrm{i} k n/N}. \tag{4.3}$$

Writing $\hat{f}[k]$ for the DFT of the sample vector, we obtain

$$\hat{f}(2\pi k/T) \approx \Delta \hat{f}[k \bmod N]. \tag{4.4}$$

The sum on the right is $2\pi/\Delta$-periodic in frequency; the continuous transform generally is not. For even N, a convenient set of frequency indices is $-N/2, \dots, N/2 - 1$. Standard FFT storage instead orders them as

$$0, 1, \dots, N/2 - 1, -N/2, \dots, -1.$$

The Nyquist frequencies $\pm\pi/\Delta$ give the same sampled exponential, so only one is stored. The function `numpy.fft.fftfreq` produces this ordering, and `fftshift` moves the negative frequencies to the left. For a window starting at a rather than zero, multiply the approximation by $e^{-\mathrm{i}\xi_k a}$.

4.1.3 Sampling and zero padding

Frequency range, frequency spacing, and the ability to resolve nearby components depend on different features of the data. Distinguishing these effects clarifies what additional measurements can achieve and what zero padding alone can provide.

Reducing Δ increases the unaliased frequency interval. Increasing the observation length T reduces the natural frequency spacing $2\pi/T$. These changes require additional measurements and have different effects. Appending zeros to an existing vector instead samples the *same finite Fourier sum* on a finer grid: padding to length M changes the spacing to $2\pi/(M\Delta)$, while the Nyquist limit remains π/Δ. It adds no information and cannot resolve features absent from the data. It is nevertheless useful for plotting spectra and designing filters. Figure 4.1 separates the effects of denser sampling and zero padding for an interval indicator. Exercise 4.9 applies the same distinction to frequency responses.

4.1.4 Time–frequency duality

Concentration in time and concentration in frequency constrain one another. We focus on compact support, where the obstruction is absolute and connects frequency localization with analyticity.

Proposition 40. *A nonzero function $f \in L^2(\mathbb{R})$ and its Fourier transform cannot both have compact support.*

Exercise 4.1 proves this by showing that a band-limited function extends to an entire function. In the finite setting, all supports are finite, so the appropriate analogue is the support-size uncertainty inequality. For example, the DFT of δ_0 is the constant vector 1: a single time sample occupies every frequency.

4.2 Filtering

Convolution models linear filtering that treats each location in the same way. Its Fourier representation lets us design the response frequency by frequency, while the convolution kernel describes its local action.

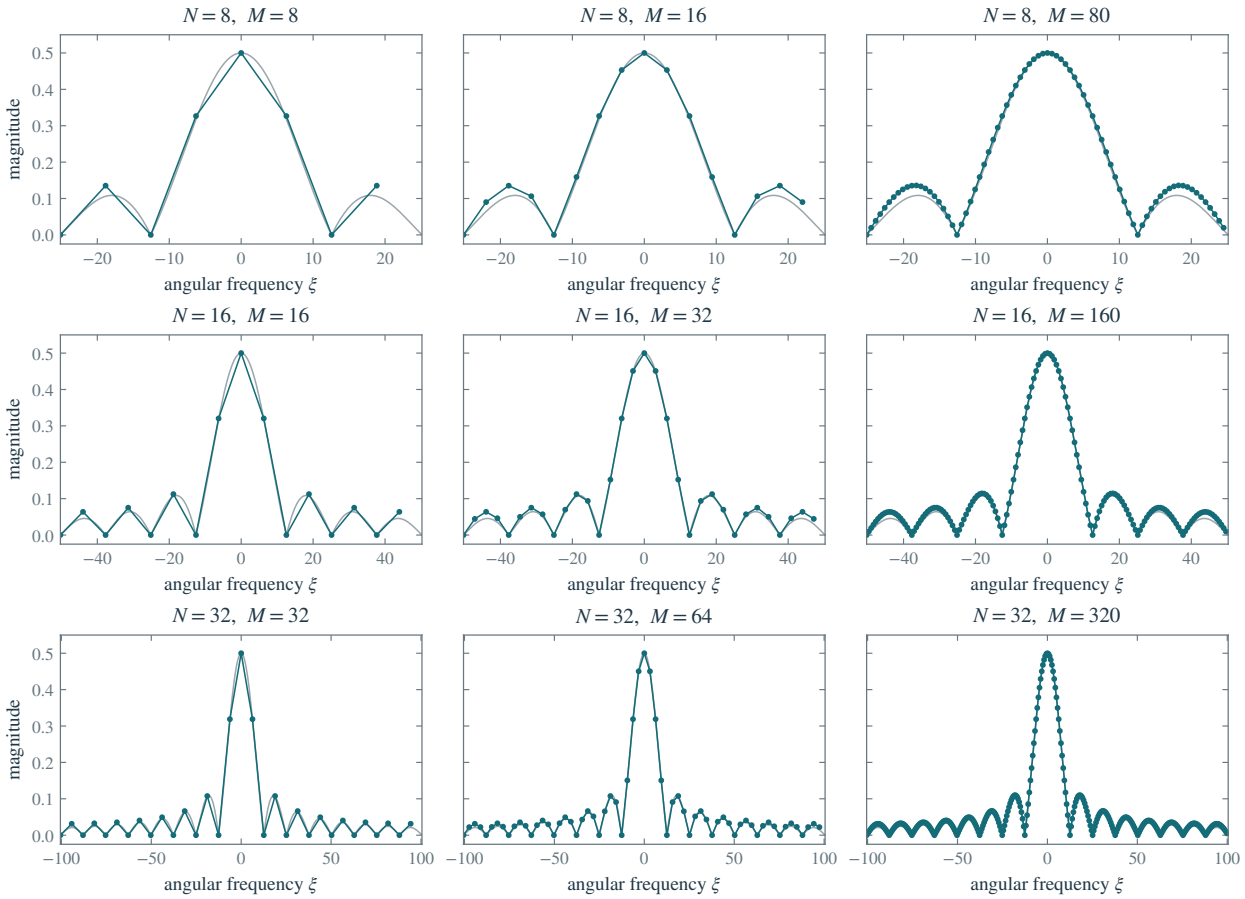


Figure 4.1. Sampling and zero padding for $f = \mathbf{1}_{[1/2,1)}$. Rows change the number N of measured samples; columns change only the padded length M. The DFT magnitudes include the quadrature factor $1/N$; the faint curve is the continuous-transform magnitude.

4.2.1 Linear filters

A translation-invariant filter is specified by how it combines shifted copies of the input. We begin with finite kernels, where this operation is well defined without convergence assumptions, and explain how to implement it on finite data.

For a finitely supported sequence g, define

$$\Phi^g(f)[n] = (f \star g)[n] = \sum_{k\in\mathbb{Z}} g[k]f[n-k].$$

This sum is well defined for every sequence f. The kernel g is the *impulse response*, since $\Phi^g(\delta_0) = g$. Its Fourier or Z transform, rather than g itself, is usually called the transfer function. Finite support makes this a *finite impulse response* (FIR) filter. Infinite kernels require a convergence condition, discussed below.

For data supported in an interval of N indices and a kernel supported in an interval of P indices, zero padding to at least $N + P - 1$ permits exact linear convolution by FFT. Store every index modulo the padded length, including negative kernel indices, then extract and order the desired output. For a short kernel, direct convolution can be cheaper than the FFT.

A useful example is a truncated discrete Gaussian,

$$g_\sigma[k] = \frac{e^{-k^2/(2\sigma^2)}}{\sum_{j=-r}^{r} e^{-j^2/(2\sigma^2)}}, \quad -r \le k \le r, \qquad g_\sigma[k] = 0 \quad (|k| > r).$$

It has unit sum and tends to δ_0 as $\sigma \downarrow 0$. Increasing σ spreads the averaging weights; the truncation radius should be enlarged with σ when an accurate Gaussian approximation is wanted. Figure 4.2 shows the resulting smoothing. A sampled sequence has no differentiability class; the familiar smoothness of continuous Gaussian convolution provides intuition rather than a literal regularity theorem for a finite vector.

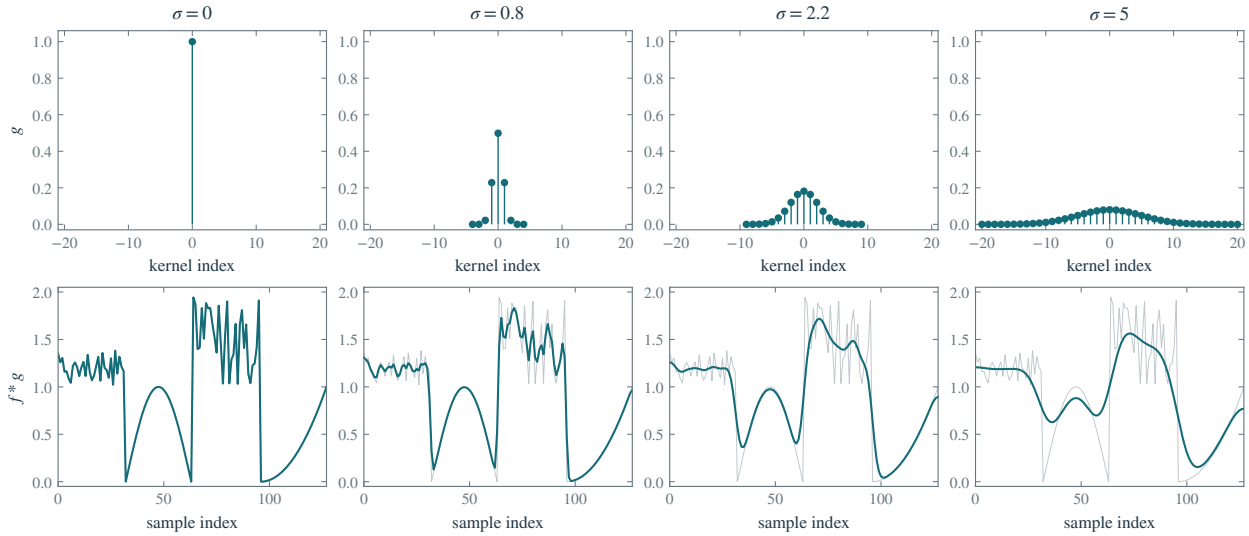


Figure 4.2. Gaussian kernels and their action on one fixed irregular signal. Widths are in sample units, and the signal is reflected at its endpoints. The first column is the exact identity kernel.

4.2.2 Responses and stability

A kernel can be understood through its action on impulses, sinusoids, or steps. These complementary descriptions guide filter design, while stability asks whether bounded input can ever produce unbounded output.

Besides its impulse response, a filter has two useful descriptions:

- Its *frequency response* is $G(\omega) = \sum_k g[k]e^{-\mathrm{i}k\omega}$. Indeed, filtering $f[n] = e^{\mathrm{i}\omega n}$ gives $G(\omega)e^{\mathrm{i}\omega n}$. The magnitude changes the amplitude and the argument changes the phase. For an FIR filter, zero padding evaluates this trigonometric polynomial on an arbitrarily fine frequency grid.
- Its *step response* is the output for $H[n] = 1$ when $n \geq 0$ and $H[n] = 0$ otherwise: $\Phi^g(H)[n] = \sum_{k \leq n} g[k]$. It reveals how a sharp transition is blurred or accompanied by oscillations. A finite vector of ones with zero extension contains two transitions, so only the part unaffected by the second transition represents this response.

Figure 4.3 compares impulse, frequency, and step responses. A filter with nonzero negative-index coefficients is noncausal. A narrow frequency transition typically requires a long or oscillatory impulse response; neither description alone captures every design tradeoff.

To control a filter's response to arbitrary bounded data, we use an operator notion of stability.

Definition 25 (Bounded-input bounded-output stability)**.** A convolution filter is stable if it defines a bounded operator on $\ell^\infty(\mathbb{Z})$.

The precise criterion is $g \in \ell^1(\mathbb{Z})$. Sufficiency follows from

$$\|\Phi^g(f)\|_\infty \leq \|g\|_1 \|f\|_\infty.$$

For necessity, choose $f[-k] = \overline{g[k]}/|g[k]|$ when $g[k] \neq 0$ and zero otherwise. The convolution at zero is then $\sum_k |g[k]|$. Equivalently, finitely supported truncations of this input show that a uniform operator bound forces these partial sums to remain bounded. The same test proves $\|\Phi^g\|_{\infty\to\infty} = \|g\|_1$. Every FIR filter is stable; recursive filters need not be. Section 5.2.2 returns to this distinction.

4.2.3 Image filtering

Smoothing an image trades noise reduction against the loss of fine detail. We extend Gaussian filtering to two dimensions and make the kernel scale and boundary treatment part of the model.

An image is an array of intensities, or several such arrays for color channels. The two-dimensional version of the Gaussian kernel is

$$g_\sigma[k,l] = C_\sigma e^{-(k^2+l^2)/(2\sigma^2)}, \qquad \sum_{k,l} g_\sigma[k,l] = 1,$$

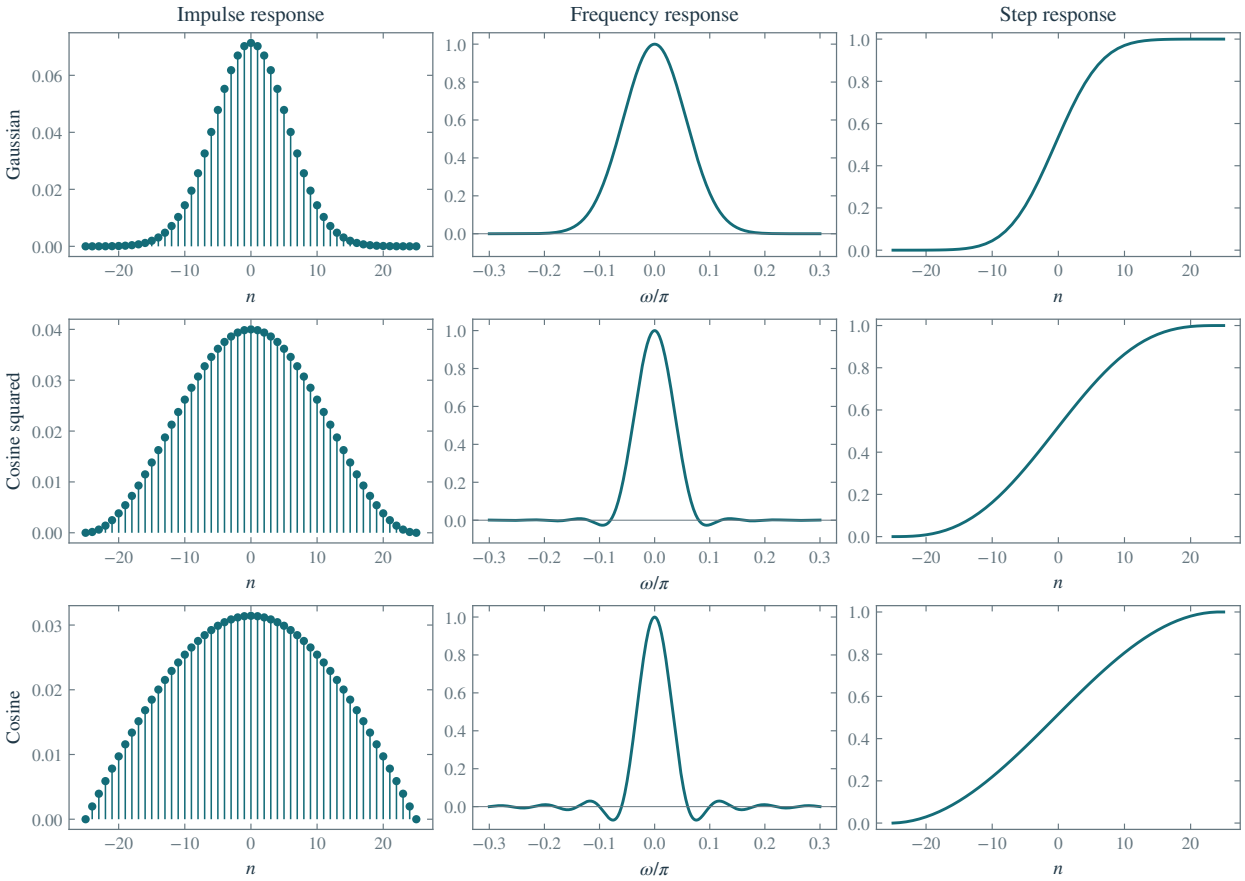


Figure 4.3. Impulse, frequency, and step responses of three unit-sum symmetric FIR kernels. The frequency plots show a neighborhood of zero; the step responses are cumulative sums of the impulse responses.

with a specified finite support and normalizing constant C_σ. Figure 4.4 compares kernels and the corresponding smoothed images. Figure 4.5 illustrates noise reduction. Averaging attenuates noise but also blurs edges and fine texture. Boundary handling is part of the model: zero extension darkens the border, periodic extension joins opposite edges, and reflection avoids such a jump when the image has no natural periodicity. Exercise 4.7 uses correlation to locate a smaller image inside a larger one.

4.3 Geometric aspects of filtering

A complex signal can also describe a polygon: its real and imaginary parts are the two coordinates of a vertex. Circular convolution then acts directly on shape, and Fourier coefficients expose its long-term behavior.

4.3.1 Polygon filtering

Repeated local averaging changes the shape of a polygon in ways that can be read from a few Fourier multipliers. We describe the iteration in complex coordinates and determine which frequencies persist, decay, or grow.

Represent a closed polygon with N ordered vertices by $\Pi \in \mathbb{C}^N$, joining $\Pi[j]$ to $\Pi[j+1 \bmod N]$. For a circular filter g, let

$$\Pi^{(0)} = \Pi, \qquad \Pi^{(m+1)} = g * \Pi^{(m)}. \tag{4.5}$$

Taking the DFT gives the exact formula

$$\widehat{\Pi^{(m)}}[k] = \hat{g}[k]^m \hat{\Pi}[k].$$

Only *active frequencies*, those with $\hat{\Pi}[k] \neq 0$, affect the result. If an active multiplier has modulus greater than one, the iterates are unbounded. If every active multiplier has modulus at most one, they are bounded. They

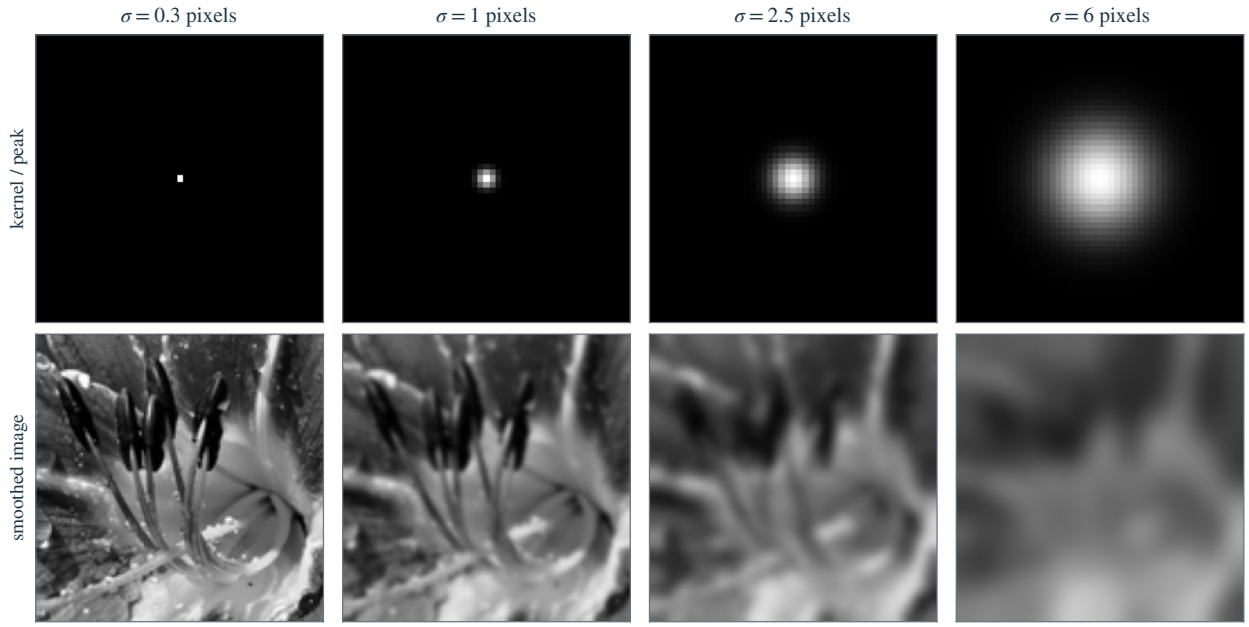


Figure 4.4. Discrete Gaussian kernels (top) and smoothing of the flower image (bottom), with reflected boundaries. Each kernel is displayed relative to its own peak; all image panels share the grayscale range $[0, 1]$.

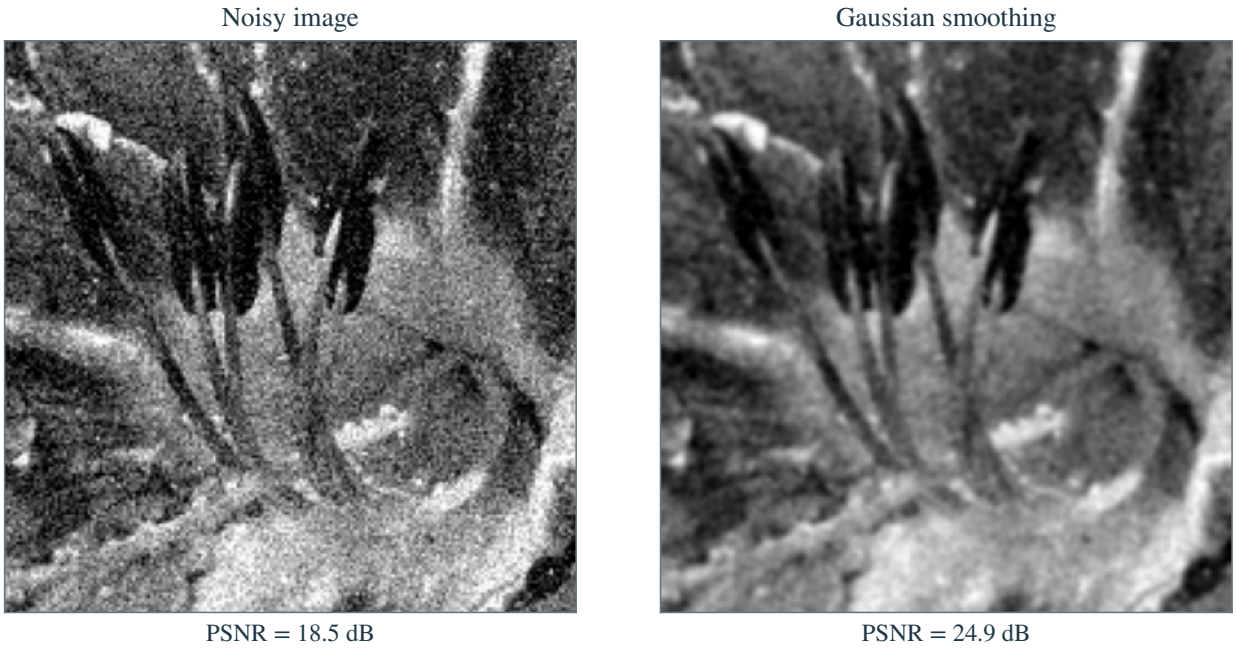


Figure 4.5. A flower image corrupted by additive Gaussian noise of standard deviation 0.12, and Gaussian smoothing with width 0.9 pixels. The displayed PSNR values use the same clean reference; intensity clipping is for display only.

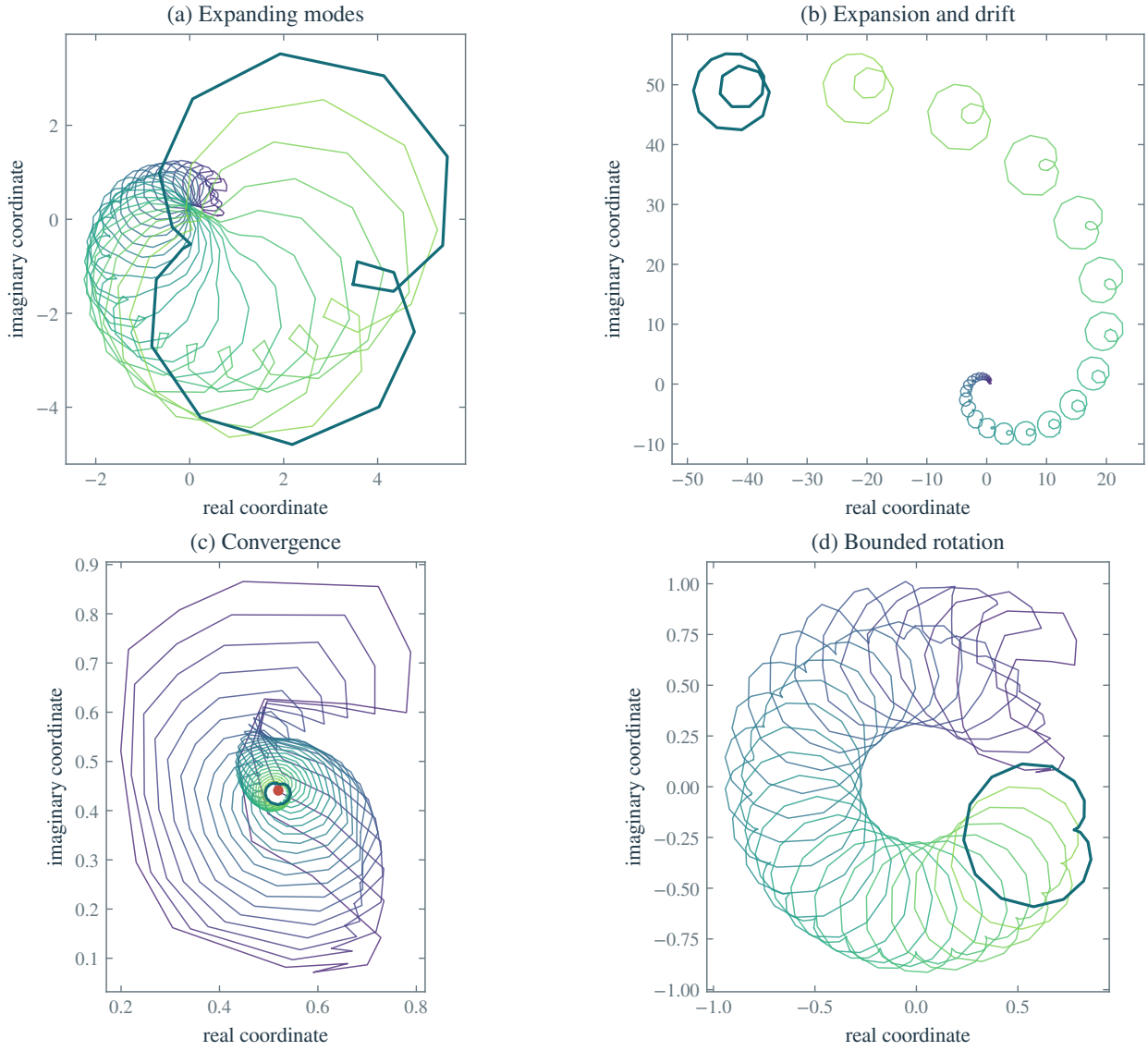


Figure 4.6. Four regimes of polygon filtering. Curves progress from the initial polygon to later iterates. Expanding active multipliers cause divergence; unit-modulus phases can keep the trajectory bounded without convergence.

converge precisely when each active multiplier either has modulus strictly less than one or equals one; in that case

$$\widehat{\Pi^{(\infty)}}[k] = \begin{cases} \hat{\Pi}[k], & \hat{g}[k] = 1, \\ 0, & \text{otherwise.} \end{cases}$$

Active multipliers on the unit circle other than one prevent convergence. When they are roots of unity, their surviving motion is periodic; decaying modes generally make the full trajectory only *asymptotically* periodic. Figure 4.6 illustrates these cases.

Repeatedly replacing adjacent vertices by their midpoint gives a simple illustration of the spectral convergence criterion.

Example 3 (Successive midpoints). For $g = (1/2, 1/2, 0, \dots, 0)$,

$$\Pi^{(1)}[j] = \tfrac{1}{2}(\Pi[j] + \Pi[j-1]), \qquad \hat{g}[k] = e^{-\mathrm{i}\pi k/N} \cos(\pi k/N).$$

The multiplier is one at $k = 0$ and has modulus less than one elsewhere. The vertices therefore converge to their arithmetic mean $c = N^{-1}\hat{\Pi}[0]$. This is the centroid of the vertices, which need not be the area centroid of the enclosed region.

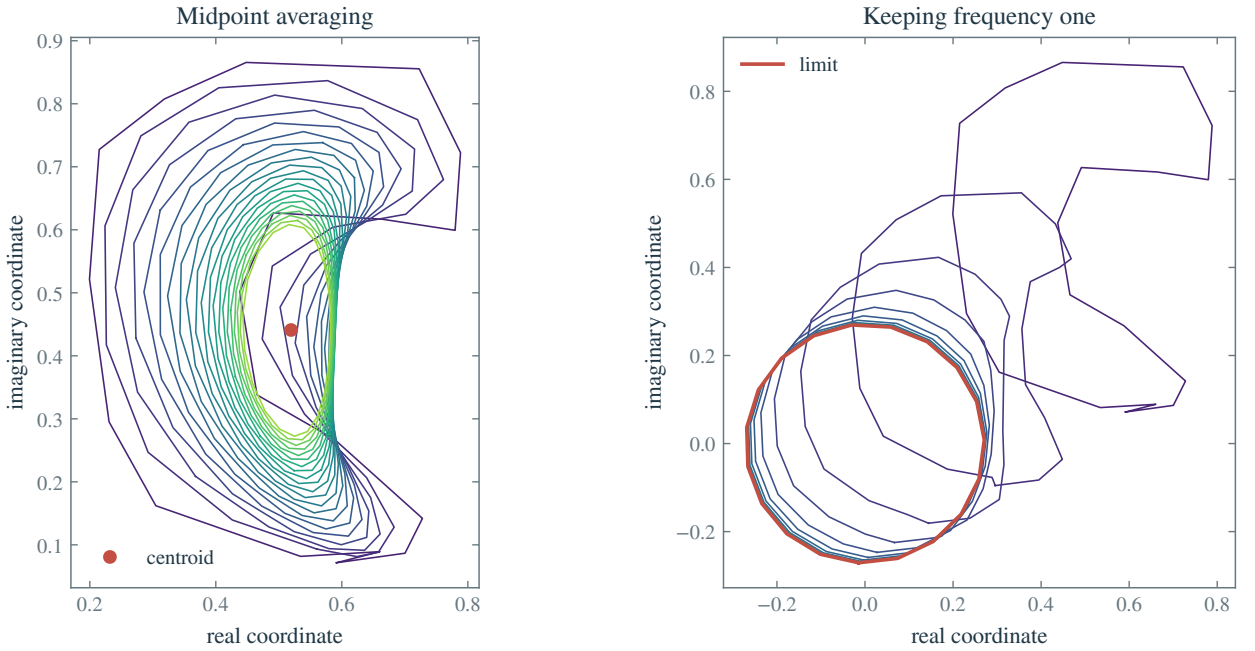


Figure 4.7. Successive midpoint averaging converges to the vertex centroid (left). Keeping only frequency one produces a regular polygon of radius $|\widehat{\Pi}[1]|/N$ (right). Red marks indicate the limits.

Retaining one nonconstant Fourier mode instead produces a nontrivial limiting shape when that mode is present initially.

Example 4 (Keeping a single frequency). If $\hat{g}[1] = 1$ and $\hat{g}[k] = 0.8$ for $k \neq 1$, then

$$\Pi^{(\infty)}[j] = N^{-1}\hat{\Pi}[1]e^{2\pi ij/N}.$$

For $\hat{\Pi}[1] \neq 0$, the limit is a regular polygon centered at zero with radius $|\hat{\Pi}[1]|/N$. If that coefficient vanishes, it collapses to zero.

4.3.2 Polygonal inequalities

Fourier coordinates relate a polygon's edge lengths, spread about its center, and signed area. We use these expressions to prove geometric bounds by comparing their contributions frequency by frequency.

The argument follows Terras [59]. For $N \geq 3$ and $c = N^{-1}\sum_j \Pi[j]$, set

$$S(\Pi) = \sum_j |\Pi[j+1] - \Pi[j]|^2,$$
$$T(\Pi) = \sum_j |\Pi[j] - c|^2,$$
$$A(\Pi) = \tfrac{1}{2}\sum_j \mathcal{I}\left(\overline{\Pi[j]}\Pi[j+1]\right).$$

The last expression is the signed area, including for self-intersecting polygons, where it counts regions with their winding multiplicities.

Expressing these three quantities in Fourier coordinates gives two bounds valid for every closed polygon.

Proposition 41. *For every such polygon,*

$$|A(\Pi)| \leq \tfrac{1}{2}T(\Pi), \qquad 4\sin^2(\pi/N)T(\Pi) \leq S(\Pi).$$

Proof. Let $z = \Pi - c$ and $Bz[j] = z[j+1]$. Translation leaves all three quantities unchanged, $\hat{z}[0] = 0$, and

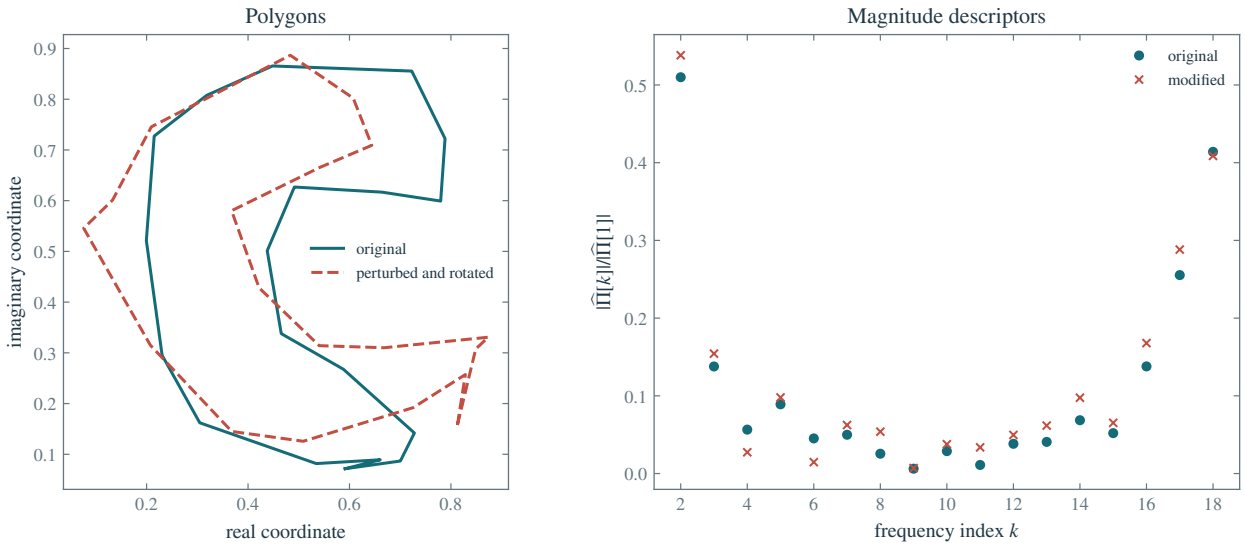


Figure 4.8. A polygon and a perturbed, rotated version, with their magnitude-based Fourier descriptors. The perturbation changes the descriptor; the rotation alone would leave it unchanged.

$\widehat{Bz}[k] = e^{2\pi ik/N}\hat{z}[k]$. Plancherel gives

$$T(\Pi) = \frac{1}{N}\sum_{k=1}^{N-1} |\hat{z}[k]|^2,$$

$$A(\Pi) = \frac{1}{2N}\sum_{k=1}^{N-1} \sin(2\pi k/N)|\hat{z}[k]|^2,$$

$$S(\Pi) = \frac{4}{N}\sum_{k=1}^{N-1} \sin^2(\pi k/N)|\hat{z}[k]|^2.$$

Use $|\sin(2\pi k/N)| \le 1$ in the second line and $\sin^2(\pi k/N) \ge \sin^2(\pi/N)$ in the third. □

4.3.3 Fourier descriptors

For shape comparison, a useful descriptor should ignore translation, scale, rotation, and the chosen starting vertex. We examine how these operations change the Fourier coefficients and identify both the resulting invariants and the information they discard.

A translation changes only $\hat{\Pi}[0]$. An orientation-preserving similarity $z \mapsto a + bz$, with $b \neq 0$, multiplies every other coefficient by b. A cyclic change of starting vertex multiplies coefficient k by a unit phase. These observations motivate the real, nonnegative descriptor

$$\mathcal{D}(\Pi) = \left(\frac{|\hat{\Pi}[2]|}{|\hat{\Pi}[1]|}, \dots, \frac{|\hat{\Pi}[N-1]|}{|\hat{\Pi}[1]|}\right), \qquad \hat{\Pi}[1] \neq 0.$$

It is invariant under these operations. A different nonzero reference coefficient can be used when the first one vanishes, provided the same choice is made in every comparison. Near a zero reference coefficient, normalizing by the total nonconstant spectral energy is more robust.

For polygons with the same number of vertices, define the descriptor *discrepancy* by $d(\Pi_1, \Pi_2) = \|\mathcal{D}(\Pi_1) - \mathcal{D}(\Pi_2)\|_2$. It is a pseudometric on this collection, not a distance that distinguishes all shapes: arbitrary changes of individual Fourier phases preserve it. It also does not automatically remove a reversal of vertex order or changes in sampling density. Figure 4.8 illustrates a useful comparison without claiming that equal descriptors imply equal shapes.

4.4 Numerical solution of partial differential equations

The Fourier transform converts differentiation in x into multiplication by $i\xi$. It therefore diagonalizes many constant-coefficient differential operators. In a computation, the boundary conditions determine which Fourier basis is appropriate: periodic exponentials for a circle, for example, or sines for homogeneous Dirichlet conditions.

4.4.1 Computing Fourier coefficients

A spectral solver needs Fourier coefficients of its initial data or source, but a computation usually starts from samples. We examine the periodic quadrature that connects the two and the aliasing error that remains.

For a continuous T-periodic function, let

$$c_k(f) = \frac{1}{T}\int_0^T f(t)e^{-2\pi ikt/T}\,\mathrm{d}t.$$

Sampling $f[n] = f(nT/N)$ gives

$$c_k(f) \approx \frac{1}{N}\sum_{n=0}^{N-1} f[n]e^{-2\pi ikn/N} = \frac{1}{N}\hat{f}[k \bmod N]. \tag{4.6}$$

For periodic data this is the composite trapezoidal rule, since the two endpoint half-weights combine. Its accuracy can be far better than a first-order rectangle estimate suggests. If the Fourier series is absolutely convergent, substitution and discrete orthogonality give the exact aliasing identity

$$\frac{1}{N}\hat{f}[k] = \sum_{\ell\in\mathbb{Z}} c_{k+\ell N}(f).$$

The error is the sum of the aliased coefficients. Smoothness gives rapid decay; analyticity in a strip gives exponential decay away from the sampling limit. See Trefethen and Weideman [60]. Increasing the sampling rate reduces this error. Zero padding alone does not produce additional independent Fourier-series coefficients of f.

4.4.2 The heat equation

Diffusion damps each Fourier mode at a rate determined by its frequency. This diagonal evolution gives an explicit solution of the periodic heat equation and a direct way to compute its smoothing effect.

On the unit circle $S^1 = \mathbb{R}/\mathbb{Z}$, consider

$$\partial_t u = \kappa\partial_{xx}u \quad (t > 0), \qquad u(0,x) = f(x), \qquad \kappa > 0. \tag{4.7}$$

For continuous f, we seek a solution continuous up to $t = 0$ and smooth for $t > 0$. Rescaling time reduces to $\kappa = 1/2$, which we use here. Writing $u(t,x) = \sum_n c_n(t)e^{2\pi inx}$ and integrating twice by parts gives

$$c_n'(t) = -2\pi^2 n^2 c_n(t), \qquad c_n(0) = c_n(f).$$

Hence

$$u(t,x) = \sum_{n\in\mathbb{Z}} c_n(f)e^{-2\pi^2n^2t}e^{2\pi inx}. \tag{4.8}$$

For $t \geq \epsilon > 0$, this series and all its space and time derivatives converge uniformly. Exercise 4.3 proves uniqueness and uniform convergence to continuous initial data as $t \downarrow 0$. Discontinuous data instead require an L^2 or pointwise interpretation; uniform convergence to a discontinuous function is impossible.

The solution is convolution on the circle with the *periodized* Gaussian heat kernel. The mean is preserved and every nonconstant mode decays, so the temperature becomes uniform. For the numerical solution, replace $c_n(f)$ by $\hat{f}[n]/N$ and multiply the stored frequencies by $e^{-2\pi^2n^2t}$. An inverse FFT evaluates the result on the original grid. No time-stepping error is introduced in this spectral evolution; the spatial sampling and truncation remain approximations. The Python routines in Section A.6 implement this method. Heat evolution is one particular family of low-pass filters. A general symmetric smoothing filter need not equal heat evolution at any time.

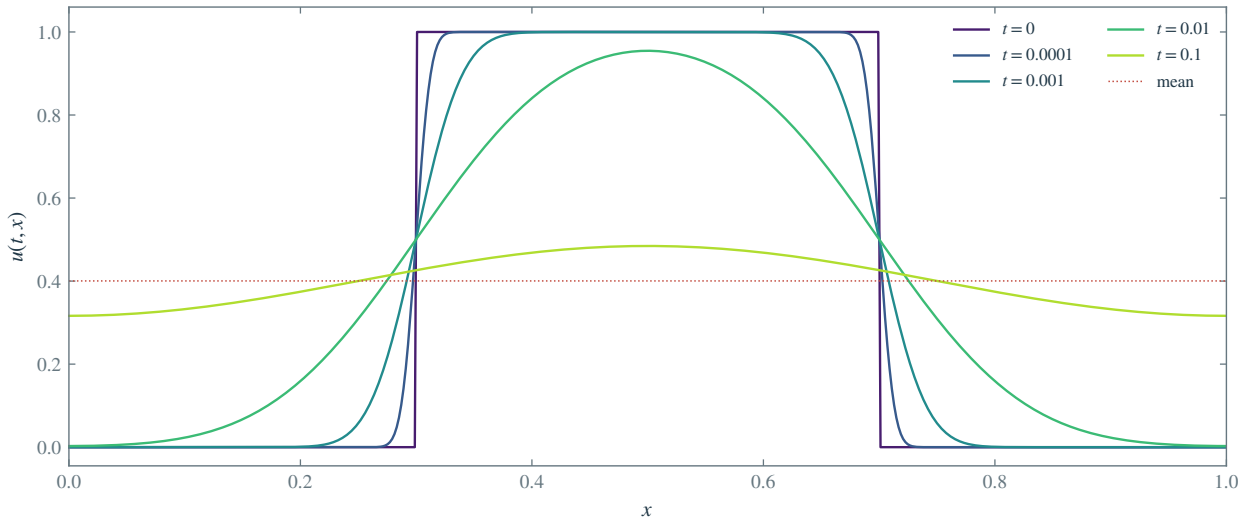


Figure 4.9. Spectral heat evolution of an interval indicator on the unit circle, with conductivity $\kappa = 1/2$. The mean is preserved and the nonconstant Fourier modes decay.

4.4.3 The Poisson equation with Dirichlet data

Prescribed boundary values change the basis needed to solve a Poisson problem. We discretize the equation on a square and use odd extensions to recover an FFT-based solver that respects the Dirichlet conditions.

On the square, consider

$$\Delta u = \partial_{xx} u + \partial_{yy} u = f \quad \text{in } (0,1)^2, \qquad u = b \quad \text{on } \partial[0,1]^2. \tag{4.9}$$

The boundary values are prescribed and agree at the corners. Poisson's equation models, among other problems, a deflected membrane or an electrostatic potential, with physical constants and signs incorporated into f. When $f = 0$, the solution is harmonic.

Let $h = 1/N$ and $U_{ij} \approx u(ih, jh)$ for $0 \le i, j \le N$. At the interior points use the five-point stencil

$$\frac{U_{i-1,j} + U_{i+1,j} + U_{i,j-1} + U_{i,j+1} - 4U_{ij}}{h^2} = F_{ij}, \qquad F_{ij} = f(ih, jh). \tag{4.10}$$

This is a second-order consistent approximation when u has four continuous spatial derivatives. Its boundary values are fixed, so it is *not* a circular convolution on the original square.

Move each boundary contribution to the right-hand side. For $1 \le i, j < N$,

$$\tilde{F}_{ij} = F_{ij} - h^{-2}\big(\mathbf{1}_{i=1} b_{0j} + \mathbf{1}_{i=N-1} b_{Nj} + \mathbf{1}_{j=1} b_{i0} + \mathbf{1}_{j=N-1} b_{iN}\big).$$

The unknown interior array now satisfies the same stencil with zero boundary values. Extend it, and $\tilde{F}$, oddly across each boundary and periodically with period $2N$ in each coordinate. Denote the extensions by U^o, F^o. For example, $F^o_{-i,j} = -F^o_{i,j}$ and $F^o_{i,-j} = -F^o_{i,j}$; values on $i = 0, N$ or $j = 0, N$ vanish. On the doubled grid the equation is exactly $\Phi * U^o = F^o$, with

$$\Phi[0,0] = -4h^{-2}, \qquad \Phi[\pm 1, 0] = \Phi[0, \pm 1] = h^{-2}, \tag{4.11}$$

and all other entries zero (indices are modulo $2N$). The convolution equation also holds on the reflection axes, where opposite neighbors cancel.

The symbol of this stencil is

$$\hat{\Phi}[k,l] = h^{-2}\big(2\cos(\pi k/N) + 2\cos(\pi l/N) - 4\big) = -4h^{-2}\big(\sin^2(\pi k/(2N)) + \sin^2(\pi l/(2N))\big).$$

Thus, except at $(k,l) = (0,0)$,

$$\widehat{U^o}[k,l] = -\frac{h^2 \widehat{F^o}[k,l]}{4\big(\sin^2(\pi k/(2N)) + \sin^2(\pi l/(2N))\big)}.$$

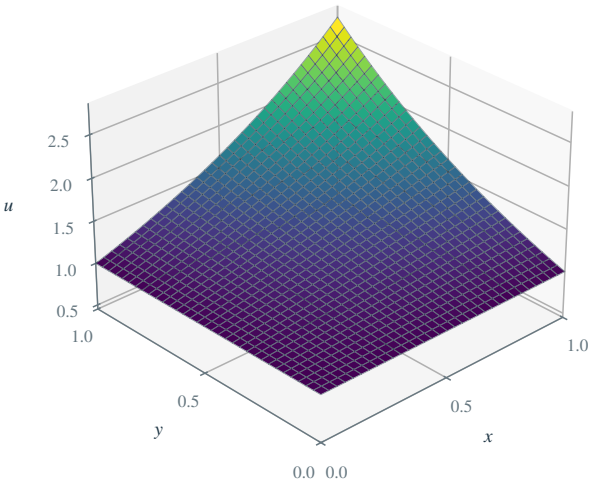


Figure 4.10. Dirichlet Poisson solution with reference $u(x, y) = e^{xy}$, on a grid with $N = 32$ intervals per axis.

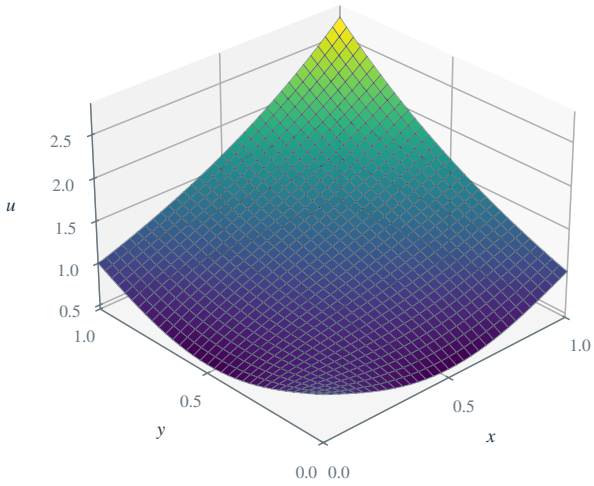


Figure 4.11. The same source with boundary values lowered by $0.45 \sin(\pi s)$ on $x = 0$ and $y = 0$. All four corner values remain compatible.

The zero mode is set to zero because an odd array has zero mean. All other denominators are positive. The inverse FFT recovers U^o; retain its interior on the original square and restore the prescribed boundary. Equivalently, a discrete sine transform works directly on the interior array; Exercise 4.5 proves the equivalence.

Section A.5 implements the solver. For validation, choose $u(x, y) = e^{xy}$, so $f = (x^2 + y^2)e^{xy}$, and prescribe its boundary values. Figure 4.10 shows this case; Figure 4.11 changes two edges while keeping compatible corners. For $u = x^2 + y^2$, the stencil gives $\Delta u = 4$ exactly, so the computed nodal solution agrees up to roundoff. In fact, centered second differences are exact for polynomials of degree at most three; this does not make them exact for general smooth data.

4.5 Fast products

Convolution also describes polynomial multiplication. The FFT therefore connects numerical signal processing to algebra and fast integer arithmetic; Bailey [5] gives historical examples involving high-precision computation.

4.5.1 Evaluation and interpolation

Polynomial multiplication is easy when polynomials are represented by their values at suitable nodes. Evaluation and interpolation provide the change of coordinates, with quotient algebras explaining exactly which product is recovered.

Choose *distinct* points $\xi_0, \ldots, \xi_{M-1}$ and set $Q(X) = \prod_{j=0}^{M-1}(X - \xi_j)$. Evaluation induces an algebra isomorphism

$$\Phi : \mathbb{C}[X]/(Q) \longrightarrow \mathbb{C}^M, \qquad [P] \longmapsto (P(\xi_0), \ldots, P(\xi_{M-1})),$$

where the target uses coordinatewise multiplication. Its kernel before passing to the quotient is (Q): a polynomial vanishes at all the distinct points if and only if their linear factors divide it. This is also the Chinese remainder theorem for the pairwise coprime ideals $(X - \xi_j)$. If points repeat, the kernel is generated by the product over the *distinct* points, and ordinary evaluations no longer give this isomorphism.

To multiply two polynomials modulo Q, evaluate both, multiply their values, then interpolate. The multiplication of values costs $O(M)$ operations; the usefulness of the method depends on the cost of the two changes of representation. Roots of unity make these changes fast.

4.5.2 Multiplication modulo $X^N - 1$

Roots of unity make both evaluation and interpolation fast through the DFT. The resulting product is naturally taken modulo $X^N - 1$, so its coefficients wrap around just as in circular convolution.

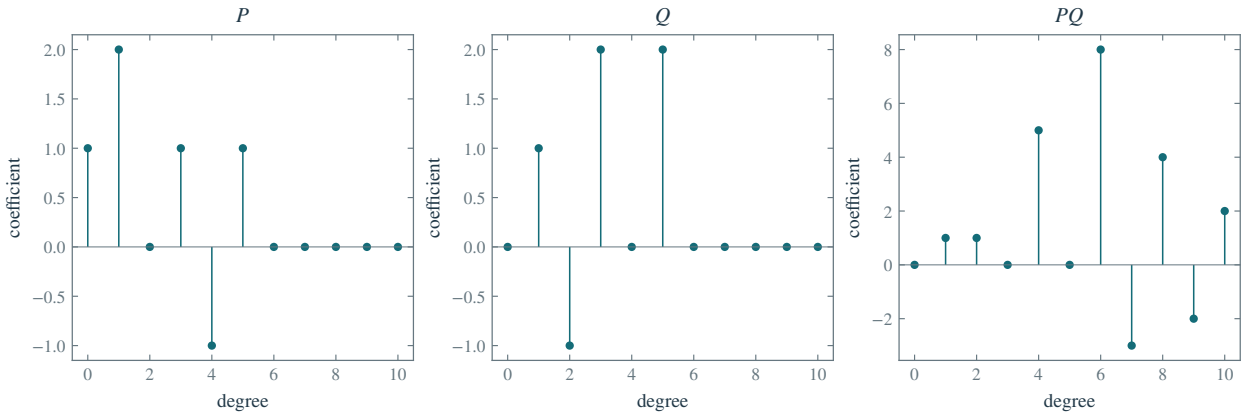


Figure 4.12. Coefficient vectors of P, Q, and their product. Length-eleven FFT convolution agrees with the ordinary degree-ten polynomial product.

For $\xi_k = e^{-2\pi ik/N}$, evaluation of a polynomial $P = \sum_{j=0}^{N-1} P_j X^j$ is exactly the DFT of its coefficient vector. Since $\prod_k (X - \xi_k) = X^N - 1$, we obtain

$$(PQ) \bmod (X^N - 1) = \mathcal{F}^{-1}\big(\mathcal{F}(P) \cdot \mathcal{F}(Q)\big). \tag{4.12}$$

The equality identifies a polynomial with its coefficient vector; on the right, multiplication is entrywise. In coefficient form the same operation is circular convolution. Figure 4.12 uses $P = 1 + 2X + X^3 - X^4 + X^5$ and $Q = X - X^2 + 2X^3 + 2X^5$. Padding to length eleven preserves their degree-ten product without wraparound.

For general distinct interpolation nodes, the Lagrange basis is

$$L_i(X) = \prod_{j \neq i} \frac{X - \xi_j}{\xi_i - \xi_j}, \qquad P(X) = \sum_i P(\xi_i) L_i(X).$$

Both products exclude $j = i$. Stable evaluation can use the barycentric form of this formula; divided differences provide another representation. The conditioning depends strongly on the nodes, as the Chebyshev example in Chapter 3 illustrates.

4.5.3 Ordinary polynomial multiplication

To recover the full polynomial product, the transform must leave room for every output coefficient. Zero padding prevents reduction modulo the transform polynomial from mixing those coefficients.

If $\deg P, \deg Q \le N - 1$, then $\deg(PQ) \le 2N - 2$ and

$$(PQ)_n = \sum_k P_k Q_{n-k}, \qquad 0 \le n \le 2N - 2, \tag{4.13}$$

where coefficients outside $0, \dots, N-1$ are zero. Pad both coefficient vectors to a transform length $M \ge 2N - 1$. Reduction modulo $X^M - 1$ then has no effect on their product, so circular convolution gives ordinary multiplication. A convenient smooth transform length yields an $O(M \log M)$ algorithm.

4.5.4 Multiplication of large integers

Integer multiplication has the same convolution structure as polynomial multiplication, followed by carry propagation. We explain this connection while distinguishing exact digit arithmetic from the rounding required by a floating-point FFT.

Write nonnegative integers in base $b \ge 2$ as

$$a = \sum_{j=0}^{N-1} a_j b^j, \qquad 0 \le a_j < b.$$

Their product is obtained by convolving the digit vectors and propagating carries. If a temporary coefficient is c_j, replace it by its remainder modulo b and add $\lfloor c_j / b \rfloor$ to the next position, continuing until the final carry vanishes. Section A.4 implements these steps in Python.

A floating-point FFT only approximates the convolution. Rounding recovers the integer coefficients when every error is strictly less than $1/2$; this needs a precision bound or independent verification, not merely a small observed distance to an integer. The examples check against Python's exact integers. Chapter 6 develops transforms over finite fields and rings, which permit exact modular arithmetic.

4.6 Exercises

These exercises connect the algebraic formulas with analytic proofs and numerical implementations. We begin by justifying the compact-support obstruction, then explore differential equations, image processing, and fast arithmetic.

A compactly supported spectrum forces analyticity, which rules out a second compact support in the original variable.

Exercise 4.1 (Compact support and analyticity)**.** Let $f \in L^2(\mathbb{R})$ and suppose $\operatorname{Supp}(\hat{f}) \subset [-A, A]$. Use Fourier inversion to construct a continuous representative of f and show that it is smooth. Express $f^{(n)}(t_0)$ as an integral, then expand the exponential to prove that

$$f(t) = \sum_{n=0}^{\infty} \frac{f^{(n)}(t_0)}{n!}(t - t_0)^n$$

for every real t. Justify the exchange of summation and integration. Deduce that a nonzero f cannot vanish on an open interval, and prove Proposition 40.

Fourier modes reveal when a time-stepping scheme for diffusion is stable and how strongly it damps each spatial frequency.

Exercise 4.2 (Finite differences for the heat equation)**.** On $S^1 = \mathbb{R}/\mathbb{Z}$, consider the conductivity-one equation

$$\partial_t u = \partial_{xx} u, \qquad u(0, x) = f(x). \tag{4.14}$$

Let $d = 1/N$, let $h > 0$ be the time step, and write $u^m[k] \approx u(mh, kd)$, with indices modulo $N \geq 3$.

1. Derive the explicit scheme, valid at *every* index $0 \leq k < N$,
$$u^{m+1}[k] = u^m[k] + s\big(u^m[k+1] + u^m[k-1] - 2u^m[k]\big), \qquad s = h/d^2.$$
2. Express one step as a circular convolution. Compare direct evaluation of the three-point stencil with applying an FFT at every step. Write a Python implementation. How can the FFT help if only the result after many steps is needed?
3. The scheme is stable if its powers are bounded for every initial vector. Determine its Fourier multipliers and the exact condition on a fixed grid. Show that $s \leq 1/2$ is necessary and sufficient for even N, and sufficient for every N. State the slightly different sharp bound for odd N.
4. For $0 \leq \theta \leq 1$, consider
$$u^{m+1} - u^m = A * \big(\theta u^{m+1} + (1-\theta)u^m\big), \qquad A[0] = -2s, \quad A[1] = A[-1] = s.$$
Find the Fourier multiplier and characterize stability. Identify the values of θ for which stability holds for every $s \geq 0$.
5. Extend the method to a periodic square grid. Derive the two-dimensional stability bound and implement the method by a two-dimensional FFT. Explain why a stable method can still exhibit sign-changing temporal oscillations of high-frequency modes.

Figure 4.13 compares explicit Euler, Crank–Nicolson, and backward Euler at a common large time step. The first is unstable; the second is stable but damps the highest frequencies poorly at that step; the third damps them strongly. Temporal oscillation and instability are not equivalent.

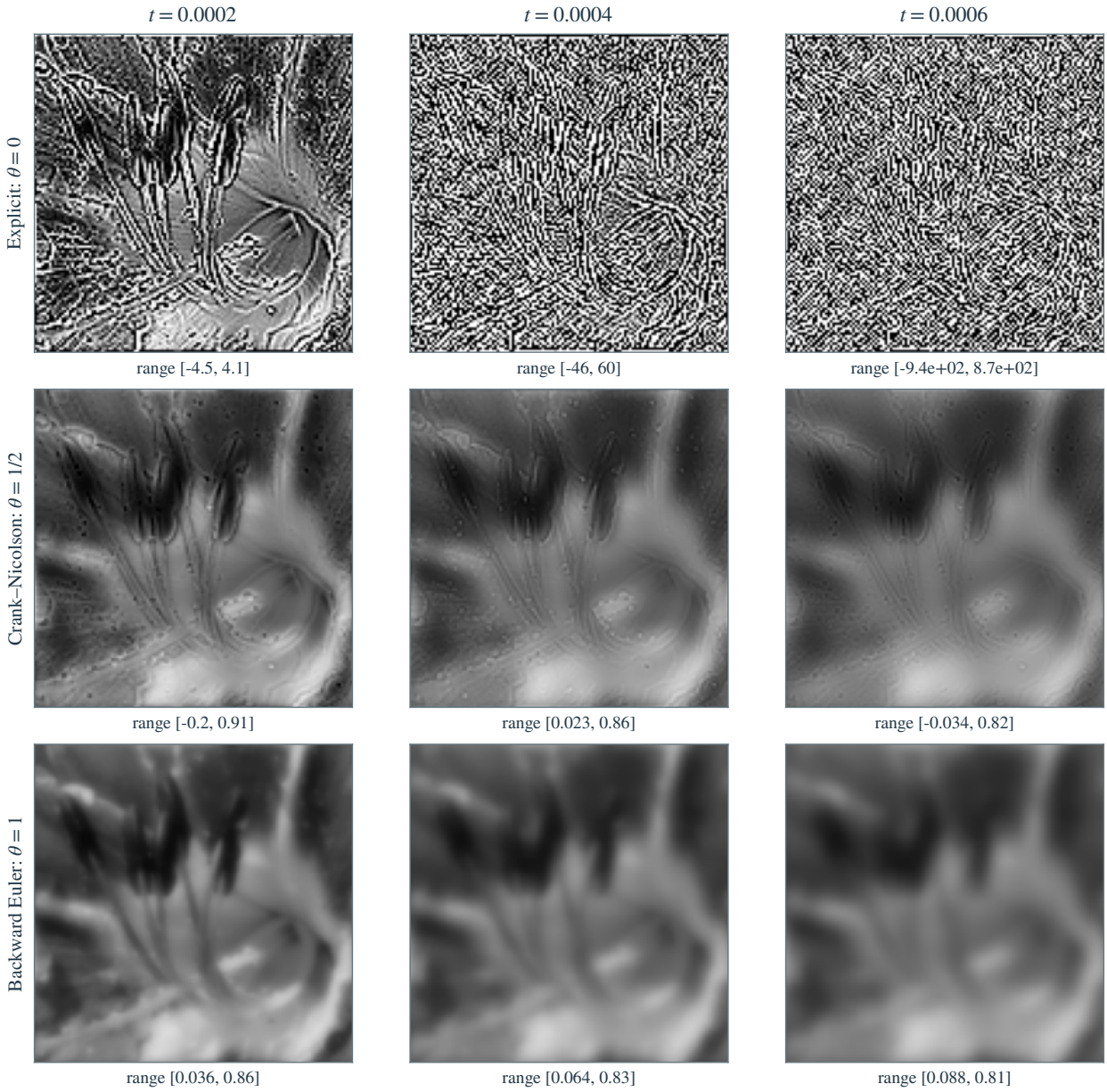


Figure 4.13. Three time steps for the same flower image, with $N = 128$ and $h = 0.0002$. Explicit Euler is unstable; Crank–Nicolson is stable but weakly damps high frequencies; backward Euler damps them strongly. All images use $[0, 1]$, and each panel reports its actual value range.

The explicit Fourier solution also supports a proof of existence and uniqueness for continuous initial data.

Exercise 4.3 (Existence and uniqueness for the heat equation)**.** Use the conductivity $\kappa = 1/2$ of (4.7), and suppose $f \in C(S^1)$. This argument follows Dym and McKean [25].

1. Show that (4.8) equals

$$u(t, x) = \int_0^1 p_t(x - y) f(y) \mathrm{d}y, \qquad p_t(x) = \sum_{n \in \mathbb{Z}} e^{-2\pi^2 n^2 t} e^{2\pi i n x}.$$

   Justify differentiation for $t > 0$.
2. For $f \in C^2(S^1)$, prove $\|u(t, \cdot) - f\|_\infty \to 0$ as $t \downarrow 0$ using the decay of its Fourier coefficients.
3. Prove positivity for any real classical solution continuous up to $t = 0$: if $f \geq 0$, then $u \geq 0$. Apply a minimum argument to $v(t, x) = e^{\beta t} u(t, x)$ with $\beta < 0$. At a negative space-time minimum (t_1, x_1) with $t_1 > 0$, use $0 \geq v_t = \beta v + \frac{1}{2} v_{xx} \geq \beta v > 0$.

4. Deduce that $p_t \geq 0$, that $\int_0^1 p_t = 1$, and that $\|p_t * f\|_\infty \leq \|f\|_\infty$. Prove uniqueness among classical solutions continuous up to their initial data, without assuming in advance that every solution has the convolution representation.
5. Approximate a continuous f uniformly by smooth periodic functions to conclude $\|u(t,\cdot) - f\|_\infty \to 0$.

The odd-extension construction extends coordinate by coordinate to a three-dimensional Dirichlet problem.

Exercise 4.4 (A three-dimensional Poisson problem)**.** Generalize the Dirichlet solver of Section 4.4.3 to $\Delta u = f$ on the unit cube. Derive the seven-point stencil, transfer the prescribed boundary values to the right-hand side, and extend the interior array oddly in all three coordinates. Give the Fourier symbol and a Python implementation of the extension. To visualize an electric potential, one may plot its level surfaces $u(x,y,z) = \lambda$, rather than level surfaces of the source f.

The same solver can be understood through the eigenvectors of a tridiagonal matrix, making its connection with sine transforms explicit.

Exercise 4.5 (Matrix formulation of the Poisson solver)**.** This exercise relates the FFT construction to matrix diagonalization; see Swarztrauber and Sweet [57]. Let $T = h^{-2}\,\mathrm{tridiag}(1,-2,1)$ have size $N-1$.

1. Show that the interior equation with zero boundary values is $TU + UT = F$. Explain which coordinate each multiplication acts on.
2. With nonzero prescribed boundary values, derive
$$T\tilde{U} + \tilde{U}T = \tilde{F}, \tag{4.15}$$
where $\tilde{F}$ includes the boundary corrections of the main text.
3. Show that $V_{ij} = \sin(\pi ij/N)$, $1 \leq i,j < N$, diagonalizes T. Find its eigenvalues λ_j, including the factor h^{-2}, and solve the transformed equation entrywise.
4. Prove $V^TV = (N/2)I$, so $Q = \sqrt{2/N}V$ is orthogonal. Relate multiplication by V to the sine transform $\mathcal{S}(x)[i] = \sum_{j=1}^{N-1} x[j]\sin(\pi ij/N)$.
5. Express $\mathcal{S}$ as a length-$2N$ DFT of an odd extension. Give its inverse and its two-dimensional version, with all normalization factors. Explain why this is equivalent to the doubled-grid solver.

A reproducible smoothing experiment must specify both the Gaussian kernel and the treatment of image boundaries.

Exercise 4.6 (Image smoothing)**.** Implement Gaussian smoothing of a grayscale image. Specify the kernel normalization, its truncation radius, and the boundary extension. Verify that a constant image remains constant with reflection boundaries, and compare the effect of several smoothing widths as in Figure 4.5.

Expanding a patch-matching error separates local energy from correlation and shows when FFT correlation is an appropriate search tool.

Exercise 4.7 (Correlation and template matching)**.** Let $f \in \mathbb{R}^{N\times N}$ be an image and $g \in \mathbb{R}^{P\times P}$ a template, with $1 \leq P \leq N$. Only consider positions $0 \leq u,v \leq N-P$, so that the entire template lies inside the image. Define
$$d(f,g)[u,v]^2 = \sum_{i,j=0}^{P-1} \big(f[u+i,v+j] - g[i,j]\big)^2.$$

1. Expand this expression using the local energy $E_f[u,v] = \sum_{i,j} f[u+i,v+j]^2$ and the correlation $\mathrm{Corr}(f,g)[u,v] = \sum_{i,j} f[u+i,v+j]g[i,j]$. When does maximizing correlation minimize d?
2. Express correlation as linear convolution with a reflected template. Explain the padding and output indices required for an FFT implementation.
3. Let $\mu_f[u,v]$ and μ_g be the patch and template means. Define normalized correlation by
$$\rho[u,v] = \frac{\sum_{i,j}(f[u+i,v+j] - \mu_f[u,v])(g[i,j] - \mu_g)}{\sqrt{\sum_{i,j}(f[u+i,v+j] - \mu_f[u,v])^2}\sqrt{\sum_{i,j}(g[i,j] - \mu_g)^2}}.$$
State when this expression is defined. Prove $|\rho| \leq 1$, characterize $\rho = 1$, and describe its behavior under affine intensity changes.

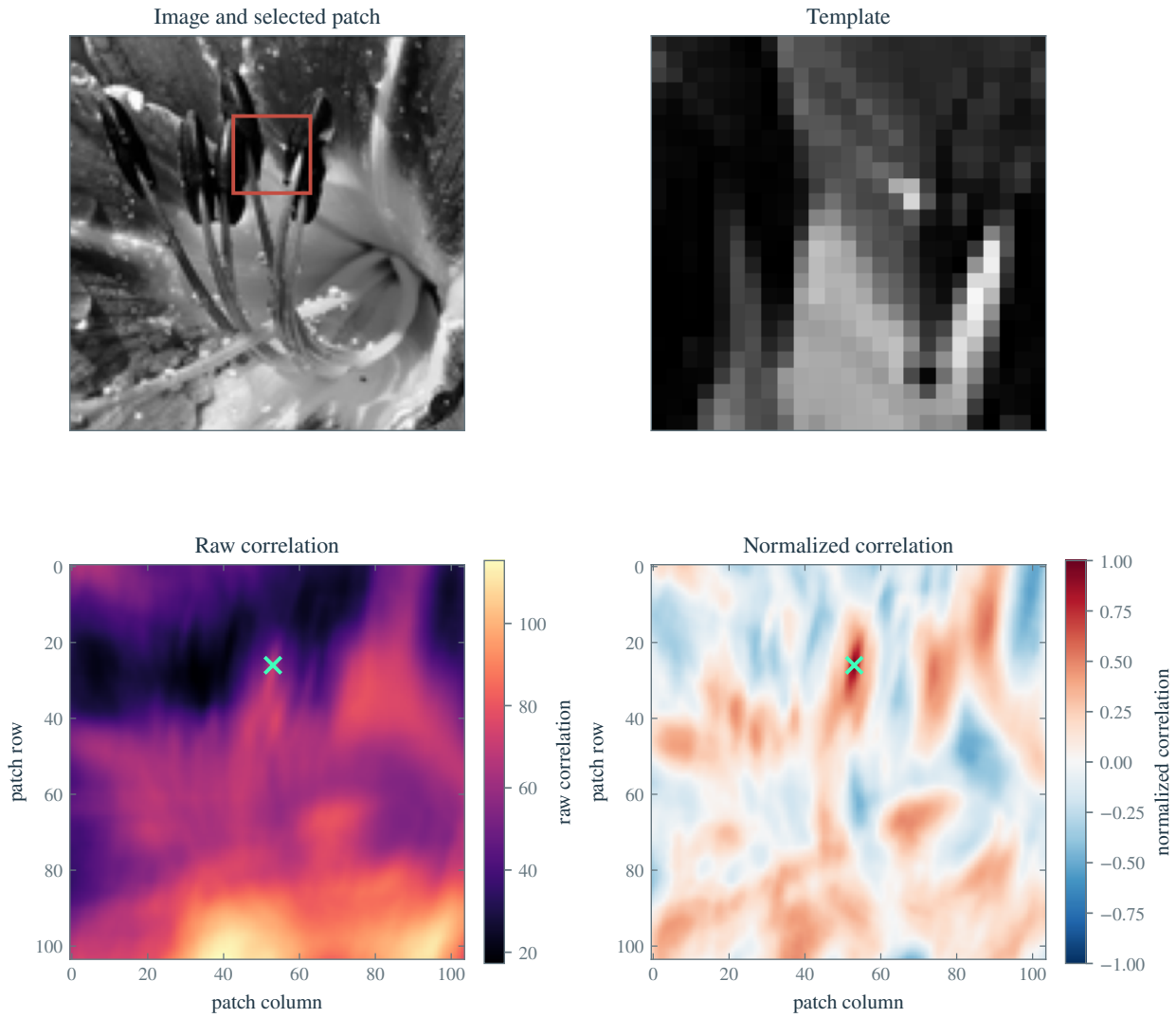


Figure 4.14. Template matching in the flower image. The red box identifies the extracted patch; crosses mark its location in the raw and normalized correlation maps. Only complete-template positions are included.

4. Express the numerator as a correlation with the centered template. For $k = 1, 2$, set $s_k[u, v] = \sum_{i,j=0}^{P-1} f[u+i, v+j]^k$. With zero extension outside the image, establish the recurrence

$$s_k[u,v] = s_k[u+1,v] + s_k[u,v+1] - s_k[u+1,v+1] \\ + f[u,v]^k + f[u+P,v+P]^k - f[u+P,v]^k - f[u,v+P]^k.$$

   Alternatively, derive a summed-area-table formula. Use these sums to compute all patch variances in $O(N^2)$ work, then implement normalized correlation. Do not assign a mathematically meaningful correlation to a constant patch.

Figure 4.14 illustrates the distinction between raw and normalized correlation. Lewis [39] discusses fast template matching.

Fractional translations implemented by Fourier phases can be combined into shears and image rotations.

Exercise 4.8 (Translation, shears, and rotation by FFT)**.** Take an odd sample count N in each coordinate. Interpret noninteger samples through periodic trigonometric interpolation on the centered grid. For a continuous image, define

$$T_v f(x,y) = f(x - v_1, y - v_2), \quad S_\lambda^{(x)} f(x,y) = f(x - \lambda y, y), \quad S_\lambda^{(y)} f(x,y) = f(x, y - \lambda x).$$

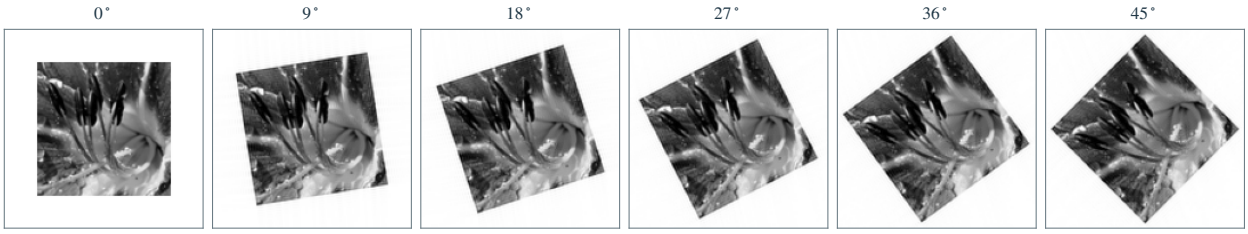


Figure 4.15. Rotations of the flower image by three Fourier shears. The odd grid avoids a Nyquist ambiguity; a white margin limits wraparound. Cropping and intensity clipping are applied only to the displayed results.

1. Derive the Fourier phase multiplier for a translation. Apply one-dimensional translations along rows or columns to implement each shear. Why does the use of signed frequency indices matter for fractional shifts?
2. For $|\theta| < \pi$, prove

$$\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} = \begin{pmatrix} 1 & -\tan(\theta/2) \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ \sin\theta & 1 \end{pmatrix} \begin{pmatrix} 1 & -\tan(\theta/2) \\ 0 & 1 \end{pmatrix}.$$

   Deduce a three-shear algorithm for the pullback rotation $R_\theta f(z) = f(\mathcal{R}_{-\theta} z)$.
3. Explain how centering and padding reduce wraparound artifacts. Show that each discrete shear is unitary and invertible when complex values are retained. Why does the finite-grid three-shear rotation generally fail the exact composition law $R_\alpha R_\beta = R_{\alpha+\beta}$? Discuss large angles and the need to avoid $\tan(\theta/2)$ near its poles.

Designing a filter in frequency coordinates exposes the tradeoff between a sharp cutoff and localization of its impulse response.

Exercise 4.9 (Designing a low-pass filter)**.** Use an odd length N and signed frequencies $\omega_k = 2\pi k/N$.

1. Construct a real circular kernel whose frequency response is one for $|\omega_k| < \pi/2$ and zero for $|\omega_k| > \pi/2$. Specify a half-value at a cutoff frequency if it is present.
2. Regard the centered kernel as a finite sequence on $\mathbb{Z}$ and plot its frequency response on a fine grid by zero padding. Compare the resulting trigonometric polynomial with the prescribed samples.
3. Replace the sharp cutoff by a raised-cosine transition of width $\epsilon\pi$ centered at each cutoff, for $0 \le \epsilon \le 1$. Compare the impulse responses and the frequency-domain oscillations.

A sharing process on a circle illustrates how averaging, periodicity, and integer rounding affect convergence.

Exercise 4.10 (A candy-sharing process)**.** Place $n \ge 3$ children around a circle, each holding a nonnegative even number of candies. At each simultaneous step, every child keeps half and gives half to the child on the right.

1. After each step, give one extra candy to every child whose new count is odd. Prove that after finitely many steps all counts are equal. Track the minimum, maximum, and the set of children attaining the minimum.
2. If fractional candies are allowed and no extra candies are supplied, express the iteration as convolution. Prove convergence to the original arithmetic mean, without claiming finite-time equality.
3. Now send half to each neighbor and retain nothing. Analyze convergence with and without the rounding rule. Distinguish odd and even n, and give an even-cycle counterexample.

Karatsuba's method offers another route to fast multiplication, based on reducing the number of recursive products.

Exercise 4.11 (Karatsuba multiplication)**.** Let P, Q have degree less than n over a field, and split their coefficient vectors after $k = \lfloor n/2 \rfloor$ entries: $P = P_0 + X^k P_1$, $Q = Q_0 + X^k Q_1$.

1. Write $PQ = R_0 + X^k R_1 + X^{2k} R_2$ and identify the three terms.

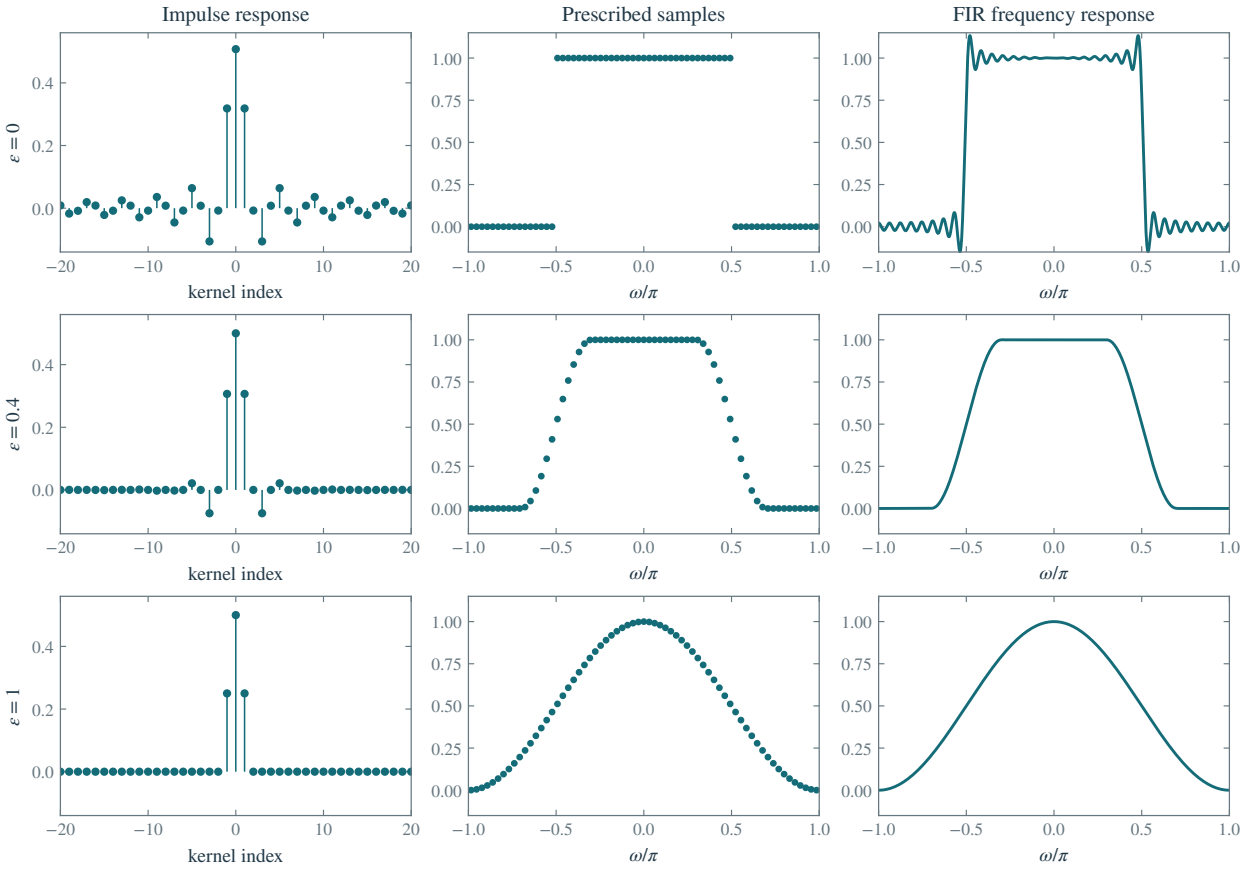


Figure 4.16. Low-pass filters with increasingly wide raised-cosine transitions. The columns show the centered impulse response, the prescribed DFT samples, and the densely sampled FIR frequency response.

2. Given $R_0 = P_0Q_0$ and $R_2 = P_1Q_1$, compute R_1 with one additional multiplication and four polynomial additions or subtractions.
3. Implement the recursion, including unequal intermediate lengths. Prove the bound $O(n^{\log_2 3})$ by padding to a power of two. Explain why the crossover with schoolbook or FFT multiplication depends on the implementation, coefficient arithmetic, and required exactness.

Spline interpolation can also be formulated as filtering: the coefficients are chosen so that the reconstructed function agrees with the samples.

Exercise 4.12 (Spline interpolation as filtering)**.** For $a \in \ell^2(\mathbb{Z})$, write $\widehat{a}(\xi) = \sum_{k\in\mathbb{Z}} a[k]e^{-\mathrm{i}k\xi}$, with convergence in $L^2(\mathbb{R}/(2\pi\mathbb{Z}))$. Given samples $u_d \in \ell^2(\mathbb{Z})$, compare

$$v(x) = \sum_{k\in\mathbb{Z}} u_d[k]\varphi(x-k), \tag{4.16}$$

$$v(x) = \sum_{k\in\mathbb{Z}} a[k]\psi(x-k), \tag{4.17}$$

where $v(j) = u_d[j]$ is required at every integer j. Assume initially that ψ is continuous and compactly supported.

1. Let $\psi_d[k] = \psi(k)$ and suppose its trigonometric polynomial $\widehat{\psi_d}$ has no zeros. Prove existence and uniqueness of $a \in \ell^2$ and derive $\widehat{a} = \widehat{u_d}/\widehat{\psi_d}$. Construct an equivalent cardinal kernel φ. Why can it have infinite support even though ψ has compact support?
2. Define centered B-splines of degree n by $\beta^0 = \mathbf{1}_{[-1/2,1/2)}$ and $\beta^n = \beta^0 * \beta^{n-1}$. Find their supports. Which degrees give direct interpolation without a prefilter? Figure 4.17 shows the first four degrees.
3. Compute the continuous transform of β^n and the periodic transform $D_n = \widehat{\beta^n_d}$ of its integer samples. For $n \geq 1$, use Poisson summation without introducing an extra factor 2π. Prove $D_n > 0$, separating

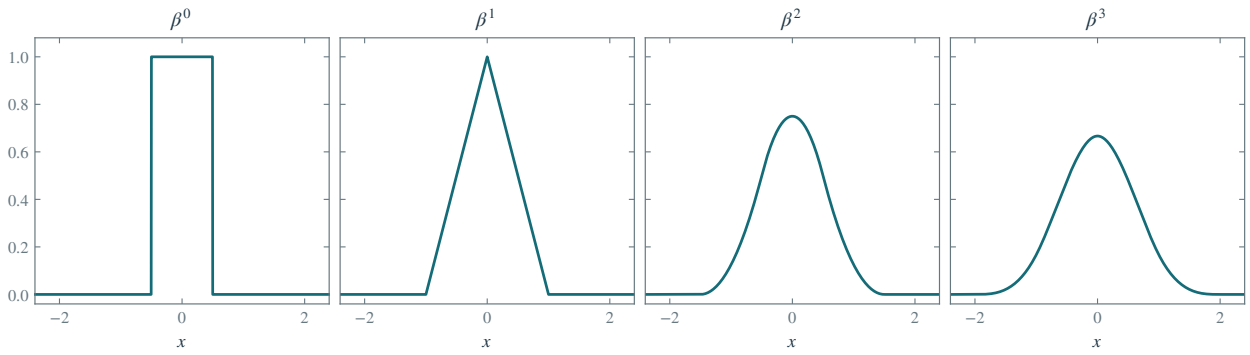


Figure 4.17. Centered B-splines of degrees zero through three. Their support is $[-(n+1)/2, (n+1)/2]$.

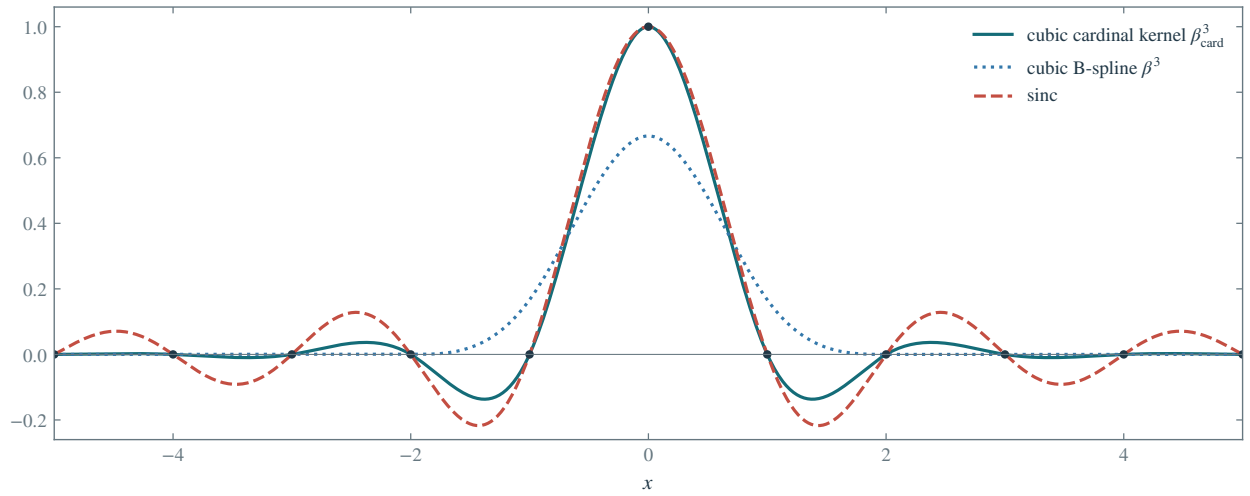


Figure 4.18. The cubic cardinal kernel, the cubic B-spline, and sinc. The cardinal kernel and sinc equal one at zero and vanish at every other integer; the cubic B-spline requires a prefilter.

even and odd values of $n+1$; for odd powers, pair positive and negative frequency shifts and use an alternating-series argument on $0 \le \xi \le \pi$.

4. Define the cardinal kernel by $\widehat{\beta^n_{\text{card}}}(\xi) = \widehat{\beta^n}(\xi)/D_n(\xi)$. Show $\beta^n_{\text{card}} \to \text{sinc}$ in $L^2(\mathbb{R})$, where $\text{sinc}(x) = \sin(\pi x)/(\pi x)$. One route is to write $\xi = \eta + 2\pi m$, $|\eta| \le \pi$, and dominate the spectral tails by $|\eta/(\eta + 2\pi m)|^{n+1}$. Explain the connection with Shannon interpolation.
5. Explain how to approximate the prefilter by FFT for a finite sample vector, and identify the boundary condition imposed by circular convolution. Compare with recursive filtering in Exercise 5.8.

For finitely many data points, boundary conditions select a unique cubic spline. A *natural* spline has zero *second* derivative at both endpoints; a spline with prescribed endpoint first derivatives is *clamped*. A *not-a-knot* spline has continuous third derivative at the first and last interior knots. Figure 4.19 compares natural and not-a-knot conditions. See Ciarlet [16] and Unser [62] for complementary viewpoints.

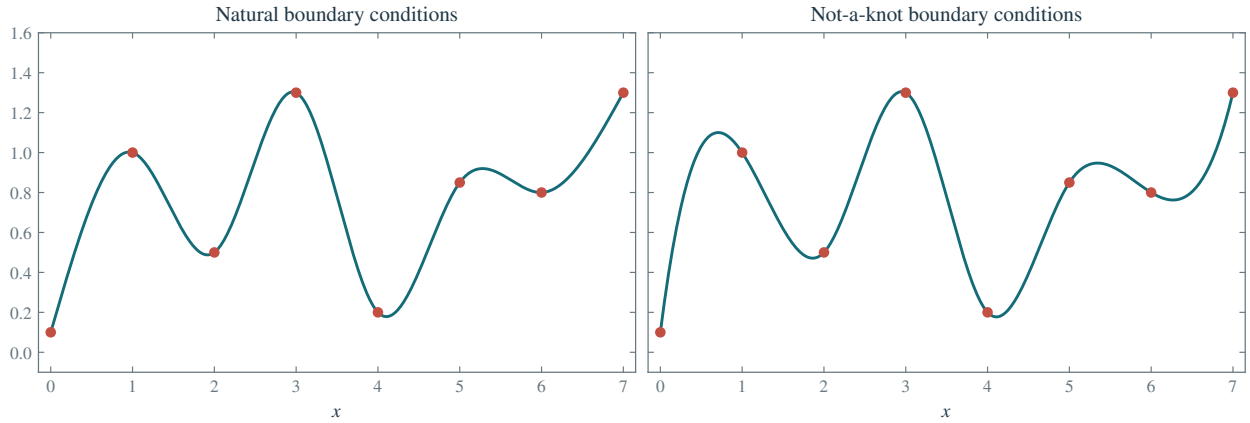


Figure 4.19. Cubic interpolation of the same eight samples with natural and not-a-knot boundary conditions. Natural conditions set the endpoint second derivatives to zero.

4.7 Solutions to the exercises

The arguments here connect spectral formulas with the analytic assumptions and boundary conventions of each application. They justify differential-equation solvers and image operations, then develop alternative multiplication and interpolation methods.

Solution to Exercise 4.1

Compact support and Cauchy–Schwarz give $\hat{f} \in L^1$. Fourier inversion therefore defines a continuous representative. Differentiating under the integral, with the bound $|\xi|^n \le A^n$, gives

$$f^{(n)}(t_0) = \frac{1}{2\pi} \int_{-A}^{A} (\mathrm{i}\xi)^n \hat{f}(\xi) e^{\mathrm{i}\xi t_0} \mathrm{d}\xi.$$

For every $z \in \mathbb{C}$, the exponential series is dominated on this interval by $e^{A|z-t_0|}|\hat{f}(\xi)|$, an integrable function. Thus

$$\frac{1}{2\pi} \int_{-A}^{A} \hat{f}(\xi) e^{\mathrm{i}\xi z} \mathrm{d}\xi = \sum_{n=0}^{\infty} \frac{f^{(n)}(t_0)}{n!} (z - t_0)^n.$$

This constructs an entire extension of f. If its real restriction vanishes on an open interval, all derivatives at an interior point vanish, so the series shows it is identically zero. If f also has compact support, its continuous representative vanishes outside that support and hence on an open interval. Therefore $f = 0$, proving the proposition.

Solution to Exercise 4.2

1. Use the forward difference $(u(t + h, x) - u(t, x))/h$ in time and the centered difference $(u(t, x + d) - 2u(t, x) + u(t, x - d))/d^2$ in space. The local consistency errors are $O(h)$ and $O(d^2)$ for smooth u.
2. The kernel is $g[0] = 1 - 2s$, $g[\pm 1] = s$. One direct step costs $O(N)$; an FFT at every step would cost $O(N \log N)$. If only step m is wanted, two FFTs and the multipliers $\hat{g}[k]^m$ avoid computing all intermediate steps. Program 21 implements the local stencil.

```
def heat_explicit(x, s, steps):
    x = np.asarray(x)
    y = np.array(x, dtype=np.result_type(x, float), copy=True)
    if y.ndim != 1 or y.size < 3 or s < 0 or steps < 0:
        raise ValueError('Use a vector of length >= 3 and nonnegative parameters.')
    for _ in range(steps):
        y += s*(np.roll(y, 1)+np.roll(y, -1)-2*y)
    return y
```

Program 21. Explicit heat evolution by a three-point stencil.

3. The multiplier is

$$r_k = 1 - 4s \sin^2(\pi k/N).$$

The DFT diagonalizes every power of the iteration. Boundedness for every initial vector is equivalent to $|r_k| \le 1$ at every frequency, hence $2s\mu_N \le 1$, where

$$\mu_N = \max_k \sin^2(\pi k/N) = \begin{cases} 1, & N \text{ even}, \\ \cos^2(\pi/(2N)), & N \text{ odd}. \end{cases}$$

This gives the stated bounds. The name CFL abbreviates Courant–Friedrichs–Lewy. For $s \le 1/2$, the stencil is a convex average, which also proves contraction in the maximum norm uniformly over grid sizes. At the even-grid endpoint, the alternating mode has multiplier -1 and remains bounded without decaying.

4. Write $a_k = -4s \sin^2(\pi k/N) \le 0$. The transformed equation gives

$$\hat{u}^{m+1}[k] = r_k \hat{u}^m[k], \qquad r_k = \frac{1 + (1 - \theta)a_k}{1 - \theta a_k}.$$

The denominator is positive, and $r_k \le 1$. The condition $r_k \ge -1$ is equivalent to $2 + (1 - 2\theta)a_k \ge 0$. Thus the method is unconditionally stable for $\theta \ge 1/2$. For $\theta < 1/2$, the exact fixed-grid condition is $2s(1 - 2\theta)\mu_N \le 1$. The threshold is $1/2$, not $1/4$. Program 22 evaluates any nonnegative integer number of steps directly in Fourier coordinates.

```
def heat_theta(x, s, theta, steps):
    x = np.asarray(x)
    if x.ndim != 1 or x.size < 3 or s < 0 or not 0 <= theta <= 1 or steps < 0:
        raise ValueError('Invalid grid, time step, or theta.')
    a = -4*s*np.sin(np.pi*np.fft.fftfreq(x.size))**2
    multiplier = (1+(1-theta)*a)/(1-theta*a)
    y = np.fft.ifft(multiplier**steps*np.fft.fft(x))
    return y.real if np.isrealobj(x) else y
```

Program 22. The one-dimensional theta method in Fourier coordinates.

5. On a square grid,
$$a_{kl} = -4s\left(\sin^2(\pi k/N) + \sin^2(\pi l/N)\right).$$
For $\theta < 1/2$, stability requires $4s(1 - 2\theta)\mu_N \le 1$. In particular, the even-grid explicit bound is $s \le 1/4$, not the one-dimensional bound $1/2$. For $\theta \ge 1/2$, stability is unconditional. At $\theta = 1/2$, r_{kl} approaches -1 as $-a_{kl}$ grows: high-frequency components alternate in sign while decaying very slowly. At $\theta = 1$, $r_{kl} = 1/(1 - a_{kl})$ approaches zero, so these components are strongly damped. Program 23 implements both.

```
def heat_theta_2d(x, s, theta, steps):
    x = np.asarray(x)
    if x.ndim != 2 or min(x.shape) < 3 or s < 0 or not 0 <= theta <= 1 or steps < 0:
        raise ValueError('Invalid grid, time step, or theta.')
    a = -4*s*(np.sin(np.pi*np.fft.fftfreq(x.shape[0]))[:, None]**2
              + np.sin(np.pi*np.fft.fftfreq(x.shape[1]))[None, :]**2)
    multiplier = (1+(1-theta)*a)/(1-theta*a)
    y = np.fft.ifft2(multiplier**steps*np.fft.fft2(x))
    return y.real if np.isrealobj(x) else y
```

Program 23. The theta method on a periodic rectangular grid.

Solution to Exercise 4.3

1. For $t > 0$, absolute uniform convergence permits exchange of the integral and the series:
$$p_t * f(x) = \sum_n e^{-2\pi^2 n^2 t} e^{2\pi i n x} \int_0^1 f(y) e^{-2\pi i n y} \mathrm{d}y.$$
On $t \ge \epsilon > 0$, every differentiated series is dominated by a polynomial in $|n|$ times $e^{-2\pi^2 n^2 \epsilon}$. Hence the convolution is smooth and solves $u_t = \frac{1}{2} u_{xx}$.
2. Two integrations by parts give $|c_n(f)| = O(|n|^{-2})$ for $n \ne 0$. Consequently,
$$\|u(t, \cdot) - f\|_\infty \le \sum_n |c_n(f)|\, |e^{-2\pi^2 n^2 t} - 1| \longrightarrow 0 \quad (t \downarrow 0)$$
by dominated convergence.
3. If $u(t_0, x_0) < 0$, then $v = e^{\beta t} u$ attains a negative minimum α on the compact cylinder $[0, t_0] \times S^1$. Since the initial values are nonnegative, its minimizing time t_1 is positive. At that point $v_{xx} \ge 0$ and $v_t \le 0$; the latter also holds at $t_1 = t_0$ by the left derivative. But $0 \ge v_t = \beta\alpha + \frac{1}{2} v_{xx} \ge \beta\alpha > 0$ when $\beta < 0$, a contradiction.

4. Apply positivity to smooth nonnegative initial functions, for which continuity at $t = 0$ was proved above. If p_t were negative somewhere, a nonnegative smooth bump supported near that point would give a negative convolution value. Thus $p_t \geq 0$. Its zeroth Fourier coefficient is one, so its integral is one, and the integral triangle inequality gives $\|p_t * f\|_\infty \leq \|f\|_\infty$. For uniqueness, apply the minimum argument directly to w and $-w$, where w is the difference of two real classical solutions with the same initial values. Both have zero initial data, so $w = 0$. For complex solutions apply this to real and imaginary parts. This avoids assuming a representation formula for an arbitrary solution.
5. Choose smooth periodic f_j converging uniformly to f. Then

$$\|p_t * f - f\|_\infty \leq 2\|f - f_j\|_\infty + \|p_t * f_j - f_j\|_\infty.$$

First choose j large, then let $t \downarrow 0$. The first term is uniformly small and the second tends to zero. This proves existence with the required initial continuity, and the preceding maximum argument proves uniqueness.

Solution to Exercise 4.4

The stencil has coefficient $-6/h^2$ at the center and $1/h^2$ at each of the six nearest neighbors. Each prescribed boundary neighbor is subtracted from the right-hand side with factor h^{-2}. Odd extension in every coordinate makes the boundary values zero and yields a $2N$-periodic array. Its symbol is

$$-\frac{4}{h^2}\left(\sin^2\frac{\pi k}{2N} + \sin^2\frac{\pi l}{2N} + \sin^2\frac{\pi m}{2N}\right).$$

Divide the transformed right-hand side by this symbol away from zero and set the zero coefficient to zero. Invert the transform, retain the interior, and restore the prescribed boundary. Program 24 constructs the odd extension in any dimension. The general solver in Program 75 likewise works in two or three dimensions.

```
def odd_extend(interior):
    y = np.asarray(interior)
    for axis in range(y.ndim):
        shape = list(y.shape); shape[axis] = 1
        zero = np.zeros(shape, dtype=y.dtype)
        y = np.concatenate([zero, y, zero, -np.flip(y, axis)], axis=axis)
    return y
```

Program 24. Odd extension along every coordinate axis.

Solution to Exercise 4.5

1. Multiplication by T on the left takes second differences in the first index; multiplication on the right takes them in the second. Their sum is the five-point stencil with zero boundary values.
2. A boundary value is a known contribution to that stencil, so move it to the right with coefficient $-h^{-2}$. Summing the contributions at corners of the interior grid gives precisely $\tilde{F}$ in the main text.
3. With $V_j[i] = \sin(\pi ij/N)$ and zero values at $i = 0, N$,

$$(TV_j)[i] = \lambda_j V_j[i], \qquad \lambda_j = -4h^{-2}\sin^2(\pi j/(2N)).$$

These eigenvalues are negative and distinct for $1 \leq j < N$. Let $U_0 = V^{-1}\tilde{U}V$ and $F_0 = V^{-1}\tilde{F}V$. Then $(\lambda_i + \lambda_j)(U_0)_{ij} = (F_0)_{ij}$, so division determines U_0 uniquely; no denominator vanishes. Finally $\tilde{U} = VU_0V^{-1}$.
4. Symmetry of T gives orthogonality between distinct eigenvectors, and the identity $\sum_{i=1}^{N-1}\sin^2(\pi ij/N) = N/2$ gives $V^TV = (N/2)I$. Since $V = V^T$, its inverse is $(2/N)V$. Thus V is *scaled orthogonal*; $Q = \sqrt{2/N}V$ is orthogonal. The product Vx is the unnormalized sine transform $\mathcal{S}x$.
5. For the length-$2N$ odd extension $x^o = (0, x[1], \ldots, x[N-1], 0, -x[N-1], \ldots, -x[1])$,

$$\widehat{x^o}[k] = \sum_{j=1}^{N-1} x[j](e^{-\pi ijk/N} - e^{\pi ijk/N}) = -2i\mathcal{S}(x)[k].$$

Hence $\mathcal{S}x = \widehat{x^o}[1:N]/(-2\mathrm{i})$ and $\mathcal{S}^{-1} = (2/N)\mathcal{S}$. The identity holds for complex data as well; taking an imaginary part alone would discard information in that case. Program 25 preserves complex input.

```
def sine_transform(x, axis=0):
    x = np.asarray(x)
    n = x.shape[axis]+1
    shape = list(x.shape); shape[axis] = 1
    zero = np.zeros(shape, dtype=x.dtype)
    odd = np.concatenate([zero, x, zero, -np.flip(x, axis)], axis=axis)
    y = np.take(np.fft.fft(odd, axis=axis), np.arange(1, n), axis=axis)/(-2j)
    return y.real if np.isrealobj(x) else y
```

Program 25. The unnormalized sine transform, preserving complex data.

For an interior matrix F, applying the transform in both coordinates gives $\mathcal{S}_2(F) = VFV$, with inverse $(2/N)^2\mathcal{S}_2$. The doubled odd array satisfies $\widehat{F^o}[k,l] = -4(VFV)_{kl}$ on the interior frequencies. Thus both constructions divide by the same eigenvalue sums and reconstruct the same solution. Program 26 applies the one-dimensional transform along the two axes.

```
def sine_transform_2d(x):
    return sine_transform(sine_transform(x, axis=0), axis=1)
```

Program 26. A two-dimensional sine transform.

Solution to Exercise 4.6

Program 27 samples a Gaussian and normalizes its finite sum to one. Its default radius is $\lceil 4\sigma \rceil$ pixels. Program 28 reads a grayscale image and convolves a reflected extension with this kernel, retaining the valid output. Reflection preserves constant images and avoids an artificial dark border. The noise is reduced at the cost of some edge and texture detail.

```
def gaussian_kernel(sigma, radius=None):
    if sigma <= 0:
        raise ValueError('Gaussian width must be positive.')
    radius = int(np.ceil(4*sigma)) if radius is None else int(radius)
    if radius < 0:
        raise ValueError('Kernel radius must be nonnegative.')
    k = np.arange(-radius, radius+1)
    g = np.exp(-(k[:, None]**2+k[None, :]**2)/(2*sigma**2))
    return g/g.sum()
```

Program 27. A finite Gaussian kernel with unit sum.

```
def smooth_image(image, sigma):
    kernel = gaussian_kernel(sigma)
    radius = kernel.shape[0]//2
    padded = np.pad(np.asarray(image, dtype=float), radius, mode='reflect')
    return fftconvolve(padded, kernel, mode='valid')
```

```
def apply_gaussian(path, sigma=1):
    with Image.open(path) as source:
        image = np.asarray(source.convert('L'), dtype=float)/255
    return image, smooth_image(image, sigma)
```

Program 28. Gaussian image smoothing with reflected boundaries.

Solution to Exercise 4.7

1. Expanding the square gives
$$d(f,g)[u,v]^2 = E_f[u,v] + \|g\|_2^2 - 2\,\mathrm{Corr}(f,g)[u,v].$$
If E_f is constant, minimizing the squared discrepancy is exactly equivalent to *maximizing* correlation. If it varies little, this is only an approximation. Full-template positions are essential here: otherwise the norm of the visible part of g also depends on the position.
2. Let $\check{g}[i,j] = g[-i,-j]$, with zero extension. Then $\mathrm{Corr}(f,g)[u,v] = (f \star \check{g})[u,v]$. When the reflected template is stored as the ordinary array $g[P-1-i, P-1-j]$, the corresponding full-convolution indices are $(u+P-1, v+P-1)$. Padding to at least $N+P-1$ in both coordinates avoids wraparound; extracting the valid part gives all complete-template positions.
3. Center the patch and template, writing their vectorized forms as a and b. For nonzero a, b, $\rho = \langle a,b\rangle/(\|a\|_2\|b\|_2)$. Cauchy–Schwarz gives $|\rho| \le 1$; equality $\rho = 1$ means $a = cb$ for some $c > 0$, not necessarily $a = b$. Offsets and positive rescalings of either intensity leave ρ unchanged; a negative rescaling of one image reverses its sign. A constant template or patch has zero variance, so normalized correlation is undefined there.
4. Since $g_0 = g - \mu_g$ has zero sum, the numerator is $\sum_{i,j} f[u+i, v+j] g_0[i,j]$. This is one FFT-based correlation. The patch variance is $s_2 - s_1^2/P^2$, and the template variance $\sum_{i,j} g_0[i,j]^2$ is computed once. The stated recurrence follows by inclusion–exclusion of four overlapping windows; the four corner terms must all contain the power k. A summed-area table gives the equivalent formula
$$s_k[u,v] = I_k[u+P, v+P] - I_k[u, v+P] - I_k[u+P, v] + I_k[u,v],$$
where $I_k[a,b] = \sum_{i<a, j<b} f[i,j]^k$. Two cumulative sums construct I_k in $O(N^2)$ work, after which each window costs four lookups. Program 29 implements this.

```
def sliding_sum(image, width, power=1):
    image = np.asarray(image, dtype=float)
    if image.ndim != 2 or not 1 <= width <= min(image.shape):
        raise ValueError('Window must fit within a two-dimensional image.')
    table = np.pad(np.cumsum(np.cumsum(image**power, axis=0), axis=1), ((1, 0), (1, 0)))
    return (table[width:, width:]-table[:-width, width:]
            - table[width:, :-width]+table[:-width, :-width])
```

Program 29. Sliding sums from a summed-area table.

Program 30 combines these sums with FFT correlation, for total cost $O(N^2 \log N)$. It uses the Frobenius norm of the template, subtracts a global image offset to reduce cancellation, and marks zero-variance patches with `NaN`. Tiny negative computed variances are clipped to zero before square roots; this corrects roundoff, not the mathematical definition.

```
def normalized_correlation(image, template):
    f, g = np.asarray(image, float), np.asarray(template, float)
    if g.ndim != 2 or g.shape[0] != g.shape[1] or g.shape[0] > min(f.shape):
        raise ValueError('Use a square template that fits inside the image.')
    f, g = f-f.mean(), g-g.mean()
    energy = np.sum(g*g)
    if energy == 0:
        raise ValueError('Normalized correlation is undefined for a constant template.')
    width = g.shape[0]
    numerator = fftconvolve(f, g[::-1, ::-1], mode='valid')
    s1, s2 = sliding_sum(f, width), sliding_sum(f, width, 2)
    variance = np.maximum(s2-s1*s1/g.size, 0)
    valid = variance > 32*np.finfo(float).eps*np.maximum(s2, 0)
    result = np.full(variance.shape, np.nan)
    np.divide(numerator, np.sqrt(variance*energy), out=result, where=valid)
    return np.clip(result, -1, 1)
```

Program 30. Normalized correlation at complete-template positions.

Solution to Exercise 4.8

1. For signed frequencies k, l,
$$\widehat{T_v f}[k,l] = e^{-2\pi i(kv_1+lv_2)/N}\hat{f}[k,l].$$
For integer shifts, this follows by changing indices. For fractional shifts, it defines translation of the chosen trigonometric interpolant. Using unsigned frequencies instead would select a different interpolation between samples. Odd sizes avoid the unmatched real Nyquist mode of even sizes. Program 31 implements the formula.

```
def fft_translate(image, shift):
    image = np.asarray(image)
    if image.ndim != 2 or any(n % 2 == 0 for n in image.shape):
        raise ValueError('Use odd sizes to avoid a Nyquist interpolation ambiguity.')
    k = np.fft.fftfreq(image.shape[0])[:, None]
    l = np.fft.fftfreq(image.shape[1])[None, :]
    phase = np.exp(-2j*np.pi*(k*shift[0]+l*shift[1]))
    y = np.fft.ifft2(np.fft.fft2(image)*phase)
    return y.real if np.isrealobj(image) else y
```

Program 31. Fractional translation using signed Fourier frequencies.

For an x-shear, translate each fixed-y column by λy; for a y-shear, translate each fixed-x row by λx. Centered coordinates place the center of the image at the origin. Program 32 implements either axis.

```
def fft_shear(image, amount, axis=0):
    image = np.asarray(image)
    if image.ndim != 2 or any(n % 2 == 0 for n in image.shape) or axis not in (0, 1):
        raise ValueError('Use odd image sizes and axis 0 or 1.')
    n, m = image.shape[axis], image.shape[1-axis]
    phase = np.exp(-2j*np.pi*np.fft.fftfreq(n)[:, None]*amount*(np.arange(m)-(m-1)/2))
    if axis == 1:
        phase = phase.T
    y = np.fft.ifft(np.fft.fft(image, axis=axis)*phase, axis=axis)
    return y.real if np.isrealobj(image) else y
```

Program 32. A centered shear along either image coordinate.

2. Multiply the three matrices with $a = -\tan(\theta/2)$ and $b = \sin\theta$. The diagonal entries are $1 + ab = \cos\theta$, the lower-left entry is b, and the upper-right entry is $2a + a^2b = -\sin\theta$. Since pullbacks compose in the corresponding matrix order, $R_\theta = S_a^{(x)} S_b^{(y)} S_a^{(x)}$ for continuous functions.
3. Each discrete line translation is unitarily diagonalized by the DFT, with unit-modulus multipliers. Its inverse negates the shift, so each shear and their product are invertible. Reversing the three steps and negating their parameters gives the inverse; the symmetric three-step pattern makes it the algorithm with angle $-\theta$.

```
def fft_rotate(image, angle):
    if abs(angle) >= np.pi:
        raise ValueError('Split large rotations into angles of modulus less than pi.')
    a = -np.tan(angle/2)
    return fft_shear(fft_shear(fft_shear(image, a, 0), np.sin(angle), 1), a, 0)
```

Program 33. Rotation by three discrete shears.

Nevertheless, intermediate sheared functions do not generally belong to the original finite tensor-product trigonometric space. Resampling and periodic wrapping therefore prevent these discrete matrices from forming an exact representation of all planar rotations: $R_\alpha R_\beta \neq R_{\alpha+\beta}$ in general. Taking real parts on

even grids or cropping intermediate results can also destroy invertibility. The odd-grid implementation preserves real data without discarding a Nyquist component, and preserves complex data as complex. Padding moves wraparound away from the image of interest; it does not make band-limited interpolation exact for arbitrary continuous images. Program 34 supplies a white margin for a small rotation gallery. Larger angles should be split into bounded steps or reduced using quarter-turn permutations to avoid very large shears. The arithmetic cost is $O(N^2 \log N)$ for a square padded grid.

```
def rotation_examples(image, angles=(0, np.pi/12, np.pi/6, np.pi/4)):
    image = np.asarray(image)
    padding = max(image.shape)//2+1
    padded = np.pad(image, padding, mode='constant', constant_values=1)
    return np.stack([fft_rotate(padded, angle) for angle in angles])
```

Program 34. A rotation gallery with a padded white margin.

Solution to Exercise 4.9

1. Form the real even sampled response $H[k] = \mathbf{1}_{|\omega_k|<\pi/2}$, with half-values at any cutoff samples, and take its inverse DFT. The resulting circular kernel is real and symmetric, and its sum is $H[0] = 1$.

```
def lowpass_kernel(n):
    if n < 3 or n % 2 == 0:
        raise ValueError('Use an odd length >= 3.')
    frequency = abs(np.fft.fftfreq(n))
    response = (frequency < .25).astype(float)
    response[frequency == .25] = .5
    return np.fft.ifft(response).real
```

Program 35. A sharp sampled low-pass response and its inverse transform.

2. Assign kernel indices to $-(N-1)/2, \ldots, (N-1)/2$, then embed them modulo a larger plotting length. Padding the negative-index terms at the wrong end would change the response. The resulting FFT samples $G(\omega) = \sum_j g[j]e^{-\mathrm{i}j\omega}$, which agrees with H at the original frequencies but oscillates between them near the cutoff. Program 36 returns the frequencies and the complex response; for this symmetric real kernel it is real up to roundoff.

```
def filter_response(kernel, length=2048):
    kernel = np.asarray(kernel)
    n = kernel.size
    if n % 2 == 0 or length < n:
        raise ValueError('Use an odd kernel length and a larger plotting grid.')
    index = np.rint(np.fft.fftfreq(n)*n).astype(int)
    padded = np.zeros(length, dtype=complex)
    padded[index % length] = kernel
    omega = 2*np.pi*np.fft.fftshift(np.fft.fftfreq(length))
    return omega, np.fft.fftshift(np.fft.fft(padded))
```

Program 36. Dense frequency-response evaluation with centered kernel indices.

3. Let $r = |\omega|$, $a = (1-\epsilon)\pi/2$, and $b = (1+\epsilon)\pi/2$. Use response one for $r \le a$, zero for $r \ge b$, and $\frac{1}{2}[1+\cos(\pi(r-a)/(b-a))]$ in the transition. At $\epsilon = 0$ use the sharp-cutoff definition separately. Increasing the transition width reduces the long oscillatory tails of the impulse response. The endpoint $\epsilon = 1$ is included without creating a negative array length.

```
def smooth_lowpass_kernel(n, epsilon):
    if not 0 <= epsilon <= 1:
        raise ValueError('Transition parameter must lie in [0, 1].')
    if epsilon == 0:
        return lowpass_kernel(n)
    if n < 3 or n % 2 == 0:
        raise ValueError('Use an odd length >= 3.')
    frequency = abs(np.fft.fftfreq(n))
    t = np.clip((frequency-(1-epsilon)/4)/(epsilon/2), 0, 1)
    return np.fft.ifft((1+np.cos(np.pi*t))/2).real
```

Program 37. A raised-cosine transition, including both endpoint parameters.

Solution to Exercise 4.10

1. The rounded update is $a_j^{m+1} = 2\lceil (a_j^m + a_{j-1}^m)/4 \rceil$. Its minimum is nondecreasing and its maximum nonincreasing, since both are even. If the current minimum is q, an updated count equals q exactly when both contributing counts equal q. Thus its new minimum set is the intersection of the old minimum set and its one-step translate. A nonempty proper subset of a cycle cannot be invariant under a one-step translation, so its cardinality strictly decreases while the minimum stays fixed. Within at most n steps, either consensus is reached or the minimum increases by at least two. It is bounded above by the initial maximum, so consensus occurs after finitely many steps. This argument excludes the already-constant case before claiming strict decrease.
2. Without rounding, the kernel is $(1/2, 1/2, 0, \ldots, 0)$. Its nonconstant Fourier multipliers have modulus less than one, so the limit is the constant vector with value $n^{-1} \sum_j a_j^0$.
3. With half sent to each neighbor, the multiplier is $\cos(2\pi k/n)$. For odd n, its modulus is less than one except at zero, so the fractional process converges to the mean. For even n, the alternating mode has multiplier -1: alternating counts $0, 2, 0, 2, \ldots$ are exchanged at every step and do not converge, even with rounding. For the rounded process on an odd cycle, a site remains at the current minimum exactly when both its neighbors were at that minimum. If the minimum set failed to shrink, it would be invariant under translation by two. Since two generates an odd cycle, a proper nonempty minimum set cannot have this property. The preceding finite-consensus argument therefore applies to odd n as well.

Solution to Exercise 4.11

1. The terms are $R_0 = P_0 Q_0$, $R_2 = P_1 Q_1$, and $R_1 = P_0 Q_1 + P_1 Q_0$.
2. Compute $R_1 = (P_0 + P_1)(Q_0 + Q_1) - R_0 - R_2$. The cross term needs one new product, two additions, and two subtractions.
3. After padding to length $n = 2^p$, the recurrence is $C(n) = 3C(n/2) + O(n)$, hence

$$C(n) = O\left(\sum_{j=0}^{p-1} 3^j \frac{n}{2^j} + 3^p \right) = O(n^{\log_2 3}).$$

Padding arbitrary lengths increases them by less than a factor two and preserves the bound. Programs 38 and 39 use Python lists, so integer coefficients retain arbitrary precision. The algebra also works over other coefficient rings supporting addition, subtraction, and multiplication. There is no universal crossover degree: benchmark comparable implementations and include conversion and precision costs.

```
def karatsuba(p, q):
    size = len(p)+len(q)-1
    if not p or not q:
        return []
    n = 1 << (max(len(p), len(q))-1).bit_length()
    p, q = list(p)+[0]*(n-len(p)), list(q)+[0]*(n-len(q))
    if n == 1:
        return [p[0]*q[0]]
    k = n//2
    r0, r2 = karatsuba(p[:k], q[:k]), karatsuba(p[k:], q[k:])
    r1 = karatsuba(polynomial_add(p[:k], p[k:]), polynomial_add(q[:k], q[k:]))
    r1 = polynomial_add(polynomial_add(r1, [-v for v in r0]), [-v for v in r2])
    result = polynomial_add(polynomial_add(r0, [0]*k+r1), [0]*(2*k)+r2)
    return result[:size]
```

Program 38. Karatsuba multiplication with exact Python integer coefficients.

```
def polynomial_add(p, q):
    n = max(len(p), len(q))
    return [(p[k] if k < len(p) else 0)+(q[k] if k < len(q) else 0) for k in range(n)]
```

Program 39. Addition of coefficient vectors of different lengths.

Solution to Exercise 4.12

1. The interpolation equations are $u_d = a * \psi_d$. Since ψ_d is finitely supported, its transform $D = \widehat{\psi_d}$ is continuous and periodic. Nonvanishing therefore implies $\min_{[-\pi,\pi]} |D| > 0$, so multiplication by D is a boundedly invertible operator on L^2 of the circle. This proves the unique solution $\widehat{a} = \widehat{u_d}/D$. Let b have Fourier series $1/D$ and set $\varphi(x) = \sum_k b[k]\psi(x-k)$. At each x, this sum has finitely many nonzero terms; moreover $b * \psi_d = \delta_0$, so $\varphi(j) = \delta_{j0}$. Then $a = b * u_d$ gives the direct form with kernel φ. The inverse filter generally has infinitely many coefficients, so compact support of ψ need not imply compact support of φ.
2. Repeated convolution gives $\mathrm{Supp}(\beta^n) = [-(n+1)/2, (n+1)/2]$ (as a closed support). Direct interpolation requires $\beta^n(j) = \delta_{j0}$. This holds for $n = 0$ and $n = 1$, giving piecewise constant and piecewise linear interpolation. It fails for $n \geq 2$; these degrees require a prefilter.
3. Put $r = n + 1$. The continuous convolution theorem gives

$$B_r(\xi) := \widehat{\beta^n}(\xi) = \left(\frac{\sin(\xi/2)}{\xi/2}\right)^r.$$

   For $n \geq 1$, Poisson summation, with absolutely convergent frequency tails, gives

$$D_n(\xi) = \sum_{k\in\mathbb{Z}} \beta^n(k) e^{-ik\xi} = \sum_{m\in\mathbb{Z}} B_r(\xi + 2\pi m).$$

   There is no factor 2π in this identity. By evenness and periodicity it suffices to take $0 \leq \xi \leq \pi$. If r is even, every summand is nonnegative and $B_r(\xi) > 0$. If $r \geq 3$ is odd, put $x = \xi/(2\pi)$. For $0 < x \leq 1/2$, pairing the terms $m = j$ and $m = -j$ gives

$$D_n(\xi) = B_r(\xi) + \left(\frac{\sin(\pi x)}{\pi}\right)^r \sum_{j=1}^{\infty} (-1)^{j+1} \left((j-x)^{-r} - (j+x)^{-r}\right).$$

   The parenthesized quantities are positive and decrease to zero: their derivative as a function of j is negative. The alternating sum is nonnegative. Thus in both parity cases $D_n(\xi) \geq B_r(\xi) > 0$ on $[-\pi, \pi]$; at zero both sides equal one. For $n = 0$, the sampled sequence is simply δ_0, so $D_0 = 1$.
4. The transform of the cardinal kernel is $H_r = B_r/D_n$. For $|\eta| \leq \pi$ and $m \in \mathbb{Z}$,

$$|H_r(\eta + 2\pi m)| \leq \left|\frac{\eta}{\eta + 2\pi m}\right|^r,$$

with the $m = 0$ ratio interpreted as one. This follows from $D_n(\eta) \geq B_r(\eta) > 0$ and the ratio of the two sinc powers. For $|\eta| < \pi$, every ratio with $m \neq 0$ is strictly less than one, and the shifted series shows $D_n(\eta)/B_r(\eta) \to 1$ as $r \to \infty$. Consequently $H_r(\eta) \to 1$ and $H_r(\eta + 2\pi m) \to 0$ for $m \neq 0$, except at a set of endpoints of measure zero.

For $r \geq 2$ and $m \neq 0$, the squared terms are bounded by $(2|m| - 1)^{-4}$. This bound is summable in m and integrable over $\eta \in [-\pi, \pi]$, while $|H_r(\eta)| \leq 1$ on the central interval. Dominated convergence on these intervals therefore proves $H_r \to \mathbf{1}_{[-\pi,\pi]}$ in $L^2(\mathbb{R})$. Since $\widehat{\operatorname{sinc}} = \mathbf{1}_{[-\pi,\pi]}$ in our convention, Plancherel gives $\beta^n_{\text{card}} \to \operatorname{sinc}$ in L^2. The argument keeps the alternating signs for even spline degrees; dropping them would give a different cardinal kernel.

For the cubic spline, $D_3(\xi) = (2 + \cos \xi)/3$ and its inverse filter is $b[k] = \sqrt{3}(\sqrt{3} - 2)^{|k|}$. Hence

$$\beta^3_{\text{card}}(x) = \sqrt{3} \sum_{k \in \mathbb{Z}} (\sqrt{3} - 2)^{|k|} \beta^3(x - k).$$

It has unbounded, exponentially decaying tails, whereas sinc decays only algebraically. The limiting cardinal interpolation uses sinc translates, as in Shannon's sampling theorem.

5. For a finite vector, division by sampled values of D_n followed by an inverse FFT computes the *periodic* prefilter exactly for that finite model. It approximates the infinite-grid problem only after a boundary model and sufficient extension have been chosen. The principal issue is the unintended periodic boundary condition, rather than an unavoidable failure of FFT convolution. Recursive causal and anticausal filters exploit the roots of the spline symbol; Exercise 5.8 develops this method for cubic splines. Natural and not-a-knot finite-interval splines solve different boundary-value problems and should not be conflated with periodic or infinite-grid interpolation.

CHAPTER 5

Transforms related to the discrete Fourier transform

The Fourier transform belongs to a wider family of useful transforms. The Hartley transform provides a real kernel, the Z transform encodes recurrences and filter stability, and the chirp algorithm evaluates spectra on flexible frequency grids. We develop their algebraic identities and fast algorithms, while distinguishing exact finite sums from numerical approximations to continuous signals.

5.1 Hartley transform

The Hartley transform represents a real signal by real coefficients without losing phase information. Like the DFT, it has a fast algorithm and can be used to compute convolutions. It is an alternative representation of the same information, rather than a uniformly faster replacement for a real-input FFT. Bracewell [10] develops this viewpoint in detail.

5.1.1 Hartley coefficients and inversion

Combining sine and cosine in one real kernel gives a transform that remains invertible. We define it, establish its normalization, and show how its symmetric and antisymmetric parts retain the complex Fourier information.

Definition 26 (Discrete Hartley transform)**.** For $f \in \mathbb{C}^N$, define

$$\mathcal{H}(f)[k] = \sum_{n=0}^{N-1} f[n] \operatorname{cas}\left(\frac{2\pi nk}{N}\right), \qquad 0 \le k < N, \tag{5.1}$$

where $\operatorname{cas}(x) = \cos x + \sin x = \sqrt{2}\cos(x - \pi/4)$. The map is complex-linear, and maps $\mathbb{R}^N$ into $\mathbb{R}^N$.

Its continuous counterpart, with frequency measured in cycles per unit, is $\mathcal{H}f(s) = \int_{\mathbb{R}} f(x)\operatorname{cas}(2\pi sx)\mathrm{d}x$ for $f \in L^1(\mathbb{R})$.

Orthogonality of the real kernel makes reconstruction possible by applying the same transform again.

Proposition 42 (Inversion)**.** *The Hartley transform satisfies $\mathcal{H}^2 = N\,\mathrm{Id}$. Thus $\mathcal{H}^{-1} = \mathcal{H}/N$, and $\mathcal{H}/\sqrt{N}$ is orthogonal on $\mathbb{R}^N$ and unitary on $\mathbb{C}^N$.*

Proof. The identity

$$\operatorname{cas}\alpha \operatorname{cas}\beta = \cos(\alpha - \beta) + \sin(\alpha + \beta) \tag{5.2}$$

and the sums of roots of unity give, for $0 \le n, n' < N$,

$$\sum_{k=0}^{N-1} \operatorname{cas}\left(\frac{2\pi nk}{N}\right) \operatorname{cas}\left(\frac{2\pi n'k}{N}\right) = N\delta_n^{n'}. \tag{5.3}$$

The sine sum is zero, and the cosine sum is N if $n = n'$ and zero otherwise. The matrix of $\mathcal{H}$ is real and symmetric, so this identity proves both assertions. □

For real input, symmetry identifies exactly how to pass between Hartley coefficients and complex DFT coefficients.

Proposition 43 (Relation to the DFT)**.** *For real f, with the negative exponential in the DFT,*

$$\mathcal{H}(f) = \mathcal{R}\mathcal{F}(f) - \mathcal{I}\mathcal{F}(f), \qquad \mathcal{F}(f) = \mathcal{H}(f)_S - \mathrm{i}\mathcal{H}(f)_A. \tag{5.4}$$

Here $g^\sharp[k] = g[-k]$, with indices modulo N, $g_S = (g + g^\sharp)/2$, and $g_A = (g - g^\sharp)/2$.

Proof. Take real and imaginary parts in the DFT formula. The cosine contribution is symmetric in k, and the sine contribution is antisymmetric. □

For complex f, the first formula must instead be written in a complex-linear form:

$$\mathcal{H}(f) = \tfrac{1}{2}\big((1+\mathrm{i})\mathcal{F}(f) + (1-\mathrm{i})\mathcal{F}(f)^\sharp\big).$$

For real input, the Fourier symmetry $\hat{f}[-k] = \overline{\hat{f}[k]}$ explains why N real Hartley coefficients suffice. Real-input FFT routines also exploit this symmetry. Their relative speed and memory use depend on the implementation; the Hartley formulation alone gives no universal factor-of-two advantage or improvement in rounding error.

5.1.2 Fast Hartley transform

The Hartley kernel admits a divide-and-conquer algorithm analogous to the FFT. Its addition formula explains how shorter transforms and reversed frequency indices combine into the full result.

Let $N = 2M$, $f^0[n] = f[2n]$, and $f^1[n] = f[2n+1]$. Splitting the sum gives

$$\mathcal{H}_N(f)[k] = \sum_{n=0}^{M-1} f^0[n]\,\mathrm{cas}\,\frac{2\pi nk}{M} + \sum_{n=0}^{M-1} f^1[n]\,\mathrm{cas}\left(\frac{2\pi nk}{M} + \frac{2\pi k}{N}\right). \tag{5.5}$$

The addition formula

$$\mathrm{cas}(\alpha+\beta) = \mathrm{cas}\,\alpha\cos\beta + \mathrm{cas}(-\alpha)\sin\beta \tag{5.6}$$

expresses the odd-index sum using the length-M Hartley transform and its reversal. More generally, set

$$(\chi_M^x a)[j] = a[j]\cos\frac{2\pi jx}{M} + a[-j]\sin\frac{2\pi jx}{M}. \tag{5.7}$$

Writing $E = \mathcal{H}_M(f^0)$ and $O = \mathcal{H}_M(f^1)$, we obtain

$$\mathcal{H}_N(f)[k] = E[k] + (\chi_M^{1/2}O)[k], \tag{5.8}$$

$$\mathcal{H}_N(f)[k+M] = E[k] - (\chi_M^{1/2}O)[k], \tag{5.9}$$

for $0 \le k < M$. Recursion, starting from $N = 1$, gives an $O(N \log N)$ algorithm when N is a power of two. The implementation in Section A.2 uses real arithmetic for real inputs. Ullmann [61] describes iterative arrangements that group the coupled frequencies into larger butterflies.

Zero padding. Appending zeros evaluates the finite-record Hartley spectrum at more frequencies. It does not add measurements or change the information in the signal. Approximating a continuous transform additionally requires a sampling model and a quadrature normalization. Figure 5.1 compares the Hartley spectrum of a triangular signal with the real part of its Fourier spectrum; their difference is the negative imaginary part.

5.1.3 Convolution by Hartley transforms

Hartley coefficients can compute convolution while keeping real data in real arithmetic. The product formula couples opposite frequencies, reflecting how the transform stores cosine and sine contributions.

Proposition 44 (Hartley convolution formula)**.** *For the unnormalized circular convolution of $a, b \in \mathbb{R}^N$, let $c = \mathcal{H}(a)$ and $d = \mathcal{H}(b)$. Then*

$$\mathcal{H}(a * b)[k] = \tfrac{1}{2}\big(c[k](d[k] + d[-k]) + c[-k](d[k] - d[-k])\big).$$

Proof. Reindex the finite sums and use the addition formula:

$$\begin{aligned}\mathcal{H}(a*b)[k] &= \sum_{l,m} a[l]b[m]\,\mathrm{cas}\,\frac{2\pi k(l+m)}{N} \\ &= d[k]\sum_l a[l]\cos\frac{2\pi kl}{N} + d[-k]\sum_l a[l]\sin\frac{2\pi kl}{N}.\end{aligned}$$

Substitute $\cos x = (\mathrm{cas}\,x + \mathrm{cas}(-x))/2$ and $\sin x = (\mathrm{cas}\,x - \mathrm{cas}(-x))/2$, then rearrange. □

The function `hartley_convolution`, in Section A.2, applies this formula and divides the final Hartley transform by N.

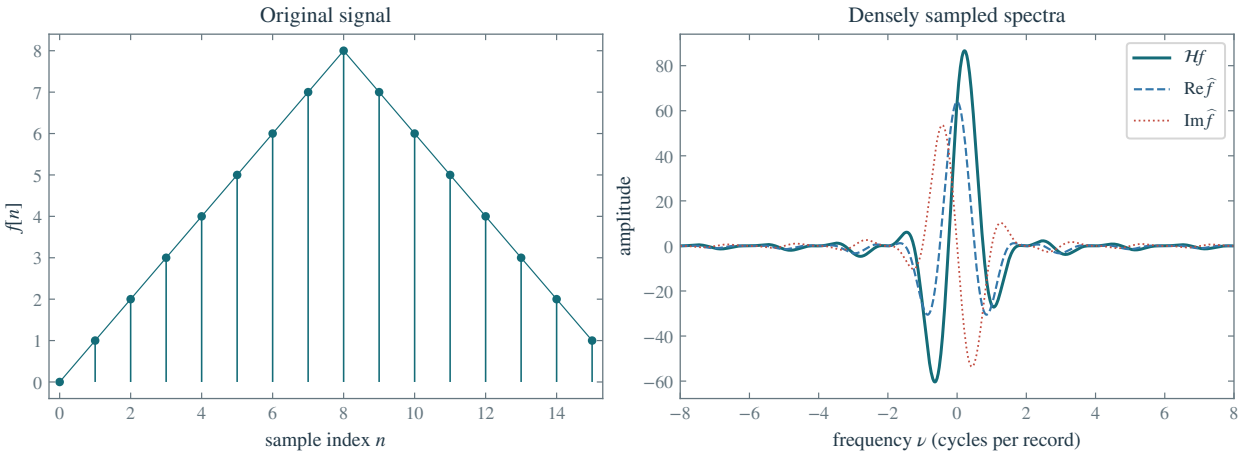


Figure 5.1. A triangular signal and its densely evaluated Fourier and Hartley spectra. The Hartley spectrum equals the real part minus the imaginary part of the DFT. Zero padding changes only the frequency grid.

Self-convolution and autocorrelation. Self-convolution and autocorrelation couple opposite frequencies differently. Comparing their Hartley formulas prevents confusing the two operations. For $c = \mathcal{H}(a)$,

$$\mathcal{H}(a * a)[k] = c[k]c[-k] + \tfrac{1}{2}(c[k]^2 - c[-k]^2),$$

whereas the real circular autocorrelation $a * a^\sharp$ satisfies

$$\mathcal{H}(a * a^\sharp)[k] = |\mathcal{F}(a)[k]|^2 = \tfrac{1}{2}(c[k]^2 + c[-k]^2).$$

5.2 Z transform and applications

The Z transform encodes a sequence as a Laurent series. Delays become powers of z^{-1}, convolutions become products, and linear recurrences become rational functions. The region of convergence remains part of the transform, even when the resulting function is rational. See [65, 21, 14] for analytic background and [31, 67] for generating-function methods.

5.2.1 The Z transform and its calculus

A Z transform records both a Laurent series and the region where it represents a function. We make this convergence information explicit before deriving the rules for delays, convolution, and coefficient extraction.

Definition 27 (Bilateral Z transform)**.** For a sequence $f \in \mathbb{C}^{\mathbb{Z}}$, define

$$\mathcal{Z}(f)(z) = \sum_{n\in\mathbb{Z}} f[n]z^{-n} \tag{5.10}$$

at the points $z \neq 0$ where the series converges absolutely.

If the convergence set has nonempty interior, that interior is an annulus $r < |z| < R$, with $0 \le r < R \le \infty$. The series converges locally uniformly there and defines a holomorphic function. Convergence on its boundary circles must be checked separately. The interior may be empty: for example, $f[n] = (1 + |n|)^{-2}$ gives absolute convergence only on $|z| = 1$. A finite sequence has a Laurent-polynomial transform and no convergence issue for $z \neq 0$.

Geometric sequences illustrate why a rational expression must be accompanied by its region of convergence.

Example 5. The causal constant sequence $u[n] = 1$ for $n \ge 0$ and zero otherwise has

$$\mathcal{Z}(u)(z) = \frac{1}{1 - z^{-1}}, \qquad |z| > 1.$$

If $b \neq 0$ and $|a| < |b|$, the sequence equal to a^n for $n \ge 0$ and b^n for $n < 0$ has transform

$$\frac{1}{1 - a/z} + \frac{1}{1 - z/b} - 1, \qquad |a| < |z| < |b|.$$

Both formulas follow by summing geometric series in their stated regions.

Absolute convergence justifies the algebraic manipulations that make the Z transform useful for filtering.

Proposition 45 (Z-transform rules)**.** *Besides linearity, the following identities hold wherever the relevant series converge absolutely; the derivative formula holds in the open annulus of convergence:*

$$\mathcal{Z}(f \star g) = \mathcal{Z}(f)\mathcal{Z}(g), \qquad (f \star g)[n] = \sum_m f[m]g[n-m],$$

$$\mathcal{Z}\big(f[\cdot - k]\big)(z) = z^{-k}\mathcal{Z}(f)(z), \qquad \mathcal{Z}\big(nf[n]\big)(z) = -z\frac{d}{dz}\mathcal{Z}(f)(z).$$

Proof. At a common point of absolute convergence,

$$\sum_{m,l} |f[m]g[l]z^{-(m+l)}| = \left(\sum_m |f[m]z^{-m}|\right)\left(\sum_l |g[l]z^{-l}|\right) < \infty.$$

Fubini's theorem therefore permits grouping the terms by $n = m + l$, which proves the convolution rule. Reindexing proves the delay rule; termwise differentiation of the Laurent series proves the last identity. □

The sequence can be recovered by coefficient extraction. For any circle $|z| = \rho$ contained in the annulus, oriented counterclockwise,

$$f[n] = \frac{1}{2\pi\mathrm{i}} \int_{|z|=\rho} \mathcal{Z}(f)(z)z^{n-1}\mathrm{d}z.$$

5.2.2 Recursive filters

A recursive filter computes each output from a few inputs and earlier outputs. Its Z transform turns the recurrence into a rational transfer function, whose poles reveal whether the zero-state response is stable.

Definition 28 (Causal recursive filter)**.** Given coefficients $a_0, \ldots, a_k$ and $b_1, \ldots, b_l$, define $y = \Phi_a^b(x)$ recursively by

$$y[n] = \sum_{j=0}^{k} a_j x[n-j] + \sum_{j=1}^{l} b_j y[n-j], \qquad n \geq 0, \tag{5.11}$$

with $x[n] = y[n] = 0$ for $n < 0$. This is the zero-state convention.

The rule is linear and causal. Its impulse response $h = \Phi_a^b(\delta_0)$ satisfies $y = h \star x$. At fixed filter order, computing the first N outputs costs $O(N)$ operations. The impulse response is often infinite, hence the term *IIR filter*; special coefficient choices or cancellations can also produce a finite impulse response. Applying the delay rule gives the transfer function

$$H(z) = \mathcal{Z}(h)(z) = \frac{A(z^{-1})}{B(z^{-1})}, \qquad A(q) = \sum_{j=0}^{k} a_j q^j, \quad B(q) = 1 - \sum_{j=1}^{l} b_j q^j.$$

For a causal impulse response, its Laurent expansion is taken about infinity. Nonzero initial conditions add a separate response; they cannot be dropped when transforming a recurrence.

A first-order recurrence shows how causality and the convergence region select the impulse response.

Example 6 (A first-order filter).

$$y[n] = x[n] + \alpha y[n-1] \tag{5.12}$$

has the causal impulse response $h[n] = \alpha^n$ for $n \geq 0$ and zero otherwise. Its transform is $H(z) = 1/(1 - \alpha z^{-1})$, with $|z| > |\alpha|$, and it is stable precisely when $|\alpha| < 1$. For $\alpha \neq 0$, the same rational expression also represents the *anticausal* sequence $h[n] = -\alpha^n$ for $n < 0$ and zero otherwise, now in $|z| < |\alpha|$. That sequence is absolutely summable precisely when $|\alpha| > 1$. The rational expression without its region of convergence does not identify the impulse response.

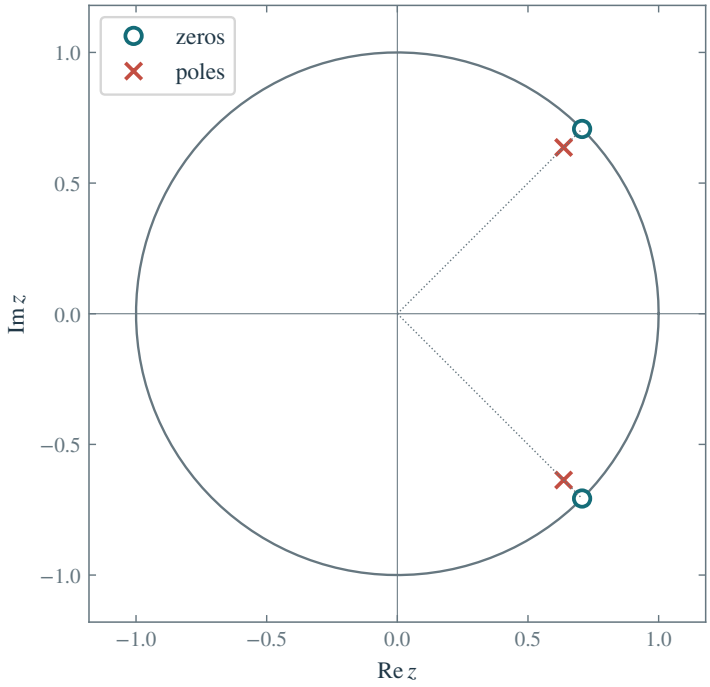


Figure 5.2. Zeros on the unit circle and poles at radius 0.9 for a notch filter at angular frequencies $\pm\pi/4$. Circles mark zeros; crosses mark poles.

The pole criterion generalizes this example to every causal rational filter after cancellation of common factors.

Proposition 46 (Poles and stability)**.** *A causal rational zero-state filter is bounded-input, bounded-output stable if and only if every pole of its* reduced *transfer function has modulus strictly less than one.*

Proof. By the convolution stability criterion in Section 4.2.2, stability is equivalent to $\sum_{n\geq 0} |h[n]| < \infty$. In that case the series for H is uniformly convergent and bounded on $|z| \geq 1$. It is holomorphic for $|z| > 1$ and continuous on the unit circle. A pole of the reduced rational function cannot lie in this region, including its boundary. Conversely, partial fractions at poles of modulus less than one express $h[n]$ as a finite sum of polynomially weighted decaying geometric sequences, together with a finitely supported sequence. Each is absolutely summable, which proves stability. □

This criterion concerns the zero-state input-output map. Cancelling an unstable pole algebraically can hide an unstable internal mode in a particular recurrence with nonzero initial state.

5.2.3 Design by poles and zeros

Pole and zero locations provide direct control over a recursive filter's frequency response. We illustrate this design principle with a notch filter, balancing frequency suppression against stability.

A notch filter suppresses selected frequencies by placing zeros on the unit circle. For $\theta = \pi/4$ and $r = 0.9$, consider

$$H(z) = \frac{(z - e^{i\theta})(z - e^{-i\theta})}{(z - re^{i\theta})(z - re^{-i\theta})} = \frac{z^2 - \sqrt{2}z + 1}{z^2 - 0.9\sqrt{2}z + 0.81}.$$

Its poles lie strictly inside the unit circle. In the notation of (5.11), its coefficients are

$$(a_0, a_1, a_2) = (1, -\sqrt{2}, 1), \qquad (b_1, b_2) = (0.9\sqrt{2}, -0.81).$$

The frequency response is $\omega \mapsto H(e^{i\omega})$. Its modulus is the gain, while its argument records the phase shift. One can compute the impulse response directly from the recurrence and then approximate the response by truncating its Fourier series. Conversely, an inverse DFT of N equally spaced response values produces the periodized impulse response $\sum_l h[n + lN]$, not an exact truncation. For a stable filter, the discrepancy decreases as the decaying tail becomes negligible.

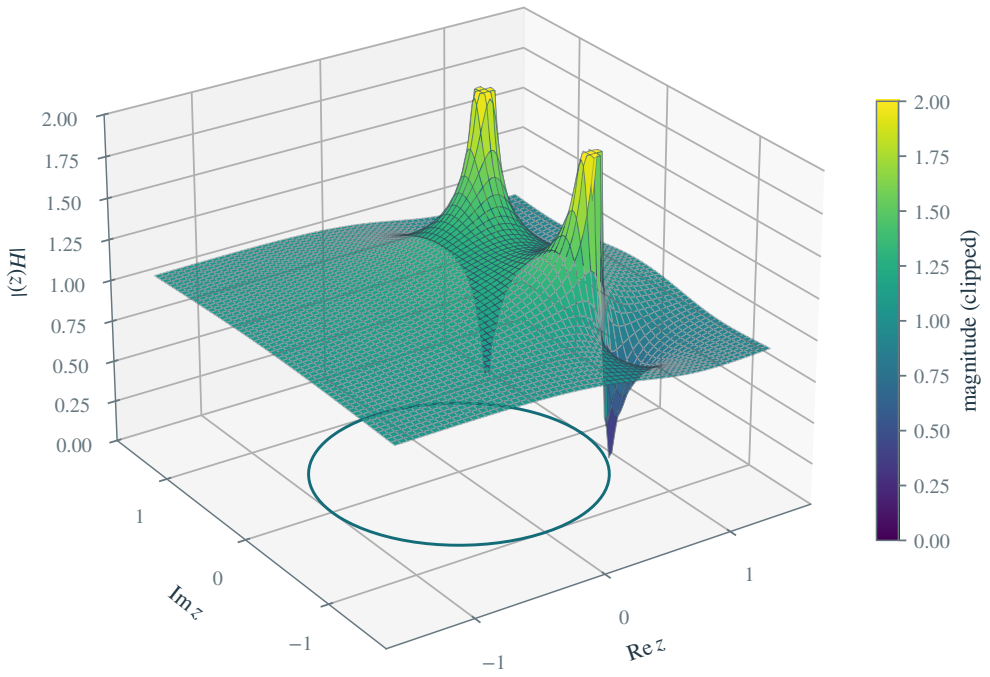


Figure 5.3. Magnitude of the rational notch-filter transfer function in the complex plane. Values above two are clipped to reveal the zeros and pole neighborhoods. The causal Laurent series converges for $|z| > 0.9$.

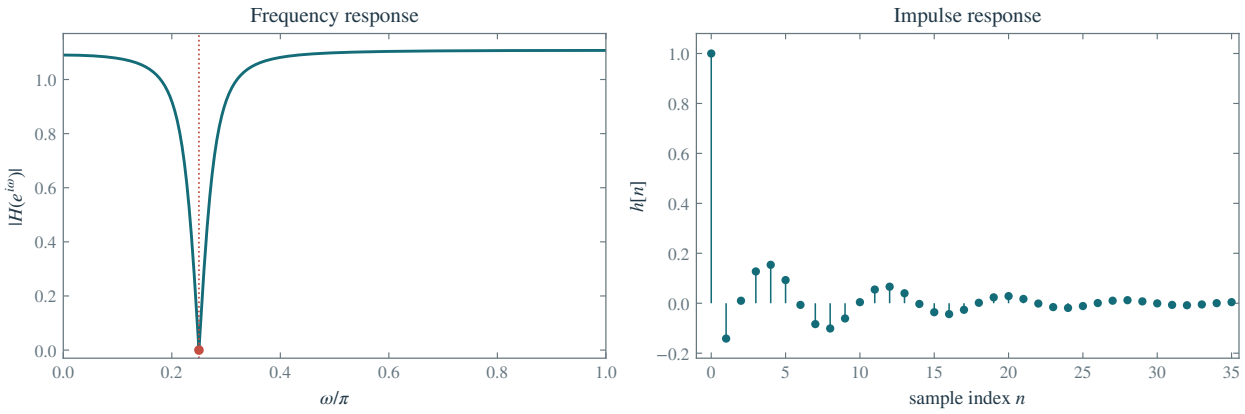


Figure 5.4. Frequency-response magnitude and causal impulse response of the notch filter. The response vanishes at angular frequency $\pi/4$.

A real rational transfer function factors into real first- and second-order sections, which can be applied in cascade. A partial-fraction decomposition instead gives parallel branches, together with any polynomial part. Repeated poles require terms with powers of their irreducible factors; these terms cannot all be treated as simple first- or second-order fractions. Exercise 5.8 illustrates both arrangements for a spline prefilter.

5.2.4 Connection with analog filtering

Analog filters act on continuous-time signals and are described by differential equations. Digital filters use difference equations. To connect the two, consider a series RLC circuit with input voltage V_e and output V_s across the resistor. Writing $V_s = RI$ and differentiating Kirchhoff's voltage law gives

$$V_e'(t) = \frac{1}{RC} V_s(t) + V_s'(t) + \frac{L}{R} V_s''(t). \tag{5.13}$$

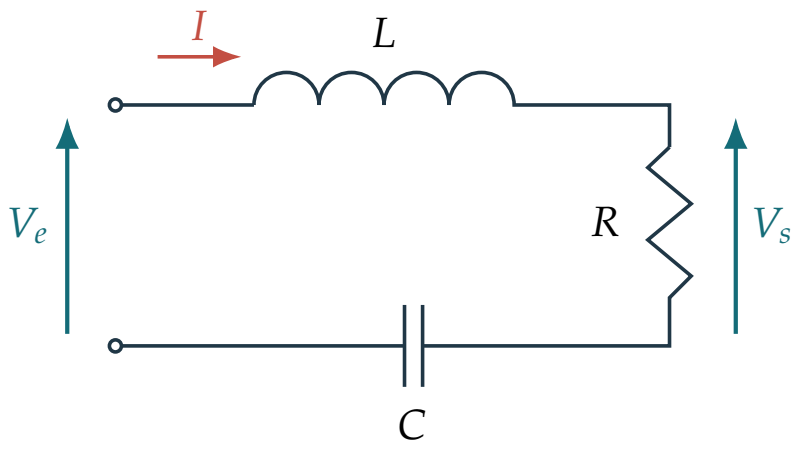


Figure 5.5. A series RLC circuit, with the output voltage V_s measured across the resistor and the input V_e across the two left terminals.

The Laplace transform plays the continuous-time role of the Z transform and will convert this differential equation into a transfer function.

Definition 29 (Laplace transform)**.** For a causal function f, its Laplace transform is $\mathcal{L}(f)(s) = \int_0^\infty f(t)e^{-st}\,\mathrm{d}t$ wherever the integral converges absolutely.

With zero initial conditions, differentiation corresponds to multiplication by s. For nonzero initial conditions it also contributes boundary terms; for instance $\mathcal{L}(f')(s) = s\mathcal{L}(f)(s) - f(0^+)$. The zero-state transfer function of the circuit is therefore

$$K(s) = \frac{s}{1/(RC) + s + (L/R)s^2}.$$

Approximate each derivative by a backward difference with time step $\Delta > 0$. If $q = z^{-1}$ denotes a one-step delay, this substitutes $s = (1 - q)/\Delta$ into K and gives

$$H(z) = \frac{1 - q}{\Delta/(RC) + 1 + L/(R\Delta) - (1 + 2L/(R\Delta))q + Lq^2/(R\Delta)}.$$

Equivalently, the sampled voltages satisfy the finite-difference equation

$$\frac{V_e[n] - V_e[n-1]}{\Delta} = \frac{V_s[n]}{RC} + \frac{V_s[n] - V_s[n-1]}{\Delta} + \frac{L}{R\Delta^2}\bigl(V_s[n] - 2V_s[n-1] + V_s[n-2]\bigr).$$

The discretization approximates the analog dynamics. Its resulting recurrence and rational transfer function are exact descriptions of the digital filter. Other discretizations give different digital filters; Exercise 5.7 examines this choice for an integrator.

5.3 Vector Z transform and the chirp algorithm

The usual DFT samples a spectrum at equally spaced points around the unit circle. The vector Z transform allows a more flexible geometric progression of evaluation points, and the chirp factorization makes these evaluations fast by reducing them to convolution.

5.3.1 A fast evaluation algorithm

The essential step is to rewrite the product of input and output indices as a difference of squares. After defining the finite evaluation problem, we use this identity to derive a convolution that an FFT can compute.

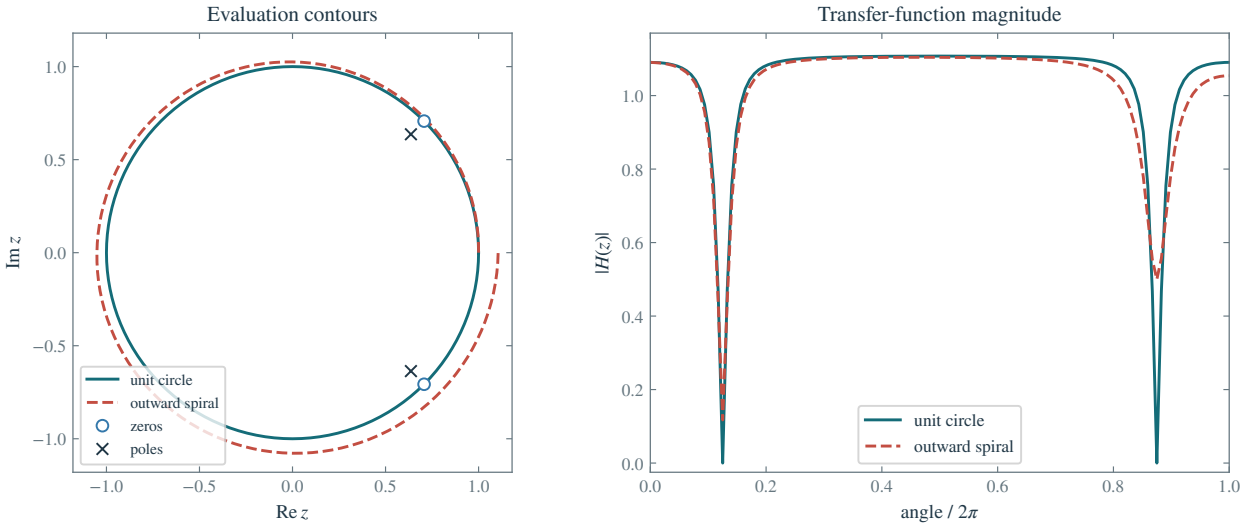


Figure 5.6. The notch-filter transfer function evaluated on a unit circle and an outward logarithmic spiral. Both contours lie in the causal region of convergence; leaving the unit circle changes the depth of the notches.

Definition 30 (Vector Z transform)**.** For $z \in \mathbb{C} \setminus \{0\}$ and $f \in \mathbb{C}^N$, define

$$\mathcal{G}_z(f)[n] = \sum_{k=0}^{N-1} f[k] z^{-nk}, \qquad 0 \le n < N. \tag{5.14}$$

These are exact evaluations of the finite Z transform at $1, z, z^2, \dots, z^{N-1}$. The associated Vandermonde matrix is invertible exactly when these points are distinct, or equivalently $z^d \neq 1$ for $1 \le d < N$. If $|z| = 1$, the evaluation points lie on a circle; otherwise they follow a logarithmic spiral.

Choose one logarithm L of z and use it consistently. The identity $nk = (n^2 + k^2 - (n-k)^2)/2$ yields

$$\mathcal{G}_z(f)[n] = e^{-Ln^2/2} \sum_{k=0}^{N-1} h[k] g[n-k], \qquad h[k] = f[k] e^{-Lk^2/2}, \quad g[j] = e^{Lj^2/2}.$$

Thus a diagonal scaling, a Toeplitz product, and a second diagonal scaling replace the dense Vandermonde product. The sign of the chirp kernel is positive. Although the factors involve half-integer exponents, the final integer powers z^{-nk} are independent of the chosen logarithm.

To compute M outputs, choose an FFT length $Q \ge N + M - 1$. Place $g[j]$ at index $j \bmod Q$ for $-(N-1) \le j \le M-1$, and pad h with zeros to length Q. The first M entries of their circular convolution are the desired sums: every difference $n - k$ lies in the stored range, with no conflicting residues. In particular $M = N$ only requires $Q \ge 2N - 1$. This embedding works for general Toeplitz matrices, without requiring an even kernel; see Exercise 5.6. Using FFTs gives the *chirp Z transform* (CZT) algorithm. Figure 5.6 illustrates these evaluation contours for the notch filter studied above. A truncated impulse response lets the CZT approximate its rational transfer function on either contour. Off the unit circle, the chirp factors can grow or decay rapidly. The algebraic identity remains exact, but floating-point overflow and cancellation can limit the accuracy of a direct chirp implementation.

5.3.2 Computing a DFT of arbitrary length

The chirp factorization supplies an FFT method even when the requested length has no useful small factors. Specializing the evaluation points to roots of unity gives Bluestein's algorithm without changing the desired DFT.

Choosing $z = e^{2\pi \mathrm{i}/N}$ gives $\mathcal{G}_z = \mathcal{F}_N$. The chirp construction therefore computes a DFT even when N is prime. With Q the smallest power of two satisfying $Q \ge 2N - 1$, it needs two forward FFTs and one inverse FFT of length Q, together with linear work. The kernel transform can be cached for repeated inputs. Since $Q < 4N$, the cost is $O(N \log N)$, although its constant differs from a radix-two FFT. This is Bluestein's algorithm; see [58]. Rader's alternative prime-length construction is developed in Exercise 5.9. Appending zeros directly to the

input also permits a longer FFT, but it changes the frequency grid and does not by itself compute the original length-N DFT.

5.4 Scaled-frequency Fourier transforms

Some applications need a detailed spectrum only within a selected frequency interval. We use the chirp algorithm to vary the sampling grid, then examine why denser evaluation does not remove the leakage caused by a finite observation window.

5.4.1 Changing the frequency grid

A single scale parameter changes the spacing of the evaluated frequencies. We define this finite transform and distinguish its invertibility properties from those of unitary fractional powers of the DFT.

For a real parameter α, define

$$G(f,\alpha)[k] = \sum_{n=0}^{N-1} f[n] e^{-2\pi i \alpha n k/N}, \qquad 0 \le k < N.$$

This is $\mathcal{G}_z(f)$ with $z = e^{2\pi i\alpha/N}$ and can therefore be computed by the CZT algorithm. It changes the spacing between sampled frequencies. It is sometimes called a fractional Fourier transform [6], but it is distinct from the unitary fractional powers of the normalized DFT in Section 3.5.2. For example,

$$G(f,1) = \mathcal{F}(f), \qquad G(f,-1) = N\mathcal{F}^{-1}(f), \qquad G(f,0)[k] = \sum_n f[n].$$

The last map is not the identity for $N > 1$. In general these transforms are neither unitary nor a group under composition, and replacing α by $-\alpha$ does not give the inverse. Their usefulness here lies in flexible spectral evaluation.

5.4.2 Finite records and spectral leakage

The spectrum of a finite record depends on the observation window as well as on the underlying frequencies. A single sinusoid makes this leakage explicit and shows what a finer frequency grid can reveal.

Consider $f[n] = e^{2\pi i\beta n/N}$ for $0 \le n < N$, and evaluate its finite-record spectrum at any real frequency ν:

$$S(\nu) = \sum_{n=0}^{N-1} e^{2\pi i(\beta-\nu)n/N}, \qquad |S(\nu)| = \left|\frac{\sin \pi(\beta-\nu)}{\sin(\pi(\beta-\nu)/N)}\right|.$$

At removable singularities, where $\beta - \nu \in N\mathbb{Z}$, the magnitude is N. If β is an integer, the DFT grid samples the zeros of the sidelobes, leaving a single nonzero bin. For noninteger β, the same windowed spectrum is sampled away from those zeros. This is spectral leakage, not a failure of the DFT to represent the finite vector.

Zooming around a spectral peak. A local spectral zoom evaluates only the neighborhood of interest, allowing finer sampling without a large transform over the full frequency range.

To evaluate $S(b + k\delta)$ for $0 \le k < M$, modulate the data and use an M-output chirp transform:

$$S(b+k\delta) = \sum_{n=0}^{N-1} \underbrace{f[n]e^{-2\pi i b n/N}}_{\widetilde{f}[n]} e^{-2\pi i \delta n k/N}.$$

A coarse FFT can locate a neighborhood of a peak; the CZT then samples that neighborhood densely without evaluating the entire frequency range. For noisy or multitone signals, the peak location need not equal an underlying component frequency exactly. Finer evaluation does not remove noise, window effects, or sampling aliases.

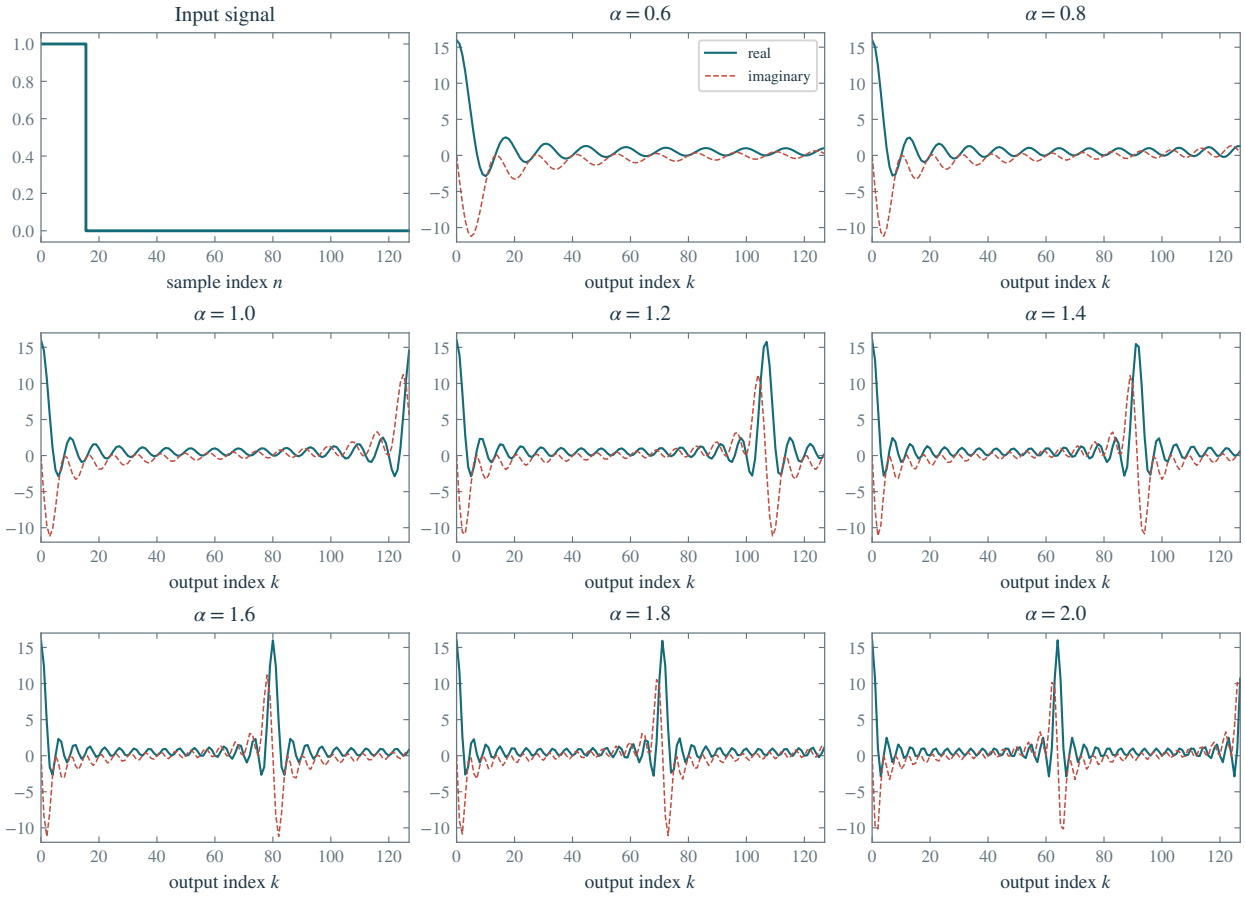


Figure 5.7. Scaled-frequency Fourier transforms of an interval indicator. The real and imaginary parts share the same scale. Changing α changes the sampled frequency grid.

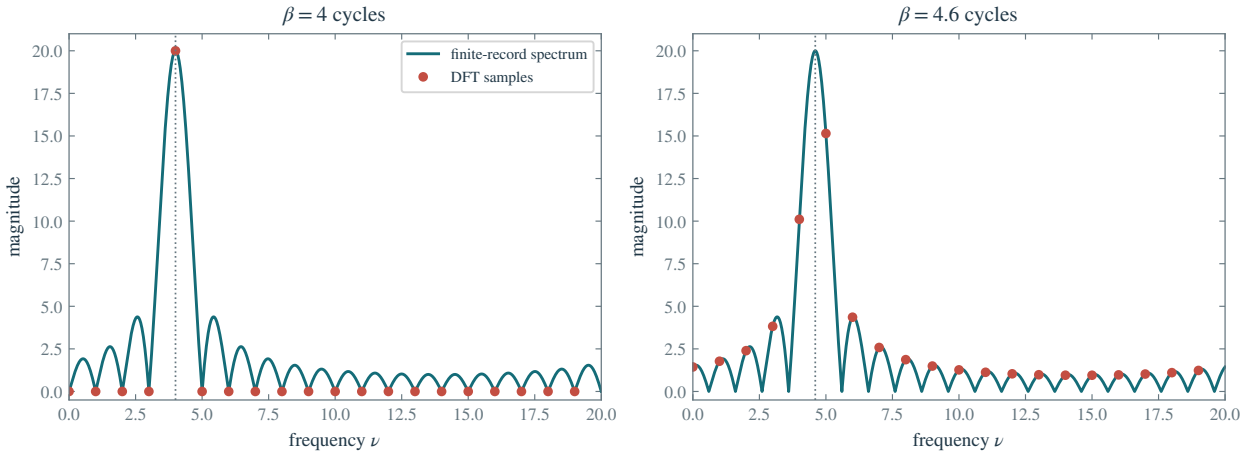


Figure 5.8. Finite-record spectra of sinusoids with integer and noninteger numbers of cycles. The continuous curves evaluate the finite sum; the dots are DFT samples. The vertical line marks the generating frequency.

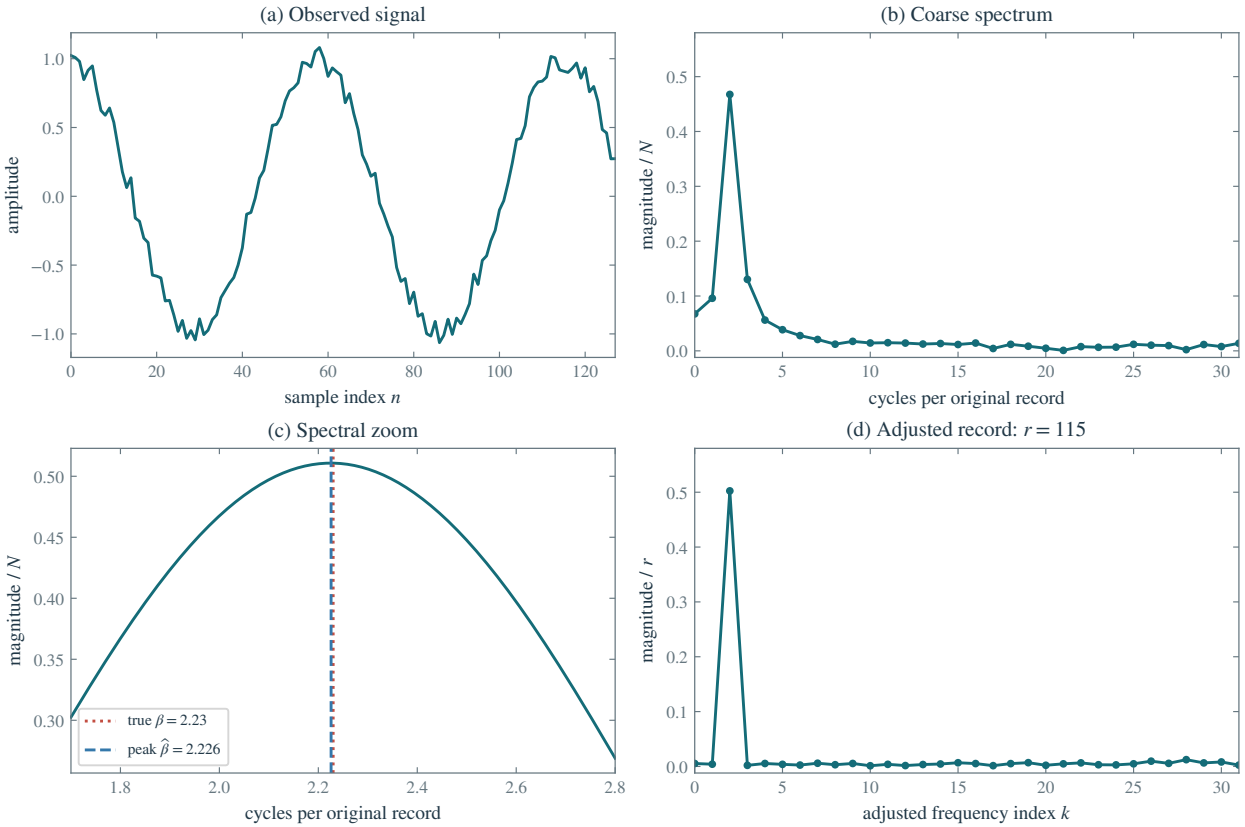


Figure 5.9. A noisy cosine, its coarse spectrum, a local spectral zoom, and an adjusted spectrum after shortening the record using the estimated peak. Spectral magnitudes are divided by the respective record lengths. The generating frequency is distinguished from its estimate.

Adjusting the observation length. Changing the observation window alters the spectrum itself, so it offers a different way to control leakage from a known pure tone.

For a pure tone with known $1 < \beta < N/2$, set $b = \lfloor \beta \rfloor$, $L = Nb/\beta$, and let r be the nearest integer to L. Retain the first $r \le N$ samples and evaluate

$$F_k = \sum_{n=0}^{r-1} f[n] e^{-2\pi i k n / L} = G\big(f[0{:}r-1], r/L\big)[k], \qquad 0 \le k < r.$$

Here $f[0{:}r-1]$ denotes the vector of those r samples. This operation *shortens* the record. It is not zero padding, and changing frequency spacing while retaining all N samples would not have the same effect. If $L = r$ is an integer, orthogonality gives $F_k = r\delta_b^k$. Otherwise, write $s = r - L$, so $|s| \le 1/2$. Then $F_b = r$, and for $k \ne b$,

$$|F_k| = \left| \frac{\sin(\pi s(b-k)/L)}{\sin(\pi(b-k)/L)} \right|,$$

with the usual limiting interpretation if the denominator vanishes. For $0 < |b-k| \le L/2$, the elementary sine bounds imply $|F_k| \le \pi|s|/2 \le \pi/4$. Close to the peak, the ratio is approximately $|s|$; farther away, small denominators can produce larger values. This adjustment reduces leakage for the chosen tone, at the cost of using fewer data and a frequency grid tailored to that tone.

5.5 Exercises

These exercises extend real-kernel transforms, recursive filtering, and flexible spectral evaluation. The first returns to spectral projectors to define powers of the Hartley transform with a consistent phase choice.

Exercise 5.1 (Eigenvectors and powers of the Hartley transform)**.** Find the possible eigenvalues of $\mathcal{H}$ and construct the orthogonal projectors onto the corresponding eigenspaces. Use these projectors to define a

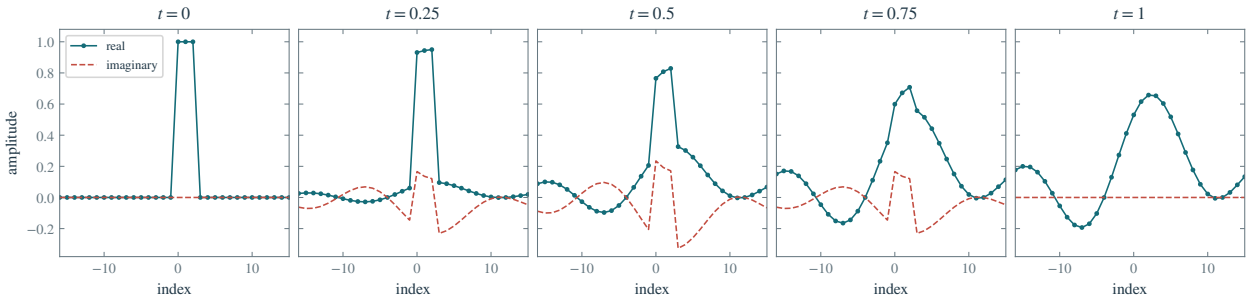


Figure 5.10. Real and imaginary parts of fixed-branch unitary Hartley powers $V^t f$, with $V = \mathcal{H}/\sqrt{N}$. All panels use the same amplitude scale and preserve the norm.

continuous one-parameter group of unitary powers V^t of $V = \mathcal{H}/\sqrt{N}$, with $V^0 = \mathrm{Id}$ and $V^1 = V$. State the branch chosen for the eigenvalue -1, and explain how to recover powers of the unnormalized transform. Compare with the construction in Section 3.5.2.

Changing the phase of the real kernel gives a different family, whose invertibility must be checked separately.

Exercise 5.2 (Generalized Hartley transform)**.** The identity

$$\operatorname{cas} x = \sqrt{2}\cos(x - \pi/4) \tag{5.15}$$

suggests, for $N \geq 3$, the family

$$\mathcal{H}_\lambda(f)[n] = \sum_{k=0}^{N-1} f[k] \cos\left(\frac{2\pi nk}{N} + \lambda\right).$$

Show that this map is invertible precisely when λ is not an integer multiple of $\pi/2$, and prove

$$\mathcal{H}_\lambda^{-1} = \frac{2}{N \sin(2\lambda)} \mathcal{H}_{\pi/2-\lambda}. \tag{5.16}$$

Distinguish this family, which changes the kernel phase, from the fractional powers in the preceding exercise.

Hartley coefficients can also represent the same trigonometric interpolant as the DFT, using only real basis functions.

Exercise 5.3 (Interpolation by the Hartley transform)**.** Let $N = 2K+1$ and $f \in \mathbb{R}^N$. Construct the trigonometric polynomial with frequencies $-K, \dots, K$ that interpolates f at the integers $0, \dots, N-1$, using its Hartley coefficients. Explain how centered frequency padding evaluates it on a finer uniform grid. Prove that it is the same interpolant as in Exercise 3.6, and specify the normalization after padding.

A tensor-product construction extends the Hartley transform to images while preserving separability.

Exercise 5.4 (A separable two-dimensional Hartley transform)**.** For $f \in \mathbb{R}^{N_1 \times N_2}$ define

$$\mathcal{H}_2(f)[n_1, n_2] = \sum_{k_1=0}^{N_1-1} \sum_{k_2=0}^{N_2-1} f[k_1, k_2] \operatorname{cas} \frac{2\pi k_1 n_1}{N_1} \operatorname{cas} \frac{2\pi k_2 n_2}{N_2}.$$

Prove $\mathcal{H}_2^2 = N_1 N_2 \,\mathrm{Id}$ and give an algorithm based on row and column transforms. This tensor-product convention uses a product of two cas functions, not cas of the sum of the two phases.

The sine and cosine formulas can be expressed algebraically, allowing exact Hartley transforms over suitable finite fields and rings.

Exercise 5.5 (Hartley transforms over finite fields)**.** Let $\mathbb{F}_q$ have odd characteristic, let ζ have order N, and suppose that the field contains ι with $\iota^2 = -1$. Define a Hartley kernel by replacing cosine and sine with

$$C_j = \frac{\zeta^j + \zeta^{-j}}{2}, \qquad S_j = \frac{\zeta^j - \zeta^{-j}}{2\iota}.$$

Prove the inversion identity and derive a radix-two FHT when N is a power of two. Extend the argument to a commutative ring in which 2 and N are units and ζ is a principal Nth root of unity. Implement an exact example in $\mathbb{F}_5[X]/(X^4 + 3)$ with $N = 16$ and $\iota = 2$. The finite-field constructions are developed in the next chapter.

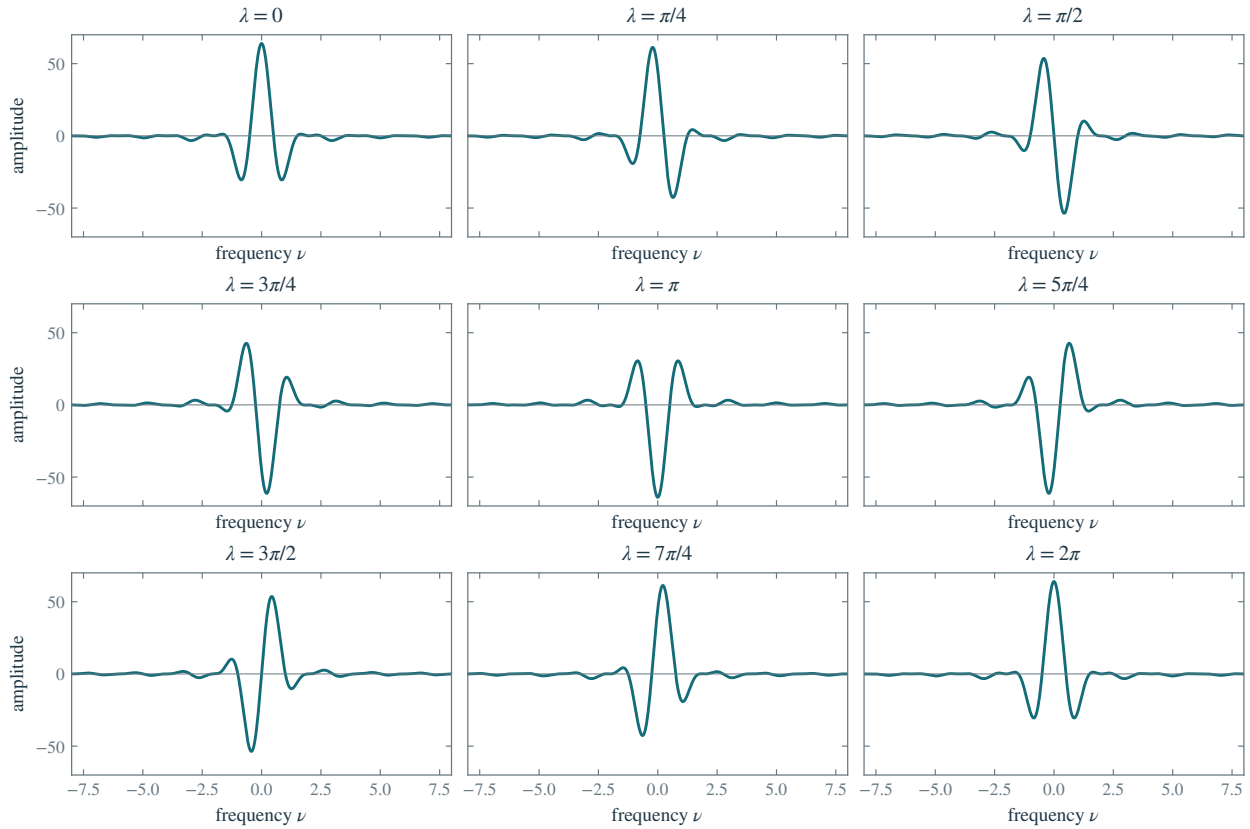


Figure 5.11. Generalized Hartley transforms of the same triangular signal. The phase λ changes the relative cosine and sine contributions; the two endpoint panels illustrate the 2π periodicity.

Toeplitz matrices expose the convolution hidden in a chirp transform and explain its embedding into a circulant matrix.

Exercise 5.6 (Chirp transforms and Toeplitz matrices)**.** Choose a logarithm L of $z \neq 0$ and set $h[k] = f[k]e^{-Lk^2/2}$ and $g[j] = e^{Lj^2/2}$. Then $y[n] = e^{Ln^2/2}\mathcal{G}_z(f)[n] = \sum_{k=0}^{N-1} g[n-k]h[k]$.

1. Show that $y = Th$ for a Toeplitz matrix T, and write T when $N = 3$.
2. A general $m \times n$ Toeplitz matrix has entries $T_{ij} = t_{i-j}$. Denote its first column by $c = (t_0, \dots, t_{m-1})$ and its first row by $r = (t_0, t_{-1}, \dots, t_{-(n-1)})$. For $Q \geq m + n - 1$, embed these data in

$$v = (c_0, \dots, c_{m-1}, \underbrace{0, \dots, 0}_{Q-m-n+1}, r_{n-1}, \dots, r_1).$$

   Locate T in the circulant matrix $C(v)$ with entries $v[i - j \bmod Q]$, and use this embedding to compute Tx by FFTs.
3. Apply the construction to the chirp kernel. Write the embedded vector for $N = 3$, $Q = 8$, and implement the algorithm for N input samples and M output values.

Different quadrature rules for a continuous integrator produce different digital recurrences and transfer functions.

Exercise 5.7 (Quadrature methods as recursive filters)**.** For a continuous input, consider the integrator

$y(t) = \int_0^t x(\tau)\mathrm{d}\tau$. Find its zero-state Laplace transfer function. Sample with spacing $\Delta > 0$, and use the rules

$$\begin{aligned} (M_0) \quad & \int_0^1 f(t)\mathrm{d}t \approx f(0), \\ (M_1) \quad & \int_0^1 f(t)\mathrm{d}t \approx \tfrac{1}{2}(f(0) + f(1)), \\ (M_2) \quad & \int_0^1 f(t)\mathrm{d}t \approx \tfrac{1}{6}(f(0) + 4f(1/2) + f(1)). \end{aligned} \tag{5.17}$$

Derive the left-rectangle and trapezoidal recurrences over one time step, and Simpson's recurrence over two steps. Determine the corresponding zero-state rational digital transfer functions. Explain the initialization needed to approximate the integral from 0, and identify the poles.

A causal pass followed by an anticausal pass can implement spline prefiltering, provided the endpoint conditions are initialized correctly.

Exercise 5.8 (Cubic spline prefiltering by recursions)**.** Use the notation of Exercise 4.12 and the centered cubic B-spline β^3.

1. Compute its integer samples β^3_d and prove $\mathcal{Z}(\beta^3_d)(z) = (z + 4 + z^{-1})/6$.
2. Factor the reciprocal of this expression and write it as a sum of a causal and an anticausal first-order transfer function, plus a constant. Identify the common annulus of convergence that gives a stable inverse.
3. For data $u[0], \ldots, u[K-1]$ with $K \geq 2$, impose whole-sample mirror extension: $u[-j] = u[j]$ and $u[K-1+j] = u[K-1-j]$. Derive exact endpoint initialization for the forward and backward recursions, then compute the interpolation coefficients from their sum.
4. Derive an alternative cascade of the two filters, with exact mirror initialization. Verify that both methods reproduce constant data and satisfy the spline interpolation equations at every sample, including the endpoints. See [63, 62] for the filtering interpretation.

For a prime transform length, multiplicative indexing provides another reduction to circular convolution.

Exercise 5.9 (Rader's prime-length FFT)**.** Let p be prime, g a generator of $\mathbb{F}_p^*$, and $\omega_p = e^{2\pi\mathrm{i}/p}$. For $f : \mathbb{F}_p \to \mathbb{C}$, show that

$$\hat{f}(g^{-b}) = f(0) + \sum_{a=0}^{p-2} f(g^a)\omega_p^{-g^{a-b}}, \qquad 0 \leq b < p-1.$$

Interpret this as a circular convolution on $\mathbb{Z}/(p-1)\mathbb{Z}$, specify its inputs and the output permutation, and implement the algorithm. Keep the zero-frequency coefficient separate.

The chirp transform can evaluate a quadrature approximation on a chosen physical frequency grid, with the phases and weights made explicit.

Exercise 5.10 (Fourier-integral quadrature on a flexible frequency grid)**.** Let f be continuous on $[a_0, a_0 + a]$ and zero outside. The desired frequencies are $\xi_k = \xi_0 + k\delta$ for $0 \leq k < M$.

1. Use the left-rectangle samples $x_j = a_0 + jh$, $0 \leq j < N$, with $h = a/N$, to approximate $\hat{f}(\xi_k) = \int f(t)e^{-\mathrm{i}\xi_k t}\mathrm{d}t$ by one chirp transform. Specify the modulation, chirp parameter, outer phase, and factor h. If $h\delta/(2\pi) = p/q$ in lowest terms, explain how the same finite sum can be read from a suitably padded DFT of length a multiple of q. Compare this with the chirp construction when q is large.
2. Replace the rule by composite Simpson quadrature on an odd number $N \geq 3$ of samples *including both endpoints*, now with $h = a/(N-1)$. Give the weights and implement the approximation with all physical scaling and phase factors included.

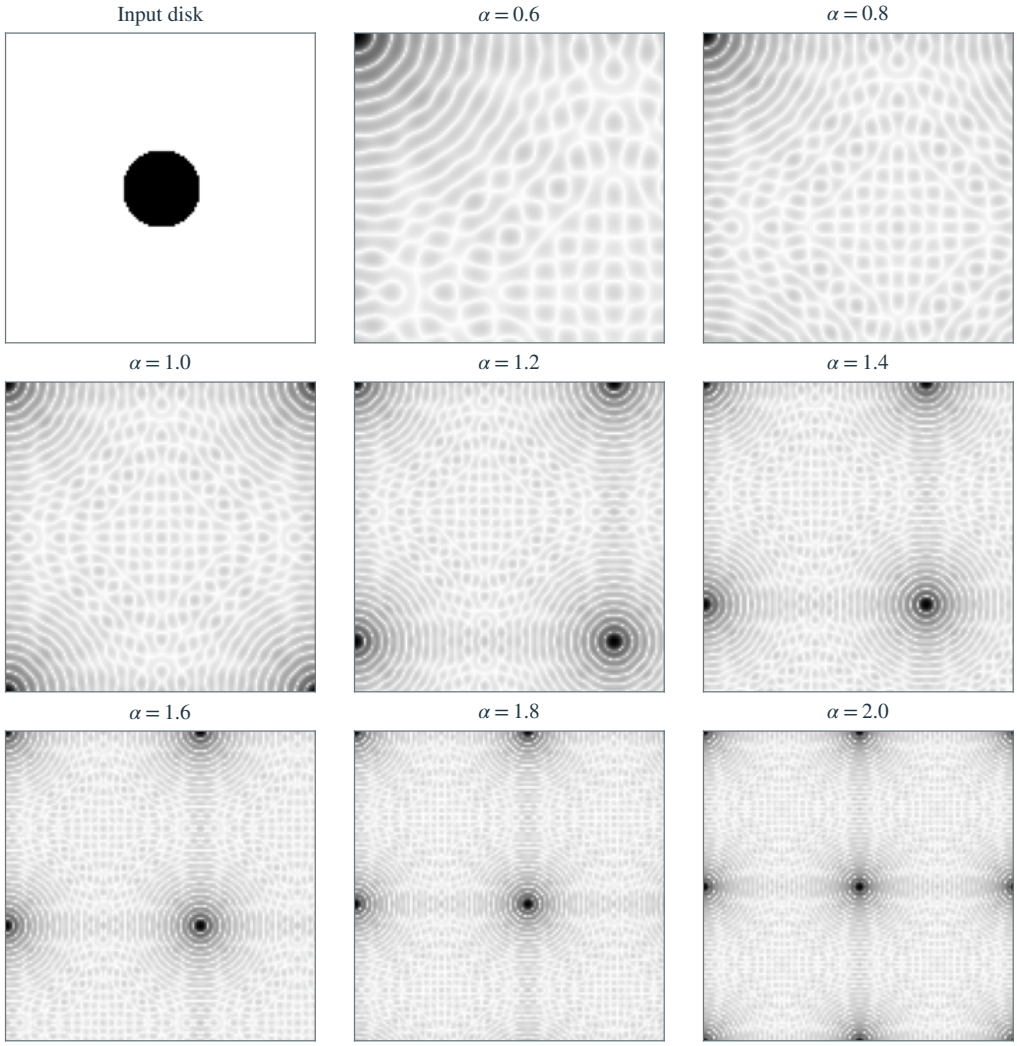


Figure 5.12. Magnitudes of scaled-frequency two-dimensional Fourier transforms of a disk. Spectral panels share the contrast mapping $\log(1 + 100|G_2|/\sum f)/\log(101)$, shown dark on white; the input panel uses ordinary grayscale.

Applying the scaled-frequency transform along each coordinate extends spectral zooming to rectangular images.

Exercise 5.11 (Scaled-frequency transforms of an image)**.** Extend $G(f, \alpha)$ to rectangular two-dimensional arrays by tensor product. Write a Python function that applies the transform along both axes, and check that $\alpha = 1$ gives the usual two-dimensional DFT. Test it on a disk indicator, displaying the magnitude of the complex output.

5.6 Solutions to the exercises

These solutions distinguish changes of kernel, spectral powers, and changes of frequency grid. They also show how convergence regions and endpoint conditions determine the correct recursive filter, and how chirp factorizations reduce flexible evaluation to convolution.

Solution to Exercise 5.1

Since $\mathcal{H}^2 = N\,\mathrm{Id}$ and $\mathcal{H}$ is real symmetric, its possible eigenvalues are $\pm\sqrt{N}$. For $V = \mathcal{H}/\sqrt{N}$, the orthogonal projectors are

$$P_+ = \tfrac{1}{2}(\mathrm{Id} + V), \qquad P_- = \tfrac{1}{2}(\mathrm{Id} - V).$$

They satisfy $P_+ + P_- = \mathrm{Id}$, $P_+P_- = 0$, and $VP_\pm = \pm P_\pm$. Thus a nonzero $P_\pm f$ is an eigenvector; zero projections must be discarded. Choosing $\log(-1) = \mathrm{i}\pi$ gives

$$V^t = P_+ + e^{\mathrm{i}\pi t}P_-, \qquad \mathcal{H}^t = N^{t/2}V^t.$$

The family V^t is unitary for real t, satisfies $V^{s+t} = V^sV^t$, and starts at the identity. The unnormalized powers have the additional factor $N^{t/2}$. Program 40 computes the unitary version used in the figure.

```
def fractional_hartley(x, power):
    """A fixed-branch unitary power of H/sqrt(N), not of unnormalized H."""
    x = np.asarray(x)
    transformed = hartley(x)/np.sqrt(x.size)
    return (x+transformed)/2 + np.exp(1j*np.pi*power)*(x-transformed)/2
```

Program 40. Unitary fractional powers of the normalized Hartley transform.

Solution to Exercise 5.2

Let $C_{nk} = \cos(2\pi nk/N)$ and $S_{nk} = \sin(2\pi nk/N)$, and let $Jf = f^\sharp$. Root-of-unity sums give

$$C^2 = \tfrac{N}{2}(\mathrm{Id} + J), \quad S^2 = \tfrac{N}{2}(\mathrm{Id} - J), \quad CS = SC = 0.$$

Now $\mathcal{H}_\lambda = \cos\lambda\, C - \sin\lambda\, S$, so

$$\mathcal{H}_\lambda\mathcal{H}_{\pi/2-\lambda} = \sin\lambda\cos\lambda(C^2 + S^2) = \tfrac{N}{2}\sin(2\lambda)\,\mathrm{Id}\,.$$

This proves the inverse formula, including the factor $1/N$. If $\sin\lambda = 0$, the antisymmetric subspace is annihilated; if $\cos\lambda = 0$, the symmetric subspace is annihilated. Both subspaces are nonzero for $N \geq 3$, proving the stated necessity as well.

Solution to Exercise 5.3

Write $N = 2K + 1$ and $g = \mathcal{H}(f)$, with indices interpreted modulo N. The interpolant is

$$F(x) = \frac{1}{N}\sum_{k=-K}^{K} g[k \bmod N]\,\mathrm{cas}\,\frac{2\pi kx}{N}.$$

At integer x, the inversion identity gives $F(x) = f[x]$. The use of *signed* frequencies matters away from those integers. For a finer grid of $P \geq N$ points, put $g[k \bmod N]$ at index $k \bmod P$ of an otherwise zero vector $\widetilde{g}$. Then

$$F(nN/P) = \frac{1}{N}\mathcal{H}_P(\widetilde{g})[n].$$

Equivalently, using an inverse Hartley transform requires the factor P/N. Both this interpolant and the Fourier interpolant belong to the N-dimensional space spanned by $e^{2\pi\mathrm{i}kx/N}$ for $-K \leq k \leq K$. Their difference has all N samples zero; DFT invertibility makes all its coefficients zero. Hence the two interpolants agree.

Solution to Exercise 5.4

Let H_1, H_2 be the Hartley matrices of sizes N_1, N_2. The separable transform is $\mathcal{H}_2(f) = H_1 f H_2^{\mathsf{T}}$. Applying it twice gives $H_1^2 f (H_2^{\mathsf{T}})^2 = N_1 N_2 f$. Thus transforming along the two axes computes both the forward map and, after division by $N_1 N_2$, its inverse. Program 41 works on rectangular arrays. Its one-dimensional routine accepts any length; radix-two FHTs can be substituted when both lengths are powers of two.

```
def hartley2d(x):
    x = np.asarray(x)
    if x.ndim != 2:
        raise ValueError('Use a two-dimensional array.')
    return hartley(hartley(x, axis=0), axis=1)
```

Program 41. A separable Hartley transform on a rectangular array.

Solution to Exercise 5.5

Define $\mathrm{cas}_j = C_j + S_j$. The algebraic addition formulas and root sums prove $H^2 = N\,\mathrm{Id}$ exactly as over $\mathbb{C}$. In a finite field, N divides $q - 1$, so it is nonzero in the field and can be inverted. The proof also applies over the stated ring: principality gives the root sums, while invertibility of 2 and N permits the required divisions. This is a transpose-orthogonality identity, not a claim about a positive inner product on a finite field.

For the radix-two recursion, replace the trigonometric twiddle factors by C_j and S_j, and replace ζ by ζ^2 in each half-length transform. Keep the same ι. In $\mathbb{F}_5[X]/(X^4+3)$, the polynomial X^4+3 is irreducible and $\zeta = X$ has order 16: $X^4 = 2$, $X^8 = -1$, and $X^{16} = 1$. Also $2^2 = -1$ in characteristic five. Program 42 performs exact arithmetic and checks the double transform. In this particular example, $16 = 1$ in the field, so the inversion scale happens to be one.

```
def finite_fht(x, root, iota):
    """Radix-two H over an odd finite field; root has order len(x), iota^2=-1."""
    field, n = type(x), x.size
    if n == 0 or n & (n-1):
        raise ValueError('The length must be a power of two.')
    if root**n != 1 or (n > 1 and root**(n//2) == 1) or iota**2 != -field(1):
        raise ValueError('The root orders or square root of -1 are incorrect.')
    if n == 1:
        return x.copy()
    even, odd = finite_fht(x[::2], root**2, iota), finite_fht(x[1::2], root**2, iota)
    a = root**np.arange(n//2)
    cosine, sine = (a+a**-1)/field(2), (a-a**-1)/(field(2)*iota)
    mixed = cosine*odd + sine*odd[(-np.arange(n//2)) % (n//2)]
    return field(np.concatenate((even+mixed, even-mixed)))
```

```
def finite_hartley_example():
    import galois
    prime = galois.GF(5, compile='python-calculate')
    polynomial = galois.Poly([1, 0, 0, 0, 3], field=prime)
    field = galois.GF(5**4, irreducible_poly=polynomial, compile='python-calculate')
    x = field([0, 1, 1, 1, 2, 2, 1, 4, 0, 1, 2, 0, 1, 0, 2, 0])
    root, iota = field(5), field(2)  # X modulo X^4+3, and a square root of -1
    y = finite_fht(x, root, iota)
    assert np.array_equal(finite_fht(y, root, iota), x)
    return x, y
```

Program 42. An exact finite-field Hartley transform and its inversion check.

Solution to Exercise 5.6

1. The entries are $T_{ij} = g[i-j]$. Since the chirp kernel is even, for $N = 3$ this gives
$$T = \begin{pmatrix} g[0] & g[1] & g[2] \\ g[1] & g[0] & g[1] \\ g[2] & g[1] & g[0] \end{pmatrix}.$$
2. The upper-left $m \times n$ block of $C(v)$ is T. Indeed, nonnegative differences $i - j$ select the first-column entries, while negative differences select the reversed row at the end of v. Pad x to length Q and compute $\widetilde{y} = \mathcal{F}_Q^{-1}(\mathcal{F}_Q(v)\mathcal{F}_Q(\widetilde{x}))$. Its first m entries give Tx.
3. For $N = 3$, $Q = 8$, the first column of the circulant is $(g[0], g[1], g[2], 0, 0, 0, g[2], g[1])$. More generally, store the kernel at residues of the differences $-(N-1), \ldots, M-1$. Program 43 includes both chirp multipliers and returns exactly the requested M outputs in exact arithmetic. It uses one consistent complex logarithm.

```
def czt(x, z, count=None):
    """Return sum_j x[j] z**(-j*k), for k=0,...,count-1."""
    x = np.asarray(x, dtype=complex)
    n, m = x.size, x.size if count is None else int(count)
    if x.ndim != 1 or n == 0 or m < 1 or z == 0:
        raise ValueError('Use a nonempty vector, nonzero z, and positive output length.')
    size = 1 << (n+m-2).bit_length()
    logarithm = np.log(complex(z))
    j, k = np.arange(n), np.arange(m)
    differences = np.arange(1-n, m)
    kernel = np.zeros(size, dtype=complex)
    kernel[differences % size] = np.exp(logarithm*differences**2/2)
    h = x*np.exp(-logarithm*j**2/2)
    convolution = np.fft.ifft(np.fft.fft(h, size)*np.fft.fft(kernel))
    return np.exp(-logarithm*k**2/2)*convolution[:m]
```

Program 43. A chirp transform with arbitrary input and output lengths.

Solution to Exercise 5.7

The zero-state Laplace transfer function is $K(s) = 1/s$. For $x_n = x(n\Delta)$, the three discrete recurrences are

$$\begin{aligned} (M_0) \quad & y_n - y_{n-1} = \Delta x_{n-1}, \\ (M_1) \quad & y_n - y_{n-1} = \tfrac{\Delta}{2}(x_n + x_{n-1}), \\ (M_2) \quad & y_n - y_{n-2} = \tfrac{\Delta}{3}(x_n + 4x_{n-1} + x_{n-2}). \end{aligned}$$

For the numerical integral from zero, initialize $y_0 = 0$ and, in Simpson's method, also supply y_1, for instance by a trapezoidal step. The two-step Simpson recurrence then evolves separate even and odd chains.

As zero-state causal filter equations, using $q = z^{-1}$, their transfer functions are

$$H_0 = \frac{\Delta q}{1-q}, \qquad H_1 = \frac{\Delta(1+q)}{2(1-q)}, \qquad H_2 = \frac{\Delta(1+4q+q^2)}{3(1-q^2)}.$$

These transfer functions correspond to zero history for the recurrences; transforming the finite-interval numerical integral with its prescribed y_0, y_1 contributes the corresponding initialization terms. All three filters have a pole at $z = 1$, reflecting the accumulation of a constant input. Simpson's filter also has a pole at $z = -1$, associated with an alternating mode. None is bounded-input, bounded-output stable. The curves in Figure 5.13 show their frequency responses away from these poles.

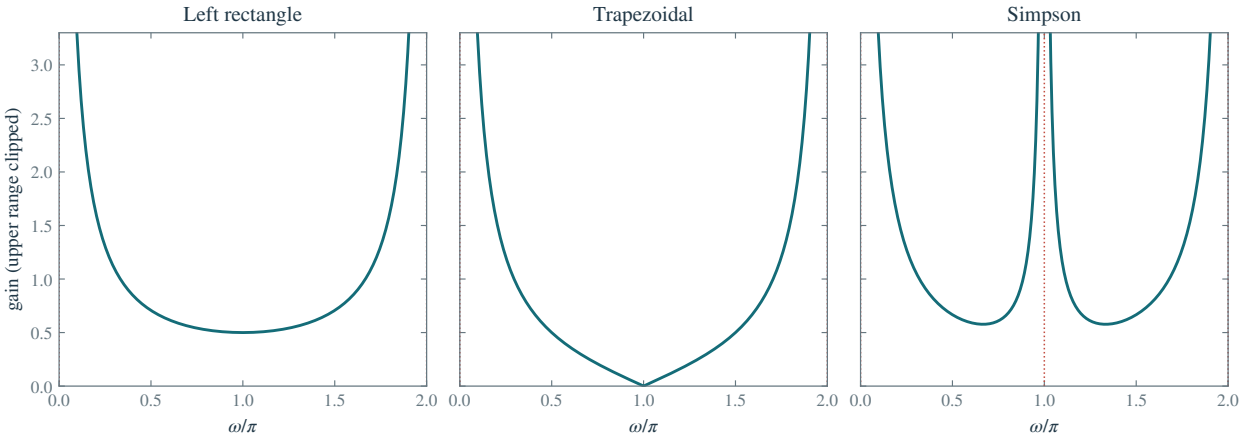


Figure 5.13. Magnitudes of the rectangle, trapezoidal, and Simpson integration filters for $\Delta = 1$. The poles at zero frequency and the additional Simpson pole at the Nyquist frequency are excluded from the plotted grid.

Solution to Exercise 5.8

1. Convolving four interval indicators gives

$$\beta^3(x) = \begin{cases} 2/3 - |x|^2 + |x|^3/2, & |x| \le 1, \\ (2 - |x|)^3/6, & 1 \le |x| \le 2, \\ 0, & |x| \ge 2. \end{cases}$$

   Its only nonzero integer samples are $1/6, 2/3, 1/6$ at $-1, 0, 1$. Thus $D(z) = \mathcal{Z}(\beta^3_d)(z) = (z + 4 + z^{-1})/6$.

2. Set $q = \sqrt{3} - 2$ and $A = -6q$. Since $|q| < 1$ and $-6q/(1 - q^2) = \sqrt{3}$, factorization and partial fractions give

$$\frac{1}{D(z)} = \frac{A}{(1 - qz^{-1})(1 - qz)} = \sqrt{3}\left(\frac{1}{1 - qz^{-1}} + \frac{1}{1 - qz} - 1\right).$$

   The causal series converges for $|z| > |q|$, and the anticausal series for $|z| < 1/|q|$. Their common annulus contains the unit circle and defines the stable inverse. Both denominators have a minus sign.

3. Write $P = 2(K - 1)$ for the period of the mirror extension. The exact initial value for the causal pass is

$$\begin{aligned} d_0 &= \sum_{j=0}^{\infty} q^j u[-j] \\ &= \frac{u_0 + q^{K-1}u_{K-1} + \sum_{j=1}^{K-2}(q^j + q^{P-j})u_j}{1 - q^P}. \end{aligned}$$

   To obtain it, group the infinite sum by periods of length P and sum the geometric series. Initialize the anticausal pass e_{K-1} by the same formula applied to the reversed data. Then compute

$$d_k = u_k + qd_{k-1}, \qquad e_k = u_k + qe_{k+1}, \qquad c_k = \sqrt{3}(d_k + e_k - u_k).$$

   The forward recursion runs from 1 to $K - 1$, and the backward recursion from $K - 2$ down to 0. Arbitrarily setting their initial values to zero would change the boundary condition and would not even preserve constant data. Program 44 includes the exact initialization. For $K = 1$, use the constant extension, which gives $c_0 = u_0$.

```
def mirror_initial(u, q):
    """Exact causal initial value for whole-sample mirror extension."""
    u = np.asarray(u)
    n = u.size
    if n == 1:
        return u[0]/(1-q)
    j = np.arange(1, n-1)
    numerator = u[0] + q**(n-1)*u[-1] + np.sum((q**j+q**(2*n-2-j))*u[j])
    return numerator/(1-q**(2*n-2))
```

```
def cubic_prefilter_parallel(u):
    """Cubic B-spline coefficients, using two parallel mirror-boundary recursions."""
    u = np.asarray(u)
    if u.ndim != 1 or u.size == 0:
        raise ValueError('Use a nonempty vector.')
    q = np.sqrt(3)-2
    forward = np.empty(u.size, dtype=np.result_type(u, float))
    backward = np.empty_like(forward)
    forward[0], backward[-1] = mirror_initial(u, q), mirror_initial(u[::-1], q)
    for k in range(1, u.size):
        forward[k] = u[k] + q*forward[k-1]
    for k in range(u.size-2, -1, -1):
        backward[k] = u[k] + q*backward[k+1]
    return np.sqrt(3)*(forward+backward-u)
```

Program 44. Cubic spline prefiltering with exact mirror initialization and parallel passes.

4. The product factorization gives a causal pass $d_k = u_k + q d_{k-1}$ with the same d_0, followed by

$$c_k = A d_k + q c_{k+1}.$$

To initialize the backward pass, use mirror symmetry $c_K = c_{K-2}$ and the two equations at $K-1$ and $K-2$. Eliminating c_{K-2} gives

$$c_{K-1} = \frac{A}{1-q^2}(d_{K-1} + q d_{K-2}).$$

This proves Program 45. Both constructions solve

$$u_0 = \frac{4c_0 + 2c_1}{6}, \quad u_k = \frac{c_{k-1} + 4c_k + c_{k+1}}{6}, \quad u_{K-1} = \frac{2c_{K-2} + 4c_{K-1}}{6}.$$

The middle formula applies to $1 \le k \le K-2$. These equations also provide a direct way to verify the coefficients. The boundary convention here is mirror extension, distinct from the natural or not-a-knot conditions in the previous chapter.

```
def cubic_prefilter_cascade(u):
    """The same mirror-boundary prefilter as a causal/anticausal cascade."""
    u = np.asarray(u)
    if u.ndim != 1 or u.size == 0:
        raise ValueError('Use a nonempty vector.')
    if u.size == 1:
        return u.astype(np.result_type(u, float))
    q = np.sqrt(3)-2
    d = np.empty(u.size, dtype=np.result_type(u, float))
    d[0] = mirror_initial(u, q)
    for k in range(1, u.size):
        d[k] = u[k] + q*d[k-1]
    c = np.empty_like(d)
    c[-1] = -6*q*(d[-1]+q*d[-2])/(1-q*q)
    for k in range(u.size-2, -1, -1):
        c[k] = -6*q*d[k] + q*c[k+1]
    return c
```

Program 45. The same mirror-boundary spline prefilter as a cascade.

Solution to Exercise 5.9

Since $a \mapsto g^a$ enumerates the nonzero residues, for $0 \le b < p-1$,

$$\hat{f}(g^{-b}) = f(0) + \sum_{a=0}^{p-2} f(g^a)\omega_p^{-g^{a-b}}.$$

Set $h[a] = f(g^a)$ and $v[a] = \omega_p^{-g^{-a}}$, both indexed modulo $p-1$. Then $\hat{f}(g^{-b}) = f(0) + (h * v)[b]$. The remaining output is $\hat{f}(0) = \sum_x f(x)$. Three length-$(p-1)$ transforms compute the convolution; the transform of v can be reused for repeated inputs. The benefit depends on how efficiently this composite length is handled. Program 46 performs the permutations explicitly and checks the prime-length and generator assumptions.

```
def rader_fft(x, generator):
    """Prime-length DFT via a cyclic convolution of length p-1."""
    from sympy import isprime
    x = np.asarray(x, dtype=complex)
    p = x.size
    indices = np.array([pow(int(generator), k, p) for k in range(p-1)])
    if not isprime(p) or set(indices) != set(range(1, p)):
        raise ValueError('Use prime length and a generator of its nonzero residues.')
    inverse = np.array([inverse_modulo(j, p) for j in indices])
    kernel = np.exp(-2j*np.pi*inverse/p)
    convolution = np.fft.ifft(np.fft.fft(x[indices])*np.fft.fft(kernel))
    y = np.empty(p, dtype=complex)
    y[0], y[inverse] = x.sum(), x[0]+convolution
    return y
```

Program 46. Rader's prime-length FFT with explicit input and output permutations.

Python's three-argument `pow` computes a modular inverse with an exponent of -1, raising an error when the inverse does not exist.

```
def inverse_modulo(x, modulus):
    return pow(int(x), -1, int(modulus))
```

Program 47. Exact modular inversion using Python integers.

Solution to Exercise 5.10

1. With $x_j = a_0 + jh$ and $\xi_k = \xi_0 + k\delta$, the rectangle rule gives

$$\begin{aligned}\hat{f}(\xi_k) &\approx h \sum_{j=0}^{N-1} f(x_j) e^{-\mathrm{i}\xi_k x_j} \\ &= h e^{-\mathrm{i}a_0\xi_k} \sum_{j=0}^{N-1} \underbrace{f(x_j) e^{-\mathrm{i}h\xi_0 j}}_{\widetilde{f}[j]} e^{-\mathrm{i}h\delta jk}.\end{aligned}$$

   The inner sum is an M-output CZT with $z = e^{\mathrm{i}h\delta}$. If $h\delta/(2\pi) = p/q$ in lowest terms, choose $Q \ge N$ a multiple of q and pad $\widetilde{f}$ to length Q. The required entries of its DFT have indices $(pQ/q)k \bmod Q$. In particular, $Q = q$ suffices when $q \ge N$. For q much larger than $N + M$, a chirp convolution of length at least $N + M - 1$ can avoid that large transform. These are equivalent ways to evaluate the chosen quadrature sum; neither removes its quadrature error.
2. With $N = 2J + 1$ endpoint-inclusive samples and $h = a/(N-1)$, Simpson's rule assigns weights

$$(w_0, \ldots, w_{N-1}) = \tfrac{1}{3}(1, 4, 2, 4, 2, \ldots, 4, 1).$$

Replace $\widetilde{f}[j]$ by $w_j f(x_j) e^{-\mathrm{i}h\xi_0 j}$ and retain the outer factor $h e^{-\mathrm{i}a_0\xi_k}$. Program 48 uses the unscaled integer weights and puts $1/3$ in the outer factor. The new spacing h and the final endpoint are essential; they differ from the rectangle grid.

```
def simpson_fourier(samples, start, step, frequency, spacing, count):
    """Approximate integral f(t) exp(-i*xi*t) dt at xi=frequency+k*spacing."""
    samples = np.asarray(samples)
    n = samples.size
    if n < 3 or n % 2 != 1 or step <= 0:
        raise ValueError('Simpson quadrature needs an odd number of at least 3 samples.')
    weights = np.ones(n)
    weights[1:-1:2], weights[2:-1:2] = 4, 2
    j = np.arange(n)
    x = weights*samples*np.exp(-1j*step*frequency*j)
    xi = frequency+spacing*np.arange(count)
    values = czt(x, np.exp(1j*step*spacing), count)
    return (step/3)*np.exp(-1j*start*xi)*values
```

Program 48. Fourier-integral quadrature with Simpson weights and physical phases.

Solution to Exercise 5.11

For $f \in \mathbb{C}^{N_1 \times N_2}$, the separable transform is

$$G_2(f,\alpha)[k,l] = \sum_{n=0}^{N_1-1} \sum_{m=0}^{N_2-1} f[n,m] \exp\left(-2\pi\mathrm{i}\alpha\left(\frac{nk}{N_1} + \frac{ml}{N_2}\right)\right).$$

Apply the one-dimensional map along each axis. The chirp parameter for an axis of length N is $e^{+2\pi\mathrm{i}\alpha/N}$, since the definition of the CZT already contains a negative exponent. At $\alpha = 1$, this recovers the usual DFT with its negative phase; the opposite sign would instead give the unnormalized inverse transform.

```
def scaled_fourier(x, alpha=1):
    x = np.asarray(x)
    return czt(x, np.exp(2j*np.pi*alpha/x.size))
```

Program 49. A scaled-frequency Fourier transform with the DFT sign convention.

```
def scaled_fourier2d(x, alpha=1):
    x = np.asarray(x)
    if x.ndim != 2:
        raise ValueError('Use a two-dimensional array.')
    y = np.apply_along_axis(scaled_fourier, 0, x, alpha)
    return np.apply_along_axis(scaled_fourier, 1, y, alpha)
```

Program 50. A scaled-frequency Fourier transform along both array axes.

CHAPTER 6

Fourier transforms over finite fields and rings

Fourier identities also work with exact arithmetic in finite fields and suitable rings. They lead to integer multiplication without rounding and to algebraic constructions of error-correcting codes. This chapter develops the required roots of unity, derives BCH encoding and decoding, and uses complex-valued Fourier duality to study code parameters.

6.1 Calculations over finite fields

A finite field can play two different roles in Fourier analysis. Earlier chapters used it as the domain of a complex-valued function. Here the *values* of the function lie in a finite field as well. The convolution and inversion identities survive when enough roots of unity are available, but statements about complex magnitudes or positive inner products do not carry over automatically.

6.1.1 Finite-field Fourier transform

Exact Fourier arithmetic requires roots of unity inside the coefficient field and an invertible transform length. We first specify the root condition, then show that it gives the same convolution and inversion formulas as over the complex numbers.

Definition 31 (Primitive root of unity)**.** An element ζ of a field K is a primitive nth root of unity if it has multiplicative order exactly n: $\zeta^n = 1$ and $\zeta^j \neq 1$ for $1 \leq j < n$.

The multiplicative group of $\mathbb{F}_q$, where $q = p^r$, is cyclic of order $q - 1$. It contains a primitive nth root if and only if $n \mid q - 1$. If ω generates $\mathbb{F}_q^*$, then $\omega^{(q-1)/n}$ is such a root. In particular, the characteristic p cannot divide n.

With such a root fixed, we can evaluate the Fourier sum entirely in the finite field.

Definition 32 (Finite-field Fourier transform)**.** Fix a primitive nth root $\zeta \in K = \mathbb{F}_q$. For $f \in K^n$, define

$$\mathcal{F}(f)[j] = \hat{f}[j] = \sum_{k=0}^{n-1} f[k]\zeta^{-kj}, \qquad 0 \leq j < n. \tag{6.1}$$

The root sums provide the cancellation needed for both convolution and reconstruction, with division performed in the field.

Proposition 47 (Convolution and inversion)**.** *The transform is an algebra isomorphism from K^n with unnormalized circular convolution to K^n with componentwise multiplication. Its inverse is*

$$\mathcal{F}^{-1}(g)[j] = n^{-1} \sum_{k=0}^{n-1} g[k]\zeta^{kj}. \tag{6.2}$$

Here n^{-1} denotes the inverse of $n\,1_K$, which is nonzero because $p \nmid n$.

Proof. The geometric sum $\sum_{k=0}^{n-1} \zeta^{ks}$ equals n if $n \mid s$ and zero otherwise: in the latter case $1 - \zeta^s$ is invertible. Expanding the two transforms proves inversion. Reindexing the finite sums in a convolution proves the product rule. □

These calculations are exact. For integer multiplication, the remaining issue is to choose moduli large enough to recover integer coefficients from their residues. This replaces the rounding analysis required by a complex FFT; it does not give a continuous-frequency interpretation to field-valued coefficients.

6.1.2 Choosing a root in the base field

A usable transform begins with a root whose order has been verified. We explain how to find one in a prime field and when the desired transform length requires a field extension.

For $n = p - 1$, any generator of $\mathbb{F}_p^*$ suffices. More generally, if $n \mid p - 1$, raise such a generator to $(p - 1)/n$. A candidate g generates $\mathbb{F}_p^*$ precisely when

$$g^{(p-1)/\ell} \neq 1 \pmod{p} \quad \text{for every prime divisor } \ell \text{ of } p - 1.$$

This reduces the order check to the prime factors of $p - 1$. See Cohen [17] for computational methods. If $n \nmid p - 1$, an extension field is needed; if $p \mid n$, no finite extension can supply a primitive nth root.

6.1.3 Cyclotomic extensions

When the base field lacks the required root, a polynomial quotient can supply it. Cyclotomic polynomials identify the primitive roots, and their factorization modulo the characteristic determines the extension degree.

Theorem-Definition 2 (Cyclotomic polynomials)**.** *For $n \geq 1$, the cyclotomic polynomial is*

$$\Phi_n(X) = \prod_{\substack{1 \leq k \leq n \\ \gcd(k,n)=1}} (X - e^{2\pi i k/n}).$$

It has integer coefficients, is irreducible over $\mathbb{Q}$, and satisfies

$$X^n - 1 = \prod_{d \mid n} \Phi_d(X). \tag{6.3}$$

See Demazure [20] for a proof. The product formula gives a recursive construction, beginning with

$$\Phi_1 = X - 1, \quad \Phi_2 = X + 1, \quad \Phi_3 = X^2 + X + 1, \quad \Phi_6 = X^2 - X + 1.$$

Program 51 computes these polynomials exactly in Python. The case $n = 105$ is the first with a coefficient of absolute value greater than one.

```
def cyclotomic_example(n=105):
    x = sp.Symbol('X')
    return sp.cyclotomic_poly(n, x, polys=True)
```

Program 51. An exact cyclotomic polynomial.

Reducing cyclotomic polynomials modulo a prime reveals the smallest field extensions that contain their roots.

Proposition 48 (Cyclotomic factorization over a finite field)**.** *Assume $n > 1$ and $\gcd(n, p) = 1$, and let r be the multiplicative order of p modulo n. The reduction of Φ_n in $\mathbb{F}_p[X]$ is a product of distinct irreducible polynomials, all of degree r.*

Proof. The derivative nX^{n-1} is relatively prime to $X^n - 1$, so the latter is square-free. Its cyclotomic factors therefore have disjoint root sets. A root ζ of Φ_n has order n: if its order were a proper divisor d, it would also be a root of $X^d - 1$, a product of other cyclotomic factors.

The Frobenius orbit of ζ is $\zeta, \zeta^p, \zeta^{p^2}, \ldots$. Its length is the least positive s for which $\zeta^{p^s} = \zeta$, namely r. The product over this orbit is fixed by Frobenius, hence has coefficients in $\mathbb{F}_p$. Every polynomial over $\mathbb{F}_p$ vanishing at ζ vanishes on its entire orbit. Thus the orbit product is its minimal polynomial and has degree r. □

The minimal extension containing a primitive nth root is consequently $\mathbb{F}_{p^r}$. If P is any irreducible factor of Φ_n modulo p, it can be realized as

$$K = \mathbb{F}_p[X]/(P), \qquad \zeta = X \bmod P.$$

The element ζ generates K as a field over $\mathbb{F}_p$; it need not generate the multiplicative group K^*. Arithmetic uses polynomials of degree less than r, with coefficients reduced modulo p and products reduced modulo P. In particular, $\mathbb{F}_{p^r}$ is not $\mathbb{Z}/p^r\mathbb{Z}$.

The fifteenth roots in characteristic two give a concrete instance of this construction.

Example 7. Over the integers, $\Phi_{15} = X^8 - X^7 + X^5 - X^4 + X^3 - X + 1$. Modulo two it factors as

$$\Phi_{15} = (X^4 + X + 1)(X^4 + X^3 + 1).$$

The order of 2 modulo 15 is 4. Either factor constructs $\mathbb{F}_{16}$, and its indeterminate class has order 15.

The same argument over $\mathbb{F}_q$ uses the order of q modulo n and the Frobenius map $x \mapsto x^q$. Integer coefficients are still reduced modulo the characteristic p, not modulo the cardinality q. Factoring over the larger field may split factors that were irreducible over $\mathbb{F}_p$.

The case in which the characteristic divides the desired order behaves differently because the polynomial acquires repeated roots.

Remark 20. If $n = mp^t$ with $p \nmid m$, then $X^n - 1 = (X^m - 1)^{p^t}$ in characteristic p. This describes the repeated roots, but it does not produce a primitive nth root when $t > 0$. The invertible DFT construction in this chapter requires $p \nmid n$.

6.1.4 Transforms in a cyclotomic extension

Constructing a suitable extension field makes the transform available for data in the base field. We explain where the transformed values live and why inversion still recovers the original data exactly.

Once K and ζ have been constructed, use the same transform formula for vectors in K^n. If the input lies in $\mathbb{F}_p^n$, its transform may leave the base field, just as the DFT of a real signal can be complex. However, inverse transformation recovers the original base-field vector. The implementation in Section B.1 constructs the field, checks the root order, and verifies inversion.

6.1.5 Exact convolution and integer products

Exact modular convolution gives an integer product only if its coefficients can be recovered uniquely from their residues. We bound these coefficients and choose the padding and moduli to avoid both cyclic overlap and modular ambiguity.

Suppose two nonnegative integers are represented by a and b digits in base B. Every coefficient of their linear digit convolution is at most

$$\min(a, b)(B - 1)^2.$$

Choose a transform length $n \geq a + b - 1$ and pad both digit vectors with zeros. A prime characteristic p larger than this bound recovers each coefficient as its unique residue in $\{0, \ldots, p - 1\}$. A larger extension cardinality p^r alone does not increase that integer-recovery range. Alternatively, use several coprime moduli and reconstruct by the Chinese remainder theorem; their product must exceed the bound. For signed coefficients bounded in absolute value by M, a modulus greater than $2M$ permits centered reconstruction.

FFT factorizations remain valid over the field: a subtransform of length n/d uses the root ζ^d. Radix-two transforms require an odd characteristic and a power-of-two length. Mixed-radix implementations can use other factorizations; prime-length leaves can be computed by a direct sum. Operation counts refer to field operations, whose own cost depends on the field representation.

6.2 Calculations over rings

A transform over $\mathbb{Z}/m\mathbb{Z}$ permits exact modular arithmetic without constructing a field extension. Zero divisors require extra care: merely choosing a root of the correct order does not guarantee invertibility. The useful dyadic examples below have odd moduli of the form $2^a + 1$. A nontrivial power-of-two transform cannot be inverted modulo 2^r, since its length is not a unit there.

6.2.1 Principal roots of unity

Over a ring, cancellation can fail even when a root has the expected multiplicative order. We impose a regularity condition on the root differences, then use it to recover the root sums and the invertibility criterion needed by Fourier analysis.

Throughout, rings are commutative, with $1 \neq 0$. An element is called *regular* if multiplication by it is injective; in particular zero is not regular. This formulation avoids ambiguity in the expression "not a zero divisor."

The required cancellation property is captured by the following definition of a principal root.

Definition 33 (Principal root of unity)**.** A principal nth root in a ring A is an element ζ such that $\zeta^n = 1$ and every $1 - \zeta^j$, $1 \le j < n$, is regular.

In a field, principal and primitive roots coincide. Over $\mathbb{Z}/15\mathbb{Z}$, by contrast, 2 has order four but is not principal, because $2^2 - 1 = 3$ is a zero divisor. Some references define principality by vanishing root sums instead. The regularity condition used here implies those sums and also gives the injectivity statement below.

We distinguish the two signs of the Fourier map before discussing whether the inverse normalization is available in the ring.

Definition 34 (Fourier maps over a ring)**.** For a principal nth root $\zeta \in A$, set

$$\mathcal{F}(f)[j] = \sum_{k=0}^{n-1} f[k]\zeta^{-kj}, \qquad \widetilde{\mathcal{F}}(f)[j] = \sum_{k=0}^{n-1} f[k]\zeta^{kj}. \tag{6.4}$$

The regularity assumption recovers the root-sum identities and also controls multiplication by the transform length.

Lemma 9 (Root sums)**.** *One has $\sum_{i=0}^{n-1} \zeta^{ki} = n\,1_A$ if $n \mid k$, and zero otherwise. Moreover, $n\,1_A$ is regular.*

Proof. Multiplying the sum by $1 - \zeta^k$ proves its vanishing when $n \nmid k$. To prove regularity of n, factor

$$X^n - 1 = \prod_{i=0}^{n-1} (X - \zeta^i).$$

Here is a justification over a ring. After removing the first j monic linear factors by polynomial division, evaluating at ζ^j leaves a product of regular differences times the remaining polynomial value. Each difference $\zeta^j - \zeta^i$ is a unit times $1 - \zeta^{j-i}$, so the remaining value is zero and the next division is exact. This induction proves the factorization. Dividing by $X - 1$ and evaluating at 1 yields

$$n\,1_A = \prod_{i=1}^{n-1} (1 - \zeta^i).$$

A product of regular elements is regular. □

These identities separate two conclusions that coincide over a field: injectivity of the Fourier map and existence of a normalized inverse.

Theorem 8 (Fourier transform over a ring)**.** *The map $\mathcal{F}$ preserves convolution products and satisfies $\widetilde{\mathcal{F}}\mathcal{F} = \mathcal{F}\widetilde{\mathcal{F}} = n\,\mathrm{Id}$. Both maps are injective. If $n\,1_A$ is a unit, $\mathcal{F}$ is an algebra isomorphism with inverse $n^{-1}\widetilde{\mathcal{F}}$.*

Proof. Expanding the composition gives

$$(\widetilde{\mathcal{F}}\mathcal{F} f)[j] = \sum_{l} f[l] \sum_{k} \zeta^{k(j-l)} = n f[j].$$

The other composition is identical with the signs reversed. The root-sum lemma and regularity of n imply injectivity. Reindexing proves the convolution identity, and division by n gives the inverse when allowed. □

For residue rings, the Chinese remainder theorem turns root existence into a condition on the prime factors of the modulus.

Proposition 49 (Existence over a residue ring)**.** *Let $m = \prod_{i=1}^{r} p_i^{a_i}$, where the p_i are distinct primes. A principal nth root exists in $\mathbb{Z}/m\mathbb{Z}$ if and only if*

$$n \mid \gcd(p_1 - 1, \dots, p_r - 1).$$

This condition implies $\gcd(n, m) = 1$, so the resulting transform is invertible.

Reduction modulo each p_i proves necessity. For sufficiency, lift a primitive nth root from each prime field to $\mathbb{Z}/p_i^{a_i}\mathbb{Z}$, then combine the lifts by the Chinese remainder theorem. The lifting argument, including verification of the nth-power identity, is detailed in Exercise 6.2.

6.2.2 A dyadic FFT over a ring

Power-of-two lengths admit a simple root criterion that also supports the recursive FFT. It yields useful modular transforms even when the chosen modulus is composite.

Proposition 50 (Principal dyadic roots)**.** *For $k \geq 1$, an element $\zeta \in A$ is a principal 2^kth root if and only if $2\,1_A$ is regular and $\zeta^{2^{k-1}} = -1$.*

Exercise 6.3 proves this criterion. It also shows that $\zeta^2, \zeta^4, \ldots$ provide the roots needed by recursive half-length transforms. For $n = 2^k$, choose

$$m = 2^{n/2} + 1, \qquad \zeta = 2 \pmod{m}.$$

The modulus is odd and $\zeta^{n/2} = -1$, so the criterion applies whether or not m is prime. A larger modulus $m = 2^{cn/2} + 1$ works with $\zeta = 2^c$ for any positive integer c; this flexibility is useful when integer coefficients require a larger recovery range.

The familiar Fermat numbers illustrate why convenient FFT moduli need not be prime.

Remark 21 (Fermat numbers). The Fermat numbers are $\mathrm{Fer}_s = 2^{2^s} + 1$. Their first five values are $3, 5, 17, 257, 65537$, all prime, whereas $\mathrm{Fer}_5 = 4294967297 = 641 \cdot 6700417$. If $2^a + 1$ is prime, then a is a power of two: an odd factor greater than one in a would yield a factorization of $2^a + 1$. Primality is unnecessary for the ring FFT.

Programs in Section B.2 implement this transform with Python integers, avoiding fixed-width overflow. Section B.3 uses it for exact integer multiplication. The FFT costs $O(n \log n)$ ring operations; the bit cost of those operations grows with the modulus size.

6.3 Error-correcting codes

An error-correcting code adds structured redundancy to a message so that small transmission errors can be detected or corrected. The competing objectives are a high information rate, reliable recovery, and efficient encoding and decoding. Finite fields supply useful alphabets and algebraic constraints; cyclic codes will connect these constraints directly to Fourier transforms. See [46, 20] for further background.

6.3.1 Words, errors, and distance

To assess error detection and correction, we need a transmission model and a distance between words. Simple parity and check-digit examples will show what information redundancy can reveal about an error.

We use fixed-length words in $\mathbb{F}_q^n$. A transmission error changes some coordinates, giving a received word $y = x + e$. The number of changed symbols, rather than their magnitudes, measures the error. This model concerns substitutions in a fixed-length word; insertion and deletion errors require a different model.

A parity bit is the simplest example of redundancy that detects some transmission errors.

Example 8 (A parity bit). To transmit $a_0, \ldots, a_{n-1} \in \mathbb{F}_2$, append $a_n = \sum_{j=0}^{n-1} a_j$. The transmitted word has even parity. An odd number of bit changes violates this condition and is detected; an even number is invisible to it. A single parity check does not locate an error, but appending such a check to another code can increase its minimum distance.

Book identifiers give a practical example in which the choice of weights and modulus determines which errors are detected.

Example 9 (ISBN check digits). The historical ISBN-10 format uses nine decimal digits followed by a check symbol $d_9 \in \{0, \ldots, 10\}$, with 10 written as X. It satisfies

$$\sum_{j=0}^{9} (10 - j) d_j = 0 \pmod{11}.$$

The ISBN-10 of the French original is 2-7298-1867-7. A change in a single symbol produces a nonzero residue. Swapping any two unequal symbols also produces a nonzero residue, because both the difference of their weights and the difference of their values are nonzero modulo eleven. These checks detect such errors but do not, in general, determine their locations.

ISBNs have thirteen digits since 2007 [32]. Their check rule is $d_0 + 3d_1 + d_2 + 3d_3 + \cdots + 3d_{11} + d_{12} = 0 \pmod{10}$. For example, 9782729818678 identifies the French original. This rule also detects every single-digit change. Unlike ISBN-10, it can miss an adjacent swap when the two digits differ by five. The different moduli and weights lead to different detection properties.

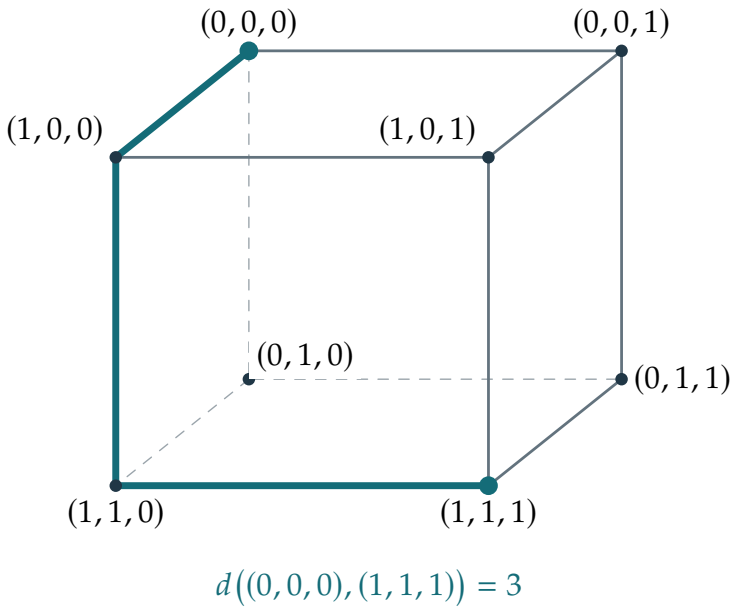


Figure 6.1. Hamming distance on $\mathbb{F}_2^3$. Each cube edge changes one coordinate; the highlighted shortest path has three edges.

These examples motivate a distance that counts altered symbols without assigning a magnitude to a field element.

Definition 35 (Hamming weight and distance)**.** For $x \in \mathbb{F}_q^n$, let $w(x)$ be its number of nonzero coordinates. The Hamming distance is $d(x, y) = w(x - y)$, the number of coordinates at which x and y differ.

This is a metric: each coordinate where x and y differ must be a coordinate where x and z differ or where z and y differ, proving the triangle inequality. On $\mathbb{F}_2^3$, it is also the shortest-path distance along the edges of the cube in Figure 6.1.

6.3.2 Linear codes and their parameters

A code is a nonempty subset $C \subseteq \mathbb{F}_q^n$. A large code carries more information, while widely separated codewords permit more errors to be corrected. Sphere-packing viewpoints help explain this tradeoff; see [26] for connections with discrete geometry.

To quantify the tradeoff between code size and separation, we record both word weights and pairwise distances.

Definition 36 (Weight and distance distributions)**.** For $0 \le i \le n$, set

$$A_i = \#\{x \in C : w(x) = i\}, \tag{6.5}$$

$$B_i = \frac{1}{|C|}\#\{(x, y) \in C^2 : d(x, y) = i\}. \tag{6.6}$$

Pairs in the second formula are ordered.

One has $B_0 = 1$ and $\sum_i A_i = \sum_i B_i = |C|$. Translating a code preserves its *distance* distribution, but generally changes its weight distribution. Translating one of its words to zero is therefore harmless for distance questions, not for arbitrary weight counts.

For a code with at least two words, define its minimum distance by $d = \min_{x \neq y \in C} d(x, y)$. It can correct every pattern of at most t substitutions precisely when its radius-t Hamming balls are disjoint, which is equivalent to $d \ge 2t + 1$. Hence its guaranteed correction radius is $\lfloor (d - 1)/2 \rfloor$. It can detect every nonzero error of weight at most $d - 1$, although detection alone need not reveal the transmitted word.

Requiring the code to be a vector subspace makes its size and encoding map accessible through linear algebra.

Definition 37 (Linear code)**.** An $[n, m, d]_q$ linear code is an m-dimensional subspace of $\mathbb{F}_q^n$ with minimum distance d. A generator matrix $G \in \mathbb{F}_q^{n\times m}$ has a basis of the code as its columns, so $C = \{Gu : u \in \mathbb{F}_q^m\}$ and $|C| = q^m$.

Changing the basis replaces G by GU for an invertible $m \times m$ matrix U and leaves the code exactly unchanged. Linearity also gives $A_i = B_i$ and $d = \min_{0\neq x\in C} w(x)$: for each difference $z \in C$, there are $|C|$ ordered pairs with $x - y = z$. Linear codes are restrictive, but their structure makes encoding and many decoding tasks tractable. There is no general rule that a linear optimum must be much worse than a nonlinear one.

An alternative matrix description tests whether a received word satisfies the code's constraints.

Remark 22 (Parity-check matrix). A full-row-rank matrix $H \in \mathbb{F}_q^{(n-m)\times n}$ with $\ker H = C$ is a parity-check matrix. It satisfies $HG = 0$, and the syndrome Hy tests membership. Both generator and parity-check matrices are nonunique.

A repetition code makes the generator and parity-check descriptions easy to compare.

Example 10 (Repetition). The binary length-four repetition code has words 0000 and 1111, with

$$G = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}, \qquad H = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{pmatrix}.$$

Its orthogonal code is the even-parity code. Generator and parity-check matrices exchange roles after transposition; this is explored in Exercise 6.6.

A length-seven Hamming code carries more information while still correcting one error, and introduces the cyclic structure studied next.

Example 11 (The cyclic Hamming code of length seven). Take the generator matrix, following the column convention,

$$G = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Its columns are successive shifts of the coefficients of $1 + X + X^3$. The sixteen codewords are listed below; the message bits are coordinates in this chosen basis.

0000000	1101000	0110100	1011100
0011010	1110010	0101110	1000110
0001101	1100101	0111001	1010001
0010111	1111111	0100011	1001011

The nonzero weights are $3, 4, 7$, with multiplicities $7, 7, 1$. Thus the code has parameters $[7, 4, 3]_2$ and corrects one error. It is cyclic because $(1 + X + X^3)(1 + X + X^2 + X^4) = X^7 + 1$ in $\mathbb{F}_2[X]$; the next section explains why this divisibility guarantees closure under all circular shifts.

No choice of linear code can avoid the basic tradeoff between dimension and minimum distance expressed by the Singleton bound.

Proposition 51 (Singleton bound)**.** *A linear code with $m \geq 1$ satisfies*

$$d \leq n - m + 1. \tag{6.7}$$

Proof. The subspace E whose last $m-1$ coordinates vanish has dimension $n-m+1$. Since $\dim E + \dim C = n+1$, their intersection contains a nonzero word. That word has weight at most $n - m + 1$. □

Equality defines a *maximum distance separable* (MDS) code. The bound also has a nonlinear form: puncturing $d - 1$ coordinates is injective on any code of minimum distance d, so $|C| \leq q^{n-d+1}$.

6.3.3 Cyclic codes

Closure under circular shifts gives a linear code additional algebraic structure. Representing words by polynomials turns these shifts into multiplication and describes the whole code through a single generator polynomial.

Identify $x = (x_0, \dots, x_{n-1})$ with $x(X) = \sum_{j=0}^{n-1} x_j X^j$ in $R = \mathbb{F}_q[X]/(X^n - 1)$. Multiplication by X sends the word to $(x_{n-1}, x_0, \dots, x_{n-2})$. A linear code is *cyclic* if it is closed under this shift. It is then an ideal of R, because closure under X and linear combinations implies closure under multiplication by every polynomial. The converse is immediate.

Every such ideal has a unique monic generator g dividing $X^n - 1$. Indeed, its inverse image in $\mathbb{F}_q[X]$ is a principal ideal containing $X^n - 1$. The code consists of the ordinary products ag with $\deg a < n - \deg g$, giving dimension $n - \deg g$. The polynomials $g, Xg, \dots, X^{n-\deg g-1}g$ provide generator columns. This also includes the zero code, with $g = X^n - 1$, and the whole space, with $g = 1$.

6.3.4 Constructing BCH codes

A BCH code is designed by forcing a block of consecutive Fourier coefficients to vanish. We choose roots compatible with the base field, construct the generator polynomial, and prove the resulting lower bound on the minimum distance.

Fix a prime p, a length n coprime to p, and a primitive nth root α in a finite extension K of $\mathbb{F}_p$. For a set $I \subseteq \mathbb{Z}/n\mathbb{Z}$, consider

$$g_I(X) = \prod_{i \in I} (X - \alpha^i). \tag{6.8}$$

This is a divisor of $X^n - 1$ in $K[X]$. To define a code over the base field, its coefficients must belong to $\mathbb{F}_p$.

The Frobenius map provides an algebraic test for whether a polynomial with extension-field coefficients descends to the base field.

Lemma 10 (Frobenius criterion)**.** *A polynomial $Q \in K[X]$ belongs to $\mathbb{F}_p[X]$ if and only if $Q(X)^p = Q(X^p)$.*

Proof. An element $a \in K$ belongs to $\mathbb{F}_p$ precisely when $a^p = a$. For $Q(X) = \sum_j a_j X^j$, the Frobenius identity gives

$$Q(X)^p = \sum_j a_j^p X^{pj}, \tag{6.9}$$

which equals $Q(X^p)$ exactly when all the coefficients are fixed. □

For a product specified by its roots, this test becomes an invariance condition on their exponents.

Proposition 52 (Admissible sets of roots)**.** *The polynomial g_I belongs to $\mathbb{F}_p[X]$ if and only if I is invariant under multiplication by p modulo n.*

Proof. A polynomial over $\mathbb{F}_p$ vanishing at α^i also vanishes at α^{pi}. Conversely, if multiplication by p permutes I, then $g_I(X)^p = \prod_{i \in I}(X^p - \alpha^{pi}) = g_I(X^p)$, so the criterion applies. □

The orbits of this permutation are the p-cyclotomic cosets

$$I_k = \{k, pk, \dots, p^{s-1}k\} \pmod{n}, \tag{6.10}$$

where s is the least positive integer with $p^s k \equiv k \pmod{n}$. They partition all residues, including $I_0 = \{0\}$. The polynomial

$$P_k(X) = \prod_{i \in I_k} (X - \alpha^i) \tag{6.11}$$

is the minimal polynomial of α^k over $\mathbb{F}_p$: its roots form one Frobenius orbit, so no proper nonempty subset can be the roots of a factor over $\mathbb{F}_p$.

Taking the required Frobenius orbits of consecutive exponents gives the standard BCH construction.

Definition 38 (Narrow-sense BCH code)**.** For $2 \le \delta \le n$, take I to be the union of the cyclotomic cosets meeting $\{1, \dots, \delta - 1\}$. The cyclic code generated by $g_I = \operatorname{lcm}(P_1, \dots, P_{\delta-1})$ is the narrow-sense BCH code of *designed distance* δ.

A union of cosets need not be a single coset. The designed distance is a lower bound, not necessarily the exact minimum distance.

Binary codes of length thirty-one illustrate how the root orbits determine concrete generator polynomials.

Example 12 (Binary codes of length 31). Fix $K = \mathbb{F}_2[\alpha]$ with $\alpha^5 + \alpha^4 + \alpha^2 + \alpha + 1 = 0$. This irreducible polynomial has degree five, and α has order 31. The cosets and minimal polynomials for this particular root are

k	I_k	$P_k(X)$
0	$\{0\}$	$X + 1$
1	$\{1, 2, 4, 8, 16\}$	$X^5 + X^4 + X^2 + X + 1$
3	$\{3, 6, 12, 24, 17\}$	$X^5 + X^3 + 1$
5	$\{5, 10, 20, 9, 18\}$	$X^5 + X^3 + X^2 + X + 1$
11	$\{11, 22, 13, 26, 21\}$	$X^5 + X^4 + X^3 + X^2 + 1$
14	$\{14, 28, 25, 19, 7\}$	$X^5 + X^2 + 1$
15	$\{15, 30, 29, 27, 23\}$	$X^5 + X^4 + X^3 + X + 1$

Their product is $X^{31} - 1$. For designed distance five, the required cosets are I_1 and I_3, giving

$$g = P_1 P_3 = 1 + X + X^2 + X^3 + X^5 + X^6 + X^8 + X^9 + X^{10}.$$

The resulting code has dimension $31 - 10 = 21$. The program in Section B.4 constructs these polynomials and verifies that their coefficients lie in the base field.

A word x belongs to the code exactly when

$$x(\alpha^i) = 0, \qquad 1 \le i \le \delta - 1. \tag{6.12}$$

In matrix form, these conditions are $Ax = 0$, where $A = (\alpha^{ij})_{1 \le i \le \delta-1,\, 0 \le j < n}$ has size $(\delta - 1) \times n$ over K. Expanding its entries in a basis of $K/\mathbb{F}_p$ gives base-field parity checks; the expanded rows need not all be independent. Equivalently, with Fourier root $\zeta = \alpha^{-1}$, these are the conditions $\hat{x}[i] = 0$ for $1 \le i < \delta$.

The consecutive vanishing conditions force every nonzero codeword to have sufficiently many nonzero coordinates.

Proposition 53 (BCH distance bound)**.** *Every nonzero word in the BCH code has weight at least δ.*

Proof. Suppose a nonzero word has $s < \delta$ nonzero coefficients, at distinct positions $b_1, \dots, b_s$. Write it as $x(X) = \sum_{j=1}^{s} a_j X^{b_j}$ with $a_j \neq 0$. The first s vanishing conditions give

$$\sum_{j=1}^{s} a_j (\alpha^{b_j})^i = 0, \qquad 1 \le i \le s.$$

The coefficient matrix is a Vandermonde matrix multiplied by a diagonal matrix with nonzero entries α^{b_j}. Its determinant is nonzero because these elements are distinct. Hence every a_j would vanish, a contradiction. □

Consecutive vanishing Fourier coefficients are essential to this argument. A support-size uncertainty bound alone does not yield the BCH distance bound. For decoding, write $\delta = 2t + 1$; then all errors of weight at most t are uniquely correctable. In the binary narrow-sense case, an even designed distance can be increased to the next odd one without changing the code, since the extra even exponent is already in the Frobenius closure of a smaller exponent.

6.3.5 Decoding through the Fourier spectrum

The syndromes of a received word reveal a short segment of the error's Fourier spectrum. Under the correction-radius assumption, a recurrence determined by the error locations lets us recover the missing spectrum and then the errors themselves.

Suppose $y = x + e$ is received, where x is a codeword and $\nu = w(e) \le t$. The known syndromes are

$$S_j = y(\alpha^j) = e(\alpha^j), \qquad 1 \le j \le 2t.$$

These are samples of the error's Fourier spectrum, using the positive phase specified above. If J is the set of error positions, define

$$\sigma(Z) = \prod_{i \in J} (1 - \alpha^i Z) = 1 + \sigma_1 Z + \cdots + \sigma_\nu Z^\nu.$$

Its degree is the *actual* number ν of errors, which may be less than t. The error positions are those i for which $\sigma(\alpha^{-i}) = 0$.

Extend $S_j = \sum_{i \in J} e_i \alpha^{ij}$ to every integer j. Substituting each root into σ proves the recurrence

$$S_j + \sigma_1 S_{j-1} + \cdots + \sigma_\nu S_{j-\nu} = 0.$$

Indeed, the left-hand side equals $\sum_{i \in J} e_i \alpha^{ij} \sigma(\alpha^{-i})$. This direct derivation fixes the direction of all indices.

Determining the number of errors. The decoder must first determine how many errors actually occurred, since this number may be smaller than the guaranteed correction radius.

Form the $t \times t$ Hankel matrix $T_t = (S_{a+b+1})_{0 \le a,b < t}$. If $u_i = \alpha^i$, then

$$T_t = V \operatorname{diag}(e_i u_i)_{i \in J} V^{\mathsf{T}}, \qquad V = (u_i^a)_{0 \le a < t,\, i \in J}.$$

The rectangular Vandermonde matrix V has column rank ν, and the diagonal matrix is invertible. Thus $\operatorname{rank} T_t = \nu$. This avoids treating a size-t locator system as invertible when fewer than t errors occur. If $\nu = 0$, all errors vanish.

Finding the locator. Once the number of errors is known, the syndrome recurrence gives a square system for the locator coefficients.

For $\nu > 0$, use the recurrence for $j = \nu + 1, \ldots, 2\nu$:

$$\sum_{l=1}^{\nu} S_{j-l} \sigma_l = -S_j.$$

The coefficient matrix is T_ν with its columns reversed, so it is invertible by the same Vandermonde factorization. It is generally dense, not triangular. Solving it determines the locator uniquely. This elementary rank-and-solve procedure is sufficient for the examples; more efficient recurrence algorithms, such as Berlekamp–Massey, avoid dense elimination for large correction radii.

Completing the spectrum. Extending the syndrome sequence recovers the full error spectrum, so inverse transformation can determine the error values directly.

The recurrence now determines $S_{2t+1}, \ldots, S_n$ successively from known values. Periodicity gives $S_0 = S_n$. Inverting the Fourier transform recovers each error value:

$$e_i = n^{-1} \sum_{j=0}^{n-1} S_j \alpha^{-ij}.$$

Subtract e from the received word. The implementation checks that the reconstructed errors belong to $\mathbb{F}_p$, that at most t are nonzero, and that the corrected word satisfies all BCH checks.

An independent recovery method finds the roots $\sigma(\alpha^{-i}) = 0$ by testing the n positions, then solves the Vandermonde system $S_j = \sum_{i \in J} e_i \alpha^{ij}$ for $1 \le j \le \nu$ to recover their magnitudes. For binary errors these magnitudes are all one, but this is not true over a general alphabet. Both methods are implemented and compared in Section B.4. The guarantee assumes at most t errors: if more occur, a nearby codeword accepted by the checks need not be the one originally transmitted.

6.4 Error-correcting codes and Fourier duality

We now return to complex-valued Fourier analysis on the additive group $G = \mathbb{F}_2^n$. Orthogonality of characters relates a linear code to its dual; for nonlinear codes, the same calculation gives positivity constraints on the distance distribution. These are the starting point of the linear programming bound. The reference [41] develops this combinatorial approach in depth.

6.4.1 Weight enumerators

A weight enumerator compresses the counting information of a code into a polynomial. We recall the dual-code construction and the MacWilliams identity, which lets one recover the enumerator of either code from the other.

Unless stated otherwise, $C \subset \mathbb{F}_2^n$ is a linear code of dimension m. The extension to $\mathbb{F}_q$ uses additive characters and the substitution $(X, Y) \mapsto (X + (q-1)Y, X - Y)$; see Exercise 2.8.

The dual code consists of the linear constraints satisfied by every codeword, expressed through the coordinate pairing.

Definition 39 (Dual code)**.** The canonical nondegenerate bilinear form is

$$\langle x,\, y\rangle = \sum_{i=0}^{n-1} x_i y_i \in \mathbb{F}_2. \tag{6.13}$$

The dual code is the orthogonal subspace

$$C^{\perp} = \{x \in \mathbb{F}_2^n : \langle x,\, y\rangle = 0 \text{ for every } y \in C\}.$$

It has dimension $n - m$, and $(C^{\perp})^{\perp} = C$.

A code and its orthogonal need not be complementary subspaces. A code is *self-orthogonal* when $C \subset C^{\perp}$, and *self-dual* when equality holds. Self-duality implies $n = 2m$, but equality of dimensions alone is insufficient. Exercise 8.9 connects self-duality to invariant polynomials.

We package the numbers of codewords at each weight in a homogeneous polynomial.

Definition 40 (Weight enumerator)**.** With $A_i = \#\{x \in C : w(x) = i\}$, the weight enumerator is

$$W_C(X, Y) = \sum_{i=0}^{n} A_i X^{n-i} Y^i.$$

It is homogeneous of degree n, with $W_C(1, 1) = |C| = 2^m$.

The MacWilliams identity then relates this polynomial to the corresponding one for the dual code.

Theorem 9 (MacWilliams identity)**.** *For a binary linear code,*

$$W_{C^{\perp}}(X, Y) = \frac{1}{|C|} W_C(X + Y, X - Y).$$

This is Theorem 5, expressed in coding terminology. Below we prove a version for arbitrary functions that includes this identity.

6.4.2 Codes as functions on a group

Representing a code by its indicator function brings it into the space where the Fourier transform acts. This viewpoint recovers linear-code duality and suggests a useful extension to functions representing nonlinear codes.

The characters of G are $\chi_a(x) = (-1)^{\langle a, x\rangle}$. Our unnormalized Fourier transform is

$$\hat{f}(a) = \sum_{x \in G} f(x)\chi_a(x).$$

For any subset $C \subset G$, its indicator $f_C = \sum_{x \in C} \delta_x$ is one on C and zero elsewhere. For a linear code,

$$a \in C^{\perp} \quad \Longleftrightarrow \quad \chi_a(x) = 1 \text{ for every } x \in C. \tag{6.14}$$

Summing a character over the code makes the annihilator relation visible directly in its indicator's Fourier transform.

Proposition 54 (Transform of an indicator)**.** *If C is linear, then $\hat{f}_C = |C| f_{C^{\perp}}$.*

Proof. For $a \in C^{\perp}$, all summands in $\sum_{x \in C} \chi_a(x)$ equal one. Otherwise, choose $x_0 \in C$ with $\chi_a(x_0) = -1$. Translation by x_0 permutes C, so this sum equals its own negative and is zero. □

For a nonlinear subset, the orthogonal subspace still exists, but it only records the linear span of the subset. To retain more information, we work with the Fourier transform of a function instead.

Definition 41 (Dual function)**.** For $f \in \mathbb{C}[G]$ with nonzero total mass $M_f = \sum_{x \in G} f(x)$, define $f^{\perp} = \hat{f}/M_f$.

Thus $(f_C)^{\perp} = f_{C^{\perp}}$ for a linear code. In general, $f^{\perp}$ need not be an indicator. The normalization uses total mass, not the average $M_f/|G|$. For any function, define

$$W_f(X, Y) = \sum_{x \in G} f(x) X^{n - w(x)} Y^{w(x)}.$$

This agrees with the code's enumerator when $f = f_C$.

The same coordinatewise character calculation extends the polynomial identity from indicators to functions of nonzero total mass.

Theorem 10 (MacWilliams identity for functions)**.** *If $M_f \neq 0$, then*

$$W_{f^\perp}(X, Y) = \frac{1}{M_f} W_f(X + Y, X - Y). \tag{6.15}$$

Proof. Interchange the two finite sums to obtain

$$W_{\hat{f}}(X, Y) = \sum_{y \in G} f(y) \sum_{x \in G} (-1)^{\langle x, y \rangle} X^{n - w(x)} Y^{w(x)}.$$

The inner sum factors coordinate by coordinate:

$$\prod_{j=0}^{n-1} (X + (-1)^{y_j} Y) = (X + Y)^{n - w(y)} (X - Y)^{w(y)}.$$

Division by M_f gives the identity. □

6.4.3 Distance distributions of arbitrary codes

For a nonlinear code, pairwise distances retain information that its linear span would lose. We encode these distances by a convolution and use Fourier positivity to constrain the possible distance distributions.

Let $C \subset \mathbb{F}_2^n$ be nonempty; linearity is no longer assumed. Its distance function, with the usual unnormalized convolution, is

$$D_C = \frac{1}{|C|} f_C * f_C = \frac{1}{|C|} \sum_{x, y \in C} \delta_{x+y}. \tag{6.16}$$

In particular, $D_C(0) = 1$, and

$$M_{D_C} = |C|. \tag{6.17}$$

If C is linear, each element has exactly $|C|$ decompositions as $x + y$ with $x, y \in C$, so $D_C = f_C$.

Grouping the values of this convolution by Hamming weight recovers the distance distribution.

Proposition 55 (Distance distribution)**.** *Writing $W_{D_C}(X, Y) = \sum_{i=0}^{n} B_i X^{n-i} Y^i$ gives*

$$B_i = \frac{1}{|C|} \#\{(x, y) \in C^2 : d(x, y) = i\}.$$

Consequently, $B_0 = 1$ and $\sum_i B_i = |C|$.

Proof. In characteristic two, $x + y = x - y$, hence $w(x + y) = d(x, y)$. Therefore

$$\sum_{w(z)=i} D_C(z) = \frac{1}{|C|} \sum_{\substack{x, y \in C \\ d(x,y)=i}} 1. \tag{6.18}$$

□

Over $\mathbb{F}_q$, replace the second indicator in the convolution by $\widetilde{f_C}(x) = f_C(-x)$. This counts differences and gives the same distance interpretation.

Define the *dual distance distribution* by

$$W_{D_C^\perp}(X, Y) = \sum_{i=0}^{n} B'_i X^{n-i} Y^i = \frac{1}{|C|} W_{D_C}(X + Y, X - Y). \tag{6.19}$$

For a linear code, it is the distance distribution of $C^\perp$. For a nonlinear code, these coefficients need not count the words of any code. They nevertheless satisfy a strong positivity constraint.

Proposition 56 (Positivity of the dual distribution)**.** *The numbers B'_i are nonnegative rationals. Moreover, $B'_0 = 1$ and $\sum_i B'_i = 2^n / |C|$.*

Proof. The convolution theorem gives

$$B'_i = \frac{1}{|C|^2} \sum_{w(a)=i} |\hat{f}_C(a)|^2 \geq 0.$$

Here $\hat{f}_C(a)$ is an integer, proving rationality. At $a = 0$ it is $|C|$, so $B'_0 = 1$. Finally, evaluate (6.19) at $(1, 1)$ and use $B_0 = 1$:

$$\sum_i B'_i = \frac{W_{D_C}(2,0)}{|C|} = \frac{2^n}{|C|}.$$

No assumption that $0 \in C$ is needed. □

Nonnegativity, together with the vanishing of $B_1, \ldots, B_{d-1}$ for a code of minimum distance d, produces the linear programming bound in Exercise 6.13.

Hadamard codes illustrate both an actual dual code and a formal dual distribution whose coefficients need not be integers.

Example 13 (Hadamard codes). The eight-word code $\mathcal{A}_8$ in (6.21) is a linear $[7, 3, 4]$ simplex code. Its distance enumerator is $X^7 + 7X^3Y^4$. Applying the substitution above gives

$$W_{D^\perp_{\mathcal{A}_8}}(X, Y) = X^7 + 7X^4Y^3 + 7X^3Y^4 + Y^7,$$

the enumerator of a Hamming code. In particular,

i	0	3	4	7
B'_i	1	7	7	1

For the twelve-word Hadamard code $\mathcal{A}_{12}$ of length eleven, distinct words are at distance six, so $W_{D_{\mathcal{A}_{12}}} = X^{11} + 11X^5Y^6$. The dual polynomial is

$$\frac{(X+Y)^{11} + 11(X+Y)^5(X-Y)^6}{12}.$$

Its nonzero coefficients are

i	0	3	4	5	6	7	8	11
B'_i	1	55/3	110/3	88/3	88/3	110/3	55/3	1

They sum to $512/3 = 2^{11}/12$, but some are nonintegral. Thus this formal dual distribution cannot be the weight distribution of a code.

6.5 Exercises

These exercises connect exact transforms, Boolean functions, and algebraic coding theory. They begin with cyclotomic computations and root-existence proofs, then develop explicit codes and bounds on their possible parameters.

Cyclotomic coefficients provide an exact-arithmetic experiment in which apparently simple root sets can produce unexpectedly large coefficients.

Exercise 6.1 (Cyclotomic coefficients)**.** Use exact Python arithmetic to find the smallest positive index n such that Φ_n has a coefficient of absolute value a, for successive integers $a \geq 1$. Verify, for $a = 1, \ldots, 7$, the values

$$1, \quad 105, \quad 385, \quad 1365, \quad 1785, \quad 2805, \quad 3135.$$

Extend the search to $a = 27$ with the explicit limit 20615. Explain why it suffices to search odd squarefree indices after $n = 1$. We seek a coefficient of absolute value *exactly* a, not merely a polynomial whose largest coefficient has magnitude at least a.

Lifting roots through prime powers and combining them by the Chinese remainder theorem proves the residue-ring existence criterion.

Exercise 6.2 (Existence of principal roots)**.** Prove Proposition 49. Write $m = \prod_{i=1}^{r} p_i^{k_i}$ with distinct primes p_i.

1. Show that a principal nth root exists in $\mathbb{F}_p$ exactly when $n \mid p-1$.
2. Let $a \in \{1, \dots, p-1\}$ represent such a root and set $\zeta_0 = a^{p^{r-1}} \pmod{p^r}$. Prove both that $\zeta_0^n = 1$ and that $\zeta_0 \equiv a \pmod{p}$.
3. Show that $1 - \zeta_0^s$ is a unit modulo p^r for $1 \le s < n$.
4. Use the Chinese remainder theorem to prove the criterion over $\mathbb{Z}/m\mathbb{Z}$, including its necessity.

The dyadic root criterion follows by analyzing how principal roots behave under taking powers.

Exercise 6.3 (Principal dyadic roots)**.** Let A be a commutative ring with identity, with the definition of a principal root used in Proposition 50.

1. Prove that ζ is a principal (nm)th root if and only if ζ^m is a principal nth root and ζ^n is a principal mth root. No coprimality assumption on n, m is imposed.
2. Show that the only possible principal square root of unity is -1, and that it is principal exactly when multiplication by 2 is injective on A.
3. Deduce that ζ is a principal 2^kth root, $k \ge 1$, exactly when 2 is regular and $\zeta^{2^{k-1}} = -1$.

Walsh coefficients measure how well a Boolean function can be approximated by an affine function, connecting Fourier analysis with coding and cryptography.

Exercise 6.4 (Boolean functions and Reed–Muller codes)**.** For a Boolean function $\widetilde{f} : \mathbb{F}_2^n \to \mathbb{F}_2$, put $f = (-1)^{\widetilde{f}}$ and $\mathcal{W}f(a) = \sum_x f(x)(-1)^{\langle a, x\rangle}$. Define its nonlinearity as the least Hamming distance of its truth table from the truth table of an affine function $\langle a, x\rangle + b$.

1. Prove that every Boolean function has a unique multilinear polynomial representation over $\mathbb{F}_2$, called its algebraic normal form. Explain why distance from affine functions measures nonlinearity.
2. Show that
$$N(f) = 2^{n-1} - \tfrac{1}{2} \max_{a \in \mathbb{F}_2^n} |\mathcal{W}f(a)|.$$
The constant coefficient $a = 0$ must be included. Describe an FWT algorithm.
3. Use Parseval's identity to prove $N(f) \le 2^{n-1} - 2^{n/2-1}$. Show that equality holds precisely when $|\mathcal{W}f(a)| = 2^{n/2}$ for all a. Such functions are called *bent*; show that they can exist only for even n.
4. For $\widetilde{h}(u, v) = \widetilde{f}(u) + \widetilde{g}(v)$, prove that $\widetilde{h}$ is bent if and only if both factors are bent. Check $\widetilde{f}(u_1, u_2) = u_1 u_2$ and construct examples for every positive even number of variables.
5. For $n \ge 1$, the first-order Reed–Muller code $R(1, n)$ consists of the truth tables of all affine Boolean functions. Determine its length, dimension, cardinality, and minimum distance. Give FWT-based encoding and nearest-word decoding algorithms, including the constant affine functions.

Bent functions were introduced in [51]. For cryptographic applications and bounds beyond the even-dimensional case, see [47].

Random samples can estimate Walsh coefficients and, when the spectrum is suitably concentrated, support a provable learning guarantee.

Exercise 6.5 (Learning Boolean functions)**.** Let $f : \mathbb{F}_2^n \to \{-1, 1\}$ and let expectations use the uniform measure. Its normalized Walsh coefficients are $a_s = \mathbb{E}[f \chi_s]$. We observe $f(X_1), \dots, f(X_m)$ at independent uniform points. The following exercise illustrates Fourier learning; the more elaborate algorithm in [33] also addresses how to find important coefficients. Recall Hoeffding's bound for independent $Z_j \in [-1, 1]$ with common mean a:
$$\Pr\left(\left| m^{-1} \sum_j Z_j - a \right| \ge \lambda \right) \le 2 \exp(-m\lambda^2/2).$$
Set $\operatorname{sign}(0) = 1$.

1. Compute $\mathbb{E}[(f - h)^2]$ for $h = a_\beta \chi_\beta$.

2. Prove, for any real-valued h, that $\Pr(f(X) \neq \operatorname{sign} h(X)) \leq \mathbb{E}[(f-h)^2]$.
3. For a fixed index β, estimate
$$\widetilde{a}_\beta = \frac{1}{m}\sum_{j=1}^{m} f(X_j)\chi_\beta(X_j). \tag{6.20}$$
If $m \geq 2\lambda^{-2}\log(2/\delta)$, prove that with probability at least $1-\delta$ over the training sample, the classifier $\varphi_0 = \operatorname{sign}(\widetilde{a}_\beta\chi_\beta)$ satisfies
$$\Pr_X(f(X) \neq \varphi_0(X)) \leq 1 - a_\beta^2 + \lambda^2.$$
Here X is a fresh uniform test point.
4. Suppose $\sum_{w(s)>d} a_s^2 \leq \alpha$, where $0 \leq d \leq n$. Let $M = \sum_{j=0}^{d}\binom{n}{j}$ and estimate all coefficients with $w(s) \leq d$ from the same sample. Show that
$$m \geq \frac{2M}{\epsilon}\log\frac{2M}{\delta} \quad\Longrightarrow\quad \Pr_X\left(f(X) \neq \operatorname{sign}\sum_{w(s)\leq d}\widetilde{a}_s\chi_s(X)\right) \leq \alpha + \epsilon$$
with training-sample probability at least $1-\delta$. This condition concerns the *Walsh spectrum*; its degree need not equal the degree of the algebraic normal form in the preceding exercise.

Generator and parity-check matrices give complementary descriptions of a code, while systematic form makes message recovery explicit.

Exercise 6.6 (Generator and parity-check matrices)**.** Let $C \subset \mathbb{F}_q^n$ have dimension m, with a full-column-rank generator $G \in \mathbb{F}_q^{n\times m}$ and a full-row-rank parity-check matrix $H \in \mathbb{F}_q^{(n-m)\times n}$.

1. Give generator and parity-check matrices for $C^\perp$.
2. If $G = \begin{pmatrix} I_m \\ A \end{pmatrix}$, find H and explain how this systematic form simplifies message recovery.
3. Show that a coordinate permutation, followed by a change of message basis, puts any generator in systematic form. Explain which operation changes the code and which only changes its parametrization.

Hamming codes use distinct nonzero syndrome columns to locate a single error and achieve a perfect packing by radius-one balls.

Exercise 6.7 (Hamming codes)**.** Let $k \geq 2$, $n = 2^k - 1$, and let the columns of $H \in \mathbb{F}_2^{k\times n}$ be the distinct nonzero vectors of $\mathbb{F}_2^k$. Set $C = \ker H$.

1. Prove that C has parameters $[2^k-1, 2^k-1-k, 3]$ and explain single-error decoding by the syndrome.
2. Construct a cyclic Hamming code as a binary BCH code of designed distance three, using a primitive element $\alpha \in \mathbb{F}_{2^k}$.
3. Prove that the radius-one balls about its words partition $\mathbb{F}_2^n$: the code is *perfect*. Check that the cyclic code in Example 11 is a Hamming code, with a parity-check matrix in the matching coordinate order.
4. Show that every nonzero word of $C^\perp$ has weight 2^{k-1}. Deduce that this dual is a simplex code.
5. Generalize the construction to $\mathbb{F}_q$ using one representative of each one-dimensional subspace of $\mathbb{F}_q^k$. Determine the length, dimension, minimum distance, and size of a radius-one ball.

The simple weight distribution of the dual simplex code determines the full Hamming-code enumerator through the MacWilliams identity.

Exercise 6.8 (Weight enumerators of Hamming codes)**.** Use the preceding exercise, with $n = 2^k - 1$ and $k \geq 2$.

1. Enumerate the length-seven code of Example 11.
2. Compute $W_{C^\perp}$ from the constant nonzero weight of the simplex code.
3. Deduce the formula
$$W_C(X,Y) = \frac{(X+Y)^n + n(X+Y)^{(n-1)/2}(X-Y)^{(n+1)/2}}{n+1}.$$
4. Calculate $A_1, \ldots, A_6$, interpreting coefficients beyond the length as zero. In particular, show that $A_3 = n(n-1)/6$.
5. Prove $A_{n-i} = A_i$ by showing that the all-ones word belongs to C.

Adding one overall parity bit changes the length-seven Hamming code into a self-dual length-eight code with larger minimum distance.

Exercise 6.9 (Extended Hamming code)**.** Append an overall parity bit to the length-seven code in Example 11. Determine the resulting length-eight code's parameters, list its words, and compute its weight enumerator. Prove self-duality by checking orthogonality of a generator's columns; invariance of the weight enumerator under the MacWilliams substitution alone does not prove equality of the two codes.

Repetition codes provide a small family in which parameters, duality, and weight enumerators can all be computed directly.

Exercise 6.10 (Repetition codes)**.** For $n \geq 2$, let $C = \{(a, \dots, a) : a \in \mathbb{F}_q\}$.

1. Determine its dimension, minimum distance, and weight enumerator.
2. Identify $C^{\perp}$, compute its parameters and enumerator, and determine exactly when the repetition code is self-dual.

Orthogonal sign matrices generate codes whose pairwise distances follow from row orthogonality, even when the codes are nonlinear.

Exercise 6.11 (Hadamard codes)**.** Let H_n, $n \geq 4$, be a Hadamard matrix whose first row and column are all ones. Replace $+1$ by 0 and -1 by 1, and delete the first column. Let $\mathcal{A}_n$ be the code formed by its n rows, and let $\mathcal{B}_n$ be their union with their bitwise complements.

1. Identify these codes for the Walsh matrix, $n = 2^k$. Are codes constructed from an arbitrary Hadamard matrix necessarily linear?
2. Compute the lengths, cardinalities, and minimum distances of $\mathcal{A}_n$ and $\mathcal{B}_n$, using orthogonality of Hadamard rows.

For reference, two constructions give the following codes:

Paley construction	Walsh construction
0000000	0000000
1001011	1010101
1100101	0110011
1110010	1100110
0111001	0001111
1011100	1011010
0101110	0111100
0010111	1101001

. (6.21)

Comparing polynomial coefficients in the MacWilliams identity gives moment constraints and determines the weight distribution of an MDS code.

Exercise 6.12 (MDS codes and Krawtchouk polynomials)**.** Let $C \subset \mathbb{F}_2^n$ be linear of dimension $1 \leq m \leq n-1$, with weight distributions (A_i) and (A_i') for C and $C^{\perp}$. Define the binary Krawtchouk polynomials by

$$P_k(x) = \sum_{j=0}^{k} (-1)^j \binom{x}{j} \binom{n-x}{k-j}, \qquad 0 \leq k \leq n. \tag{6.22}$$

For integer arguments, a binomial coefficient is zero when its lower index lies outside its usual range.

1. Compare coefficients in the MacWilliams identity to show $A_k' = 2^{-m} \sum_i A_i P_k(i)$.
2. Prove the moment identities

$$\sum_{i=0}^{n-k} \binom{n-i}{k} A_i = 2^{m-k} \sum_{i=0}^{k} \binom{n-i}{n-k} A_i', \qquad 0 \leq k \leq n.$$

One route differentiates the inverse MacWilliams identity k times in X and then sets $X = Y = 1$.

3. Assume C is MDS, so its minimum distance is $d = n - m + 1$. Prove that $C^\perp$ is MDS with minimum distance $m + 1$, and deduce

$$\sum_{i=d}^{n-k} \binom{n-i}{k} A_i = \binom{n}{k}(2^{m-k} - 1), \qquad 0 \le k < m.$$

4. Explain why these equations determine the entire weight distribution, and show that $A_d = \binom{n}{d}$.

Fourier positivity turns constraints on distance distributions into a linear program that bounds the size of an arbitrary binary code.

Exercise 6.13 (The linear programming bound)**.** Let $R(n, d)$ be the largest cardinality of a binary code of length n and minimum distance at least d, with $1 \le d \le n$.

1. Prove that the distance distribution of any nonempty code obeys $\sum_{i=0}^{n} B_i P_k(i) \ge 0$ for $0 \le k \le n$.
2. Show that $R(n, d)$ is bounded by the following linear program and prove that its feasible set is bounded. Distance d itself is allowed.

$$\begin{aligned} &\text{maximize } \sum_{i=0}^{n} x_i, \\ &\text{subject to } x_0 = 1, \quad x_1 = \cdots = x_{d-1} = 0, \quad x_d, \ldots, x_n \ge 0, \\ &\qquad \sum_{i=0}^{n} x_i P_k(i) \ge 0 \quad (0 \le k \le n). \end{aligned}$$

6.6 Solutions to the exercises

Exact arithmetic and character orthogonality play complementary roles in these solutions. Root calculations justify modular transforms, while polynomial and Fourier arguments determine code parameters, Boolean-function properties, and bounds on distance distributions.

Solution to Exercise 6.1

For $n > 1$, write $r = \mathrm{rad}(n)$ for the product of its distinct prime factors. Then $\Phi_n(X) = \Phi_r(X^{n/r})$, so repeated prime factors only insert zero coefficients. For odd $r > 1$, $\Phi_{2r}(X) = \Phi_r(-X)$, which preserves absolute values. Thus a first occurrence of an absolute value greater than one must have an odd squarefree index. The value one first occurs at $n = 1$, since $\Phi_1 = X - 1$; there is no cyclotomic polynomial Φ_0 in this convention. For distinct primes p, q, the coefficients of Φ_{pq} also have absolute value at most one. Indeed,

$$\Phi_{pq}(X) = \frac{(1 - X^{pq})(1 - X)}{(1 - X^p)(1 - X^q)}.$$

Below degree pq, its coefficient at k is $N(k) - N(k-1)$, where $N(k)$ counts nonnegative solutions of $up + vq = k$. There is at most one such solution when $k < pq$: two would differ by $(q, -p)$ times a nonzero integer and force one representation to have value at least pq. Since $\deg \Phi_{pq} < pq$, all coefficients belong to $\{-1, 0, 1\}$. Prime indices give coefficients equal to one.

Program 52 can therefore skip indices with fewer than three distinct prime factors when seeking magnitudes above one. The exact identity $\Phi_n(X) = \prod_{d|n}(1 - X^d)^{\mu(n/d)}$ for $n > 1$ allows efficient coefficient computation: multiply or divide by $1 - X^d$ in a power series truncated at degree $\varphi(n)$. The two loop directions in the program implement these operations without symbolic polynomial expansion.

Searching through 20615 gives the first indices for $a = 1, \ldots, 27$:

a	1	2	3	4	5	6	7	8	9
n	1	105	385	1365	1785	2805	3135	6545	6545
a	10	11	12	13	14	15	16	17	18
n	10465	10465	10465	10465	10465	11305	11305	11305	11305
a	19	20	21	22	23	24	25	26	27
n	11305	11305	11305	15015	11305	17255	17255	20615	20615

These values need not increase: the first index for magnitude 23 is smaller than that for magnitude 22. The explicit search limit also distinguishes an incomplete search from a proof of nonexistence.

```
def cyclotomic_coefficients(n):
    """Ascending exact coefficients from the truncated Mobius product."""
    if n < 1:
        raise ValueError('The cyclotomic index must be positive.')
    if n == 1:
        return [-1, 1]
    degree = int(sp.totient(n))
    coefficients = [1]+[0]*degree
    for divisor in sp.divisors(n):
        sign = sp.mobius(n//divisor)
        if sign == 1:
            for k in range(degree, divisor-1, -1):
                coefficients[k] -= coefficients[k-divisor]
        elif sign == -1:
            for k in range(divisor, degree+1):
                coefficients[k] += coefficients[k-divisor]
    return coefficients
```

```
def first_cyclotomic_coefficients(largest=7, limit=3200):
    """First positive index with a coefficient of each absolute value 1,...,largest."""
    if largest < 1 or limit < 1:
        raise ValueError('Use positive bounds.')
    found = {1: 1}
    if largest == 1:
        return found
    for n in range(3, limit+1, 2):
        factors = sp.factorint(n)
        if any(exponent != 1 for exponent in factors.values()):
            continue  # Repeated prime factors only insert zeros; 2 changes signs.
        if len(factors) < 3:
            continue  # One or two distinct primes give only 0, 1, and -1.
        coefficients = cyclotomic_coefficients(n)
        for value in set(abs(int(c)) for c in coefficients):
            if 1 <= value <= largest:
                found.setdefault(value, n)
        if len(found) == largest:
            return found
    return found  # Missing values have not been found within the explicit limit.
```

Program 52. First occurrences of cyclotomic coefficient magnitudes.

Solution to Exercise 6.2

1. Over a field, every nonzero element is regular, so principality means exact order n. The cyclic group $\mathbb{F}_p^*$ has an element of that order precisely when $n \mid p-1$.
2. The integer a is prime to p, hence invertible modulo p^r. As $a^n = 1 + pc$ for some integer c, repeated use of $(1+p^j u)^p \equiv 1 \pmod{p^{j+1}}$ gives

$$\zeta_0^n = (1+pc)^{p^{r-1}} \equiv 1 \pmod{p^r}.$$

   Frobenius is the identity on $\mathbb{F}_p$, so $a^{p^{r-1}} \equiv a \pmod{p}$. These are separate requirements: merely showing $\zeta_0^{p-1} = 1$ would not establish its order divides n.
3. For $1 \le s < n$, $a^s \not\equiv 1 \pmod{p}$. Thus $1 - \zeta_0^s$ is prime to p and therefore a unit modulo p^r. This proves principality.
4. If $n \mid p_i - 1$ for every i, combine the lifted roots by the Chinese remainder theorem. Their differences from lower powers of one are units in every component, hence units modulo m. Conversely, in a finite ring a regular element is a unit. Reducing a principal root modulo each p_i therefore preserves all the nonvanishing differences $1 - \zeta^s$, so its order remains exactly n. Thus every $p_i - 1$ is divisible by n.

Solution to Exercise 6.3

1. If ζ is principal of order nm, the powers in the question are principal by direct substitution into the definition. Conversely, the two assumptions imply $\zeta^{nm} = 1$. Suppose $a\zeta^i = a$ with $1 \le i < nm$, and write $i = nq + r$, $0 \le r < n$. If $r = 0$, then $1 \le q < m$, so principality of ζ^n gives $a = 0$. If $r > 0$, raising the equality through m successive multiplications gives $a(\zeta^m)^r = a$; principality of ζ^m again gives $a = 0$.
2. If $\zeta^2 = 1$ and $\zeta - 1$ is regular, then $(\zeta - 1)(\zeta + 1) = 0$ forces $\zeta = -1$. This root is principal exactly when -2 is regular. A general ring can have other square roots of unity, but they are not principal in this sense.
3. Necessity follows by taking the 2^{k-1}st power and applying the preceding part. For sufficiency, use induction. When $k > 1$, ζ^2 is a principal 2^{k-1}st root by induction, while $\zeta^{2^{k-1}} = -1$ is a principal square root. Part 1 concludes.

Solution to Exercise 6.4

1. The point indicator at $u \in \mathbb{F}_2^n$ is the multilinear polynomial

$$\delta_u(x) = \prod_{j:u_j=1} x_j \prod_{j:u_j=0} (1 + x_j).$$

   Thus such polynomials span all functions. There are 2^n monomials, so both the polynomial space and the function space have dimension 2^n over $\mathbb{F}_2$; the representation is unique. Each space contains 2^{2^n} elements. Distance zero from the affine subspace means exactly that the function is affine.
2. The signed correlation with the affine function (a, b) is

$$\sum_x (-1)^{\widetilde{f}(x)+\langle a,x\rangle+b} = (-1)^b \mathcal{W}f(a) = 2^n - 2d(\widetilde{f}, \widetilde{f}_{a,b}).$$

   Optimize first over b, then over all a, including zero. The FWT computes these correlations in $O(n2^n)$ operations.
3. Parseval gives $\sum_a |\mathcal{W}f(a)|^2 = 2^{2n}$. Hence the largest squared coefficient is at least 2^n, with equality precisely when all squared magnitudes equal 2^n. Walsh coefficients are integers, so the magnitude $2^{n/2}$ is possible only for even n.
4. Since $h(u, v) = f(u)g(v)$, its transform factors as $\mathcal{W}h(a, b) = \mathcal{W}f(a)\mathcal{W}g(b)$. Flat spectra give a flat product. Conversely, if the product is flat and nonzero, fixing either index shows that each factor has constant magnitude; Parseval fixes their magnitudes to the bent values. The truth table of u_1u_2 has signed Walsh transform $(2, 2, 2, -2)$. Consequently $u_1u_2 + u_3u_4 + \cdots + u_{n-1}u_n$ is bent for even n.
5. The truth-table code has length 2^n, dimension $n + 1$, and 2^{n+1} words. A nonconstant affine function is balanced; the constant-one function has weight 2^n. Thus its minimum distance is 2^{n-1}. If a is stored as a binary integer, the Walsh transform of δ_a is the sign table of $\langle a, x\rangle$. Program 53 converts this to bits and adds b. For decoding, transform the received sign table, maximize absolute correlation over every index, and use the sign to choose b. Ties correspond to multiple nearest codewords; the program chooses the first maximizing index.

```
def encode_rm(a, b, variables):
    if not 0 <= a < 2**variables or b not in (0, 1):
        raise ValueError('Use a valid linear part and binary constant.')
    impulse = np.zeros(2**variables, dtype=int)
    impulse[a] = 1
    signs = fwt(impulse).real.astype(int)
    return ((1-signs)//2 + b) % 2
```

Program 53. Encoding a first-order Reed–Muller word.

```
def decode_rm(word):
    word = np.asarray(word)
    n = word.size
    if word.ndim != 1 or n < 2 or n & (n-1) or not np.all((word == 0) | (word == 1)):
        raise ValueError('Use a binary vector of dyadic length at least two.')
    spectrum = fwt(1-2*word.astype(int)).real
    a = int(np.argmax(abs(spectrum)))
    b = int(spectrum[a] < 0)
    return encode_rm(a, b, n.bit_length()-1)
```

Program 54. Nearest-word Reed–Muller decoding by Walsh correlation.

Solution to Exercise 6.5

1. Orthogonality of the Walsh basis and $\mathbb{E}[f^2] = 1$ give $\mathbb{E}[(f - a_\beta \chi_\beta)^2] = 1 - a_\beta^2$.
2. If the predicted sign is wrong, f and h lie on opposite sides of zero, or $h = 0$ and $f = -1$. In either case $|f - h| \geq 1$. Thus the error indicator is bounded pointwise by $(f - h)^2$; take expectations.
3. Apply Hoeffding to $Z_j = f(X_j)\chi_\beta(X_j)$, whose mean is a_β. The sample-size condition makes the failure probability at most δ. Conditional on $|\widetilde{a}_\beta - a_\beta| \leq \lambda$, orthogonal projection gives

$$\mathbb{E}_X[(f - \widetilde{a}_\beta \chi_\beta)^2] = 1 - a_\beta^2 + (a_\beta - \widetilde{a}_\beta)^2 \leq 1 - a_\beta^2 + \lambda^2.$$

   The sign bound then proves the assertion. The fixed-index assumption matters: choosing an index from the data requires a simultaneous bound.
4. Use $\lambda = \sqrt{\epsilon/M}$. For each of the M retained coefficients, Hoeffding bounds the probability of an error greater than λ by δ/M. A union bound implies that *all* errors are at most λ with probability at least $1 - \delta$; these coefficient errors need not be independent. On that event, Parseval gives

$$\mathbb{E}[(f - \varphi)^2] = \sum_{w(s)>d} a_s^2 + \sum_{w(s)\leq d} (a_s - \widetilde{a}_s)^2 \leq \alpha + M\lambda^2 = \alpha + \epsilon.$$

   Apply part 2. The exact count $M = \sum_{j=0}^{d} \binom{n}{j}$ also covers $d = 0$ and small n; replacing it indiscriminately by n^d does not.

Solution to Exercise 6.6

1. A generator of $C^\perp$ is H^T and a parity-check matrix is G^T, since $HG = 0$ and both have the required rank.
2. If $G = \begin{pmatrix} I_m \\ A \end{pmatrix}$, then $H = \begin{pmatrix} -A & I_{n-m} \end{pmatrix}$. The first m coordinates of Gu are the message u. The minus sign disappears only in characteristic two.
3. Choose m independent rows of G and move them to the top by a coordinate permutation. If the resulting top block is B, multiply G on the right by B^{-1}. The permutation gives an equivalent code; the invertible change of message basis preserves the code itself.

Solution to Exercise 6.7

1. The matrix H contains every coordinate vector, so it has rank k and nullity $2^k - 1 - k$. No column is zero and no two are equal; therefore no nonzero codeword has weight one or two. For distinct nonzero vectors a, b, the three distinct columns $a, b, a + b$ sum to zero, giving minimum distance three. If a single error occurs in position j, its syndrome is column j of H. With columns ordered by the binary integers $1, \ldots, n$, the syndrome directly encodes $j + 1$ when positions are numbered from zero.

```
def hamming_check(bits):
    if bits < 2:
        raise ValueError('Use at least two parity-check coordinates.')
    return np.array([binary_digits(j, bits) for j in range(1, 2**bits)]).T
```

```
def decode_hamming(word):
    word = np.asarray(word)
    n = word.size
    if word.ndim != 1 or n < 3 or (n+1) & n or not np.all((word == 0) | (word == 1)):
        raise ValueError('Use a binary Hamming word of length 2^k-1, k>=2.')
    bits = (n+1).bit_length()-1
    syndrome = hamming_check(bits) @ word.astype(int) % 2
    position = sum(int(bit) << j for j, bit in enumerate(syndrome))
    corrected = word.astype(int).copy()
    if position:
        corrected[position-1] ^= 1
    return corrected
```

Program 55. Syndrome decoding with columns ordered by binary integers.

```
def binary_digits(x, width):
    if not 0 <= x < 2**width:
        raise ValueError('The nonnegative integer must fit the requested width.')
    return np.array([(x >> j) & 1 for j in range(width)], dtype=int)
```

Program 56. Binary digits, least significant coordinate first.

2. Let α generate $\mathbb{F}_{2^k}^*$. A binary polynomial P satisfies $P(\alpha^2) = P(\alpha)^2$, so the BCH conditions for designed distance three reduce to $P(\alpha) = 0$. Expanding $\sum_{j=0}^{n-1} P_j \alpha^j = 0$ in a basis of the extension gives a binary check matrix whose columns enumerate every nonzero vector of $\mathbb{F}_2^k$. This is a Hamming code; requiring all n roots to vanish would instead give the zero code.
3. Each radius-one ball has $n + 1 = 2^k$ elements. The balls are disjoint because the minimum distance is three, and their total size is $2^{n-k}2^k = 2^n$. For the coordinate order of Example 11, a matching matrix is

$$H = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{pmatrix}.$$

   Its columns are the coordinates of $1, \alpha, \ldots, \alpha^6$ modulo $\alpha^3 + \alpha + 1$, and direct multiplication gives $HG = 0$. It is a column permutation of the canonical binary-integer check matrix.
4. Every nonzero dual word is $(\langle u, v\rangle)_{v \neq 0}$ for some $u \neq 0$. A nonzero linear functional on $\mathbb{F}_2^k$ takes each value 2^{k-1} times. Removing the zero input does not remove a one, so its weight is 2^{k-1}. Differences of distinct dual words are nonzero dual words, hence all pairwise distances have this value.
5. Use one nonzero representative from each projective line in $\mathbb{F}_q^k$. There are $n = (q^k - 1)/(q - 1)$ such lines. The check matrix has rank k; no two columns are proportional, while three columns in a common two-dimensional subspace are dependent. Thus the parameters are $[n, n - k, 3]$. A radius-one ball has $1 + n(q - 1) = q^k$ elements, which again proves perfection.

Solution to Exercise 6.8

1. Enumeration gives $W_C = X^7 + 7X^4Y^3 + 7X^3Y^4 + Y^7$.
2. The simplex dual has one zero word and $2^k - 1 = n$ words of weight $2^{k-1} = (n + 1)/2$. Hence

$$W_{C^\perp} = X^n + nX^{(n-1)/2}Y^{(n+1)/2}.$$

3. Apply the MacWilliams identity to $C^\perp$, whose cardinality is $n + 1$, and use $(C^\perp)^\perp = C$.

4. Expanding at $X = 1$ gives

$$A_1 = A_2 = 0, \qquad A_3 = \frac{n(n-1)}{6},$$
$$A_4 = \frac{n(n-1)(n-3)}{24}, \qquad A_5 = \frac{n(n-1)(n-3)(n-7)}{120},$$
$$A_6 = \frac{n(n-1)(n-3)(n-5)(n-7)}{720}.$$

   Program 57 computes the entire enumerator with exact symbolic arithmetic. Alternatively, each pair of distinct columns of H has a unique third column equal to their sum. Every weight-three word is counted by three such pairs, giving $A_3 = \binom{n}{2}/3$.

```
def hamming_weight_distribution(bits=3):
    x = sp.Symbol('Y')
    n = 2**bits-1
    polynomial = sp.Poly(sp.expand(((1+x)**n+n*(1-x)**(2**(bits-1))
                                   *(1+x)**(2**(bits-1)-1))/2**bits), x)
    return [int(polynomial.nth(k)) for k in range(n+1)]
```

Program 57. Exact weight distribution of a binary Hamming code.

5. Each row of H contains 2^{k-1} ones, an even number for $k \geq 2$. Thus $H\mathbf{1} = 0$, and $x \mapsto x + \mathbf{1}$ bijects words of weight i with words of weight $n - i$.

Solution to Exercise 6.9

Appending parity maps weights three and four to four, and weight seven to eight. The code therefore has parameters $[8, 4, 4]$ and enumerator $X^8 + 14X^4Y^4 + Y^8$. Its words, in the order of the messages used in Example 11, are

00000000	11010001	01101001	10111000
00110101	11100100	01011100	10001101
00011011	11001010	01110010	10100011
00101110	11111111	01000111	10010110

Each column of the generator G in that example has weight three, so the extended generator is $G_8 = \begin{pmatrix} G \\ 1\,1\,1\,1 \end{pmatrix}$. Directly, $G_8^{\mathsf{T}} G_8 = 0$ over $\mathbb{F}_2$. Its rank is four, so its image is self-orthogonal of dimension half the length and is therefore self-dual. The enumerator's invariance is a consequence of this argument; by itself it would only show that the code and its dual have the same weight distribution.

Solution to Exercise 6.10

1. The code has dimension one, minimum distance n, and enumerator $W_C = X^n + (q-1)Y^n$.
2. The dual is $\{x : \sum_i x_i = 0\}$, of dimension $n - 1$ and minimum distance two for $n \geq 2$: $(1, -1, 0, \ldots, 0)$ is a word, whereas no weight-one word satisfies the check. Its enumerator is

$$W_{C^\perp} = \frac{(X + (q-1)Y)^n + (q-1)(X - Y)^n}{q}.$$

   Self-duality first forces $1 = n - 1$, hence $n = 2$. In that case it holds exactly when $(1, 1)$ is orthogonal to itself, namely in characteristic two.

Solution to Exercise 6.11

1. For the Walsh matrix of order 2^k, the rows after conversion are the truth tables of the linear forms $x \mapsto \langle a, x \rangle$. Removing the coordinate $x = 0$ gives the simplex code of dimension k. Adding complements supplies all affine forms and gives the punctured Reed–Muller code of dimension $k + 1$. Both are linear. Linearity is not guaranteed for a general Hadamard matrix: for order twelve, the two cardinalities are twelve and twenty-four, neither a power of two. One Paley construction gives the following twelve words and their complements:

$\mathcal{A}_{12}$	$\mathcal{B}_{12} \setminus \mathcal{A}_{12}$
00000000000	11111111111
11011100010	00100011101
01101110001	10010001110
10110111000	01001000111
01011011100	10100100011
00101101110	11010010001
00010110111	11101001000
10001011011	01110100100
11000101101	00111010010
11100010110	00011101001
01110001011	10001110100
10111000101	01000111010

2. Orthogonal sign rows agree in $n/2$ positions and disagree in $n/2$. The removed first position was common to all rows, so the code $\mathcal{A}_n$ has length $n - 1$, cardinality n, and all distinct-word distances $n/2$. For distinct $u, v \in \mathcal{A}_n$,

$$d(u, \overline{v}) = (n - 1) - d(u, v) = n/2 - 1, \qquad d(u, \overline{u}) = n - 1.$$

The original and complemented sets are disjoint, since $n/2 < n - 1$ for $n \geq 4$. Thus $\mathcal{B}_n$ has length $n - 1$, cardinality $2n$, and minimum distance $n/2 - 1$.

Solution to Exercise 6.12

1. The coefficient of Y^k in $(1+Y)^{n-i}(1-Y)^i$ is $P_k(i)$. Comparing coefficients in $2^m W_{C^\perp}(1, Y) = W_C(1+Y, 1-Y)$ gives the formula.
2. Start with $2^{n-m} W_C(X, 1) = \sum_i A'_i (X + 1)^{n-i}(X - 1)^i$. After k derivatives, the left side at $X = 1$ is $2^{n-m} k! \sum_{i=0}^{n-k} \binom{n-i}{k} A_i$. For the summand indexed by i on the right, only the Leibniz term that differentiates $(X - 1)^i$ exactly i times can survive at $X = 1$. For $i \leq k$ its value is

$$k! \binom{n-i}{k-i} 2^{n-k} A'_i.$$

Divide by $2^{n-m} k!$ and use $\binom{n-i}{k-i} = \binom{n-i}{n-k}$.
3. Let H be a full-rank $(n - m) \times n$ check matrix of C. The MDS distance means that every $n - m$ columns are independent. A nonzero word of the dual has the form $u^\mathsf{T} H$, with $u \neq 0$. If it had $n - m$ zero coordinates, u would annihilate an invertible submatrix of H, a contradiction. Its weight is therefore at least $m + 1$, and Singleton's bound gives equality. In the moment identity for $k < m$, only $A'_0 = 1$ survives on the right; on the left, isolate $A_0 = 1$ and use $A_1 = \cdots = A_{d-1} = 0$.
4. Take the equations in the order $k = m - 1, m - 2, \ldots, 0$. The first contains only A_d; each following equation introduces one new coefficient, with coefficient one. They determine $A_d, \ldots, A_n$ successively. The first gives $A_d = \binom{n}{m-1} = \binom{n}{d}$.

Solution to Exercise 6.13

1. The coefficient comparison in the preceding exercise applies to W_{D_C} as well: $|C|B'_k = \sum_i B_i P_k(i)$. Proposition 56 makes this nonnegative.
2. For minimum distance at least d, $B_1, \ldots, B_{d-1}$ vanish, $B_0 = 1$, all other B_i are nonnegative, and $\sum_i B_i = |C|$. Every code therefore supplies a feasible point with objective value its cardinality, proving the upper bound. To see boundedness, put $z_k = \sum_i x_i P_k(i) \geq 0$. The generating identity $\sum_k P_k(i) Y^k = (1+Y)^{n-i}(1-Y)^i$ at $Y = 1$ gives
$$\sum_k z_k = 2^n x_0 = 2^n.$$
Since $z_0 = \sum_i x_i$ and the z_k are nonnegative, the objective is at most 2^n. Each nonnegative x_i is bounded by the same value. The feasible set is closed and nonempty, so the maximum is attained.

CHAPTER 7

Linear representations of finite groups

For a noncommutative group, scalar characters alone no longer resolve every function. Linear representations replace scalar frequencies by matrix blocks. This chapter develops irreducible representations, character orthogonality, and Fourier inversion, then uses these tools to study invariants, counting problems, and wavelets on finite fields.

A representation realizes an abstract group through linear transformations. This connects group structure with invariant subspaces, matrix spectra, and geometry. Our main references are [54, 29]; [30] emphasizes Fourier analysis, while [1, 18] provide further examples and character tables. The historical accounts [34, 35] explain how group determinants led to the theory.

7.1 Representations and their decomposition

Matrix-valued actions provide the additional components missing from scalar Fourier analysis on a noncommutative group. We introduce representations and their basic constructions, then show when they split into irreducible pieces.

7.1.1 Linear representations

A representation lets a group act by linear transformations while preserving its multiplication law. We give the definition, relate it to the group algebra, and construct examples that will support the decomposition theory.

Throughout this chapter G is finite, except in an exercise explicitly examining an infinite group. We initially allow a general field K; the character and Fourier theory will be over $\mathbb{C}$.

We define the action on the vector space itself, without choosing matrix coordinates.

Definition 42 (Linear representation)**.** A finite-dimensional representation is a homomorphism $\rho : G \to GL(V)$, where V is a finite-dimensional K-vector space. Thus $\rho(gh) = \rho(g)\rho(h)$ and $\rho(1) = \mathrm{Id}_V$. We also write $gv = \rho(g)v$. The representation is *faithful* if ρ is injective.

For example, the orientation-preserving symmetries of a cube form a group isomorphic to $\mathfrak{S}_4$, acting faithfully on $\mathbb{R}^3$. The full isometry group of the cube has twice as many elements, so the restriction to orientation-preserving symmetries is essential.

Extending group multiplication bilinearly gives an algebra on which representations can also act.

Definition 43 (Group algebra)**.** The group algebra $K[G]$ has basis $(\delta_g)_{g\in G}$ and multiplication $\delta_g * \delta_h = \delta_{gh}$, extended bilinearly. Its identity is δ_1. Identifying a basis vector with a point indicator gives

$$f = \sum_{g\in G} f(g)\delta_g, \tag{7.1}$$

and the convolution formula

$$(f * h)(x) = \sum_{g\in G} f(g)h(g^{-1}x).$$

The order of the factors matters when G is noncommutative.

To express this extension precisely, we specify what it means for an algebra to act on a vector space.

Definition 44 (Algebra representation)**.** A representation of a unital associative K-algebra A on V is a *unital* algebra homomorphism $A \to \mathrm{End}(V)$.

Every group representation extends uniquely to one of its group algebra:

$$\rho(f) = \sum_{g \in G} f(g)\rho(g).$$

Conversely, restriction to the units δ_g recovers the group representation. The unital condition ensures $\rho(\delta_1) = \mathrm{Id}_V$.

The same action is often described in the language of modules.

Remark 23 (Modules). Equivalently, V is a unital left $K[G]$-module, with action $f \cdot v = \rho(f)v$. Group representations and such modules are two languages for the same structure.

The regular representation

Every finite group acts on functions on itself by translation. This regular representation is a faithful model of the group and will later contain every irreducible complex representation.

Left multiplication defines the regular representation on $K[G]$:

$$\rho_r(g)f = \delta_g * f, \qquad (\rho_r(g)f)(x) = f(g^{-1}x).$$

It permutes the basis by $\rho_r(g)\delta_h = \delta_{gh}$, and is faithful because $\rho_r(g)\delta_1 = \delta_g$. Its dimension is $|G|$.

Direct sums and spaces of linear maps

New representations can be built from old ones by combining their spaces or acting on the maps between them. Direct sums describe decompositions, while the action on linear maps will let us compare the resulting components.

For representations on V and W, their direct sum acts by

$$\rho_{V \oplus W}(g)(v, w) = (\rho_V(g)v, \rho_W(g)w).$$

Its matrices are block diagonal. On the space $\mathcal{L}(V, W)$, define

$$g \cdot T = \rho_W(g)T\rho_V(g^{-1}).$$

This is a representation since $\rho_W(g)\rho_W(h)T\rho_V(h^{-1})\rho_V(g^{-1}) = \rho_W(gh)T\rho_V((gh)^{-1})$. It is also the tensor representation on $W \otimes V^*$. More generally, tensor products act by $g \cdot (v \otimes w) = \rho_V(g)v \otimes \rho_W(g)w$.

The dual representation

An action on vectors induces an action on linear forms that preserves evaluation. The inverse in this construction is essential for obtaining a representation on the dual space.

The dual space $V^* = \mathcal{L}(V, K)$ carries

$$\rho^*(g)\ell = \ell \circ \rho(g^{-1}), \qquad \rho^*(g) = \rho(g^{-1})^{\mathsf{T}}$$

in dual bases. The transpose here is algebraic; it is not a Hermitian adjoint. The evaluation pairing satisfies

$$(\rho^*(g)\ell)(v) = \ell(\rho(g^{-1})v), \qquad (\rho^*(g)\ell)(\rho(g)v) = \ell(v).$$

This is the representation on $\mathcal{L}(V, K)$ when K carries the trivial action.

Actions on polynomials

Linear changes of variables act on polynomials by substitution. Restricting to each degree gives finite-dimensional representations whose fixed vectors will describe polynomial invariants.

For a finite subgroup $G \subset GL_n(K)$, regard X as a column of indeterminates and set

$$(\rho(A)P)(X) = P(A^{-1}X).$$

If $A^{-1} = (a_{ij})$, the substitution is $X_i \mapsto \sum_j a_{ij}X_j$. The inverse and this row/column convention ensure $\rho(A)\rho(B) = \rho(AB)$. Each homogeneous degree is preserved. Thus both polynomials of degree at most s, denoted $K_s[X_1, \ldots, X_n]$, and homogeneous polynomials of degree s, denoted $K_s^0[X_1, \ldots, X_n]$, are finite-dimensional representations. The entire polynomial ring is an infinite-dimensional representation. Its invariant subring will be studied in Exercises 7.5 and 7.6.

One-dimensional representations

One-dimensional representations recover the scalar characters used earlier in the book. They also clarify the distinction between multiplicative characters and the trace characters introduced below.

Over $\mathbb{C}$, a one-dimensional representation is a homomorphism $G \to \mathbb{C}^*$, hence a character in the sense of Chapter 1. Even when G is nonabelian, $g \mapsto \det \rho(g)$ is one such representation. We will shortly extend the word *character* to traces of higher-dimensional representations; those traces are generally not homomorphisms.

7.1.2 Irreducibility and complete reducibility

To identify the elementary components of a representation, we examine its invariant subspaces. Over the complex numbers, averaging an inner product makes invariant complements available and leads to complete reducibility.

Definition 45 (Isomorphism and subrepresentation)**.** An isomorphism between representations on V and W is a linear isomorphism $T : V \to W$ with $T\rho_V(g) = \rho_W(g)T$ for every g. A subrepresentation is the restriction to an invariant subspace. A representation is *irreducible* if $V \neq 0$ and its only invariant subspaces are 0 and V. A nonzero representation is *indecomposable* if it is not a direct sum of two nonzero subrepresentations.

Irreducibility implies indecomposability over every field. The converse requires a theorem: an invariant subspace need not have an invariant complement. We now work over $\mathbb{C}$, unless another field is explicitly specified.

For finite groups over the complex numbers, averaging supplies an inner product compatible with every group element.

Proposition 57 (An invariant inner product)**.** *Every complex representation of a finite group admits a positive-definite Hermitian inner product for which all $\rho(g)$ are unitary.*

Proof. Average any positive-definite Hermitian inner product:

$$\langle v,\, w\rangle_G = \frac{1}{|G|}\sum_{g\in G}\langle \rho(g)v,\, \rho(g)w\rangle.$$

The term $g = 1$ guarantees strict positivity for $v \neq 0$. Replacing g by gh in the sum proves invariance under $\rho(h)$. □

With this inner product, the orthogonal complement of an invariant subspace is invariant as well.

Theorem 11 (Complete reducibility)**.** *Every invariant subspace has an invariant complement. Consequently a nonzero representation is irreducible if and only if it is indecomposable.*

Proof. Let W be invariant and use the averaged inner product. If $v \perp W$, then for $w \in W$, $\langle \rho(g)v,\, w\rangle_G = \langle v,\, \rho(g^{-1})w\rangle_G = 0$. Thus $W^\perp$ is invariant and $V = W \oplus W^\perp$. □

An algebraic version of the averaging argument shows exactly which assumption on the coefficient field is needed.

Remark 24 (Maschke's averaging argument). The conclusion holds over any field whose characteristic does not divide $|G|$; algebraic closure is unnecessary. Choose a linear projection $P : V \to W$ and average its conjugates:

$$P_0 = \frac{1}{|G|}\sum_{g\in G}\rho(g)P\rho(g^{-1}).$$

Each summand maps into W and is the identity on W. Therefore $P_0^2 = P_0$ and $\operatorname{im} P_0 = W$. Reindexing shows that P_0 commutes with the group action, so $\ker P_0$ is the required invariant complement. This proof explains precisely why the characteristic matters.

Repeatedly taking invariant complements decomposes the whole representation into elementary pieces.

Proposition 58 (Decomposition into irreducibles)**.** *Every complex representation of a finite group is a direct sum of irreducible representations; the zero representation is the empty sum.*

Proof. Induct on the dimension. An irreducible space needs no decomposition; otherwise split off a proper nonzero invariant subspace and its invariant complement, and apply the induction hypothesis to both. □

The decomposition raises a distinction between the chosen summand subspaces and their intrinsic multiplicities.

Remark 25 (Uniqueness). The summand subspaces need not be unique. Their isomorphism classes and multiplicities are unique, as the character formulas below will show.

7.1.3 Permutation representations

An action on a finite set becomes a linear representation by permuting basis vectors. For the symmetric group, cycle structure lets us compare group conjugacy with similarity of the resulting matrices.

The natural action of $\mathfrak{S}_n$ on K^n is $\rho_p(\sigma)e_j = e_{\sigma(j)}$. Its permutation matrix M_σ has trace equal to the number of fixed points of σ. Labeling the elements of a finite group identifies its regular representation with this construction through the embedding $G \hookrightarrow \mathfrak{S}_{|G|}$ given by left multiplication.

We first recall the group-theoretic criterion for conjugacy, expressed entirely in terms of cycles.

Lemma 11 (Conjugacy in the symmetric group)**.** *Two permutations are conjugate exactly when they have the same number of cycles of each length. Thus conjugacy classes are indexed by partitions of n.*

Proof. Conjugation relabels each cycle and preserves its length. Conversely, match cycles of equal length and choose a permutation matching their entries in cyclic order; it conjugates one permutation to the other. □

In characteristic zero, passing to permutation matrices preserves this classification even when arbitrary changes of basis are allowed.

Theorem 12 (Similarity of permutation matrices)**.** *Over a field of characteristic zero, M_σ and M_τ are similar if and only if σ and τ are conjugate in $\mathfrak{S}_n$.*

Proof. Conjugating permutations conjugates their matrices. Conversely, let $c_k(\sigma)$ count the k-cycles. A cycle block has characteristic polynomial $X^k - 1$, so similarity implies

$$\prod_k (X^k - 1)^{c_k(\sigma)} = \prod_k (X^k - 1)^{c_k(\tau)}. \tag{7.2}$$

In an algebraic closure, a primitive mth root is a simple root of $X^k - 1$ exactly when $m \mid k$. Its multiplicity in the product is $\sum_{m|k} c_k$. Starting at $m = n$ and descending determines every c_m, so the cycle counts agree. This proof was communicated by Daniel Ferrand. □

For $n \geq 2$, the complex permutation representation splits as

$$H_0 = \mathbb{C}(1, \dots, 1), \tag{7.3}$$

$$H_1 = \{x \in \mathbb{C}^n : x_1 + \cdots + x_n = 0\}. \tag{7.4}$$

The action on H_0 is trivial; that on H_1 is the *standard representation*. Exercise 7.7 proves its irreducibility using double transitivity. The other one-dimensional representation is the sign $\epsilon(\sigma) = \det M_\sigma$. For $n \geq 2$, these are the only one-dimensional representations; a complete classification of higher-dimensional ones uses partitions and Young tableaux [29].

7.2 Invariant vectors and averaging

Averaging over a finite group extracts the part of a vector or linear map that respects the symmetry. We develop this operation and combine it with Schur's lemma, preparing the orthogonality relations for characters.

7.2.1 Invariants and intertwining operators

Fixed vectors encode quantities unchanged by the group action. Applied to a space of linear maps, the same idea defines intertwiners, which compare representations without breaking their symmetry.

Definition 46 (Invariant vectors)**.** For a representation on V, let $V^G = \{v : \rho(g)v = v \text{ for all } g \in G\}$.

For the polynomial action of $\mathfrak{S}_n$, invariants are precisely the symmetric polynomials. For the action on $\mathcal{L}(V, W)$, they are intertwining operators.

Definition 47 (Intertwining operator)**.** The space of G-equivariant linear maps is

$$\mathrm{Hom}_G(V, W) = \{T \in \mathcal{L}(V, W) : T\rho_V(g) = \rho_W(g)T \text{ for all } g\}.$$

Its invertible elements are representation isomorphisms. In module language, these are precisely the $K[G]$-module homomorphisms.

7.2.2 Schur's lemma

Irreducibility sharply restricts the linear maps that can respect a group action. Schur's lemma makes this restriction precise and will turn averaged matrix identities into scalar formulas.

Lemma 12 (Schur's lemma)**.** *A nonzero intertwiner between irreducible representations is an isomorphism. Over $\mathbb{C}$, every equivariant endomorphism of an irreducible representation is a scalar multiple of the identity.*

Proof. The kernel and image of an intertwiner are invariant. Irreducibility therefore forces the kernel to be zero and the image to be the whole target when the map is nonzero. For an endomorphism T over $\mathbb{C}$, choose an eigenvalue λ. Then $T - \lambda \,\mathrm{Id}$ is an intertwiner with nontrivial kernel, so it is zero. □

The first assertion holds over any field. Without algebraic closure, $\mathrm{End}_G(V)$ is a division algebra, not necessarily the scalar field or even commutative.

Over the complex numbers, Schur's lemma consequently determines the dimension of the intertwiner space between two irreducibles.

Corollary 5. *For irreducible complex representations,*

$$\dim \mathrm{Hom}_G(V, W) = \begin{cases} 1 & V \simeq W, \\ 0 & V \not\simeq W. \end{cases}$$

Proof. If $S : V \to W$ is an equivariant isomorphism, any intertwiner T satisfies $S^{-1}T = \lambda \,\mathrm{Id}$, hence $T = \lambda S$. The nonisomorphic case follows from the lemma. □

For an irreducible representation, an element of the center $Z(G)$ acts by a scalar, since its action commutes with every $\rho(g)$. In particular, all irreducible complex representations of an abelian group are one-dimensional: if every group element acts by a scalar, every line is invariant.

7.2.3 The Reynolds operator

Averaging the orbit of a vector removes its noninvariant component. The resulting Reynolds operator gives both a projection onto the fixed subspace and a trace formula for its dimension.

Definition 48 (Reynolds operator)**.** For a representation on V, set $R_G = |G|^{-1} \sum_{g \in G} \rho(g)$.

The average has precisely the projection and trace properties needed to extract and count invariant vectors.

Theorem 13 (Averaging projects onto invariants)**.** *The operator R_G is a projection with image V^G, and $\dim_{\mathbb{C}} V^G = \mathrm{tr}\, R_G$. For an invariant Hermitian inner product it is the orthogonal projection onto V^G.*

Proof. Reindexing gives $\rho(h)R_G = R_G$, so its image is invariant pointwise. It fixes every vector of V^G, proving $R_G^2 = R_G$ and the image claim. A projection has only eigenvalues zero and one, so its trace is its rank. Finally, unitarity and the bijection $g \mapsto g^{-1}$ give $R_G^* = R_G$. □

7.2.4 Averaging linear maps

Averaging can similarly turn any linear map into an intertwiner. For irreducible representations, Schur's lemma then determines the dimension of the image of this averaging operator.

Definition 49 (Averaged map)**.** For $T \in \mathcal{L}(V, W)$, the Reynolds operator of the Hom representation is

$$\widetilde{T} = \frac{1}{|G|} \sum_{g \in G} \rho_W(g) T \rho_V(g^{-1}).$$

It is an intertwiner, and every intertwiner is fixed by this averaging.

For irreducibles, its trace on the space of maps distinguishes equivalent representations from inequivalent ones.

Proposition 59. *If V, W are irreducible, the trace of this operator on $\mathcal{L}(V, W)$ is one when $V \simeq W$ and zero otherwise.*

This follows by combining the projection theorem with Schur's lemma. It is the link between averaging matrices and orthogonality of characters.

7.3 Characters

Traces compress representation matrices into scalar functions that are constant on conjugacy classes. Despite this compression, complex characters retain enough information to identify representations up to isomorphism and to compute their multiplicities.

7.3.1 Traces and character identities

The character records the trace of each group element's action. We establish how it behaves under the basic representation constructions, including the action on linear maps needed for orthogonality.

Definition 50 (Character)**.** The character of a representation ρ is $\chi_\rho(g) = \operatorname{tr} \rho(g)$.

A higher-dimensional character need not be multiplicative and can vanish. Nevertheless, it contains all eigenvalue data: the values $\chi_\rho(g^j)$ are the power sums of the eigenvalues of $\rho(g)$. Newton's identities recover its characteristic polynomial from the first $\dim V$ power sums. We will show that the whole character determines the representation up to isomorphism.

A trace identity for left and right matrix multiplication will express the character of a space of linear maps in terms of the two original characters.

Lemma 13 (Trace on a space of linear maps)**.** *For $u \in \operatorname{End}(W)$ and $v \in \operatorname{End}(V)$, the endomorphism $\Phi : \mathcal{L}(V, W) \to \mathcal{L}(V, W)$, $T \mapsto uTv$, has trace* $\operatorname{tr}(u) \operatorname{tr}(v)$.

Proof. Use the matrix-unit basis E_{ji} of $\mathcal{L}(V, W)$. The coefficient of E_{ji} in $uE_{ji}v$ is $u_{jj}v_{ii}$. Summing these diagonal coefficients proves the formula. □

Together with trace invariance, this identity gives the character rules for the constructions introduced above.

Proposition 60 (Character identities)**.** *For complex representations of a finite group,*

$$\begin{aligned} \chi_V(1) &= \dim V, & \chi_V(g^{-1}) &= \overline{\chi_V(g)}, \\ \chi_V(hgh^{-1}) &= \chi_V(g), & \chi_{V \oplus W} &= \chi_V + \chi_W, \\ \chi_{\mathcal{L}(V,W)} &= \overline{\chi_V} \chi_W, & \chi_{V^*} &= \overline{\chi_V}. \end{aligned}$$

Isomorphic representations have equal characters, and $\chi_{V \otimes W} = \chi_V \chi_W$.

Proof. The first, conjugacy, direct-sum, and isomorphism identities follow from trace invariance and block matrices. Choose invariant inner products; then $\rho(g^{-1}) = \rho(g)^*$, giving conjugation of the trace. Apply the preceding lemma to the Hom action. The dual case takes $W = \mathbb{C}$ with the trivial action, and the tensor identity follows from the trace of a tensor product. □

The regular representation has an especially simple character because a nonidentity translation fixes no group element.

Example 14 (Regular character). The regular character is $\chi_r = |G|\delta_1$. Indeed, left multiplication by g fixes a basis element δ_h exactly when $gh = h$, or $g = 1$.

7.3.2 Orthogonality

Characters of distinct irreducible representations are orthogonal for the uniform inner product on the group. Averaging on a space of linear maps explains this identity and shows why Schur's lemma supplies its normalization.

Use the normalized Hermitian product, linear in its first argument,

$$\langle f, h\rangle = \frac{1}{|G|}\sum_{g\in G} f(g)\overline{h(g)}.$$

With this normalization, the irreducible characters satisfy the following orthonormality relation.

Theorem 14 (Character orthogonality)**.** *The characters of pairwise nonisomorphic irreducible representations form an orthonormal family in* $\mathbb{C}[G]$.

Proof. The trace of the Reynolds operator on $\mathcal{L}(V, W)$ is

$$\frac{1}{|G|}\sum_{g\in G}\overline{\chi_V(g)}\chi_W(g) = \langle \chi_W, \chi_V\rangle.$$

It also equals $\dim \mathrm{Hom}_G(V, W)$, which is zero or one by Schur's lemma. □

There are therefore finitely many irreducible isomorphism classes: their characters form a linearly independent family in the $|G|$-dimensional space $\mathbb{C}[G]$.

7.4 Multiplicities and dimension formulas

Character orthogonality turns abstract decomposition into a calculation of integer multiplicities. Applying it to the regular representation also gives a dimension identity that will establish completeness of the matrix-valued Fourier transform.

7.4.1 Decomposing a representation

An irreducible character acts as a test for how many copies of its representation occur in a given action. We express these multiplicities as inner products, which also gives criteria for isomorphism and irreducibility.

Let $(V_i, \rho_i)_{i=1}^r$ represent all irreducible isomorphism classes, and write $d_i = \dim V_i$ and $\chi_i = \chi_{V_i}$. The set of these classes is the dual $\hat{G}$. It is generally a set, not a group.

The inner product with each irreducible character extracts its multiplicity uniquely.

Proposition 61 (Uniqueness of multiplicities)**.** *Every representation has a decomposition*

$$V \simeq \bigoplus_{i=1}^{r} V_i^{\oplus a_i}, \qquad a_i = \langle \chi_V, \chi_i\rangle \in \mathbb{N}. \tag{7.5}$$

Moreover, $\langle \chi_V, \chi_V\rangle = \sum_i a_i^2$.

Proof. Complete reducibility gives

$$V = W_1 \oplus \cdots \oplus W_q \tag{7.6}$$

with irreducible summands. Additivity gives $\chi_V = \sum_j \chi_{W_j}$. Taking its inner product with χ_i counts the summands isomorphic to V_i by orthogonality. Taking its squared norm gives the final identity. □

These multiplicities show that characters determine isomorphism classes and detect irreducibility.

Corollary 6. *Two representations are isomorphic exactly when their characters agree. A representation is irreducible exactly when its character has squared norm one.*

The first statement follows because the character determines every multiplicity; the second uses their nonnegative integer values.

7.4.2 The regular representation

The regular representation records all irreducibles at once. Its particularly simple character determines their multiplicities and gives a useful check that a proposed list is complete.

Proposition 62. *Each irreducible V_i occurs in the regular representation with multiplicity d_i. Thus $\mathbb{C}[G] \simeq \bigoplus_i V_i^{\oplus d_i}$ as a G-module.*

Proof. Since $\chi_r = |G|\delta_1$, its inner product with χ_i is $\overline{\chi_i(1)} = d_i$. □

Evaluating the regular-character decomposition at the identity and away from it yields two useful counting identities.

Corollary 7. *The irreducible degrees and characters satisfy*

$$\sum_i d_i^2 = |G|, \qquad \sum_i d_i \chi_i(g) = 0 \quad (g \neq 1).$$

These are the values at the identity and elsewhere of $\chi_r = \sum_i d_i \chi_i$. They also check whether a proposed list of pairwise nonisomorphic irreducibles is complete.

7.5 Fourier analysis with matrix coefficients

Scalar characters describe class functions, but arbitrary functions on a noncommutative group require full matrix blocks. We construct this Fourier transform, prove inversion, and then recover scalar formulas for the center of the group algebra.

7.5.1 The Fourier transform

The algebra extension of each irreducible representation supplies one Fourier block. Taking all blocks preserves the order of convolution products and provides enough coordinates to reconstruct any function on the group.

Definition 51 (Matrix-valued Fourier transform)**.** For $f \in \mathbb{C}[G]$, let

$$\mathcal{F} f(\rho) = \sum_{g \in G} f(g)\rho(g), \qquad \mathcal{F} : \mathbb{C}[G] \longrightarrow \bigoplus_{i=1}^{r} \mathrm{End}(V_i). \tag{7.7}$$

This is the algebra extension of each representation. With this convention, $\mathcal{F}\delta_g(\rho) = \rho(g)$ and multiplication keeps its order:

$$\mathcal{F}(f * h)(\rho) = \mathcal{F} f(\rho)\mathcal{F} h(\rho).$$

Indeed, substitute $x = gy$ into the convolution sum and use $\rho(gy) = \rho(g)\rho(y)$. For an abelian group, every irreducible representation is one-dimensional, and the block indexed by $\rho = \chi$ is exactly $\hat{f}(\chi) = \sum_g f(g)\chi(g)$ from (1.10). For the numerical DFT of Chapter 3, we take the cyclic characters $\chi_k(n) = e^{-2\pi i k n/N}$, as in (3.2). The negative exponential comes from this indexing choice; no extra conjugation is inserted in the group transform.

The regular representation ensures that no information is lost by collecting all the irreducible blocks.

Proposition 63 (Fourier isomorphism)**.** *The map $\mathcal{F}$ is an algebra isomorphism.*

Proof. If all blocks of f vanish, its action on every direct sum of irreducibles vanishes. In particular it acts as zero in the regular representation. Applying this action to δ_1 gives $f * \delta_1 = f = 0$, proving injectivity. The source and target both have dimension $|G| = \sum_i d_i^2$, so the map is also surjective. □

The same regular-character identity gives an explicit reconstruction formula using matrix traces.

Theorem 15 (Fourier inversion)**.** *For every $g \in G$,*

$$f(g) = \frac{1}{|G|} \sum_{i=1}^{r} d_i \operatorname{tr}\Big(\rho_i(g^{-1}) \mathcal{F} f(\rho_i)\Big).$$

Proof. By linearity it suffices to take $f = \delta_h$. The right side becomes $|G|^{-1} \sum_i d_i \chi_i(g^{-1}h) = |G|^{-1} \chi_r(g^{-1}h)$, which is one if $g = h$ and zero otherwise. □

7.5.2 Class functions and the center

Functions constant on conjugacy classes have a simpler spectral description. We identify them with the center of the convolution algebra and use Schur's lemma to show why their Fourier blocks are scalar.

A conjugacy class is $C_h = \{ghg^{-1} : g \in G\}$. A *class function* (or central function) is constant on each conjugacy class. We denote their space by $\mathbb{C}[G]^G$. If the classes are $C_1, \dots, C_s$, their indicators

$$f_{C_j}(g) = \begin{cases} 1 & g \in C_j, \\ 0 & g \notin C_j \end{cases} \tag{7.8}$$

form a basis, so the dimension is s. The notation refers to invariants for the pullback action $(h \cdot f)(x) = f(h^{-1}xh)$.

Conjugation invariance of a function is equivalent to commuting with every element of the group algebra.

Proposition 64. *Class functions are precisely the center of the convolution algebra.*

Proof. One has $(\delta_h * f * \delta_{h^{-1}})(x) = f(h^{-1}xh)$. Thus f is a class function exactly when it commutes with every δ_h, and by linearity exactly when it commutes with all of $\mathbb{C}[G]$. □

Schur's lemma therefore reduces each irreducible Fourier block to a scalar, which its trace determines.

Lemma 14 (Central Fourier blocks)**.** *For a class function f and an irreducible representation of degree d,*

$$\mathcal{F} f(\rho) = \frac{|G|}{d} \langle f, \overline{\chi_\rho} \rangle \operatorname{Id}.$$

Proof. Centrality makes the block commute with every $\rho(g)$, so Schur's lemma makes it scalar. Taking the trace determines the scalar from $d\lambda = \sum_g f(g)\chi_\rho(g)$. □

These scalar blocks contain all the information in a class function, establishing completeness of the character family.

Theorem 16 (Characters form a basis of class functions)**.** *The irreducible characters form an orthonormal basis of $\mathbb{C}[G]^G$.*

Proof. Orthogonality is established. If a class function is orthogonal to all conjugate characters, the lemma makes every Fourier block zero, hence the function is zero. The conjugate characters therefore span; complex conjugation shows that the characters themselves span as well. □

Counting the elements in this basis identifies the number of irreducibles with the number of conjugacy classes.

Corollary 8. *The number of irreducible isomorphism classes equals the number of conjugacy classes.*

Equality of these counts does not by itself give a preferred correspondence between the two sets.

Together with the dimension formula, this count characterizes the groups for which scalar Fourier analysis is sufficient.

Corollary 9. *A finite group is abelian if and only if all its irreducible complex representations have degree one.*

Proof. If there are r conjugacy classes, the group is abelian exactly when $r = |G|$. Since $\sum_i d_i^2 = |G|$ and every $d_i \geq 1$, this is equivalent to every d_i being one. □

7.5.3 Fourier series of class functions

For class functions, the irreducible characters themselves are a complete set of scalar Fourier coordinates. We write the corresponding expansion and energy identity with the normalized inner product.

For any function, define the character coefficients

$$c_f(\rho_i) = \langle f, \chi_i \rangle = \frac{1}{|G|} \sum_{g \in G} f(g)\overline{\chi_i(g)}. \tag{7.9}$$

If f is a class function, the orthonormal-basis theorem gives $f = \sum_i c_f(\rho_i)\chi_i$.

Orthonormality transfers the inner product of class functions directly to their character coefficients.

Proposition 65 (Plancherel identity for class functions)**.** *For class functions f, h,*

$$\frac{1}{|G|} \sum_{g \in G} f(g)\overline{h(g)} = \sum_i c_f(\rho_i)\overline{c_h(\rho_i)}.$$

Expand both functions in the character basis and use orthogonality. For a general function, the character expansion is only its orthogonal projection onto class functions. The next chapter uses matrix entries to obtain a basis of the whole space.

7.5.4 Central idempotents and isotypic projections

A decomposition into individual irreducible subspaces may depend on choices, but the sum of all copies of a fixed irreducible is intrinsic. Central idempotents give explicit projectors onto these isotypic components.

A central idempotent of an algebra is a central element e with $e^2 = e$. In a direct sum of full matrix algebras, the elements E_i that are the identity in block i and zero in all other blocks are the primitive central idempotents. They sum to the identity and satisfy $E_iE_j = \delta_{ij}E_i$. Fourier inversion gives their counterparts:

$$e_i(g) = \frac{d_i}{|G|}\chi_i(g^{-1}) = \frac{d_i}{|G|}\overline{\chi_i(g)}, \qquad \sum_i e_i = \delta_1, \quad e_i * e_j = \delta_{ij}e_i.$$

For any representation ρ_U, extend the action to $\mathbb{C}[G]$ and set

$$P_i = \rho_U(e_i) = \frac{d_i}{|G|} \sum_{g \in G} \overline{\chi_i(g)}\rho_U(g).$$

The block identities of the central idempotents identify the image of each projector independently of any chosen decomposition.

Proposition 66 (Isotypic projection)**.** *The operator P_i projects onto the sum U_i of all irreducible subrepresentations isomorphic to V_i. These intrinsic isotypic components satisfy $U = \bigoplus_i U_i$ and $U_i \simeq V_i^{\oplus a_i}$.*

Proof. On an irreducible summand of type V_j, the operator P_i acts by the ith central idempotent: it is the identity for $i = j$ and zero otherwise. Complete reducibility proves all assertions. In an invariant Hermitian inner product these projections are orthogonal, since $P_i^* = P_i$. □

Individual copies inside an isotypic component may depend on a chosen decomposition; the component and its projector do not.

7.6 Exercises

These exercises test the hypotheses behind complete reducibility and use averaging to study symmetry, counting, and invariants. The first exhibits a small representation in which an invariant subspace has no invariant complement.

Exercise 7.1 (Reducible but indecomposable representations)**.** For the additive group of a field K, consider

$$\rho(x) = \begin{pmatrix} 1 & x \\ 0 & 1 \end{pmatrix}.$$

Show that this representation is reducible but indecomposable. Explain which hypothesis of complete reducibility fails when $K = \mathbb{C}$, and which fails when K is finite. Lack of algebraic closure is not the relevant obstruction in the finite case.

Commuting with every left translation forces an operator to be a right convolution, with the side of multiplication fixed by the group law.

Exercise 7.2 (Translation-invariant operators)**.** For a finite group G, let $\tau_h f(g) = f(h^{-1}g)$. If a linear operator $A : \mathbb{C}[G] \to \mathbb{C}[G]$ commutes with all τ_h, prove that there is a unique $\varphi \in \mathbb{C}[G]$ with $Af = f * \varphi$ for every f. Explain why the convolution factor is on the right.

Multiplying a representation by a scalar character preserves irreducibility but may leave its isomorphism class unchanged.

Exercise 7.3 (Twisting by a one-dimensional representation)**.** Let $\eta : G \to \mathbb{C}^*$ be a one-dimensional character and let ρ be irreducible. Prove that $g \mapsto \eta(g)\rho(g)$ is irreducible, with character $\eta\chi_\rho$. Show that a nontrivial twist need not produce a new isomorphism class. Find examples where it equals ρ, and where it is the one-dimensional representation η.

Tensor products describe all irreducible representations of a product of finite groups.

Exercise 7.4 (Representations of a product group)**.** For finite groups G, H, construct every irreducible representation of $G \times H$ from those of its two factors. Give the characters and prove both pairwise inequivalence and completeness of your list.

Averaging monomials constructs polynomial invariants, and Noether's bound limits the degrees needed to generate the invariant ring.

Exercise 7.5 (Invariant polynomials and Noether's bound)**.** Let $G \subset GL_n(K)$ be finite and K have characteristic zero. The action is $P(X) \mapsto P(A^{-1}X)$. For a multi-index $\alpha \in \mathbb{N}^n$, write $X^\alpha = \prod_j X_j^{\alpha_j}$ and $|\alpha| = \sum_j \alpha_j$. We seek finitely many invariants $P_1, \dots, P_s$ such that

$$K[X_1, \dots, X_n]^G = K[P_1, \dots, P_s]. \tag{7.10}$$

1. For permutation matrices representing $\mathfrak{S}_n$, identify the invariants and give generators.
2. Determine minimal generators for the invariant rings of $V_4 = \{\mathrm{diag}(\pm 1, \pm 1)\}$ and $C_2 = \{I, -I\}$ on $K[X, Y]$. Is the polynomial expression in the generators unique?
3. Let $R_G(P) = |G|^{-1} \sum_{A \in G} P(A^{-1}X)$. Show that it suffices to express each $R_G(X^\alpha)$ in terms of the averages with $|\beta| \le |G|$.
4. Put $N = |G|$ and introduce $u_1, \dots, u_n$. For $A \in G$, set $U_A = \sum_j u_j (A^{-1}X)_j$. Prove

$$S_k := \sum_{A \in G} U_A^k = N \sum_{|\alpha| = k} \binom{k}{\alpha} R_G(X^\alpha) u^\alpha.$$

   Use Newton's identities to express S_k, for $k > N$, as a polynomial in $S_1, \dots, S_N$. Compare coefficients of u^α and conclude Noether's bound:

$$K[X_1, \dots, X_n]^G = K[R_G(X^\beta) : |\beta| \le |G|].$$

5. Apply this method to the quarter-turn group generated by $\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$. Reduce the resulting list of generators and exhibit a relation among them.

Molien's series counts homogeneous invariants degree by degree using only the original representation matrices.

Exercise 7.6 (Molien's theorem)**.** For a finite subgroup $G \subset GL_n(\mathbb{C})$, let V_s be the space of homogeneous polynomials of degree s, with action $P(X) \mapsto P(A^{-1}X)$. Set $a_s = \dim V_s^G$ and $\Phi(t) = \sum_{s \ge 0} a_s t^s$. Prove the formal-series identity

$$\Phi(t) = \frac{1}{|G|} \sum_{A \in G} \frac{1}{\det(I - tA)}. \tag{7.11}$$

1. Show that invariance of a polynomial is equivalent to invariance of all its homogeneous components.

2. Let $A^{[s]}$ be the matrix of the degree-s action. Show $a_s = |G|^{-1} \sum_A \operatorname{tr} A^{[s]}$. As a convention check, compute $A^{[2]}$ in the basis (X^2, XY, Y^2) when $A^{-1} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$.
3. Express the eigenvalues of $A^{[s]}$ using those of A^{-1}, then sum over s. Explain why inversion of the group elements converts the denominator $\det(I - tA^{-1})$ into the displayed formula.
4. Compute the Molien series for the groups G_1, G_2 defined in Exercise 8.9. Explain how comparing homogeneous dimensions can certify a proposed generating set, and why a Hilbert series alone does not establish algebraic independence or generation.

For permutation representations, averaging fixed-point counts gives orbit counts and an irreducibility test for doubly transitive actions.

Exercise 7.7 (The Cauchy–Frobenius counting lemma)**.** A finite group G acts on a finite set X. Let X_g be the fixed-point set of g and let $\pi(g)e_x = e_{gx}$ on $V = \mathbb{C}[X]$.

1. Express $\langle \chi_\pi, 1 \rangle$ in terms of $|X_g|$.
2. Prove that this inner product counts the orbits, either by identifying invariant functions or by counting pairs (g, x) with $gx = x$.
3. Count binary necklaces of six beads, identifying rotations and reflections. Use all twelve elements of the dihedral group D_6.
4. Suppose $|X| \geq 2$ and the action is doubly transitive. Show $\dim \operatorname{End}_G(V) = 2$ and deduce that the complement of the constants in $\mathbb{C}[X]$ is irreducible. Apply this to the standard representation of $\mathfrak{S}_n$ for $n \geq 2$.

Combining character orthogonality with algebraic integers imposes a divisibility condition on every irreducible degree.

Exercise 7.8 (Irreducible degrees and algebraic integers)**.** Prove that the degree d of an irreducible complex representation ρ divides $|G|$. You may use basic properties of algebraic integers [53].

1. For a conjugacy class C, show that $\sum_{g \in C} \rho(g) = r_C \operatorname{Id} \in \operatorname{End}(V)$ and compute $r_C = |C|\chi_\rho(C)/d$. This scalar can be zero.
2. Prove $|G|/d = \sum_C r_C \chi_\rho(C^{-1})$.
3. Show that the number of factorizations $x = ab$ with $a \in C, b \in D$ is constant as x ranges over a fixed conjugacy class E. Call it a_{CD}^E.
4. Prove $r_C r_D = \sum_E a_{CD}^E r_E$. Deduce that $\sum_C \mathbb{Z} r_C$ is a ring containing one and finitely generated as an additive abelian group.
5. Conclude that all r_C are algebraic integers. Explain why character values are sums of roots of unity, and deduce $d \mid |G|$. Distinguish finite generation as a module from finite generation merely as an algebra.

The decomposition of the regular representation factors the group determinant into contributions from the irreducible representations.

Exercise 7.9 (The group determinant)**.** For indeterminates $(X_g)_{g \in G}$, define

$$\Theta(G) = \det(X_{gh^{-1}})_{g,h \in G}, \qquad \Theta_\rho(G) = \det\left(\sum_{g \in G} X_g \rho(g) \right).$$

This generalizes the circulant determinant in Exercise 1.1; see [34] for its historical role.

1. Show that $\Theta(G)$ is the determinant associated with the regular representation. Extract matrix entries as coordinates, remembering that the point indicators have squared norm $1/|G|$ in our normalized inner product.
2. Show $\Theta_{U \oplus V} = \Theta_U \Theta_V$.
3. Use the Fourier isomorphism to prove that the algebra map $\rho_i : \mathbb{C}[G] \to \operatorname{End}(V_i)$ is surjective. Deduce that its matrix-entry linear forms in the variables X_g are independent.
4. Prove that the generic $n \times n$ determinant is irreducible over $\mathbb{C}$. In an induction using a row expansion, justify primitivity of the coefficients before applying Gauss's lemma.

5. Complete the independent matrix-entry forms to linear coordinates on $\mathbb{C}^{|G|}$ and deduce that Θ_{ρ_i} is irreducible.
6. Show that its leading terms as a polynomial in X_1 are

$$X_1^{d_i} + X_1^{d_i-1} \sum_{g \neq 1} \chi_i(g) X_g.$$

   Conclude that these factors determine the character and are pairwise nonproportional for nonisomorphic irreducibles.
7. Prove the irreducible factorization $\Theta(G) = \prod_i \Theta_{\rho_i}(G)^{d_i}$.

The affine group of a finite field gives a concrete irreducible action on mean-zero functions.

Exercise 7.10 (The affine group of a finite field)**.** For prime p, write the affine map $x \mapsto ax + b$ as (b, a), with $b \in \mathbb{F}_p$ and $a \in \mathbb{F}_p^*$. Let G_p be this group.

1. Give its product, identity, and inverse, and identify the semidirect product $\mathbb{F}_p \rtimes \mathbb{F}_p^*$.
2. Prove that $\pi(b, a)f(x) = f(a^{-1}(x - b))$ is a unitary representation on $\mathbb{C}[\mathbb{F}_p]$ for the normalized inner product.
3. Prove that the mean-zero subspace

$$E = \{f : \sum_{x \in \mathbb{F}_p} f(x) = 0\} \tag{7.12}$$

   is invariant and irreducible. You may use double transitivity or compute its character explicitly.

Translations and dilations of one vector in that action produce a redundant wavelet transform with an explicit reconstruction formula.

Exercise 7.11 (Wavelets on a finite field)**.** With the preceding action, set $\psi_{b,a} = \pi(b, a)\psi$ and define

$$\mathcal{W}_\psi f(b, a) = \sum_{x \in \mathbb{F}_p} f(x)\overline{\psi_{b,a}(x)}.$$

The transform has $p(p - 1)$ coefficients and generally has a proper range.

1. Express the transform using the ordinary additive DFTs $\hat{f}, \hat{\psi}$, with negative exponential in the forward DFT.
2. If $0 \neq \psi \in E$, prove for every $f \in E$ that

$$f(x) = \frac{1}{c_\psi} \sum_{a \neq 0} \sum_b \mathcal{W}_\psi f(b, a)\psi_{b,a}(x), \qquad c_\psi = p \sum_x |\psi(x)|^2 = p^2 \|\psi\|_2^2.$$

   What does this reconstruction return when f has nonzero mean?
3. Suppose

$$d_\psi := (p - 1)|\hat{\psi}(0)|^2 = \sum_{k \neq 0} |\hat{\psi}(k)|^2 > 0.$$

   Prove the same formula with d_ψ in place of c_ψ for every f. Determine the isometry factor first for unnormalized sums of squares, then for the normalized inner products on both finite groups.
4. Implement forward and inverse transforms in Python. Compare the FFT implementation with direct sums, including complex-valued signals, and reproduce Figure 7.1.

This finite-field construction is developed in [27]. Its algebraic dilations are permutations of residues, rather than a nested family of Euclidean scales.

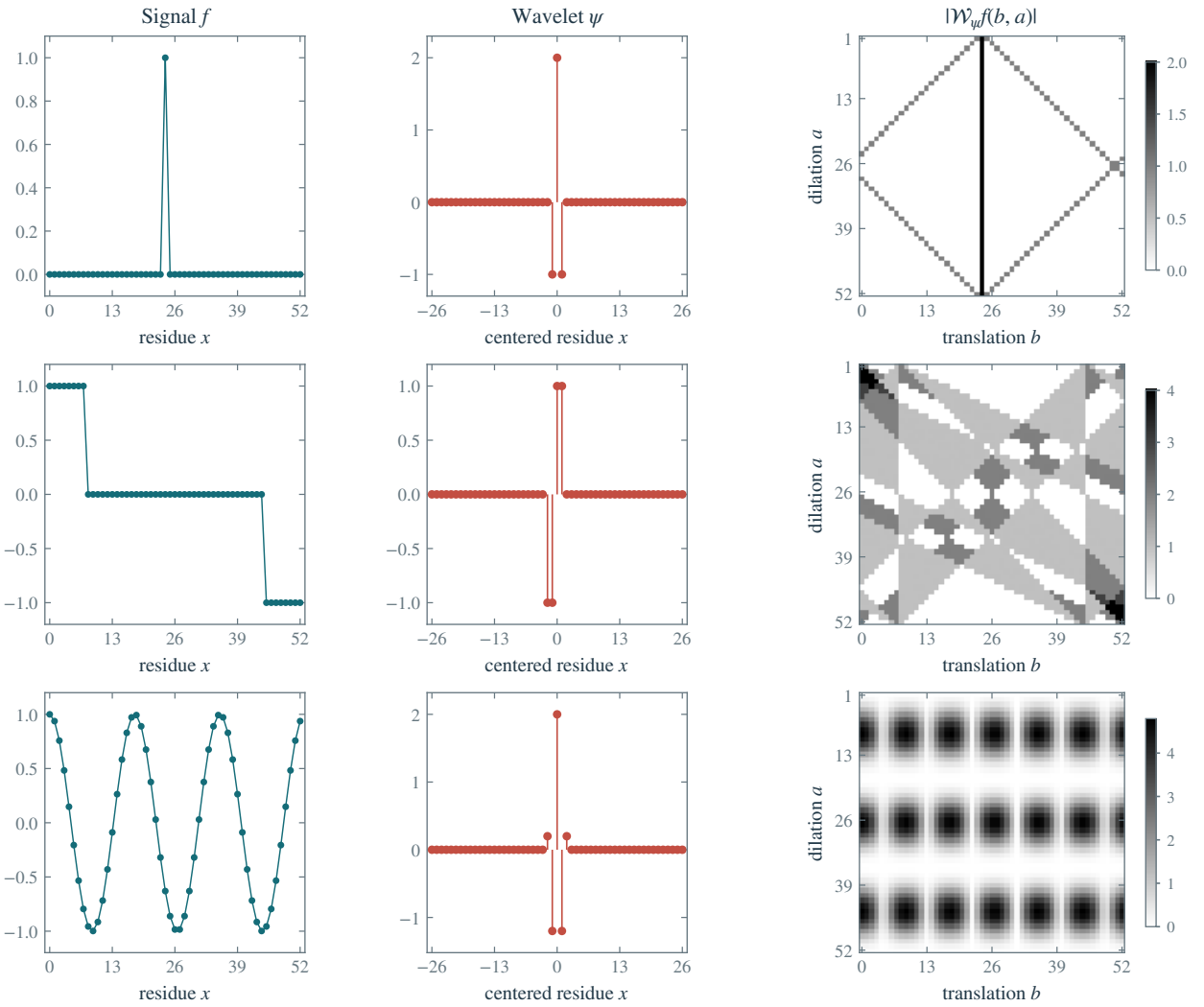


Figure 7.1. Wavelets on $\mathbb{F}_{53}$: the signal, the centered wavelet, and the magnitude of its affine-group transform. Translation b is horizontal and nonzero dilation a is vertical. Each row has its own indicated magnitude scale. The first signal has nonzero mean, which the mean-zero wavelet removes.

7.7 Solutions to the exercises

These solutions use invariant subspaces, averaging, and character identities to analyze representations and their applications. Particular attention is given to the hypotheses behind decomposition, the construction of polynomial invariants, and the normalization of finite-field wavelet reconstruction.

Solution to Exercise 7.1

The line Ke_1 is invariant. It is the only invariant line: if $v = ae_1 + be_2$ with $b \neq 0$, invariance under $\rho(1)$ would force $\rho(1)v = v$, by comparing second coordinates, whereas their difference is $be_1 \neq 0$. Thus no direct sum of two invariant lines is possible. For $K = \mathbb{C}$, the acting additive group is infinite. For finite K of characteristic p, its order is a power of p, so division by $|G|$ in Maschke's averaging argument is impossible. This is a characteristic obstruction, not an algebraic-closure obstruction.

Solution to Exercise 7.2

Set $\varphi = A\delta_1$. Since $\delta_g = \tau_g \delta_1$,

$$Af = \sum_g f(g) A\delta_g = \sum_g f(g)\tau_g \varphi = f * \varphi.$$

Evaluating this identity at $f = \delta_1$ proves uniqueness. Right convolution commutes with the left regular action by associativity. Replacing it by left convolution would require additional commutation conditions on the kernel in a nonabelian group.

Solution to Exercise 7.3

The twisted matrices satisfy the representation law because η is a scalar homomorphism. A subspace is invariant under them exactly when it is invariant under ρ, since $\eta(g) \neq 0$. The twist is therefore irreducible, with character $\eta\chi_\rho$. Equivalently, $|\eta(g)| = 1$ makes the character norm unchanged. For $G = \mathfrak{S}_3$, the standard character has values $(2, 0, -1)$ on the identity, transpositions, and three-cycles. Multiplying by sign leaves it unchanged, so the twist is isomorphic to the same representation. If ρ is trivial, its twist is η itself. Thus nontriviality of η alone does not ensure a new representation.

Solution to Exercise 7.4

Let ρ_i and σ_j be the irreducibles of G and H. The external tensor products

$$(g, h) \longmapsto \rho_i(g) \otimes \sigma_j(h)$$

have characters $\chi_i(g)\eta_j(h)$. The normalized inner product on the product group factors, so these characters are orthonormal. They therefore give pairwise nonisomorphic irreducibles. Finally,

$$\sum_{i,j} (\dim V_i \dim W_j)^2 = \left(\sum_i \dim V_i^2\right)\left(\sum_j \dim W_j^2\right) = |G||H|.$$

The degree-squares identity proves completeness.

Solution to Exercise 7.5

1. Permutation matrices identify the action with permutation of indeterminates. The invariant ring is generated freely by the elementary symmetric polynomials $\sigma_j = \sum_{i_1 < \cdots < i_j} X_{i_1} \cdots X_{i_j}$, $1 \leq j \leq n$.
2. Independent sign changes force both exponents to be even, giving $K[X, Y]^{V_4} = K[X^2, Y^2]$. The two generators are algebraically independent. For C_2, only the total degree must be even, so $K[X, Y]^{C_2} = K[X^2, XY, Y^2]$. These three degree-two elements are needed, but expressions in them are not unique because $(XY)^2 = X^2Y^2$.
3. If P is invariant and $P = \sum_\alpha c_\alpha X^\alpha$, then $P = R_G(P) = \sum_\alpha c_\alpha R_G(X^\alpha)$.
4. Expand each U_A^k by the multinomial theorem and sum over A. This gives the asserted identity for S_k. For N variables, Newton's identities in characteristic zero express every symmetric polynomial in the

first N power sums. Applied to the N quantities (U_A), they give $S_k = F_k(S_1, \dots, S_N)$ with coefficients in K. Compare the coefficient of u^α for $|\alpha| = k$: the left side is $N\binom{k}{\alpha} R_G(X^\alpha)$; the right side is a polynomial in averages of degrees at most N. Division by the nonzero integer $N\binom{k}{\alpha}$ proves the bound. Characteristic zero justifies both divisions in Newton's identities and this final division.

5. Odd-degree averages vanish because $-I \in C_4$. At degree two, $R(X^2) = R(Y^2) = (X^2 + Y^2)/2$ and $R(XY) = 0$. At degree four,

$$\begin{aligned} R(X^4) = R(Y^4) &= (X^4 + Y^4)/2, \\ R(X^3Y) &= (X^3Y - XY^3)/2, \\ R(X^2Y^2) = X^2Y^2, &\qquad R(XY^3) = -R(X^3Y). \end{aligned}$$

Noether's bound therefore gives generators

$$P = X^2 + Y^2, \quad Q = X^2Y^2, \quad R = XY(X^2 - Y^2),$$

since $X^4 + Y^4 = P^2 - 2Q$. They satisfy $R^2 = Q(P^2 - 4Q)$. One degree-two generator and two additional degree-four generators are necessary: the invariant spaces at these degrees have dimensions one and three, respectively, whereas P alone produces only P^2 at degree four.

Solution to Exercise 7.6

1. Linear substitutions preserve every homogeneous degree. Equality of two polynomials is equivalent to equality of their homogeneous components, which proves the claim.
2. The Reynolds operator on V_s projects onto V_s^G, hence its trace is a_s. With the specified inverse matrix and monomial basis,

$$A^{[2]} = \begin{pmatrix} a^2 & ac & c^2 \\ 2ab & ad + bc & 2cd \\ b^2 & bd & d^2 \end{pmatrix}.$$

These entries come from $(aX + bY)^2$, $(aX + bY)(cX + dY)$, and $(cX + dY)^2$. Using entries of A in place of A^{-1} would describe a different substitution convention.

3. A finite-order complex matrix is diagonalizable. If $\mu_1, \dots, \mu_n$ are the eigenvalues of A^{-1}, those on degree s are μ^α for $|\alpha| = s$. Therefore

$$\sum_{s \ge 0} \operatorname{tr}(A^{[s]}) t^s = \prod_{j=1}^{n} (1 - t\mu_j)^{-1} = \det(I - tA^{-1})^{-1}.$$

Average over the group and replace A by A^{-1} to obtain Molien's formula, including its factor $1/|G|$.

4. For the reflection group G_1 of order two,

$$\Phi_1(t) = \frac{1}{2}\left(\frac{1}{(1-t)^2} + \frac{1}{1-t^2}\right) = \frac{1}{(1-t)(1-t^2)}.$$

The group G_2 in the self-dual-code exercise includes the even-weight reflection $\operatorname{diag}(1, -1)$ and has order sixteen. Its eight rotations and eight reflections give

$$\Phi_2(t) = \frac{1}{16}\left(\sum_{j=0}^{7} \frac{1}{1 - 2t\cos(j\pi/4) + t^2} + \frac{8}{1-t^2}\right) = \frac{1}{(1-t^2)(1-t^8)}.$$

After verifying that proposed invariants are algebraically independent, the subalgebra they generate has a known homogeneous dimension in each degree. Equality with the Molien coefficients then proves generation. A denominator resembling that of a polynomial ring is not by itself such a proof; explicit invariants and their independence are required.

Solution to Exercise 7.7

1. The trace of a permutation matrix is its number of fixed points, so $\langle \chi_\pi, 1\rangle = |G|^{-1}\sum_g |X_g|$.
2. The invariant functions are constant on each orbit, and the orbit indicators form a basis. Thus $\dim V^G$ equals the number of orbits; the Reynolds trace gives the stated average. Alternatively,
$$\sum_g |X_g| = \#\{(g,x) : gx = x\} = \sum_x |G_x| = \sum_{\text{orbits } O} |O|\frac{|G|}{|O|}.$$
3. A rotation by $2\pi j/6$ has $\gcd(6,j)$ cycles and fixes $2^{\gcd(6,j)}$ binary colorings. The six rotation counts are $64, 2, 4, 8, 4, 2$. Three reflections through opposite vertices fix $2^4 = 16$ colorings each; three through opposite edge midpoints fix $2^3 = 8$ each. Hence the number of necklaces is
$$\frac{64 + 2\cdot 2 + 2\cdot 4 + 8 + 3\cdot 16 + 3\cdot 8}{12} = 13.$$
The half-turn contribution 8 must be included, and adjacent rotations are separated by $\pi/3$, not $\pi/6$.
4. A matrix commutes with the permutation action precisely when its entries are constant on the orbits of the diagonal action on $X \times X$. Double transitivity gives exactly two orbits, the diagonal and its complement, hence $\dim \mathrm{End}_G(V) = 2$ and $\langle \chi_\pi, \chi_\pi\rangle = 2$. Transitivity gives $\dim V^G = 1$. Write $V = \mathbb{C}\mathbf{1} \oplus W$ with no invariants in W; character orthogonality then gives $\|\chi_W\|_2^2 = 2 - 1 = 1$. Thus W is irreducible. The natural action of $\mathfrak{S}_n$ is doubly transitive for $n \geq 2$.

Solution to Exercise 7.8

1. Conjugating the class sum permutes its terms, so it is an intertwiner. Schur's lemma makes it scalar; taking its trace yields $r_C = |C|\chi_\rho(C)/d$.
2. Character orthogonality gives
$$|G| = \sum_C |C|\chi_\rho(C)\chi_\rho(C^{-1}) = d\sum_C r_C\chi_\rho(C^{-1}).$$
3. If $x' = hxh^{-1}$, simultaneous conjugation $(a,b) \mapsto (hah^{-1}, hbh^{-1})$ bijects the factorizations of x with those of x'. The number is therefore constant on the class.
4. Multiply two class sums and collect products by their conjugacy class. Applying ρ gives $r_C r_D = \sum_E a_{CD}^E r_E$. Their integer span is consequently closed under multiplication and contains $r_{\{1\}} = 1$. It is finitely generated as a $\mathbb{Z}$-module, not just as a ring.
5. This module is torsion-free inside $\mathbb{C}$, hence free of finite rank. Multiplication by r_C has an integer matrix in a $\mathbb{Z}$-basis. Its monic characteristic polynomial annihilates the operator and, upon application to one, annihilates r_C. Thus r_C is an algebraic integer. Each $\rho(g)$ has finite order, so its eigenvalues are roots of unity; its trace is a *sum* of such roots and is an algebraic integer. The formula in part 2 makes $|G|/d$ an algebraic integer. Since it is rational, it is an integer, proving divisibility.

Solution to Exercise 7.9

1. Acting on δ_h, the element $X = \sum_a X_a\delta_a$ gives $\sum_a X_a\delta_{ah}$. Its coordinate at g is $X_{gh^{-1}}$, so this is exactly the regular-action matrix. With the normalized inner product, that coordinate is $|G|\langle X * \delta_h, \delta_g\rangle$, not the inner product alone.
2. Direct sums give block-diagonal matrices, whose determinants multiply.
3. The map ρ_i is the composition of the Fourier isomorphism with projection onto its ith full matrix block, hence is surjective. A linear relation among its entry forms would then vanish on every matrix unit, forcing all coefficients to be zero.
4. Induct on n, the case $n = 1$ being immediate. Expand along the last row, regarding its entries as indeterminates over the polynomial ring R in the first $n-1$ rows. Each cofactor is a generic determinant of size $n-1$, irreducible by induction, and two cofactors obtained by deleting different columns are

nonassociate. Their greatest common divisor in R is therefore one. The row expansion is a primitive polynomial over this UFD. Over the fraction field of R, it is a nonconstant linear polynomial and hence irreducible. Gauss's lemma concludes. Primitivity is essential: a polynomial linear in a variable may still have a nonconstant common factor in its coefficients.

5. Complete the d_i^2 independent entry forms to a basis of all linear forms in (X_g). In the resulting invertible linear coordinates, Θ_{ρ_i} is the generic determinant in the first d_i^2 variables, independent of the remaining variables. Irreducibility is preserved by adjoining these variables and by the invertible substitution.
6. Since $\rho_i(1) = I$, write $\rho_i(X) = X_1 I + B$, with $B = \sum_{g \neq 1} X_g \rho_i(g)$. The coefficient of $X_1^{d_i - 1}$ in $\det(X_1 I + B)$ is $\operatorname{tr} B = \sum_{g \neq 1} \chi_i(g) X_g$. The degree gives $d_i = \chi_i(1)$, so the polynomial determines the entire character, hence the isomorphism class. Proportional such factors must have the same degree and, because they are monic in X_1, must be equal. Therefore distinct irreducible classes give nonproportional factors.
7. The regular representation contains V_i with multiplicity d_i. Apply the direct-sum determinant formula and the preceding irreducibility and distinctness results.

Solution to Exercise 7.10

1. Composition gives

$$(b, a)(d, c) = (b + ad, ac), \quad 1 = (0, 1), \quad (b, a)^{-1} = (-a^{-1}b, a^{-1}).$$

This is the semidirect product with $\mathbb{F}_p^*$ acting on the additive group by multiplication. Its order is $p(p-1)$.

2. The action on points is $x \mapsto ax + b$, and the action on functions is its inverse pullback. Thus it respects composition and permutes coordinates, preserving the normalized Hermitian product.
3. The mean is unchanged by any coordinate permutation, so E is invariant. Write $e_k(x) = \omega_p^{kx}$, with $\omega_p = e^{2\pi \mathrm{i}/p}$. Then

$$\pi(b, a) e_k = \omega_p^{-ka^{-1}b} e_{ka^{-1}}.$$

The nonzero k form a basis of E. If $a \neq 1$, no such frequency is fixed, so the character is zero. For $a = 1$, its value is $p - 1$ when $b = 0$ and -1 when $b \neq 0$. Hence its squared character norm is

$$\frac{(p-1)^2 + (p-1)}{p(p-1)} = 1,$$

proving irreducibility. Alternatively, an affine map is uniquely determined by its values at two distinct points, so the action is doubly transitive and the preceding counting exercise applies.

Solution to Exercise 7.11

1. A change of variable gives $\hat{\psi}_{b,a}(k) = \omega_p^{-bk} \hat{\psi}(ak)$. Plancherel then yields

$$\mathcal{W}_\psi f(b, a) = \frac{1}{p} \sum_{k \in \mathbb{F}_p} \hat{f}(k) \overline{\hat{\psi}(ak)} \omega_p^{bk}.$$

For each fixed dilation, this is an inverse ordinary DFT.

2. Define the unnormalized synthesis $Sy = \sum_{a \neq 0, b} y(b, a) \psi_{b,a}$. Taking its Fourier transform and summing over translations gives

$$\widehat{S\mathcal{W}_\psi f}(k) = \hat{f}(k) \sum_{a \neq 0} |\hat{\psi}(ak)|^2.$$

For $k \neq 0$, multiplication by k permutes $\mathbb{F}_p^*$, so the last sum is $\sum_{u \neq 0} |\hat{\psi}(u)|^2 = c_\psi$ when ψ has mean zero. Parseval gives $c_\psi = p \sum_x |\psi(x)|^2 > 0$. At $k = 0$ the result vanishes. Thus $S\mathcal{W}_\psi f = c_\psi f$ on E, and for arbitrary f the reconstruction is $f - p^{-1} \sum_x f(x)$.

3. For a general wavelet, the synthesis-analysis multiplier equals $(p-1)|\hat{\psi}(0)|^2$ at zero and $\sum_{u \neq 0} |\hat{\psi}(u)|^2$ elsewhere. The assumed equality makes it the same positive value d_ψ at all frequencies. Therefore

$S\mathcal{W}_\psi = d_\psi \,\mathrm{Id}$. For unnormalized Euclidean sums, S is the adjoint of $\mathcal{W}_\psi$, so $\sum_{b,a} |\mathcal{W}_\psi f(b,a)|^2 = d_\psi \sum_x |f(x)|^2$. For our normalized group inner products, the adjoint is instead $S/(p-1)$ and

$$\|\mathcal{W}_\psi f\|^2_{\mathbb{C}[G_p]} = \frac{d_\psi}{p-1}\|f\|^2_{\mathbb{C}[\mathbb{F}_p]}.$$

Accordingly the isometry factors multiplying $\mathcal{W}_\psi$ are $1/\sqrt{d_\psi}$ and $\sqrt{(p-1)/d_\psi}$, respectively. Inversion is a left inverse, or an inverse restricted to the transform's range.

4. Program 58 uses one FFT convolution per dilation, for $O(p^2 \log p)$ operations and $O(p^2)$ output storage. The literal triple sum would cost $O(p^3)$. Program 59 performs synthesis in frequency coordinates and divides by the frame constant. Its default mode recovers the mean-zero component; the full-space mode checks the tight-frame condition. Direct complex-valued inner products and independent reconstruction tests verify the conjugations and scaling.

```
def finite_wavelet(f, psi):
    f, psi = np.asarray(f, dtype=complex), np.asarray(psi, dtype=complex)
    p = psi.size
    if psi.ndim != 1 or f.shape != (p,) or not sp.isprime(p):
        raise ValueError('Use two vectors of the same prime length.')
    F, P = np.fft.fft(f), np.fft.fft(psi)
    k = np.arange(p)
    return np.array([np.fft.ifft(F*np.conj(P[a*k % p])) for a in range(1, p)])
```

Program 58. Affine-group wavelet analysis by ordinary FFTs.

```
def inverse_finite_wavelet(coefficients, psi, mode='mean-zero'):
    psi = np.asarray(psi, dtype=complex)
    p = psi.size
    coefficients = np.asarray(coefficients, dtype=complex)
    if psi.ndim != 1 or not sp.isprime(p) or coefficients.shape != (p-1, p):
        raise ValueError('Use p-1 dilation rows and p translation columns.')
    P, k = np.fft.fft(psi), np.arange(p)
    c = np.sum(abs(P[1:])**2)
    if c <= 0:
        raise ValueError('The wavelet needs a nonzero nonconstant component.')
    if mode == 'mean-zero':
        if abs(P[0]) > 1e-10*np.linalg.norm(P):
            raise ValueError('The mean-zero mode requires a mean-zero wavelet.')
    elif mode == 'tight':
        if not np.isclose((p-1)*abs(P[0])**2, c, rtol=1e-10, atol=0):
            raise ValueError('The full-space mode requires the stated tight-frame condition.')
    else:
        raise ValueError('Choose mean-zero or tight.')
    spectrum = np.zeros(p, dtype=complex)
    for a in range(1, p):
        spectrum += np.fft.fft(coefficients[a-1])*P[a*k % p]
    if mode == 'mean-zero':
        spectrum[0] = 0  # The analysis contains no information about the signal mean.
    return np.fft.ifft(spectrum/c)
```

Program 59. Wavelet reconstruction with explicit mean-zero and full-space conventions.

CHAPTER 8

Applications of linear representations

Character tables turn geometric group actions into concrete algebraic information. They reveal representation multiplicities and normal subgroups, while matrix coefficients provide coordinates for data indexed by a noncommutative group. This chapter develops these applications through cyclic, dihedral, symmetric, and quaternion groups, and a ranking-data example.

8.1 Representations of familiar groups

Familiar symmetry groups make the abstract character identities concrete. We learn to read and check a character table, then construct the tables of cyclic and dihedral groups and the symmetric group on four letters.

8.1.1 Reading a character table

A character table organizes the irreducible representations against the conjugacy classes. Its orthogonality relations allow unknown entries to be constrained and completed, provided the class-size weights are handled correctly.

Let $C_1, \dots, C_r$ be the conjugacy classes, with $C_1 = \{1\}$ and $k_j = |C_j|$. The character table is the square matrix $\Phi = (\chi_i(C_j))_{i,j}$, with the trivial character in the first row. Its first column gives the irreducible degrees $d_i = \chi_i(1)$. Row orthogonality is weighted by class size:

$$\sum_j k_j \chi_i(C_j)\overline{\chi_l(C_j)} = |G|\delta_{il}.$$

After including these column weights, row orthogonality also yields orthogonality between conjugacy classes.

Proposition 67 (Column orthogonality)**.** *For conjugacy classes C, D,*

$$\sum_{i=1}^{r} \chi_i(C)\overline{\chi_i(D)} = \begin{cases} |G|/|C| & C = D, \\ 0 & C \neq D. \end{cases}$$

Proof. The indicator of C has coefficient $c_{f_C}(\rho_i) = |C|\overline{\chi_i(C)}/|G|$. Apply Plancherel to the indicators of C, D, whose inner product is $|C|\delta_{CD}/|G|$, and simplify. Equivalently, use the unitary matrix $|G|^{-1/2}\Phi \operatorname{diag}(\sqrt{k_j})$. □

Complex conjugation is essential: character values need not be real.

8.1.2 Cyclic groups

For a cyclic group, the character table reduces to the ordinary Fourier matrix. This example connects the representation-theoretic notation with the scalar transform developed earlier.

For $G = \langle g_0 \rangle$ of order n, every irreducible is one-dimensional. With $\omega_n = e^{2\pi \mathrm{i}/n}$, all characters are

$$\chi_j(g_0^k) = \omega_n^{jk}, \qquad 0 \le j, k < n.$$

Thus the table is the Vandermonde matrix (ω_n^{jk}), and its normalization by $1/\sqrt{n}$ is unitary. Choosing g_0 identifies the dual with $\mathbb{Z}/n\mathbb{Z}$; this recovers the ordinary discrete Fourier theory.

8.1.3 Dihedral groups

Adding reflections to cyclic rotations produces two-dimensional irreducible representations. We construct them geometrically and use the dimension count to check completeness, distinguishing odd and even polygon sizes.

For $n \geq 3$, the dihedral group D_n has order $2n$ and presentation

$$D_n = \langle r, s : r^n = s^2 = 1,\ srs = r^{-1} \rangle.$$

Here r is a rotation through $2\pi/n$ and s is a reflection of a regular n-gon. Every element has a unique form $s^\epsilon r^k$, with $\epsilon \in \{0, 1\}$ and $0 \leq k < n$.

A one-dimensional representation must send both r and s to signs, with the additional condition $\rho(r)^n = 1$. For even n this gives four characters, and for odd n only the first two:

	r^k	sr^k
ψ_1	1	1
ψ_2	1	-1
ψ_3	$(-1)^k$	$(-1)^k$
ψ_4	$(-1)^k$	$(-1)^{k+1}$

.

These columns describe two families of elements, not individual conjugacy classes. The last two rows are used only for even n. For the two-dimensional representations, set

$$\rho_h(r) = \begin{pmatrix} \omega_n^h & 0 \\ 0 & \omega_n^{-h} \end{pmatrix}, \qquad \rho_h(s) = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

They satisfy the defining relations, and in particular

$$\rho_h(sr^k) = \begin{pmatrix} 0 & \omega_n^{-hk} \\ \omega_n^{hk} & 0 \end{pmatrix}.$$

Swapping coordinates intertwines ρ_h with ρ_{n-h}. At $h = 0$ the representation splits as $\psi_1 \oplus \psi_2$; for even n, the case $h = n/2$ splits as $\psi_3 \oplus \psi_4$. For

$$1 \leq h \leq \lfloor (n-1)/2 \rfloor,$$

the eigenvalues of $\rho_h(r)$ are distinct. Its only invariant lines are the coordinate axes, which $\rho_h(s)$ exchanges. These representations are therefore irreducible, with characters

$$\chi_h(r^k) = 2\cos(2\pi hk/n), \qquad \chi_h(sr^k) = 0.$$

The listed values of h give distinct characters, since the frequencies $\{h, -h\}$ are distinct modulo n. Finally, the degree-squares sums are $4 + 4(n/2 - 1) = 2n$ for even n and $2 + 4(n-1)/2 = 2n$ for odd n. The lists are complete.

8.1.4 The symmetric group on four letters

The permutation action on four letters, its sign twist, and a quotient action provide the irreducibles of $\mathfrak{S}_4$. Constructing their table also prepares a geometric interpretation through the rotations of a cube.

Conjugacy is determined by cycle type. In the order used below, the five classes have sizes $1, 6, 8, 6, 3$: identity, transpositions, three-cycles, four-cycles, and double transpositions. Let V_s be the standard three-dimensional representation and ϵ the sign. Since a permutation character counts fixed letters, subtracting the trivial character gives $\chi_s = (3, 1, 0, -1, -1)$. Its sign twist, denoted W, has character $(3, -1, 0, 1, -1)$ and is inequivalent to V_s. Together with the trivial and sign representations, these account for $1 + 1 + 9 + 9 = 20$ of the degree-squares sum 24. There are five irreducibles, so the remaining one, W', has degree two. The regular-character identity determines its row:

class size	1	6	8	6	3
	1	(12)	(123)	(1234)	(12)(34)
χ_1	1	1	1	1	1
χ_ϵ	1	−1	1	−1	1
χ_s	3	1	0	−1	−1
χ_W	3	−1	0	1	−1
$\chi_{W'}$	2	0	−1	0	2

.

Each row has weighted squared norm one, and the rows are mutually orthogonal. The representation W' is the pullback of the standard representation of $\mathfrak{S}_3$ under a quotient map, as the exercises show.

The cube. Cube rotations give a spatial realization of $\mathfrak{S}_4$, allowing its characters to be interpreted through rotation angles and permutations of diagonals.

The orientation-preserving symmetries of a cube act on its four body diagonals. This action identifies the group with $\mathfrak{S}_4$. There are 24 such rotations: an oriented orthonormal coordinate frame can be sent to any signed coordinate frame of determinant one. The diagonal action is faithful; a rotation fixing all four diagonal lines can only be the identity. For example, this follows from the fact that three independent such directions would be eigenvectors, while their pairwise nonorthogonality forces the corresponding real eigenvalues to agree; $-I$ reverses orientation.

A half-turn about an axis through opposite edge midpoints has cycle type (12) on the diagonals. A third-turn about a body diagonal has type (123); quarter-turns and half-turns about face-center axes have types (1234) and (12)(34). On the six faces, these fix respectively $0, 0, 2, 2$ faces, giving

$$\chi_{\text{faces}} = (6, 0, 0, 2, 2) = \chi_1 + \chi_W + \chi_{W'}.$$

The three-dimensional spatial rotation representation itself is W: its trace is $1 + 2\cos\theta$, yielding $(3, -1, 0, 1, -1)$. The actions on vertices and edges are treated in Exercise 8.4.

8.2 Normal subgroups from character values

Character values can reveal group structure as well as representation multiplicities. We recover representation kernels from their traces, then show that intersections of irreducible kernels determine every normal subgroup.

8.2.1 The kernel of a character

A representation identifies elements that lie in the same coset of its kernel. For a finite group over the complex numbers, the character detects this kernel by attaining the degree itself, a stronger condition than merely attaining its maximal modulus.

Proposition 68. *Let ρ be a complex representation of degree d. If g has order k, then $\rho(g)$ is diagonalizable with kth roots of unity as eigenvalues. Consequently,*

$$|\chi_\rho(g)| \le d, \qquad K_{\chi_\rho} := \{g : \chi_\rho(g) = d\} = \ker \rho.$$

In particular, K_{χ_ρ} is a normal subgroup.

Proof. The minimal polynomial divides $X^k - 1$, which has distinct complex roots. The trace is therefore a sum of d numbers of modulus one. Its modulus is at most d. If the trace equals the real number d, equality in the triangle inequality forces all eigenvalues to equal one, hence $\rho(g) = I$. The converse is immediate. □

The weaker equality $|\chi_\rho(g)| = d$ only says that $\rho(g)$ is scalar; it need not be the identity.

A representation that is trivial on a normal subgroup can be understood entirely as an action of the quotient.

Lemma 15 (Representations of a quotient)**.** *For $N \triangleleft G$, composition with $\pi : G \to G/N$ identifies representations of G/N with representations of G that are trivial on N. Invariant subspaces and irreducibility are preserved.*

Proof. A representation of the quotient pulls back as $\widetilde{\rho}(g) = \rho(\pi(g))$. Conversely, if $N \subset \ker \widetilde{\rho}$, its value depends only on the coset, defining a representation of G/N. Surjectivity of π makes the invariant-subspace conditions identical. □

8.2.2 Recovering normal subgroups

A single irreducible kernel need not give every normal subgroup. Passing to quotients and decomposing their regular representations shows how intersections recover the full collection and yields a criterion for simplicity.

Proposition 69. *Every normal subgroup is an intersection of kernels of irreducible characters, and every such intersection is normal. The empty intersection is understood to be G.*

Proof. For $N \triangleleft G$, pull back the regular representation of G/N. Since that representation is faithful, its pullback has kernel exactly N. Decompose it into irreducibles. A direct-sum matrix is the identity exactly when each of its nonzero summand blocks is the identity, so its kernel is the intersection of the kernels of the irreducibles that occur. The converse follows because kernels and their intersections are normal. □

A nontrivial group is therefore simple precisely when no nontrivial irreducible representation has a nontrivial kernel.

Corollary 10 (A simplicity criterion)**.** *A nontrivial finite group is simple exactly when each nontrivial irreducible representation is faithful. Equivalently,*

$$\chi_i(g) \neq \chi_i(1) \quad \text{for every nontrivial irreducible } i \text{ and every } g \neq 1.$$

Proof. In a simple group, a nontrivial irreducible has a proper normal kernel, which must be the identity subgroup. Conversely, a proper nontrivial normal subgroup is an intersection of irreducible kernels and cannot arise if every nontrivial irreducible kernel is trivial. □

For $\mathfrak{S}_4$, the nontrivial proper normal subgroups are

$$V_4 = \{1, (12)(34), (13)(24), (14)(23)\} \subset \mathfrak{A}_4.$$

They are $\ker \chi_{W'}$ and $\ker \chi_\epsilon$, respectively. The subgroup generated by the *single* double transposition (12)(34) has only two elements and is not V_4; its normal closure is V_4.

8.3 Spectral analysis on a noncommutative group

Data indexed by group elements need not be constant on conjugacy classes, so traces alone do not provide enough coordinates. We establish orthogonality for individual matrix entries and use the resulting Fourier basis to analyze ranking data.

8.3.1 Matrix-entry orthogonality

Choosing orthonormal bases in the irreducible spaces turns their matrix entries into functions on the group. Schur orthogonality determines the scale at which these functions form a complete orthonormal basis.

Choose an invariant Hermitian inner product and an orthonormal basis in each irreducible space V_k, of dimension d_k. Write the unitary representation matrices as $R_k(g) = (r^k_{ab}(g))$. Each entry is a complex-valued function on G.

The following identity gives both pairwise orthogonality and the dimension factor needed to normalize the entries.

Theorem 17 (Schur orthogonality for matrix entries)**.** *For valid indices in the respective representations,*

$$\langle r^k_{ab},\, r^l_{cd} \rangle = \frac{\delta_{kl}\delta_{ac}\delta_{bd}}{d_k}.$$

Consequently $(\sqrt{d_k}\, r^k_{ab})_{k,a,b}$ *is an orthonormal basis of* $\mathbb{C}[G]$ *for the normalized inner product.*

Proof. Average a linear map $T : V_l \to V_k$:

$$\widetilde{T} = \frac{1}{|G|} \sum_{g \in G} R_k(g) T R_l(g)^*. \tag{8.1}$$

It is zero for $k \neq l$. For $k = l$, Schur's lemma makes it scalar, and preservation of trace gives $\widetilde{T} = (\operatorname{tr} T/d_k)I$. Take $T = E_{bd}$, the matrix with its only nonzero entry in row b, column d, and compare entry (a, c). The left side is exactly the inner product in the assertion. This proves orthogonality and the norm. There are $\sum_k d_k^2 = |G|$ entries, proving completeness. □

Characters are sums of diagonal entries, so their orthogonality is consistent with this stronger result. The character is independent of basis; individual entries depend on the chosen orthonormal basis.

8.3.2 Fourier coordinates and matrix blocks

The orthonormal matrix-entry basis gives scalar coefficients with a direct energy interpretation. We relate them explicitly to the matrix-valued transform, keeping track of conjugation and normalization before using them on data.

Set $\Delta_{kab} = \sqrt{d_k}\, r^k_{ab}$. For any $f \in \mathbb{C}[G]$,

$$c_f(k,a,b) = \langle f,\, \Delta_{kab} \rangle, \qquad f = \sum_{k,a,b} c_f(k,a,b)\Delta_{kab}, \qquad \|f\|_2^2 = \sum_{k,a,b} |c_f(k,a,b)|^2.$$

The matrix-valued transform of the preceding chapter uses $R_k(g)$ without conjugation. Its exact relation to these coordinates is

$$(\mathcal{F}f(\rho_k))_{ab} = \sum_{g\in G} f(g) r^k_{ab}(g) = \frac{|G|}{\sqrt{d_k}} \overline{c_{\overline{f}}(k,a,b)}. \tag{8.2}$$

Both the conjugation and the factor $|G|$ are needed. Equivalently, conjugated matrix entries provide a Fourier basis directly matching this transform convention. Plancherel becomes

$$\sum_{g\in G} |f(g)|^2 = \frac{1}{|G|} \sum_k d_k \|\mathcal{F}f(\rho_k)\|_{\mathrm{F}}^2,$$

where $\|M\|_{\mathrm{F}}^2 = \sum_{a,b} |M_{ab}|^2$. A unitary basis change within an irreducible representation conjugates its Fourier block and preserves this block's energy, although individual entries and their interpretation change.

A direct evaluation costs $O(|G|^2)$ operations because there are $|G|$ matrix entries, each defined by a sum of $|G|$ terms. Structured algorithms exploit subgroup chains and compatible representation bases; [43] surveys such methods. Orthogonality guarantees stable analysis and synthesis, but does not by itself make a signal sparse. For data analysis, the group structure and basis should reflect meaningful symmetries of the data.

8.3.3 An example with rankings

A ranking is a permutation, so a distribution of rankings is a function on a symmetric group. We analyze a three-item example to show how a chosen Fourier coefficient can measure a specific preference contrast.

Following the statistical viewpoint of [22], consider rankings of city, suburb, and countryside. We make the encoding explicit: $\sigma(j)$ is the *rank assigned to item* j, where items $1, 2, 3$ are city, suburb, and countryside, and rank one is best. Thus $(2, 3, 1)$ means countryside first, city second, suburb third. The data table retained from the earlier edition is

city rank	suburb rank	countryside rank	count
1	2	3	242
2	1	3	170
3	2	1	359
1	3	2	28
2	3	1	12
3	1	2	628

.

Its total is 1439. The most frequent row, $(3, 1, 2)$, ranks suburb first and city last. Regard the counts as a function $f : \mathfrak{S}_3 \to \mathbb{N}$. Here triples are one-line permutation notation, not cycle notation.

Besides the trivial and sign representations, use the standard representation on the sum-zero plane in $\mathbb{C}^3$. In the orthonormal basis

$$u = (e_1 - e_2)/\sqrt{2}, \qquad v = (e_1 + e_2 - 2e_3)/\sqrt{6},$$

let $R(\sigma) = B^{\mathsf{T}} P_\sigma B$, where $B = (u, v)$ and $P_\sigma e_j = e_{\sigma(j)}$. In table order, the matrices are

$(1,2,3)$	$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$	$(2,1,3)$	$\begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}$
$(3,2,1)$	$\frac{1}{2}\begin{pmatrix} 1 & -\sqrt{3} \\ -\sqrt{3} & -1 \end{pmatrix}$	$(1,3,2)$	$\frac{1}{2}\begin{pmatrix} 1 & \sqrt{3} \\ \sqrt{3} & -1 \end{pmatrix}$
$(2,3,1)$	$\frac{1}{2}\begin{pmatrix} -1 & -\sqrt{3} \\ \sqrt{3} & -1 \end{pmatrix}$	$(3,1,2)$	$\frac{1}{2}\begin{pmatrix} -1 & \sqrt{3} \\ -\sqrt{3} & -1 \end{pmatrix}$

.

They satisfy the usual permutation composition law. Using r_{ab} for their entry functions, the exact orthogonal expansion is

$$f = \frac{1}{6}\left(1439\,1 + 325\epsilon - 109r_{11} + 285\sqrt{3}\,r_{12} - 947\sqrt{3}\,r_{21} - 203r_{22}\right).$$

For each two-dimensional entry, its squared norm is $1/2$, which accounts for the factor two relative to an ordinary normalized inner product. The largest nonconstant coordinate in the normalized basis is the one associated with

$$r_{21} = \frac{\sqrt{3}}{2}(0, 0, -1, 1, 1, -1).$$

This function groups rankings according to the item ranked last: it is negative for city, positive for suburb, and zero for countryside. Those three totals are 987, 40, and 412, respectively. Its coefficient therefore measures the strong imbalance between placing city and suburb last. This interpretation depends on the explicitly chosen rank encoding and basis; it is not a general property of every matrix coefficient.

8.4 Exercises

These exercises use character tables to decompose geometric actions, detect kernels, and construct alternative Fourier bases. The first derives column orthogonality from the weighted unitary form of a character table.

Exercise 8.1 (Orthogonality of a character table)**.** Let $\Phi = (\chi_i(C_j))$ and $K = \operatorname{diag}(|C_1|, \dots, |C_r|)$. Show that $\Phi K \Phi^* = |G| I$, and hence that $U = |G|^{-1/2}\Phi K^{1/2}$ is unitary. Deduce column orthogonality. The class-size matrix multiplies on the right because classes index columns.

The dihedral group can also act by rotations in three dimensions, giving a geometric representation to decompose by its character.

Exercise 8.2 (A spatial representation of the dihedral group)**.** For $n \geq 3$, realize D_n by rotations about the z-axis through angles $2\pi k/n$ and half-turns about lines in the xy-plane making angles $k\pi/n$ with the x-axis. Show that these transformations form a representation in $SO_3(\mathbb{R})$. Compute its character and decompose its complexification into irreducibles. A half-turn about a line is a three-dimensional rotation, whose restriction to the xy-plane is a reflection.

The symmetries of a labeled triangle provide explicit matrices and the full character table of $\mathfrak{S}_3$.

Exercise 8.3 (Representations of the symmetric group on three letters)**.** Label the triangle vertices $z_1 = 1$, $z_2 = e^{2\pi \mathrm{i}/3}$, and $z_3 = e^{-2\pi \mathrm{i}/3}$. In the real basis $(1, \mathrm{i})$ of its plane, compute the matrices of (12) and (123); also identify the reflection (23). Deduce the full character table of $\mathfrak{S}_3$. Here parentheses denote cycles, unlike the one-line triples in the ranking example.

Counting fixed vertices and edges of a cube gives two permutation characters whose irreducible components can be read from the same table.

Exercise 8.4 (Actions on cube vertices and edges)**.** The cube rotation group acts on its eight vertices and twelve edges. Compute both permutation characters in the class order of Section 8.1.4, and use the character table to decompose the representations. Edges are considered as unordered vertex pairs.

The two-dimensional irreducible of $\mathfrak{S}_4$ becomes transparent when viewed as the pullback of a quotient representation.

Exercise 8.5 (The two-dimensional representation of the symmetric group)**.** Let W' be the irreducible representation of $\mathfrak{S}_4$ with character $(2, 0, -1, 0, 2)$ in the established class order.

1. Show that every double transposition acts as the identity.
2. Recall why representations trivial on a normal subgroup H are precisely pullbacks of representations of G/H.
3. Let $H = \{1, (12)(34), (13)(24), (14)(23)\}$. Use the action on the three partitions of four letters into two unordered pairs to show $\mathfrak{S}_4/H \simeq \mathfrak{S}_3$.
4. Deduce that W' is the pullback of the standard representation of $\mathfrak{S}_3$.

Faithfulness, degree divisibility, and the center together obstruct two-dimensional irreducibles for nonabelian simple groups.

Exercise 8.6 (Two-dimensional representations and simple groups)**.** Let G be a finite nonabelian simple group. Prove that it has no irreducible complex representation of degree two.

1. Prove Cauchy's theorem: if prime p divides $|G|$, some element has order p. Use cyclic rotation on
$$X = \{(g_1, \dots, g_p) \in G^p : g_1 \cdots g_p = 1\}.$$
Check that this subset is rotation-invariant, even for a nonabelian group.
2. If a degree-two irreducible ρ existed, use divisibility of irreducible degrees to find an involution $t \in G$.
3. Prove that ρ is faithful and takes values in $SL_2(\mathbb{C})$. Show that $\rho(t) = -I$, and derive a contradiction from the center of G.

The quaternion group supplies a noncommutative Fourier basis on eight points, giving an alternative to the Walsh basis.

Exercise 8.7 (Quaternions and a Fourier basis on eight points)**.** The real quaternion algebra $\mathbb{H}$ is the four-dimensional real vector space with basis $(1, i, j, k)$ and bilinear multiplication satisfying
$$i^2 = j^2 = k^2 = -1, \qquad ij = k = -ji, \quad jk = i = -kj, \quad ki = j = -ik.$$
The group $H_8 = \{\pm 1, \pm i, \pm j, \pm k\}$ consists of eight units. The group algebra $\mathbb{R}[H_8]$ has dimension eight and must be distinguished from $\mathbb{H}$.

1. For $q = a + bi + cj + dk$, verify the multiplication table through the faithful real matrix model
$$L(q) = \begin{pmatrix} a & -b & -c & -d \\ b & a & -d & c \\ c & d & a & -b \\ d & -c & b & a \end{pmatrix}.$$
Deduce associativity and a real representation of H_8. Describe the surjective algebra map $\mathbb{R}[H_8] \to \mathbb{H}$ and its kernel.
2. Verify the real-algebra embedding
$$M(q) = \begin{pmatrix} a + \mathrm{i}b & -c - \mathrm{i}d \\ c - \mathrm{i}d & a - \mathrm{i}b \end{pmatrix}.$$
Compute $M(q)^* M(q)$ and deduce a unitary complex representation of H_8 of dimension two. Prove its irreducibility.
3. Find the four one-dimensional representations and prove that these five irreducibles form a complete list. Give an explicit orthonormal basis of $\mathbb{C}[H_8]$ in the order $(1, -1, i, -i, j, -j, k, -k)$.
4. Transport the group law to $\{0,1\}^3$ by a fixed bijection, then use the product group H_8^n to construct a Fourier basis on $\{0,1\}^{3n}$. Specify the dimension 8^n and the conjugations and normalization needed to compute coefficients by tensor products.

This gives an alternative to the abelian Walsh basis. Connections with learning Boolean functions are discussed in [8].

Geometric symmetries of the cube suggest explicit polynomial invariants whose behavior can be checked by signed coordinate permutations.

Exercise 8.8 (Polynomial invariants of cube rotations)**.** Let G be the orientation-preserving symmetry group of the cube with vertices $(\pm 1, \pm 1, \pm 1)$, acting on $\mathbb{R}[X, Y, Z]$.

1. Prove that $X^2 + Y^2 + Z^2$ is invariant.
2. Show that a rotation is a signed coordinate permutation and sends XYZ to either XYZ or $-XYZ$. Deduce invariance of $(XYZ)^2$.

3. Define
$$F = (X+Y+Z)(X+Y-Z)(X-Y+Z)(-X+Y+Z),$$
$$Q = (X^2-Y^2)(X^2-Z^2)(Y^2-Z^2).$$
Show that F^2 and Q^2 are invariant. In fact, show that F itself and the degree-nine polynomial $XYZ\,Q$ are invariant. Interpret the linear factors through planes associated with the cube's diagonals.

The weight enumerators of self-dual binary codes obey polynomial symmetries, linking the MacWilliams identity with invariant theory.

Exercise 8.9 (Self-dual binary codes and invariant polynomials)**.** Let $C \subset \mathbb{F}_2^n$ be a self-dual code of dimension k. Its integer weight enumerator is viewed as a polynomial in $\mathbb{C}[X, Y]$.

1. Prove $n = 2k$.
2. Use homogeneity and the MacWilliams identity to prove invariance under
$$A = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$
3. For $G_1 = \langle A \rangle$, show that
$$\mathbb{C}[X, Y]^{G_1} = \mathbb{C}[X + (\sqrt{2}-1)Y,\ Y(X-Y)].$$
Explain why these two generators are algebraically independent.
4. Self-orthogonality makes all codeword weights even. Deduce invariance under $D = \mathrm{diag}(1, -1)$ and identify $G_2 = \langle A, D \rangle$ as a dihedral group of order sixteen. The transformation $-I$ alone only encodes even total degree, a weaker condition than even word weights.
5. Use exact Python Reynolds averages to compute a generating set. Then show that the two algebraically independent invariants
$$P = X^2 + Y^2, \qquad Q = X^2Y^2(X^2-Y^2)^2$$
generate $\mathbb{C}[X, Y]^{G_2}$. You may use the complex coordinates $z = X + \mathrm{i}Y$, $w = X - \mathrm{i}Y$, or the Molien series after verifying independence. Deduce the form of a homogeneous self-dual weight enumerator, and recover the extended Hamming enumerator as $P^4 - 4Q$.

For larger groups, Reynolds averaging up to Noether's bound can be expensive; Gröbner-basis methods provide additional tools [19].

8.5 Solutions to the exercises

The solutions translate geometric group actions into character values and then into decomposition or kernel information. The final examples connect quaternionic Fourier coordinates, cube invariants, and the polynomial symmetries of self-dual codes.

Solution to Exercise 8.1

Row orthogonality says $\Phi K \Phi^* = |G| I$. The square matrix $U = \Phi K^{1/2}/\sqrt{|G|}$ therefore satisfies $UU^* = I$ and is unitary. The identity $U^*U = I$ yields

$$\Phi^*\Phi = |G|K^{-1}, \qquad \sum_i \overline{\chi_i(C_j)}\chi_i(C_l) = \frac{|G|}{|C_j|}\delta_{jl}.$$

The conjugation cannot be omitted: characters of a cyclic group of order three already provide nonreal entries.

Solution to Exercise 8.2

Let r rotate the xy-plane through $2\pi/n$ and fix the z-axis, and let $s = \operatorname{diag}(1, -1, -1)$. These matrices are proper rotations satisfying $r^n = s^2 = I$ and $srs = r^{-1}$. They give the specified $2n$ rotations. Their characters are

$$\chi(r^k) = 1 + 2\cos(2\pi k/n), \qquad \chi(sr^k) = -1.$$

The complexified xy-plane carries ρ_1, while the z-axis carries ψ_2, which is 1 on rotations r^k and -1 on sr^k. Thus the decomposition is $\rho_1 \oplus \psi_2$ for every $n \geq 3$, whether n is odd or even. Both multiplicities are one.

Solution to Exercise 8.3

The cycle (123) rotates through $2\pi/3$. The transposition (12) reflects about the line at angle $\pi/3$, whereas (23) fixes the real axis. Consequently,

$$\rho((123)) = \frac{1}{2}\begin{pmatrix} -1 & -\sqrt{3} \\ \sqrt{3} & -1 \end{pmatrix}, \quad \rho((12)) = \frac{1}{2}\begin{pmatrix} -1 & \sqrt{3} \\ \sqrt{3} & 1 \end{pmatrix}, \quad \rho((23)) = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

In the class order identity, transposition, three-cycle, the class sizes are $1, 3, 2$ and the character table is

	1	(12)	(123)
1	1	1	1
ε	1	-1	1
χ_ρ	2	0	-1

.

The last row has squared norm $(4 + 2)/6 = 1$, proving irreducibility. The degree squares sum to $1 + 1 + 4 = 6$, proving completeness.

Solution to Exercise 8.4

In the class order $1, (12), (123), (1234), (12)(34)$, the vertex and edge permutation characters are respectively

$$\chi_V = (8, 0, 2, 0, 0), \qquad \chi_E = (12, 2, 0, 0, 0).$$

A nontrivial rotation fixes vertices only when its axis is a body diagonal, in which case it fixes its two endpoints. A half-turn about an axis joining opposite edge midpoints fixes those two edges as unordered pairs, although it exchanges their endpoints. No other nonidentity rotation fixes an edge. Taking weighted inner products with the character table gives

$$V \simeq 1 \oplus \varepsilon \oplus s \oplus W, \qquad E \simeq 1 \oplus s^{\oplus 2} \oplus W \oplus W'.$$

The dimensions are $1 + 1 + 3 + 3 = 8$ and $1 + 6 + 3 + 2 = 12$.

Solution to Exercise 8.5

A double transposition g has order two, so its representing matrix is diagonalizable with eigenvalues in $\{1, -1\}$. Trace two in dimension two forces both eigenvalues to be one. The representation is therefore trivial on $H = \{1, (12)(34), (13)(24), (14)(23)\}$. A representation trivial on H defines a well-defined representation of G/H by $gH \mapsto \rho(g)$; pullback reverses this construction and preserves invariant subspaces.

The action on 12|34, 13|24, and 14|23 has kernel H. Its image contains a transposition and a three-cycle, and hence is all of $\mathfrak{S}_3$. Thus $\mathfrak{S}_4/H \simeq \mathfrak{S}_3$. The pullback of the standard two-dimensional representation has character $(2, 0, -1, 0, 2)$ and is W'.

Solution to Exercise 8.6

1. The first $p-1$ entries of a tuple in X determine the last, so $|X| = |G|^{p-1}$. Cyclic rotation preserves X: the product after one rotation is $g_1^{-1}(g_1 \cdots g_p)g_1 = 1$. Each orbit has size one or p. The fixed tuples are $(g, \ldots, g)$ with $g^p = 1$. Their number is therefore congruent to $|X|$ modulo p. It is a positive multiple of p, since the identity gives one fixed tuple and $p \mid |X|$. There is a nonidentity g with $g^p = 1$, necessarily of order p.
2. If ρ had degree two, divisibility of irreducible degrees would imply $2 \mid |G|$, and the preceding argument would give an element t of order two.
3. The kernel is normal, so simplicity and nontriviality of ρ make it trivial. The determinant homomorphism also has a normal kernel. If this kernel were trivial, G would embed in the abelian group $\mathbb{C}^*$ and be abelian. Hence $\det \rho(g) = 1$ for all g. The minimal polynomial of $\rho(t)$ divides $X^2 - 1$, so this matrix is diagonalizable with eigenvalues ± 1. Determinant one and faithfulness exclude I and force $\rho(t) = -I$. Since this matrix commutes with every $\rho(g)$, faithfulness makes t central. The center of a nonabelian simple group is trivial, a contradiction.

Solution to Exercise 8.7

1. The matrices $L(i), L(j), L(k)$ satisfy the displayed multiplication table. Bilinearity then gives $L(qq') = L(q)L(q')$ for all q, q'. The first column of $L(q)$ recovers q, so this map is injective; associativity of matrices proves associativity of quaternion multiplication. The map $\mathbb{R}[H_8] \to \mathbb{H}$ sends the formal basis vector δ_g to the quaternion g. Its kernel has basis
$$\delta_{-g} + \delta_g, \qquad g = 1, i, j, k.$$
It is the ideal generated by $\delta_{-1} + \delta_1$ and has dimension four. In the group algebra, δ_{-1} is an independent basis vector; it becomes $-\delta_1$ only in this quotient.
2. Direct multiplication verifies the same relations for M and gives
$$M(q)^* M(q) = (a^2 + b^2 + c^2 + d^2)I.$$
Thus the eight group elements have unitary images. A common invariant complex line would be an eigenspace of $M(i) = \operatorname{diag}(\mathrm{i}, -\mathrm{i})$, hence a coordinate line, but $M(j)$ exchanges these two lines. The representation is irreducible, with character $(2, -2, 0, 0, 0)$ on the classes $\{1\}, \{-1\}, \{\pm i\}, \{\pm j\}, \{\pm k\}$.
3. Every one-dimensional character factors through $H_8/\{\pm 1\} \simeq \mathbb{F}_2^2$. The signs assigned to i and j are independent and their product is the sign assigned to k. These four characters and M are complete because $4 \cdot 1^2 + 2^2 = 8$. In the prescribed element order, the rows of the following matrix A are an orthonormal basis for the normalized inner product on $\mathbb{C}[H_8]$:
$$A = \operatorname{diag}(1, 1, 1, 1, \sqrt{2}, \sqrt{2}, \sqrt{2}, \sqrt{2}) \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 \\ 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 \\ 1 & 1 & -1 & -1 & -1 & -1 & 1 & 1 \\ 1 & -1 & \mathrm{i} & -\mathrm{i} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 1 & -\mathrm{i} & \mathrm{i} \\ 0 & 0 & 0 & 0 & 1 & -1 & -\mathrm{i} & \mathrm{i} \\ 1 & -1 & -\mathrm{i} & \mathrm{i} & 0 & 0 & 0 & 0 \end{pmatrix}.$$
The last four rows are $\sqrt{2}$ times the four matrix entries of M. In particular, $AA^* = 8I$.

4. Products of these basis functions give an orthonormal basis on H_8^n, which has 8^n points. After fixing a compatible product ordering, the normalized Fourier coefficients of a signal f are

$$c = 8^{-n}\,\overline{A}^{\otimes n} f.$$

   For Euclidean unitary coordinates, use instead $U^{\otimes n} f$ with $U = \overline{A}/\sqrt{8}$. The conjugation comes from taking inner products with the basis functions. Successive transforms along the n array axes compute the tensor product, as in Program 9.

Solution to Exercise 8.8

A cube rotation is a signed coordinate permutation with determinant one. The sum of coordinate squares is therefore invariant. If its underlying permutation is σ and its three signs are $\epsilon_1, \epsilon_2, \epsilon_3$, then $\epsilon_1\epsilon_2\epsilon_3 = \operatorname{sgn}(\sigma)$. Consequently XYZ transforms by $\operatorname{sgn}(\sigma)$ and its square is invariant.

Expanding the four factors gives

$$F = 2(X^2Y^2 + X^2Z^2 + Y^2Z^2) - X^4 - Y^4 - Z^4.$$

This expression is invariant under all signed coordinate permutations, so both F and F^2 are invariant. The polynomial Q is alternating in X^2, Y^2, Z^2; it also transforms by $\operatorname{sgn}(\sigma)$. Thus Q^2 and $XYZ\,Q$ are invariant under proper cube rotations. The four planes occurring in F are perpendicular to the four body diagonal lines. Factoring Q into $X \pm Y$, $X \pm Z$, and $Y \pm Z$ gives six planes perpendicular to face-diagonal directions.

Solution to Exercise 8.9

1. Since $\dim C^\perp = n - k$, self-duality gives $n = 2k$.
2. MacWilliams gives $W_C(X, Y) = 2^{-k} W_C(X + Y, X - Y)$. Homogeneity of degree $n = 2k$ absorbs the factor as $2^{-n/2}$, proving invariance under A.
3. Put $t = \sqrt{2} - 1$, $\ell = X + tY$, and $m = Y - tX$. The substitution A fixes ℓ and negates m. Every invariant is therefore a polynomial in ℓ, m^2, which are algebraically independent. The quadratic $q = Y(X - Y)$ is invariant and is not a multiple of ℓ^2. Since the invariant quadratics have basis ℓ^2, m^2, it follows that m^2 is a linear combination of ℓ^2, q with a nonzero coefficient of q. Thus $\mathbb{C}[X, Y]^{G_1} = \mathbb{C}[\ell, q]$, again with independent generators.
4. For $c \in C = C^\perp$, the identity $c \cdot c = \sum_i c_i = 0$ in $\mathbb{F}_2$ makes its weight even. Hence $W_C(X, -Y) = W_C(X, Y)$. The matrix AD is rotation through $\pi/4$, of order eight, and D is a reflection conjugating it to its inverse. The resulting group has eight rotations and eight reflections. Although $-I = (AD)^4$ belongs to this group, invariance under $-I$ alone does not impose even powers of Y.
5. In the coordinates $z = X + \mathrm{i}Y$, $w = X - \mathrm{i}Y$, the rotation multiplies z, w by inverse primitive eighth roots and D exchanges them. A rotation-invariant monomial has exponent difference divisible by eight. After imposing the exchange symmetry, all invariants are generated by $t = zw$ and $u = z^8 + w^8$: the sums $z^{8r} + w^{8r}$ satisfy the recurrence with initial values $2, u$ and characteristic polynomial $T^2 - uT + t^8$. Direct expansion gives

$$t = P = X^2 + Y^2, \qquad u = 2P^4 - 64Q, \quad Q = X^2Y^2(X^2 - Y^2)^2.$$

   These generators are algebraically independent. For example, the Jacobian of $(zw, z^8 + w^8)$ is $8(w^8 - z^8)$, a nonzero polynomial. Thus $\mathbb{C}[X, Y]^{G_2} = \mathbb{C}[P, Q]$, in agreement with the Molien series $1/((1 - t^2)(1 - t^8))$. Every homogeneous self-dual weight enumerator consequently has the form

$$W_C = \sum_{j=0}^{\lfloor n/8 \rfloor} c_j P^{n/2-4j} Q^j.$$

   For the extended Hamming code, expansion gives $P^4 - 4Q = X^8 + 14X^4Y^4 + Y^8$, the enumerator found earlier.

Program 60 implements Reynolds averages of all monomials of degrees one through the group order. The helper `exponents_of_degree` enumerates their exponent tuples, and `matrix_group` in the same module constructs the finite group by exact matrix multiplication. In characteristic zero, Noether's bound makes

the output a generating set. It is generally redundant: normalizing and removing duplicate polynomials does not remove all algebraic dependencies. The two generators P, Q above are obtained by the additional mathematical argument, not by an automatic minimality claim for this routine.

```
import sympy as sp
from invariants import matrix_group, reynolds_generators
X, Y = sp.symbols("X Y")
A = sp.Matrix([[1, 1], [1, -1]]) / sp.sqrt(2)
D = sp.diag(1, -1)
generators = reynolds_generators(matrix_group([A, D]), (X, Y))
```

```
def exponents_of_degree(variables, degree):
    if variables == 1:
        yield (degree,)
    else:
        for first in range(degree+1):
            for rest in exponents_of_degree(variables-1, degree-first):
                yield (first,)+rest
```

```
def reynolds_generators(group, variables):
    """An overcomplete generating set from Noether's degree bound."""
    forms = [A*sp.Matrix(variables) for A in group]  # Inverses give the same average.
    found = {}
    for degree in range(1, len(group)+1):
        for powers in exponents_of_degree(len(variables), degree):
            average = sum(sp.prod(u**j for u, j in zip(row, powers)) for row in forms)/len(group)
            polynomial = sp.Poly(sp.expand(average), *variables, extension=True)
            if not polynomial.is_zero:
                normalized = polynomial.monic().as_expr()
                found.setdefault(normalized, None)
    return list(found)
```

Program 60. Exact Reynolds averages giving an overcomplete set of invariant generators.

APPENDIX A

Python programs for numerical computation

The programs in this appendix connect transform identities with working numerical algorithms. They implement fast transforms, integer products, and Fourier solvers for differential equations. Explicit conventions and independent reference calculations make it possible to understand both the algorithms and the limits of floating-point arithmetic.

Using the programs

The printed code is intended to be run alongside the mathematical derivations. This section explains how to use the accompanying modules and check a first transform against an independent implementation.

The accompanying `listings/` directory contains ordinary Python modules with their imports and shared helpers. The printed functions are extracted directly from these files; functions in the same module can call one another without being copied into separate files. From the source directory, this example makes the modules available and checks an FFT:

```
import sys
sys.path.insert(0, "listings")
import numpy as np
from fft_algorithms import fft_dit
x = np.array([1, 2, 0, -1], dtype=complex)
assert np.allclose(fft_dit(x), np.fft.fft(x))
```

The repository README gives installation and verification instructions. NumPy arrays use zero-based indices, `a*b` is entrywise multiplication, and `a @ b` is matrix multiplication. Numerical Fourier transforms use the negative exponential and an unnormalized forward transform; the inverse includes $1/N$. Unless otherwise stated, vectors are one-dimensional arrays, polynomial coefficients increase with degree, and real data may produce complex intermediate arrays. All random demonstrations fix their seed.

The implementations favor readable algorithms. Recursive versions allocate temporary arrays; they are not in-place routines. For large numerical computations, NumPy and SciPy provide compiled transforms; FFTW is another established implementation [28]. The checks supplied here compare the teaching algorithms with direct mathematical formulas and independent numerical or exact references.

A.1 Fast Walsh transform

The Walsh recursion provides a compact example of a transform built entirely from additions and subtractions. The implementation makes the tensor factorization visible and keeps the inverse normalization explicit.

The function `fwt`, in `transforms.py`, follows the tensor factorization of Section 2.2.2. Its length must be a positive power of two. The transform satisfies $H_N^2 = NI$, so applying the same function and dividing by N gives its inverse. The displayed recursion uses $O(N \log N)$ arithmetic and creates temporary arrays at each level.

```
def fwt(x):
    x = np.asarray(x, dtype=complex)
    n = x.size
    if x.ndim != 1 or n == 0 or n & (n-1):
        raise ValueError('Expected a nonempty vector of power-of-two length.')
    if n == 1:
        return x.copy()
    u, v = fwt(x[:n//2]), fwt(x[n//2:])
    return np.r_[u + v, u - v]
```

Program 61. A recursive fast Walsh transform.

A.2 Fast Hartley transform

The fast Hartley transform shows how paired frequencies allow a real-kernel transform to be evaluated recursively. The accompanying convolution routine demonstrates how those coefficients can be used directly in a calculation.

The module `hartley.py` implements the real-kernel transform of Section 5.1.2. The radix-two function `fht` calls `hartley_twiddle`, which combines the paired frequencies k and $-k$. Its inverse is again obtained by dividing the forward transform by N. For real input, this recursion uses real arithmetic throughout.

The convolution function accepts two nonempty vectors of the same length. It uses `hartley`, an FFT-based helper valid at arbitrary lengths, and combines the transforms of *both* inputs. The result is circular convolution. To compute linear convolution, first pad both vectors to a common length at least the sum of their lengths minus one.

```
def fht(x):
    """Radix-two fast Hartley transform; no complex arithmetic for real x."""
    x = np.asarray(x)
    n = x.size
    if x.ndim != 1 or n == 0 or n & (n-1):
        raise ValueError('Use a nonempty vector of power-of-two length.')
    if n == 1:
        return x.copy()
    even = fht(x[::2])
    odd = hartley_twiddle(fht(x[1::2]))
    return np.concatenate((even+odd, even-odd))
```

Program 62. A radix-two fast Hartley transform.

```
def hartley_twiddle(x, shift=.5):
    x = np.asarray(x)
    n = x.size
    angle = 2*np.pi*shift*np.arange(n)/n
    return np.cos(angle)*x + np.sin(angle)*x[(-np.arange(n)) % n]
```

Program 63. The paired-frequency twiddle operator for the FHT.

```
def hartley_convolution(a, b):
    """Circular convolution of equal-length vectors using Hartley transforms."""
    a, b = np.asarray(a), np.asarray(b)
    if a.ndim != 1 or a.shape != b.shape or a.size == 0:
        raise ValueError('The inputs must be nonempty vectors of equal length.')
    c, d = hartley(a), hartley(b)
    reverse = (-np.arange(a.size)) % a.size
    spectrum = (c*(d+d[reverse]) + c[reverse]*(d-d[reverse]))/2
    return hartley(spectrum)/a.size
```

Program 64. Circular convolution from Hartley transforms of both inputs.

A.3 Fast Fourier transforms

Different FFT factorizations compute the same transform but organize the data differently. These implementations place recursive, time-decimation, and frequency-decimation algorithms side by side so their indexing can be compared.

The module `fft_algorithms.py` provides three radix-two FFTs: `fft_recursive` follows the time-decimation identity directly; `fft_dit` iterates butterflies after bit reversal; `fft_dif` performs frequency-decimation butterflies before the final reordering. All return frequencies in the usual order and accept `inverse=True`, which reverses the phase and includes $1/N$. Their mathematical derivations are in Sections 3.2.1, 3.2.5, and 3.2.6.

The remaining helpers multiply by a linear phase, construct the full bit-reversal permutation, and reverse the specified number of bits of a single integer. The two iterative FFTs keep a working array and use temporary vectors for each butterfly block; no claim of strict in-place operation is needed.

```
def fft_recursive(x, inverse=False):
    x = np.asarray(x, dtype=complex)
    n = x.size
    if x.ndim != 1 or n == 0 or n & (n-1):
        raise ValueError('Expected a vector of power-of-two length.')
    if inverse:
        return np.conj(fft_recursive(np.conj(x)))/n
    if n == 1:
        return x.copy()
    a = fft_recursive(x[::2])
    b = phase_multiply(fft_recursive(x[1::2]), .5)
    return np.r_[a+b, a-b]
```

Program 65. A recursive radix-two FFT, with inverse support.

```
def fft_dit(x, inverse=False):
    x = np.asarray(x, dtype=complex)
    if x.ndim != 1:
        raise ValueError('Expected a vector.')
    n = x.size
    y = x[bit_reversal_indices(n)].copy()
    size = 2
    while size <= n:
        half = size//2
        w = np.exp((-1 if not inverse else 1)*2j*np.pi*np.arange(half)/size)
        for start in range(0, n, size):
            a = y[start:start+half].copy()
            b = w*y[start+half:start+size]
            y[start:start+half], y[start+half:start+size] = a+b, a-b
        size *= 2
    return y/n if inverse else y
```

Program 66. An iterative FFT with decimation in time.

```
def fft_dif(x, inverse=False):
    y = np.array(x, dtype=complex, copy=True)
    if y.ndim != 1:
        raise ValueError('Expected a vector.')
    n, size = y.size, y.size
    indices = bit_reversal_indices(n)
    while size > 1:
        half = size//2
        w = np.exp((-1 if not inverse else 1)*2j*np.pi*np.arange(half)/size)
        for start in range(0, n, size):
            a, b = y[start:start+half].copy(), y[start+half:start+size].copy()
            y[start:start+half], y[start+half:start+size] = a+b, w*(a-b)
        size //= 2
    return y[indices]/n if inverse else y[indices]
```

Program 67. An iterative FFT with decimation in frequency.

```
def phase_multiply(x, alpha):
    x = np.asarray(x, dtype=complex)
    if x.ndim != 1 or x.size == 0:
        raise ValueError('Expected a nonempty vector.')
    return x * np.exp(-2j*np.pi*alpha*np.arange(x.size)/x.size)
```

Program 68. Multiplication by a linear phase.

```
def bit_reversal_indices(n):
    if n < 1 or n & (n-1):
        raise ValueError('Expected a positive power of two.')
    indices = np.array([0])
    while indices.size < n:
        indices = np.r_[2*indices, 2*indices+1]
    return indices
```

Program 69. Bit-reversal indices in linear total work.

```
def reverse_index(i, bits):
    if bits < 0 or not 0 <= i < 2**bits:
        raise ValueError('Index does not fit in the specified bit count.')
    result = 0
    for _ in range(bits):
        result, i = 2*result+(i & 1), i >> 1
    return result
```

Program 70. Reversing the bits of a single index.

A.4 Integer multiplication by FFT

FFT multiplication connects an approximate numerical transform with an exact integer output. The code separates digit conversion, convolution, rounding, and carry propagation, then checks the final product independently.

The module `polynomials.py` stores the base-b digits of a nonnegative integer from least to most significant. The conversion functions use exact Python integers, whose precision is not fixed. The function `multiply_digits` pads to cover the linear convolution, rounds its floating-point coefficients, and propagates the carries. It rejects an excessive coefficient bound and independently verifies the reconstructed result using Python's exact multiplication. This verification is deliberate: rounding near an integer alone cannot certify a floating-point computation.

The short example compares a transform product with the exact result. For an algorithm whose transform arithmetic is itself exact, use the modular method in Section B.3. Neither teaching implementation is intended to improve on Python's built-in large-integer multiplication.

```
def multiply_digits(x, y, base):
    exact = digits_to_integer(x, base)*digits_to_integer(y, base)
    if not x or not y or min(len(x), len(y))*(base-1)**2 > 2**40:
        raise ValueError('Use nonempty digit vectors with a moderate coefficient bound.')
    length = 1 << (len(x)+len(y)-2).bit_length()
    product = np.fft.irfft(np.fft.rfft(x, length)*np.fft.rfft(y, length), length)
    coefficients = [int(v) for v in np.rint(product[:len(x)+len(y)-1])]
    if min(coefficients) < 0:
        raise ArithmeticError('FFT error produced a negative coefficient.')
    digits, carry = [], 0
    for coefficient in coefficients:
        carry, digit = divmod(coefficient+carry, base)
        digits.append(digit)
    while carry:
        carry, digit = divmod(carry, base)
        digits.append(digit)
    while len(digits) > 1 and digits[-1] == 0:
        digits.pop()
    if digits_to_integer(digits, base) != exact:
        raise ArithmeticError('FFT product failed exact verification.')
    return digits
```

Program 71. FFT digit convolution and carry propagation, with exact verification.

```
def integer_to_digits(value, base):
    if not isinstance(value, int) or value < 0 or not isinstance(base, int) or base < 2:
        raise ValueError('Use a nonnegative integer and an integer base >= 2.')
    digits = []
    while value:
        value, digit = divmod(value, base)
        digits.append(digit)
    return digits or [0]
```

Program 72. Exact conversion from a nonnegative integer to base-b digits.

```
def digits_to_integer(digits, base):
    if not isinstance(base, int) or base < 2:
        raise ValueError('Base must be an integer >= 2.')
    value = 0
    for digit in reversed(digits):
        if int(digit) != digit or not 0 <= digit < base:
            raise ValueError('Digits must be integers in range(base).')
        value = base*value+int(digit)
    return value
```

Program 73. Reconstruction of an integer from its digits.

```
def integer_product_example():
    a, b, base = 262122154512154212, 314134464653513212, 20
    digits = multiply_digits(integer_to_digits(a, base), integer_to_digits(b, base), base)
    return digits_to_integer(digits, base) == a*b
```

Program 74. Verification of a large-integer product.

A.5 The Dirichlet Poisson equation

The Poisson solver turns a boundary-value problem into a transform calculation on an extended grid. Its interface makes the source, boundary values, and grid spacing explicit so that the numerical result can be checked against a manufactured solution.

The routines in `pde.py` implement the finite-difference problem of Section 4.4.3. The full source and boundary arrays share a grid; only the boundary entries of the second array are used. Boundary contributions are subtracted from the interior right-hand side, which is solved by an odd extension and a multidimensional FFT. The helper `poisson_zero_boundary` includes the grid-spacing factor in the discrete Laplacian symbol.

The manufactured solution $u(x, y) = e^{xy}$ gives $\Delta u = (x^2 + y^2)e^{xy}$ and supplies exact boundary values on the unit square. Its left and bottom edges are constant one, while the right and top edges are e^y and e^x. The driver computes the grid, numerical solution, and reference array. Refining the grid checks second-order spatial convergence; the manufactured solution is a verification device, not information used in the interior solve.

```
def poisson_zero_boundary(rhs, h):
    rhs = np.asarray(rhs)
    if h <= 0 or rhs.ndim < 1 or min(rhs.shape) == 0:
        raise ValueError('Use a nonempty interior grid and positive spacing.')
    extended = odd_extend(rhs)
    symbol = np.zeros(extended.shape)
    for axis, length in enumerate(extended.shape):
        shape = [1]*rhs.ndim; shape[axis] = length
        symbol -= 4*np.sin(np.pi*np.arange(length)/length).reshape(shape)**2/h**2
    zero = (0,)*rhs.ndim
    symbol[zero] = 1
    spectrum = np.fft.fftn(extended)/symbol
    spectrum[zero] = 0
    solution = np.fft.ifftn(spectrum)
    interior = tuple(slice(1, n+1) for n in rhs.shape)
    return solution[interior].real if np.isrealobj(rhs) else solution[interior]
```

```
def poisson_dirichlet(source, boundary, h):
    source, boundary = np.asarray(source), np.asarray(boundary)
    if source.shape != boundary.shape or min(source.shape) < 3:
        raise ValueError('Source and boundary arrays must share a grid with an interior.')
    center = (slice(1, -1),)*source.ndim
    result = np.array(boundary, dtype=np.result_type(source, boundary, float), copy=True)
    result[center] = 0
    rhs = np.array(source[center], dtype=result.dtype, copy=True)
    for axis in range(source.ndim):
        neighbors = np.roll(result, 1, axis)+np.roll(result, -1, axis)
        rhs -= neighbors[center]/h**2
    result[center] = poisson_zero_boundary(rhs, h)
    return result
```

```
def poisson_example(n=32):
    grid = np.linspace(0, 1, n+1)
    X, Y = np.meshgrid(grid, grid, indexing='ij')
    exact = poisson_exact(X, Y)
    solution = poisson_dirichlet(poisson_source(X, Y), exact, 1/n)
    return grid, solution, exact
```

Program 75. A Dirichlet Poisson solver and a manufactured-solution example.

```
def poisson_source(x, y):
    return (x*x+y*y)*np.exp(x*y)
```

Program 76. The source term for the exact solution e^{xy}.

```
def poisson_exact(x, y):
    return np.exp(x*y)
```

Program 77. An exact reference solution for the Poisson test.

```
def poisson_left_boundary(y):
    return poisson_exact(0, y)
```

Program 78. The reference boundary value at $x = 0$.

A.6 The periodic heat equation

The periodic heat equation is especially simple in Fourier coordinates because each mode evolves independently. The program evaluates this evolution directly at selected times, making the effect of diffusion visible without a time-stepping loop.

The function `heat_periodic`, also in `pde.py`, evolves a periodic trigonometric interpolant for $\partial_t u = \kappa\, \partial_{xx} u$. On a period L, each Fourier coefficient is multiplied by $\exp(-\kappa t(2\pi k/L)^2)$. The grid omits the repeated endpoint. This is spectral evolution at the requested time, rather than a time-stepping approximation.

The driver starts from the interval indicator returned by `heat_initial` and returns the solutions at several times. Changing the initial array, conductivity, period, or time does not change the transform algorithm. Section 4.4.2 also discusses finite-difference time stepping and its separate stability conditions.

```
def heat_example(n=256, times=(0, .001, .003, .01, .03, .1)):
    grid = np.arange(n)/n
    initial = heat_initial(grid)
    return grid, np.stack([heat_periodic(initial, t) for t in times])
```

Program 79. Heat evolution at several times from the same initial data.

```
def heat_periodic(x, time, kappa=.5, period=1):
    x = np.asarray(x)
    if x.ndim != 1 or x.size == 0 or time < 0 or kappa < 0 or period <= 0:
        raise ValueError('Invalid heat equation data.')
    omega = 2*np.pi*np.fft.fftfreq(x.size, d=period/x.size)
    y = np.fft.ifft(np.exp(-kappa*time*omega**2)*np.fft.fft(x))
    return y.real if np.isrealobj(x) else y
```

Program 80. Spectral heat evolution with an explicit period and conductivity.

```
def heat_initial(x):
    return ((np.asarray(x) >= .3) & (np.asarray(x) < .7)).astype(float)
```

Program 81. An interval indicator as initial temperature.

APPENDIX B

Python programs for exact arithmetic

Exact arithmetic separates algebraic identities from rounding error. These programs construct finite fields, carry out modular Fourier transforms, and implement the encoding and decoding procedures of Chapter 6. Their small examples are checked against direct transforms, integer products, and code constraints.

The modules `finite_fields.py` and `error_correcting_codes.py` use SymPy for integer and polynomial arithmetic, and `galois` for finite-field arrays. Their imports, shared helpers, and executable checks are supplied with the complete Python sources. Array coordinates increase with polynomial degree; a polynomial library may instead print its coefficients in descending order, so the conversion is explicit. All random examples use a fixed seed.

B.1 Fourier transforms over finite fields

Finite-field transforms make the algebraic inversion and convolution identities executable without rounding. The programs construct a suitable coefficient field, verify the root order, and compare fast evaluation with the direct sum.

Program 82 factors Φ_n over $\mathbb{F}_p$ and uses one irreducible factor to construct a field containing a root of *exact* order n. It rejects $p \mid n$. The example takes $p = 2$ and $n = 15$, so the transform values lie in an extension of the binary input alphabet. The module also supplies a mixed-radix FFT, checked against the direct transform here.

In `galois`, an integer used to construct an extension-field element encodes its polynomial-basis coordinates. Consequently, the scalar $n\,1_K$ is constructed as `field(n % p)`, not `field(n)`. This matters in the inverse transform's normalization.

```
def field_with_root(p, n, modulus=None):
    """A minimal extension of F_p containing a root of exact order n."""
    if not sp.isprime(p) or n < 1 or gcd(p, n) != 1:
        raise ValueError('Use prime p and positive n coprime to p.')
    prime = galois.GF(p, compile='python-calculate')
    if modulus is None:
        coefficients = [int(c) % p for c in cyclotomic_example(n).all_coeffs()]
        factors, _ = galois.Poly(coefficients, field=prime).factors()
        polynomial = factors[0]
    else:
        polynomial = galois.Poly(modulus, field=prime)
    if polynomial.degree == 1:
        field, root = prime, -polynomial.coeffs[1]/polynomial.coeffs[0]
    else:
        field = galois.GF(p**polynomial.degree, irreducible_poly=polynomial,
                          compile='python-calculate')
        root = field(p)  # The class of X in the polynomial-basis representation.
    if root**n != 1 or any(root**(n//ell) == 1 for ell in sp.factorint(n)):
        raise ValueError('The selected polynomial does not give a root of order n.')
    return field, root
```

```
def finite_dft(x, root, inverse=False):
    field = type(root)
    x = field(x)
    n = x.size
    if x.ndim != 1 or n == 0 or root**n != 1:
        raise ValueError('The input length must match the root order.')
    if any(root**(n//ell) == 1 for ell in sp.factorint(n)):
        raise ValueError('The root must have exact order n.')
    powers = np.outer(np.arange(n), np.arange(n))
    y = (root**(powers if inverse else -powers)) @ x
    return y/field(n % field.characteristic) if inverse else y
```

```
def finite_transform_example():
    field, root = field_with_root(2, 15)
    x = field(np.arange(15) % 2)
    y = finite_dft(x, root)
    assert np.array_equal(finite_dft(y, root, inverse=True), x)
    assert np.array_equal(finite_fft(x, root), y)
    return x, y
```

Program 82. Constructing and checking an exact finite-field Fourier transform.

B.2 Fourier transforms over residue rings

Residue-ring transforms offer exact modular arithmetic without an extension-field representation. The implementation checks the conditions needed for a dyadic FFT and makes the modular inverse normalization explicit.

Program 83 uses Python integers reduced modulo m. For a dyadic transform length $n > 1$, it checks that n is invertible and $\zeta^{n/2} = -1 \pmod{m}$. These hypotheses make ζ a principal root by Proposition 50. A modulus need not be prime: the transform works, for instance, modulo the composite Fermat number $2^{32} + 1$. The inverse uses the inverse root and multiplication by $n^{-1} \pmod{m}$.

```
def _modular_fft(x, root, modulus):
    n = len(x)
    if n == 1:
        return x.copy()
    even = _modular_fft(x[::2], root*root % modulus, modulus)
    odd = _modular_fft(x[1::2], root*root % modulus, modulus)
    inverse_root = pow(root, -1, modulus)
    mixed = [value*pow(inverse_root, k, modulus) % modulus for k, value in enumerate(odd)]
    return ([(a+b) % modulus for a, b in zip(even, mixed)]
            + [(a-b) % modulus for a, b in zip(even, mixed)])
```

```
def modular_fft(x, modulus, root, inverse=False):
    x = [int(a) % modulus for a in x]
    n = len(x)
    if modulus < 2 or n == 0 or n & (n-1) or gcd(n, modulus) != 1:
        raise ValueError('Use dyadic length invertible modulo the modulus.')
    if pow(root, n, modulus) != 1 or (n > 1 and pow(root, n//2, modulus) != modulus-1):
        raise ValueError('The root must be a principal dyadic root.')
    y = _modular_fft(x, pow(root, -1, modulus) if inverse else root, modulus)
    return [a*pow(n, -1, modulus) % modulus for a in y] if inverse else y
```

Program 83. A radix-two Fourier transform with modular integer arithmetic.

B.3 Multiplication of large integers

Recovering an integer product from modular convolution requires a modulus large enough to prevent coefficient ambiguity. The program chooses that modulus from a bound and then performs exact carry propagation.

The digit base is an input parameter, and the transform length covers the full linear convolution. Program 84 chooses a modulus $m = 2^{cn/2} + 1$ larger than the coefficient bound; 2^c is then a principal nth root. The inverse transform returns the exact nonnegative convolution coefficients, after which carry propagation recovers the product. Signed inputs and zero are handled separately. The digit-conversion helpers are given in Programs 72 and 73.

Python's built-in integers already support arbitrary precision. The purpose of this implementation is to expose the transform-based algorithm and its bounds, rather than to extend Python's range or promise a faster product. The tests compare the result with independent built-in multiplication, including operands with hundreds of decimal digits.

```
def multiply_integers_modular(a, b, base=10000):
    """Exact integer multiplication via an NTT, without floating-point rounding."""
    if a == 0 or b == 0:
        return 0
    x, y = integer_to_digits(abs(a), base), integer_to_digits(abs(b), base)
    size = max(2, 1 << (len(x)+len(y)-2).bit_length())
    bound = min(len(x), len(y))*(base-1)**2
    step = max(1, (bound.bit_length()+size//2-1)//(size//2))
    modulus, root = (1 << (step*size//2))+1, 1 << step
    X = modular_fft(x+[0]*(size-len(x)), modulus, root)
    Y = modular_fft(y+[0]*(size-len(y)), modulus, root)
    coefficients = modular_fft([u*v % modulus for u, v in zip(X, Y)], modulus, root, True)
    digits, carry = [], 0
    for value in coefficients:
        carry, digit = divmod(value+carry, base)
        digits.append(digit)
    while carry:
        carry, digit = divmod(carry, base)
        digits.append(digit)
    return (1 if (a > 0) == (b > 0) else -1)*digits_to_integer(digits, base)
```

Program 84. Exact integer multiplication with a sufficiently large modular transform.

B.4 BCH encoding and decoding

The BCH programs connect generator-polynomial construction with recovery of a corrupted message. They follow the syndrome and locator argument of the chapter, including the case in which fewer errors occur than the code can correct.

The implementation has four parts:

1. Program 85 takes the union of the cyclotomic cosets meeting $1, \ldots, \delta - 1$ and constructs the generator polynomial. The alphabet is $\mathbb{F}_p$, while root and syndrome computations take place in a suitable extension.
2. Program 86 multiplies a message polynomial by the generator, then illustrates correction of three errors in a binary length-31 code of designed distance seven.
3. Program 87 determines the actual error count from a Hankel matrix and solves for the minimal locator. This includes the zero-error case and fewer errors than the permitted maximum.
4. Program 88 either completes the syndrome spectrum or finds the locator roots and solves for error magnitudes. Both routes verify the alphabet, error count, and all code constraints.

The rank calculation is exact finite-field elimination. Outside the stated correction radius, successful checks do not identify the transmitted word uniquely; this is the usual limitation of bounded-distance decoding.

```
def cyclotomic_coset(p, n, index):
    orbit = []
    while index not in orbit:
        orbit.append(index)
        index = p*index % n
    return orbit
```

```
def bch_code(p=2, n=31, distance=7, modulus=None):
    if not 2 <= distance <= n:
        raise ValueError('The designed distance must lie between 2 and n.')
    field, root = field_with_root(p, n, modulus)
    indices = sorted({j for k in range(1, distance) for j in cyclotomic_coset(p, n, k)})
    polynomial = galois.Poly.Roots(root**np.array(indices))
    assert np.all(polynomial.coeffs**p == polynomial.coeffs)
    prime = galois.GF(p, compile='python-calculate')
    generator = prime([int(c) for c in polynomial.coeffs[::-1]])
    return dict(p=p, n=n, k=n-polynomial.degree, t=(distance-1)//2,
                field=field, root=root, generator=generator, distance=distance)
```

Program 85. Constructing the generator of a prime-alphabet BCH code.

```
def encode_bch(message, code):
    prime = type(code['generator'])
    message = prime(message)
    if message.shape != (code['k'],):
        raise ValueError('The message length must equal the code dimension.')
    word = prime.Zeros(code['n'])
    g = code['generator']
    for j, value in enumerate(message):
        word[j:j+g.size] += value*g
    return word
```

```
def bch_example():
    code = bch_code()
    message = np.random.default_rng(61).integers(0, 2, code['k'])
    word = encode_bch(message, code)
    received = word.copy()
    received[[0, 12, 30]] += type(word)(1)
    corrected = decode_bch(received, code)
    assert np.array_equal(corrected, word)
    assert np.array_equal(corrected, decode_bch(received, code, method='roots'))
    return code, word, received, corrected
```

Program 86. BCH encoding and an independently checked decoding example.

```
def error_locator(syndromes):
    """Minimal locator from S_1,...,S_(2t), including fewer-than-t errors."""
    field = type(syndromes)
    t = syndromes.size//2
    indices = np.add.outer(np.arange(t), np.arange(t))
    hankel = syndromes[indices]
    degree = int(np.count_nonzero(np.any(hankel.row_reduce() != 0, axis=1)))
    if degree == 0:
        if np.any(syndromes):
            raise ValueError('The syndromes are inconsistent with the error bound.')
        return field([1])
    indices = degree+np.arange(degree)[:, None]-np.arange(1, degree+1)[None, :]
    tail = np.linalg.solve(syndromes[indices], -syndromes[degree:2*degree])
    locator = field(np.r_[1, tail])
    for j in range(degree, 2*t):
        if syndromes[j]+np.sum(tail*syndromes[j-np.arange(1, degree+1)]) != 0:
            raise ValueError('No locator fits the supplied syndromes.')
    return locator
```

Program 87. Recovering the actual locator degree and its coefficients.

```
def decode_bch(received, code, method='spectrum'):
    field, root, n, t = code['field'], code['root'], code['n'], code['t']
    prime = type(code['generator'])
    received = prime(received)
    if received.shape != (n,) or t < 1:
        raise ValueError('Use a code with positive correction radius and matching word length.')
    S = (root**np.outer(np.arange(1, 2*t+1), np.arange(n))) @ field(received)
    locator = error_locator(S)
    degree = locator.size-1
    if method == 'spectrum':
        spectrum = field.Zeros(n+1)
        spectrum[1:2*t+1] = S
        for j in range(2*t+1, n+1):
            if degree:
                spectrum[j] = -np.sum(locator[1:]*spectrum[j-np.arange(1, degree+1)])
        spectrum[0] = spectrum[n]
        errors = finite_dft(spectrum[:n], root**-1, inverse=True)
    elif method == 'roots':
        values = (root**(-np.outer(np.arange(n), np.arange(degree+1)))) @ locator
        positions = np.flatnonzero(values == 0)
        if positions.size != degree:
            raise ValueError('The locator does not have the required number of roots.')
        errors = field.Zeros(n)
        if degree:
            matrix = root**np.outer(np.arange(1, degree+1), positions)
            errors[positions] = np.linalg.solve(matrix, S[:degree])
    else:
        raise ValueError('Choose spectrum or roots.')
    if np.any(errors**code['p'] != errors) or np.count_nonzero(errors) > t:
        raise ValueError('The reconstructed error is outside the permitted alphabet or radius.')
    corrected = field(received)-errors
    check = (root**np.outer(np.arange(1, code['distance']), np.arange(n))) @ corrected
    if np.any(check):
        raise ValueError('The corrected word does not satisfy the BCH checks.')
    return prime([int(c) for c in corrected])
```

Program 88. BCH decoding by spectrum completion or locator roots.

References

[1] Jonathan L. Alperin and Rowen B. Bell. *Groups and representations*. Springer-Verlag, 1995.

[2] Jörg Arndt. *Algorithms for programmers*. 2002.

[3] Michael Artin. *Algebra*. Prentice Hall, 1991.

[4] László Babai. The Fourier transform and equations over finite abelian groups. *University of Chicago, Technical Report TR-2001-01*, 2001.

[5] David H. Bailey. The computation of π to 29,360,000 decimal digits using Borweins' quartically convergent algorithm. *Mathematics of Computation*, 50(181):283–296, 1988.

[6] David H. Bailey and Paul N. Swarztrauber. The fractional Fourier transform and applications. *SIAM Review*, 33(3):389–404, 1991. See also the author's errata.

[7] Riccardo Bernardini and Jelena Kovačević. Designing local orthogonal bases on finite groups I: abelian case. *Journal of Fourier Analysis and Applications*, 6(1):1–23, 2000.

[8] Dan Boneh. Learning using group representations. *Proc. COLT 1995*, pages 418–426, 1995.

[9] John P. Boyd. *Chebyshev and Fourier Spectral Methods*. Dover Publications, 2000.

[10] Ronald Bracewell. *The Hartley transform*. Oxford University Press, 1986.

[11] E. Oran Brigham. *Fast Fourier Transform and its Applications*. Prentice Hall, 1988.

[12] Charles S. Burrus. Notes on the FFT. 1997.

[13] Gianfranco Cariolaro, Tomaso Erseghe, Peter Kraniauskas, and Nicola Laurenti. Multiplicity of fractional Fourier transforms and their relationships. *IEEE Transactions on Signal Processing*, 48(1):227–241, 2000.

[14] Henri Cartan. *Théorie élémentaire des fonctions d'une variable complexe*. Hermann, 1961.

[15] Çağatay Candan, M. Alper Kutay, and Haldun M. Özaktaş. The discrete fractional Fourier transform. *IEEE Trans. on Signal Processing*, 48:1329–1337, 2000.

[16] Philippe G. Ciarlet. *Introduction à lánalyse numérique et à lóptimisation*. Dunod, 1990.

[17] Henri Cohen. *A course in computational algebraic number theory*. Springer-Verlag, 1993.

[18] Michael J. Collins. *Representations and characters of finite groups*. Cambridge University Press, 1990.

[19] David Cox, John Little, and Donal O'Shea. *Ideals, Varieties, and Algorithms: An Introduction to Computational Algebraic Geometry and Commutative Algebra*. Springer-Verlag, second edition, 1997.

[20] Michel Demazure. *Cours d'algèbre. Primalité, divisibilité, codes.* Cassini, 1997.

[21] Gilbert Demengel. *Transformées de Fourier généralisées*. Ellipses, 1999.

[22] Persi Diaconis. *Group Representations in Probability and Statistics*. IMS Lecture Series 11, Institute of Mathematical Statistics, Hayward, California, 1988.

[23] Bradley W. Dickinson and Kenneth Steiglitz. Eigenvectors and functions of the discrete Fourier transform. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 30(1):25–31, 1982.

[24] David L. Donoho and Philip B. Stark. Uncertainty principles and signal recovery. *SIAM Journal on Applied Mathematics*, 49:906–931, 1989.

[25] Harry Dym and Henry P. McKean. *Fourier series and integrals*. Academic Press, 1972.

[26] Noam D. Elkies. Lattices, linear codes, and invariants, part I. *Notices of the AMS*, 47(10):1238–1245, 2000.

[27] Kristin Flornes, Alex Grossmann, Matthias Holschneider, and Bruno Torrésani. Wavelets on discrete fields. *Applied and Computational Harmonic Analysis*, 1:137–147, 1994.

[28] Matteo Frigo and Steven G. Johnson. FFTW: An adaptive software architecture for the FFT. *ICASSP conference proceedings*, 3:1381–1384, 1998.

[29] William Fulton and Joe Harris. *Representation theory : a first course*. Springer-Verlag, 1991.

[30] Roe Goodman and Nolan R. Wallach. *Representations and invariants of the classical groups*. Cambridge University Press, 1999.

[31] Ronald L. Graham, Donald E. Knuth, and Oren Patashnik. *Concrete Mathematics*. Addison-Wesley, 1994.

[32] International ISBN Agency. What is an ISBN? https://www.isbn-international.org/index.php/content/what-isbn/10. Accessed September 2026.

[33] Eyal Kushilevitz and Yishay Mansour. Learning decision trees using the Fourier spectrum. *SIAM Journal on Computing*, 22(6):1331–1348, 1993.

[34] Tsit Yuen Lam. Representations of finite groups: A hundred years, part I. *Notices of the AMS*, 45(3):368–372, 1998.

[35] Tsit Yuen Lam. Representations of finite groups: A hundred years, part II. *Notices of the AMS*, 45(4):465–474, 1998.

[36] Serge Lang. *Algebra*. Addison-Wesley, 1965.

[37] Philippe Langevin. *Les sommes de caractères et la formule de Poisson dans la théorie des codes, des séquences et des fonctions booléennes*. Université de Toulon, 1999.

[38] Franz Lemmermeyer. *Reciprocity Laws: From Euler to Eisenstein*. Springer-Verlag, 2000.

[39] J.P. Lewis. Fast normalized cross-correlation. *Vision Interface*, pages 120–123, 1995.

[40] Rudolf Lidl and Harald Niederreiter. *Finite fields*, volume 20 of *Encyclopedia of Mathematics and its Applications*. Addison-Wesley, 1983.

[41] F. Jessie MacWilliams and Neil J. A. Sloane. *The theory of error-correcting codes, Part I*. North-Holland, 1977.

[42] Stéphane Mallat. *Une exploration des signaux en ondelettes*. Editions de l'école Polytechnique, 2000.

[43] David K. Maslen and Daniel N. Rockmore. Generalized FFTs—a survey of some recent results. In *Groups and Computation II*, volume 28 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, pages 183–237. American Mathematical Society, 1997.

[44] Ewa Matusiak, Murad Özaydin, and Tomasz Przebinda. The Donoho–Stark uncertainty principle for a finite abelian group. *Acta Mathematica Universitatis Comenianae. New Series*, 73(2):155–160, 2004.

[45] Jean-Yves Ouvrard. *Probabilité 2*. Cassini, 2000.

[46] Odile Papini and Jacques Wolfmann. *Algèbre discrète et codes correcteurs*. Springer-Verlag, 1995.

[47] Enes Pasalić and Thomas Johansson. Further results on the relation between nonlinearity and resiliency of Boolean functions. *Cryptography and Coding, Lecture Notes in Computer Science*, 1746:35–44, 1999.

[48] William H. Press, Brian P. Flannery, Saul A. Teukolsky, and William T. Vetterling. *Numerical Recipes in C: The Art of Scientific Computing*. Cambridge University Press, 1988.

[49] Ramis, Deschamps, and Odoux. *Tome 1 : algèbre*. Masson, 1979.

[50] Daniel N. Rockmore. The FFT - an algorithm the whole family can use. *Computing in Science and Engineering*, 2(1):60–64, 2000.

[51] Oscar S. Rothaus. On bent functions. *Journal of Combinatorial Theory, Series A*, 20(3):300–305, 1976.

[52] Walter Rudin. *Analyse réelle et complexe*. Dunod, 1987.

[53] Pierre Samuel. *Théorie des nombres*. Hermann, 1967.

[54] Jean-Pierre Serre. *Représentations linéaires des groupes finis*. Hermann, 1966.

[55] Jean-Pierre Serre. *Cours d'arithmétique*. PUF, 1970.

[56] Gilbert Strang. The discrete cosine transform. *SIAM Review*, 41(1):135–147, 1999.

[57] Paul N. Swarztrauber and Roland A. Sweet. The Fourier and cyclic reduction for solving Poisson's equation. *Handbook of Fluid Dynamics and Fluid Machinery*, 1996.

[58] Paul N. Swarztrauber, Roland A. Sweet, William L. Briggs, Van-Emden Henson, and James Otto. Bluestein's FFT for arbitrary N on the hypercube. *Parallel Computing*, Vol.17:607–617, 1991.

[59] Audrey Terras. *Fourier analysis on finite groups and applications*, volume 43 of *London Mathematical Society Student Texts*. Cambridge University Press, 1999.

[60] Lloyd N. Trefethen and J. A. C. Weideman. The exponentially convergent trapezoidal rule. *SIAM Review*, 56(3):385–458, 2014.

[61] Ronald F. Ullmann. An algorithm for the fast Hartley transform. *Stanford Exploration Project*, SEP-38, 1984.

[62] Michael Unser. Splines: A perfect fit for signal and image processing. *IEEE Signal Processing Magazine*, 16(6):22–38, 1999. See the author's online erratum for the anticausal initialization sign.

[63] Michael Unser, Akram Aldroubi, and Murray Eden. Fast B-spline transforms for continuous image representation and interpolation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 13(3):277–285, 1991.

[64] Martin Vetterli and Pierre Duhamel. Split-radix algorithms for length-p^m DFT's. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 37(1):57–64, 1989.

[65] Robert Vích. *Z Transform Theory and Applications*. D. Reidel, 1987.

[66] André Warusfel. *Structures algébriques finies*. Hachette, 1971.

[67] Herbert S. Wilf. *Generatingfunctionology*. Academic Press, 1990.